



RapidResponse Scripting Guide

SU 2208

Kinaxis Confidential Information - for use by Kinaxis and authorized customers and partners of Kinaxis only.

All Specifications, claims, features, representations, and/or comparisons provided are correct to the best of our knowledge as of the date of publication, but are subject to change without notice. While we will always strive to ensure our documentation is accurate and complete, this document may also contain errors and omissions of which we are not aware.

THIS INFORMATION IS PROVIDED BY KINAXIS ON AN "AS IS" BASIS, WITHOUT ANY OTHER WARRANTIES OR CONDITIONS, EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, WARRANTIES OF MERCHANTABILITY, SATISFACTORY QUALITY, MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE, OR THOSE ARISING BY LAW, STATUTE, USAGE OF TRADE, COURSE OF DEALING OR OTHERWISE. YOU ASSUME THE ENTIRE RISK AS TO THE RESULTS OF THE INFORMATION PROVIDED. WE SHALL HAVE NO LIABILITY TO YOU OR ANY OTHER PERSON OR ENTITY FOR ANY INDIRECT, INCIDENTAL, SPECIAL, OR CONSEQUENTIAL DAMAGES WHATSOEVER, INCLUDING, BUT NOT LIMITED TO, LOSS OF REVENUE OR PROFIT, LOST OR DAMAGED DATA OR OTHER COMMERCIAL OR ECONOMIC LOSS, EVEN IF WE HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES, OR THEY ARE FORESEEABLE. WE ARE ALSO NOT RESPONSIBLE FOR CLAIMS BY A THIRD PARTY. OUR MAXIMUM AGGREGATE LIABILITY TO YOU AND THAT OF OUR DEALERS AND SUPPLIERS SHALL NOT EXCEED THE COSTS PAID BY YOU TO PURCHASE THIS DOCUMENT. SOME STATES/COUNTRIES DO NOT ALLOW THE EXCLUSION OR LIMITATION OF LIABILITY FOR CONSEQUENTIAL OR INCIDENTAL DAMAGES, SO THE ABOVE LIMITATIONS MAY NOT APPLY TO YOU. All product, font and company names are trademarks or registered trademarks of their respective owners and use of them does not imply any affiliation with or endorsement by them.

Copyright © 2011-2022 Kinaxis Inc. All rights reserved. This document may not be reproduced or transmitted in any form (whether now known or hereinafter discovered or developed), in whole or in part, without the express prior written consent of Kinaxis.

Kinaxis RapidResponse contains technology which is protected under US Patents #7,610,212, #7,698,348, #7,945,466, #8,015,044, #9,292,573, #9,710,501, #10,467,337, #10,776,260, #10,832,196, #10,846,651, and #10,936,501 by the United States Patent and Trademark Office. It is also protected under the Japan Patent Office, Japanese patent #4393993 and the Intellectual Property India Office, patents #255768 and #279101. Other patents are pending.

This document may include examples of fictitious companies, products, Web addresses, email addresses, and people. No association with any real company, product, Web address, email address, and person is intended or should be inferred.

RapidResponse Scripting Guide

Date of Publication: Tuesday, August 16, 2022

Published in Canada.

Support: support@kinaxis.com

Web site: <http://www.kinaxis.com>

Contents

Part 1: Overview	17
CHAPTER 1: Welcome	19
CHAPTER 2: About Kinaxis	21
Customer Support	21
CHAPTER 3: Accessing help and documentation	23
Determining which help system or guide to use	25
Documentation conventions	27
Finding the content you need in HTML help	27
Your feedback	29
Help embedded in resources	29
Other ways to learn about RapidResponse	34
CHAPTER 4: New and changed features	37
2022 scripting changes	38
2021 scripting changes	38
2020 scripting changes	41
2019 scripting changes	43
2018 scripting changes	45
2017 scripting changes	48
2016 scripting changes	49
Part 2: RapidResponse Scripting	51
CHAPTER 5: About RapidResponse scripts	53
Knowledge requirements for script authoring	54
About forms	55
CHAPTER 6: Developing scripts	59
Create a script	62
Script versioning	63
Create script code	64
Define arguments for a script	68

Specify dependencies	73
Test a script	74
Upgrade a script	75
Calling other scripts	77
Guidelines for creating scripts to be used by other scripts	77
Call a function from an included script	78
Call another script	78
Adding help for scripts	79
Create help for a script	80
Add author notes	81
Editing and managing scripts	82
Edit a script	82
Managing scripts	84
Custom messages in forms	84
Notices	85
Control errors	87
Using scripts to communicate with external systems	88
Sample scripts available for RapidResponse	89
Example: creating a scenario	89
Managing historical scenarios	90
Modify worksheet data	91
Create a new order using a form	91
CHAPTER 7: Running scripts	95
Schedule a script to run automatically	97
Script logging	97
Running scripts from Web services	99
Part 3: Script reference	101
CHAPTER 8: RapidResponse script object model	103
CHAPTER 9: Resource objects	109
HelpInformation object	110
Resource collections	110
Collection indexing	111
Collection iteration functions	112
get method	112
exists method	113
create method	114
remove method	115
Resource object methods	116
give method	116
makePublic method	117
rename method	119

copy method	120
accessControlList collection	121
remove method	121
add method	122
Principal objects	123
allow method	124
deny method	125
hasPermission method	126
Permission object	126
CHAPTER 10: Script objects	129
scripts collection	129
get method	130
exists method	130
remove method	131
Script object methods	132
give method	133
makePublic method	134
rename method	134
copy method	135
call method	136
CHAPTER 11: Scenario objects	139
ActivityLog objects	141
scenarios collection	142
get method	142
exists method	144
create method	145
remove method	147
createTemporary method	148
createWorkflowScenario method	149
Obtaining scenario sequence numbers	150
Scenario object methods	153
give method	154
makePublic method	155
rename method	156
update method	157
reset method	158
commit method	159
addResponse method	161
setProperties method	163
cdcInstances collection	164
get method	165
CDCInstance objects	165

captureChanges method	166
clearChanges method	167
resetChanges method	167
CHAPTER 12: Workbook objects	169
WorkbookPreferences objects	172
workbooks collection	172
get method	172
exists method	176
remove method	176
Workbook object methods	177
give method	178
makePublic method	179
rename method	180
copy method	181
generateReport method	181
dependencies collection	185
WorkbookDependency objects	186
variables collection	186
WorkbookVariable objects	186
VariableValueQuantity object	187
VariableValueBoolean object	188
VariableValueList object	188
NameValuePair object	189
commands collection	189
get method	189
exists method	190
Command object	191
DataUpdateCommand object	192
ScriptCommand object	192
OpenFormCommand object	193
execute method	193
CHAPTER 13: Worksheet objects	197
CrosstabWorksheet object	199
Worksheet arguments	199
worksheets collection	200
get method	200
exists method	201
Worksheet object methods	202
getAvailableHierarchies method	203
getChart method	203
getChartDefinition method	204
getData method	205

getFilterManager method	206
getPossibleControlValues method	206
getTreemapQuery method	208
importData method	208
convertToJSON method	210
columns collection	216
WorksheetColumn object	216
imageMap collection	216
WorksheetColumnImageMapping object	217
Worksheet data collections	217
Worksheet row and cell objects	217
Worksheet data objects	219
Worksheet data import objects	220
Worksheet data methods	221
get method	221
slice method	222
close method	223
WorksheetDataModification object	224
execute method	224
Worksheet bucketing objects	225
BasicBucketSettings object	226
AdvancedBucketSettings object	226
BucketInterval object	227
BucketOption object	227
BucketAnchorDate object	228
BucketDateOffset object	228
BucketStartDate object	229
BucketCalendarToDate object	229
AutobucketInfo object	230
DateBucket object	230
WorksheetColumnDrillToDetail object	231
columnMappings collection	232
DrillToDetailColumnMapping object	232
WorksheetColumnDrillToDetailBucketVariableMappings object	232
WorksheetColumnDrillToDetailColumnMapping object	233
DrillTarget object	233
CHAPTER 14: TreemapQuery object	235
TreemapQueryColorMeasure object	236
TreemapQueryDimension object	236
dimensions collection	236
Treemap data objects	237
Treemap data collections	238

TreemapDrillToDetail object	238
columnMappings collection	239
TreemapDrillToDetailMapping object	239
Treemap methods	239
getData method	240
CHAPTER 15: Filter objects	241
filters collection	242
get method	242
exists method	243
remove method	244
Filter object methods	245
give method	245
makePublic method	246
rename method	247
copy method	248
CompositeFilterData object	249
FilterItemGroup object	249
ScenarioGroup object	249
HierarchyGroup object	250
BucketSettingsGroup object	250
FilterManagerFactory object	250
createFilterManager method	250
createCompositeFilterManager method	251
FilterManager object	253
getWorkbookBaseTable method	254
getUsesSelectedFilter method	255
getMultiScenarioCount method	256
getWorkbookVariables method	257
getBucketSettings method	257
getFilterItems method	258
getFilterValues method	259
getHierarchyNode method	260
isMultiScenario method	261
validate method	261
CompositeFilterManager object	262
getData method	262
CHAPTER 16: Hierarchy objects	265
hierarchies collection	265
get method	265
exists method	266
Hierarchy object methods	267
give method	268

makePublic method	268
rename method	269
copy method	270
CHAPTER 17: SiteGroup object	273
siteGroups collection	273
get method	274
exists method	275
remove method	276
SiteGroup object methods	276
give method	277
makePublic method	278
rename method	278
copy method	279
CHAPTER 18: Alert objects	281
alerts collection	281
get method	281
exists method	282
remove method	283
Alert object methods	284
give method	285
makePublic method	285
rename method	286
copy method	287
runAsync method	288
CHAPTER 19: ScheduledTask objects	291
scheduledTasks collection	291
get method	291
exists method	292
remove method	293
ScheduledTask object methods	294
give method	295
makePublic method	295
rename method	296
copy method	297
runAsync method	298
CHAPTER 20: AutomationChain objects	301
automationChains collection	301
remove method	302
get method	302
exists method	303
AutomationChain object methods	304
give method	305

makePublic method	306
rename method	307
copy method	308
runAsync method	309
CHAPTER 21: Schedule objects	311
schedules collection	311
get method	311
exists method	312
Schedule object methods	313
runAutomationTasks method	313
CHAPTER 22: TaskFlow objects	315
taskFlows collection	315
get method	315
exists method	316
TaskFlow object methods	317
give method	318
makePublic method	318
rename method	319
copy method	320
CHAPTER 23: Exception object	323
CHAPTER 24: ProfileVariable objects	325
profileVariables collection	325
set method	326
setConstantVariable method	327
get method	327
CHAPTER 25: ProcessingRule object	329
processingRules collection	329
get method	330
CHAPTER 26: Dashboard objects	331
widgetReferences collection	332
WidgetReference objects	332
dashboards collection	333
get method	333
exists method	334
Dashboard object methods	334
give method	335
makePublic method	336
rename method	337
copy method	338
getLayout method	338
createEmbeddedLinkUrl method	339
createLink method	339

CHAPTER 27: Widget objects	341
ControlSettings objects	342
widgets collection	343
exists method	343
get method	344
Widget object methods	345
give method	345
makePublic method	346
rename method	347
copy method	348
getAttachments method	348
CHAPTER 28: Form objects	351
forms collection	351
get method	352
exists method	353
remove method	353
Form object methods	354
give method	355
makePublic method	356
rename method	356
copy method	357
okCancelNotice method	358
yesNoNotice method	360
yesNoCancelNotice method	361
autoCloseNotice method	362
formDependencies collection	363
FormDependency object	363
Action objects	363
dismissNoticeAction method	364
runScriptAction method	364
closeFormAction method	365
Response object	365
ControlMessage object	366
Notice object	366
ErrorResponse object	366
CHAPTER 29: Scorecard objects	367
Scorecards collection	367
get method	368
remove method	368
exists method	369
Scorecard object methods	370
give method	370

makePublic method	371
rename method	372
copy method	373
CHAPTER 30: Collaboration objects	375
collaborations collection	375
get method	375
remove method	376
collaborationScenario collection	377
get method	377
has method	378
add method	379
remove method	380
setProperties method	381
CHAPTER 31: Responsibility objects	383
responsibilities collection	383
get method	383
exists method	384
Responsibility object methods	385
give method	385
makePublic method	386
CHAPTER 32: Workflow objects	389
workflows collection	389
get method	389
exists method	390
remove method	391
Workflow object methods	392
give method	392
makePublic method	393
rename method	394
copy method	395
startExecution method	395
CHAPTER 33: Query object	397
execute method	397
executeSingleton method	399
query object	400
query object methods	401
queryParameters object	401
close method	401
commit method	402
query data collections	403
columns collection	404
rows collection	404

cells collection	404
query data methods	405
get method	405
deleteRow method	406
deleteRows method	407
insertRow method	408
CHAPTER 34: ProcessTemplate objects	411
processTemplates collection	411
get method	411
exists method	412
ProcessTemplate object methods	413
give method	414
makePublic method	414
rename method	415
copy method	416
CHAPTER 35: charting property	419
JsChart objects	419
ChartEngine object methods	419
generate method	420
CHAPTER 36: integrationPlatformService property	423
Real-time message objects	424
runTask method	425
sendMessage method	426
getEndpointIds method	427
getEndpointPathParameterNames method	428
invokeEndpoint method	429
CHAPTER 37: Link object	435
links collection	435
get method	436
Link object methods	436
getResource method	437
createEmbeddedLinkUrl method	437
CHAPTER 38: dateTime property	439
toCalendarDate method	440
add method	440
subtract method	441
difference method	442
CHAPTER 39: console property	445
writeLine method	445
CHAPTER 40: Message objects	447
SendMessage object	448
SendBroadcastMessage object	448

messages collection	449
sendMessage method	449
sendBroadcastMessage method	451
CHAPTER 41: User objects	453
UserProfile object	453
ContactInfo object	455
users collection	455
create method	456
generatePassword method	458
remove method	458
UserCollectionObject object	459
setProperties method	461
setPassword method	462
LoggedInUser object	463
Permissions object	464
hasPermission method	464
setProperties method	465
CHAPTER 42: Group object	467
GroupProfile object	467
groups collection	468
create method	468
remove method	469
GroupCollectionObject object	470
Group object methods	470
setProperties method	470
add method	471
remove method	472
CHAPTER 43: server property	475
executeCheckpoint method	477
executeSnapshot method	477
executeCapture method	478
executeCondense method	479
executeCreateDataPackage method	480
executeDataExpiry method	480
executeDataIntegrityCheck method	481
executeDataSynchronize method	482
executeDataUpdate method	483
executeCommitDataUpdate method	485
executeCancelDataUpdate method	485
executeDeleteFiles method	486
executeExtractData method	488
executeExtractNetChangeData method	490

executeRestart method	493
executeVersionReclaim method	494
Defining tables for extracting data	495
Object index	499
Method index	505
Property index	513

Part 1: Overview

- [Welcome](#)
- [About Kinaxis](#)
- [Accessing help and documentation](#)
- [New and changed features](#)

CHAPTER 1: Welcome

Thank you for choosing and using RapidResponse® from Kinaxis®. RapidResponse helps manufacturers manage increasing business complexity and achieve operations performance breakthroughs with its proven solution for demand and supply chain planning, monitoring and response.

This guide provides information about developing custom applications that allow you to perform operations beyond standard RapidResponse capabilities.

Scripts allow you to automate operations and customize processes. You can design a script to perform operations on scenarios, run workbook commands, perform date calculations, and run RapidResponse server commands. For more information, see ["About RapidResponse scripts" on page 53](#).

Forms provide a user-focused interface for scripts that form authors can customize. For more information, see ["About forms" on page 55](#).

CHAPTER 2: About Kinaxis

Kinaxis® delivers cloud-based S&OP and supply chain applications for discrete manufacturers and brand owners with complex supply chain networks and volatile business environments.

Leaders across multiple industry verticals, including A&D, Automotive, High Tech, Industrial and Life Sciences rely on RapidResponse® applications to create a foundation for concurrent planning, continuous performance monitoring, and coordinated responses to plan variances across multiple areas of the business. All founded on a single product, RapidResponse's configurable applications encompass a full spectrum of supply chain processes, including such functions as S&OP, supply planning, capacity planning, demand planning, inventory management, MPS, and order fulfillment.

As a result, Kinaxis customers have replaced disparate planning and performance management tools and are realizing significant operations performance breakthroughs in planning cycles, supply chain response times and decision accuracy. From a single product, customers are able to easily model varying supply chain conditions to make both long-term and real-time demand and supply balancing decisions quickly, collaboratively, and in line with the shared business objectives of multiple stakeholders.

For more information, visit www.kinaxis.com or the Kinaxis Knowledge Network at <http://knowledge.kinaxis.com/>.

Customer Support

The Kinaxis Customer Support team understands demand and supply chain planning, monitoring and response, and is experienced in applying RapidResponse to real-world business scenarios. Areas of expertise include:

- RapidResponse functionality
- Data extraction and mapping issues
- Technical software compatibility

- Coordination of integration
- Product implementation

If you are a key support contact, you can submit support cases. To submit a support case, log into the Kinaxis Knowledge Network. You can also contact Kinaxis Customer Support by phoning 1-866-463-7877 or by sending a message to support@kinaxis.com.

Kinaxis Knowledge Network

The Kinaxis Knowledge Network is the place to find knowledge about RapidResponse. In one site, you can:

- Search all knowledge about RapidResponse, including product documentation, knowledge base articles, and threaded discussions about common questions.
- Review your company's Customer Support requests (and, if you are a key support contact, create new Support requests).
- Engage with a community of RapidResponse experts in the Discussion Groups.

Go to <https://knowledge.kinaxis.com> and log in today. You can also access the Kinaxis Knowledge Network from RapidResponse by clicking **Kinaxis Knowledge Network** on the **Help** menu.

CHAPTER 3: Accessing help and documentation

Determining which help system or guide to use	25
Documentation conventions	27
Finding the content you need in HTML help	27
Your feedback	29
Help embedded in resources	29
Other ways to learn about RapidResponse	34

The RapidResponse documentation set includes several different help systems and guides, each dealing with a different area of knowledge. Help for individual resources, such as workbooks is also available.

In addition to the RapidResponse documentation, you can access a variety of other resources to help you learn about the capabilities and features in RapidResponse. These include an online community where you can view how-to videos and find advice, a mailing list that you can use to keep up to date on supply chain news and upcoming events, and instructor-led training courses.

Accessing RapidResponse help systems and guides

You can access RapidResponse help systems and guides two ways:

- Access documentation directly from the Help menu in RapidResponse.
- For links to all of the available help systems and guides, see "[Determining which help system or guide to use](#)" on page 25.

Account permissions and the contents of your Help menu

The list of options you see when you click the Help menu in RapidResponse depends on the permissions associated with your RapidResponse account. The more permissions you have in RapidResponse, the more options you see on the menu. For example, resource authors have more permission than general users.

Links to documentation that you are less likely to need, based on your permissions, are not shown on the Help menu. For example, users without administrator permissions do not see Administration Help.

Documentation formats

Most RapidResponse documentation resources are available as both HTML help systems and PDF documents. Some documentation is only available in PDF format.

HTML help systems are optimized for viewing in a web browser. While it is possible to print individual topics from HTML help, you will get better results if you print the topic or topics from the equivalent PDF guide.

PDF guides are optimized for printing. They can be viewed and printed using Adobe Reader[®]. You can download Adobe Reader from the Adobe site at www.adobe.com.

Accessing help for RapidResponse resources

Help is often available in the RapidResponse application window for individual resources of the following types:

- Dashboards and widgets
- Worksheets and workbooks
- Scorecards
- Forms
- Scripts
- Responsibility definitions

For more information, see "[Help embedded in resources](#)" on page 29.

Determining which help system or guide to use

The RapidResponse documentation set includes several help systems and guides, each dealing with a different area of knowledge. The following table summarizes the available documentation, some of which you can access from the Help menu in RapidResponse.

Title	Description
RapidResponse User Guide (Java Client)	This guide provides RapidResponse users with basic reference and procedural information. It covers topics such as viewing data, modifying data as part of simulation, solving business problems, and customizing the user interface.
RapidResponse User Guide (Web Client)	This guide provides information on how to view and edit data in the Web client.
RapidResponse User Guide (Mobile Client)	This guide provides information on how to view data in the Mobile client.
RapidResponse Applications Guide (Java Client)	This guide provides information about the RapidResponse applications that support supply chain planning processes across different functional areas.
RapidResponse Applications Guide (Web Client)	This guide provides information about the RapidResponse applications that support supply chain planning processes across different functional areas in the Web client.
RapidResponse Fundamental Concepts Guide	RapidResponse product overview, which includes key capabilities and the deployment methodology.
RapidResponse Resource Authoring Guide	This guide provides information on creating and managing resources, such as dashboards, workbooks, and filters. Detailed information about RapidResponse Query language is also included.
RapidResponse Data Model and Algorithm Guide	Description of the RapidResponse data model and associated algorithms. All tables and fields in the data model are listed and described. This guide also notes changes to the data model corresponding to the RapidResponse release.
Data model posters	A series of posters illustrating the structure of tables and fields in RapidResponse, and the relationships between fields. For more information, see " Data Model Posters " on page 26.

Title	Description
RapidResponse Administration Guide	Information for administrators, covering installation, upgrades, maintenance, system settings, and user administration.
RapidResponse Scripting Guide	Information about building custom applications using scripting language objects, functions, and methods to automate some RapidResponse processes.
RapidResponse Web Services Guide	Information on using RapidResponse Web services to create users, resources, and Web service client programs.
RapidResponse Data Integration Guide	Information about integrating enterprise data sources with RapidResponse, including mapping data from source tables and fields, customizing the RapidResponse database, extracting data from enterprise sources, performing data updates to bring new and updated data into RapidResponse, moving data between RapidResponse instances, and moving data changes between RapidResponse and business partners in real-time.
Global Help	If you are not sure which help system contains the information you need, you can search for it in this help system, which includes information from all of the other help systems.
RapidResponse Release Summary	The latest release summary outlines all of the changes that have been made in RapidResponse 2208, including new features and defect resolutions in service updates. Release summaries can be found in the Kinaxis Knowledge Network in the Documentation Center.
Custom Help	Your company might have also created custom help specific to your RapidResponse implementation.

The permissions associated with your RapidResponse account affect the list of guides that you can access from the Help menu. For more information, see ["Account permissions and the contents of your Help menu" on page 24](#).

Data Model Posters

Users who have permission to create resources such as workbooks and dashboards have access to the following posters on their RapidResponse Help menus:

- *RapidResponse Data Model for Import poster*: displays the relationship between the tables and fields used in the RapidResponse data import process.
- *RapidResponse Calculated Data Model poster*: displays the relationship between the main RapidResponse database calculated tables and fields. Calculated fields in the RapidResponse data model input tables are also displayed.
- *RapidResponse Sales & Operations Planning poster* : displays the relationship between the tables used to support the Sales & Operations Planning application.

- *RapidResponse Inventory Planning and Optimization poster* : displays the relationship between the tables used to support Inventory Planning and Optimization application.
- *RapidResponse Integrated Project Management poster* : displays the relationship between the tables used to support Integrated Project Management application.
- *RapidResponse Historical Supply poster* and *RapidResponse Historical Demand poster*: displays the relationship between all tables (input, control, and calculated) that are used to store historical supply data and historical demand data, respectively.

Documentation conventions

To help you quickly locate and interpret information, Kinaxis guides use conventions noted in the following table.

Convention	Description
bold	Bold is used for user-interface elements in procedures. For example: On the File menu, click New .
<i>italic</i>	Italic is used when citing other documents. For example: For more information, see the <i>RapidResponse Administration Guide</i> .
Courier New	Courier New is used for programming examples and text that is entered in Microsoft Windows Command Prompt window or command lines.
 Caution:	Identifies a caution message (critical information).
 Note:	Identifies a note.
 Tip:	Identifies a tip.

If you are using a Mac OS, procedures that use the Windows CTRL key should be replaced with the Mac OS Command key.

Finding the content you need in HTML help

You can find information in any RapidResponse HTML help system by searching or by browsing in the contents or the index.



Searching the help

You can find the information you require by entering a term in the Search box. You can also refine your search by using AND or OR to search for a combination of words, and you can put quotation marks around your search word or phrase to search for that exact phrasing only. For example, "sort" would return any topics with the word "sort", but not the words "sorted" or "sorting".

► Search for content

- Type a search term in the search bar and then click .



Note: If you do not want the words you searched for highlighted in the help topic, click **Remove Highlights** .



Tip: If the browser window is small, the search bar might not be displayed. To see a search bar click  or enlarge the window.

Browsing the contents and the index

You can also find information by browsing in either of these panes:

- **Contents:** Browse through the topics in the table of contents.
- **Index:** Find a topic through the index. Scroll through the index or type a word or phrase in the Search Index box to find an index entry.



Tip: If the browser window is small, the Contents and Index panes might not be displayed. To see them, click  or enlarge the window.

Your feedback

Kinaxis takes great pride in developing user-friendly applications and we hope that our documentation resources ensure a high level of usability. We welcome your feedback about the help topics you access.

If you have comments or suggestions about Kinaxis documentation or training materials, you can email them to education@kinaxis.com.

Help embedded in resources

Many openable RapidResponse resources have help embedded within the resource. The form the help takes depends on the type of resource.

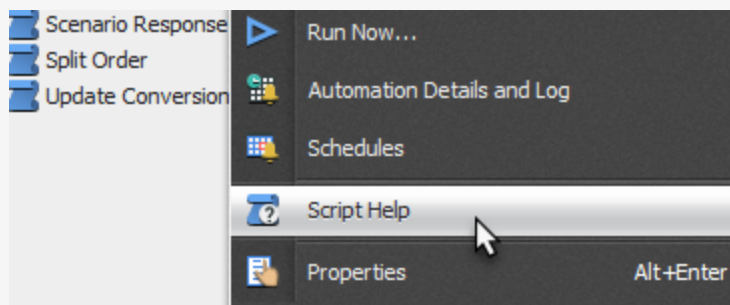
When help is available for a resource, you can view it from the open resource. You can also view the help for some types of resources from the Explorer, without having to open the resource first. These resources include workbooks, scorecards, widgets, forms, and scripts. Viewing resource help from the Explorer can help you to determine whether a resource is the right one to use, without having to wait for it to load.

► View resource help without opening the resource

- In the **Explorer**, select a resource and then click  beside the name of the resource.

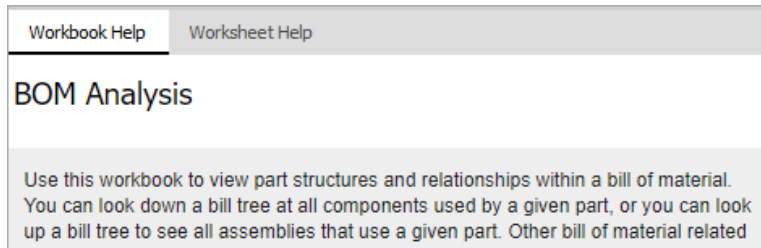


Tip: You can also view resource help from the Explorer by right-clicking the name of the resource and then selecting the Help option for the resource from the menu. For example, to view help for a script, click Script Help.



View and print worksheet and workbook help

RapidResponse worksheets can include a help page that is displayed in a pane on the right-hand side of the worksheet. Workbooks might also include a help page, in which case, the help pane includes two tabs: Workbook Help and Worksheet Help.



► Show or hide the help pane

- On the **Help** menu, click **Show Workbook/Worksheet Help**.



Note: This command is not available for workbooks that do not have workbook help or help for at least one worksheet defined.



Tip: You can also show or hide the help pane by clicking **Show Workbook/Worksheet Help**  on the toolbar.

► Print the worksheet or workbook help

You can print the worksheet or workbook help when the help pane is displayed.

1. On the **File** menu, click **Print**.
2. Click **Worksheet help** or **Workbook Help**, and then click **OK**.



Note: The worksheet and workbook help are printed as they appear in the help pane. Widen the help pane before printing to maximize the amount of information printed on a page.



Tip: You can also print the worksheet help by right-clicking in the help pane, and then clicking **Print**.

View and print scorecard help

Each scorecard in RapidResponse includes a help page that is displayed in a pane on the right-hand side of the scorecard. You can show and hide the scorecard help pane.

- On the **Help** menu, click **Show Scorecard Help**.



Note: This command is available only for scorecards that have help defined.



Tip: You can also show or hide the scorecard help pane by clicking **Show Scorecard Help**  on the toolbar.

► Print the scorecard help

You can print the scorecard help when it is displayed.

1. On the **File** menu, click **Print**.
2. Click **Scorecard help**, and then click **OK**.



Note: The scorecard help is printed as it appears in the help pane. Widen the worksheet help pane before printing to maximize the amount of information printed on a page.



Tip: You can also print the scorecard help by right-clicking in the help pane, and then clicking **Print**.

View dashboard and widget help

Dashboard help is often available for a dashboard as a whole, and you might also be able to view help for individual widgets.

Dashboard help

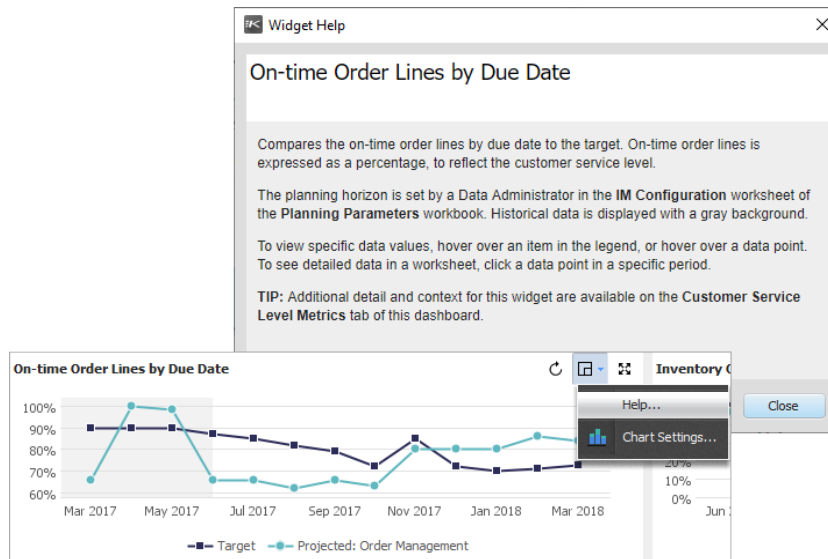
Some dashboards might have an Information tab with a text widget that shows dashboard help. Dashboard help contains information about how the dashboard is intended to be used, and can include links to related resources, websites, or help videos.

Widget help

Most widgets on dashboards that are supplied with RapidResponse have widget help, and some custom widgets might also include help. Widget help typically describes the metrics displayed in a widget, and might include links to other resources such as workbooks and websites. You can access help for widgets directly from the widget controls on a widget on your dashboard.

► View widget help

- On a widget title bar, click **Actions** , and then click **Help**.
The widget help displays in a dialog box.



View form help

Forms can display two types of embedded help:

- Text on the form itself can provide information such as what action is run by the underlying script, the purpose of the form, or how best to use the form.
- Individual controls on the form might also display help specific to the control, such as the purpose of the control or what types of values it accepts.

Resolve Case *i*

Case
Display case resolutions.

Current Status
Type the summary of the resolution and include the case number.

Resolution Details * *i*

More information

For more complex cases, upload the case file below.

Upload *
Upload more details about the case

Label controls can also display form-level help and information.

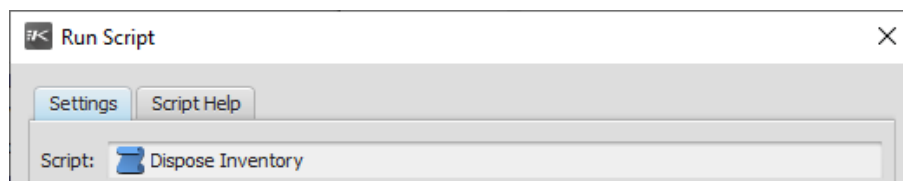
OK CANCEL

On an open form:

- To view form help, click the information icon at the top right corner.
- To view help for a control, next to the control title click the information icon.

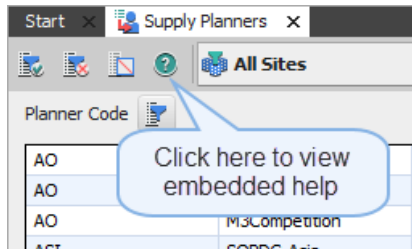
View script help

Some scripts might have embedded help. If a script has embedded help, a Script Help tab is shown when you run the script.



View help for a responsibility definition

You might be able to access embedded help for an open responsibility definition. If help is available, a Responsibility Help  button is available near the top left of the tab.



Resources without embedded help

Some types of resources, such as filters, cannot have embedded help, and not every resource that could have embedded help does have embedded help. Resource authors decide whether or not to include help when they create new resources that can have embedded help.

Sometimes, the information needed to use that resource might be available elsewhere. For example, if you access a dashboard by opening it from a task flow, the information you need to understand the purpose of the dashboard might be included in the task flow. Another example is a custom resource created for your company, which might be documented in company procedures that are maintained outside of RapidResponse.

Other ways to learn about RapidResponse

In addition to the RapidResponse documentation set, there are several other ways to learn about RapidResponse.

Kinaxis Knowledge Network

Visit the Kinaxis Knowledge Network at knowledge.kinaxis.com to access features including:

- [Upgrade Center](#): Learn what's new in each version of RapidResponse.
- [Training videos](#): Familiarize yourself with the basic capabilities of RapidResponse by taking an introductory course, delivered as a series of short videos. You can also view videos on some more advanced topics.
- [Discussion Forums](#): Discuss RapidResponse with other users.
- [Blogs](#): Learn how others are using RapidResponse and solving supply chain problems.

Some content is only available after you create a Kinaxis Knowledge Network user account and sign in.

News mailing list

Subscribe to the mailing list to stay up-to-date on news and upcoming supply chain events, such as Kinexions, the annual Kinaxis training and user conference. You can find the subscription form at <http://info.kinaxis.com/customer-subscription-page>.

Training courses

Kinaxis offers instructor-led courses and self-paced online training courses to help you develop in-depth RapidResponse knowledge. Instructor-led courses are usually delivered online in a virtual classroom, but on-site instruction can also be arranged. To see a list of available courses, visit <https://www2.kinaxis.com/rapidresponse-training/courses.cfm>.

CHAPTER 4: New and changed features

2022 scripting changes	38
2021 scripting changes	38
2020 scripting changes	41
2019 scripting changes	43
2018 scripting changes	45
2017 scripting changes	48
2016 scripting changes	49

This section describes all the new scripting features introduced in recent RapidResponse releases. There are similar chapters in other guides. For more information, see ["Determining which help system or guide to use" on page 25](#).

You can also learn more about the new features and changes included in RapidResponse from the *RapidResponse Release Summary Guide*.

A new version of the *RapidResponse Release Summary Guide* is released with each new version of RapidResponse, including service updates.

This document is available for download from the Kinaxis Knowledge Network and summarizes the changes to RapidResponse. It contains a compilation of changes gathered from every RapidResponse guide and help system. It also contains lists of defects resolved in each version.

► To access the *RapidResponse Release Summary Guide*

1. On the **Help** menu, click **Kinaxis Knowledge Network**.
2. Sign into the Kinaxis Knowledge Network.

You can create a new Kinaxis Knowledge Network account if you do not have one. An account is required.

3. On the **Knowledge** menu, click **Documentation**.
4. Under **Release Summary Documents**, select a release to download the PDF document.



Tip: You can also find release summary content in the [Global Help](#) system.

2022 scripting changes

This section outlines new and changed scripting features in 2022 RapidResponse releases.

- ["Service Update 2208" on page 38](#)
- ["Service Update 2204" on page 38](#)

Service Update 2208

Data extraction commands

The methods that run the `ExtractData` and `ExtractNetChangeData` commands now return information about the extracted data, including the extract identifier and details about the data. This allows you to use these commands in processes that create and then use the extracted data, or require the extract identifier for future steps in a process. See ["executeExtractData method" on page 488](#) and ["executeExtractNetChangeData method" on page 490](#).

Service Update 2204

Administrator permissions

You must now be a system administrator to run any script that assigns an administrative permission to a user or group, or that adds a user to a group with any of the administrator permissions. This ensures any administrative changes are made by a system administrator. See ["Permission object" on page 126](#).

2021 scripting changes

This section outlines new and changed scripting features in 2021 RapidResponse releases.

- [Service Update 2112](#)
- ["Service Update 2111" on page 39](#)
- [Service Update 2108](#)
- [Service Update 2105](#)

- [Service Update 2104](#)
- [Service Update 2103](#)

Service Update 2112

createWorkflowScenario method for Scenario objects

You can use this new method to create scenarios in workflows. See [createWorkflowScenario method](#).

New parameter for Scenario objects

Scenario objects can now include the scoped name of a scenario that identifies if the scenario is private or shared. See [Scenario objects](#).

Service Update 2111

Sending data to endpoints

You can now generate a JSON representation of a worksheet's data, which you can include in outbound endpoint calls. This allows you to construct a payload directly from a worksheet's data without requiring you to manually construct each row of the payload. You can specify the rows to include and the formatting for this JSON object, and customize it as a series of objects, arrays of values, or single strings, depending on your endpoint's requirements. See ["convertToJSON method" on page 210](#).

Service Update 2108

Running scripts in workflows

You can now run scripts in workflows by making the script available to workflows in the script properties. See ["Create a script" on page 62](#).

In addition, new context variables (#Workflow and #WorkflowExecutionID) have been added that you can use to ensure that values required by the script from the workflow are passed to the script. See ["Resource Context" on page 70](#).

Workflow object

The Workflow object defines workflows that build out automated processes in RapidResponse. See [Workflow objects](#).

For more about workflows, see the [RapidResponse Resource Authoring Guide](#).

Query object

The Query object that defines queries that run in RapidResponse is now available. This object implements query object methods in addition to query data collections and query data methods. See ["Query object" on page 397](#).

Error reporting for endpoints

If you use a script to access an external endpoint, any errors encountered by the endpoint call are now reported in the response. This replaces the previous behavior of only reporting the HTTP status code when an error occurs, and allows you to diagnose issues with your scripts or endpoints. See ["invokeEndpoint method" on page 429](#).

Service Update 2105

Real-time Integration Service deprecation

As of this Service Update, the Real-time Integration Service is no longer available, and any operations that use methods defined by the BusinessMessage object no longer function.

Service Update 2104

Real-time Integration Service deprecation

In an upcoming RapidResponse Service Update, support for the Real-time Integration Service and the BusinessMessage object that provides an interface to it will end. If you use this object or its related methods in scripts, those scripts are not removed, however, these methods will no longer function.

Service Update 2103

Dynamic path parameters

Endpoints defined for accessing external systems or services can now support dynamic path parameters. For endpoints with dynamic paths, you must specify a value for each parameter, which is typically used to define a path for a single call to the external system or service. These dynamic paths allow you to connect to services that do not use a fixed path.

Values for path parameters can be specified when you invoke an endpoint, and a new `getEndpointPathParameterNames` method allows you to retrieve the names of the parameters defined for an endpoint.

For more information, see ["integrationPlatformService property" on page 423](#).

2020 scripting changes

This section outlines new and changed scripting features in 2020 RapidResponse releases.

- [Service Update 2012](#)
- [Service Update 2011](#)
- [Service Update 2007](#)
- [Service Update 2005](#)
- [Service Update 2001](#)

Service Update 2012

Run scripts using the new Web client

The new HTML5-based Web client is now available. It runs in a web browser, without the need for a Java Runtime Environment. You can run scripts in this client from the Automation panel. See the [RapidResponse User Guide \(Web Client\)](#).

Script authoring is still done using the Java client.

Service Update 2011

Communicating with external systems

You can now use predefined endpoints to send requests and data to an external system or service. This allows you to define processes that notify external systems about processes in RapidResponse, or to notify RapidResponse about processes in external systems. You can also include a payload to send data to external systems, without requiring you to export data or use a file to transfer data between systems.

Endpoints are defined for you by your RapidResponse administrator. Typically these endpoints provide access to a RESTful service, and each call to the endpoint specifies the operation to perform through that endpoint using the HTTP methods defined for that service. Requests sent to endpoints have requirements defined by the endpoint, and these depend on whether the request is used for notifications, for sending data, or for retrieving data. See ["Using scripts to communicate with external systems" on page 88](#).

Methods to retrieve the endpoints defined by your administrator and to send requests to an endpoint are defined as part of the `integrationPlatformService` property. For more information, see ["integrationPlatformService property" on page 423](#).

Service Update 2007

Managing profile variables for other users

You can now retrieve and specify values for profile variables that are defined for other users and groups. This allows you to manage the profile variables for groups you are responsible for, and to specify values required for a process.

The `profileVariables` collection's `get()` method now takes an optional parameter to specify the user or group the variable belongs to. You can use this to verify the correct value is set for each user or group a variable is defined for.

A new method, `setConstantVariable()`, to set the variable values has been added to the `profileVariables` collection, which allows you to set or create a Constant profile variable for another user or group. Only Constant variables can be modified using this method. You can continue to set values for your own profile variables of any type using the `set()` method, but you must set a Constant variable for another user or group.

Retrieving or specifying variable values for other users requires the user running the script to have permission to modify macros and profile variables. For more information about this permission, see the [RapidResponse Administration Guide](#).

See "[profileVariables collection](#)" on page 325.

Service Update 2005

Running tasks on schedules

A new `Schedule` object that represents automation task schedules is now available. You can use this to run all automation tasks that use a particular schedule. This replaces a similar capability supported by the RapidResponse OpenAPI, which has been deprecated. You can use the `Schedule` object to replicate the behavior you require. See "[Schedule objects](#)" on page 311.

Service Update 2001

The .xls file format is no longer supported

RapidResponse has been upgraded to use a faster, more modern engine to import and export data, such as when generating a report from a worksheet. The new engine supports the current Microsoft Excel file formats, .xlsx and .xlsm. However, the .xls file format, which was used by versions of Microsoft Excel that were released in 2003 and earlier, is no longer supported.

Scripts that previously used the [generateReport](#) method to create a ReportOutput object with a .xls file format will now throw an exception. These scripts must be updated to specify a valid file format for the report.

User visibility limitations on sending messages

Scripts that send messages are now limited by the visibility level of the user running the script. Messages include information about the sender, including their name and contact card, and messages sent by scripts could include this information without the user's permission or knowledge. Now, users who have their visibility level set to None cannot send messages, and users with the Limited visibility level can send messages only to users with the All visibility level. This ensures scripts do not send information about users who want to be hidden without their consent. See "[sendMessage method](#)" on [page 449](#).

2019 scripting changes

This section outlines new and changed scripting features in 2019 RapidResponse releases.

- [Service Update 1907](#)
- [Service Update 1905](#)
- [Service Update 1902](#)
- [Service Update H1901.1](#)
- [Service Update 1901](#)
- [Version 2016.2, Service Update 27](#)

Service Update 1907

Worksheet record limit deprecated

The worksheet record limit feature, which was used to limit the number of records retrieved when a worksheet query ran, has been deprecated. This can affect scripts in two ways:

- For the `getData` method, the `overrideRecordLimit` and `returnIncompleteResults` parameters no longer have any effect. Existing scripts that include these parameters continue to function, ignoring the parameters.
- For `WorksheetData` objects, the status value of `RecordLimitReached` no longer occurs.

Service Update 1905

Upgraded JavaScript engine

RapidResponse scripts are now executed using version 7.4 of the Google V8 JavaScript engine. This provides performance improvements and access to JavaScript and ECMAScript features and language updates, including private class fields and locale settings. Your existing scripts continue to function and not require any changes to use the updated engine.

Service Update 1902

Delete collaborations

System and user administrators can delete collaborations using scripts.

For more information, see ["remove method" on page 376](#).

Automation resources locked after ownership transfer

When an alert, scheduled task, or automation chain is given to another user, the new owner or an administrator must unlock it before it can run. This new requirement could result in a LockedOut exception when using the following methods after an automation resource has been given to a new owner:

- [runAsync method](#) (alerts)
- [runAsync method](#) (automation chains)
- [runAsync method](#) (scheduled tasks)

For information about unlocking automation resources, see the [RapidResponse User Guide \(Java Client\)](#).

Service Update H1901.1

Obsolete server commands

If you have migrated to RapidResponse H1901.1, server commands that manage database files are no longer required, and are no longer supported. These commands, Capture, Condense, Snapshot, and VersionReclaimSweep, no longer perform any operations, but also do not cause scripts that use them to fail.

For more information, see ["executeCapture method" on page 478](#), ["executeCondense method" on page 479](#), and ["executeSnapshot method" on page 477](#).

Service Update 1901

Version numbering

As of this release of RapidResponse, the service update numbering now reflects which release number of the year the service update is. This supports the continuous delivery of RapidResponse, and allows you to correlate the service update number of your RapidResponse installation with the order of releases for that year.

For example, RapidResponse G1901 is the first release of 2019 using this numbering.

The letter designation in the version number (G) is used to identify the major platform version. The letter is only included where it is relevant.

Improved resource name validation

When renaming and copying resources, the new name for the resource is now validated for length, and resource names must contain 250 characters or fewer. This ensures resources you create in scripts are valid.

Version 2016.2, Service Update 27

This section outlines new and changed features in RapidResponse 2016.2, Service Update 27.

Improved resource name validation

When renaming and copying resources, the `ArgumentError` exception is now returned if the new resource name specified is not valid. Previously this exception was only returned if the name was blank or null.

2018 scripting changes

This section outlines new and changed scripting features in 2018 RapidResponse releases.

- [Version 2016.2, Service Update 26](#)
- [Version 2016.2, Service Update 21](#)
- [Version 2016.2, Service Update 20](#)
- [Version 2016.2, Service Update 15](#)

Version 2016.2, Service Update 26

This section outlines new and changed features in RapidResponse 2016.2, Service Update 26.

Light user type added

A new user license type, Light, has been added to RapidResponse. For more information about user types, see the [RapidResponse Administration Guide](#).

"Light" is now a valid value for the `userLicenseType` property of a [UserCollectionObject](#) or [UserProfile](#) object.

In existing scripts that specify a value for the deprecated `licenseType` property of a [UserCollectionObject](#) or [UserProfile](#) object, a value of `External` now corresponds to the new license type, `Light`.

Limitations on user information visibility

A new user permission, `Visibility`, allows administrators to limit users' ability to see information about each other, such as name or email address. For more information about this feature, see the [RapidResponse Administration Guide](#).

New property

To support this change, a new property, `visibilityLevel`, has been added to [UserCollectionObject](#) and [UserProfile](#) objects. If `visibilityLevel` is not specified when creating a new user using a script, the new user is granted the `All` visibility level.

Effect on existing scripts

If a script is run using the credentials of a user who has a visibility level of `Limited` or `None`, the users whose information they do not have permission to see are not included in the [users collection](#).

Version 2016.2, Service Update 21

This section outlines new and changed features in RapidResponse 2016.2, Service Update 21.

New user license types

Previously, users could be created with three types of user licenses: `Internal`, `External`, and `Alert`. The valid user license types are now `Full`, `Report`, and `Alert`. For more information about the naming of user license types, see the [RapidResponse Administration Guide](#).

Existing scripts that refer to the `licenseType` property of a [UserCollectionObject](#) or [UserProfile](#) object will continue to work. A value of `Internal` corresponds to the new license type, `Full`, and a value of

External corresponds to the new license type, Report. For example, a script that previously created user accounts with the Internal license type will now create user accounts with the Full license type.

However, the `licenseType` properties are deprecated and replaced with new `userLicenseType` properties. If a `UserCollectionObject` or `UserProfile` object has both `licenseType` and `userLicenseType` properties, the `userLicenseType` property takes precedence.

Version 2016.2, Service Update 20

This section outlines new and changed features in RapidResponse 2016.2, Service Update 20.

Real-time data transactions

The Real-time Integration Service is replaced by the Real-time Integration Platform. As a result, the `BusinessMessage` object is now deprecated, and the `integrationPlatformService` object has new methods and properties to be used for sending messages to your external systems.

The `integrationPlatformService`'s `sendMessage()` method takes a worksheet and sends its data to the Real-time Integration Platform using a message definition name specified in the method call. For complete information about the Real-time Integration Platform, see the [RapidResponse Data Integration Guide](#).

When you send a message to your external systems through the Real-time Integration Platform, you can specify worksheets that contain the data to send and any sets of records that relate to the data, such as supply orders with a set of scheduled receipt records. This replaces the usage of business message definitions specified through the `BusinessMessage` object, and allows you to send data from any worksheet that meets your requirements.

For more information, see "[integrationPlatformService property](#)" on page 423

Version 2016.2, Service Update 15

This section outlines new and changed features in RapidResponse 2016.2, Service Update 15.

Resource collections and management

A number of RapidResponse resources have been added to the script object model, allowing you to enumerate and manage them. You can now enumerate and manage task flows, responsibilities, hierarchies, and processes. These resources implement only the enumeration, access control, `get()`, and `exists()` methods from the resource collection, and the `give()` and `makePublic()` methods from the resource object.

You can use these methods to perform audits of resources available to users, and to provide access to the resources a user requires. For more information, see "[Hierarchy objects](#)" on page 265,

["ProcessTemplate objects" on page 411](#), ["Responsibility objects" on page 383](#), or ["TaskFlow objects" on page 315](#).

2017 scripting changes

This section outlines new and changed scripting features in 2017 RapidResponse releases.

- [Version 2016.2, Service Update 12](#)
- [Version 2016.2, Service Update 11](#)
- [Version 2016.2, Service Update 7](#)
- [Version 2016.2, Service Update 6](#)

Version 2016.2, Service Update 12

This section outlines new and changed features in RapidResponse 2016.2, Service Update 12.

Workbook initialization changes

When you initialize a workbook using the workbooks collection `get()` method, you can now specify the currency used for monetary values in that workbook. This value is optional, and should be specified only for workbooks that allow users to select the currency used to display values. For more information, see ["Workbook objects" on page 169](#).

Version 2016.2, Service Update 11

This section outlines new and changed features in RapidResponse 2016.2, Service Update 11.

Integration Platform Service

If you use the Integration Platform Service for integrating data, you can use a new property of the `rapidResponse` object to trigger a task to run on your hosted Talend Administration Center. This allows you to run a task from within RapidResponse, and can be used as part of a closed loop process to move data changes back to your data source. For more information, see ["integrationPlatformService property" on page 423](#).

Managing user accounts that use single sign-in

If you have implemented single sign-in (SSI), you can create or modify user accounts to indicate they can sign in only using the SSI interface. These user accounts do not have passwords in RapidResponse, which can improve security because a user cannot sign in using another user's password, and all passwords are managed in your SSI infrastructure. You cannot set passwords for these user accounts. For more information, see ["UserProfile object" on page 453](#).

In addition, you can now specify a password using the User object's `setProperties` method, which allows you to remove a password at the same time as you set the user as only able to sign in using SSI, or the converse. Because an SSI-only user cannot have a password and a standard user must have a password, these properties must be set together. For more information, see ["setProperties method" on page 461](#).

Version 2016.2, Service Update 7

This section outlines new and changed features in RapidResponse 2016.2, Service Update 7.

executeCheckpoint method

The `executeCheckpoint` method was deprecated and should no longer be used. For more information see ["executeCheckpoint method" on page 477](#).

Version 2016.2, Service Update 6

This section outlines new and changed features in RapidResponse 2016.2, Service Update 6.

executeRestart method

The `executeRestart` server method no longer requires a parameter. This reflects a change in the underlying server command the method executes, and allows you to create scripts that restart the server without executing a Checkpoint command first. For more information, see ["executeRestart method" on page 493](#).

2016 scripting changes

This section outlines new and changed scripting features in 2016 RapidResponse releases.

Version 2016.2 initial release

This section outlines new and changed scripting features in the initial release of RapidResponse 2016.2.

Workbook filtering

For scripts that include part or reference part data, you can now use the new `partType` property to specify whether the `part` or `referencePart` property of the `WorkbookInitialization` object should be populated. This allows you to properly initialize workbooks that display one type of part, and allows you to use the same script to initialize regardless of the part type it displays. You can populate this property from the `#Worksheet` script argument if you are calling the script from a workbook command, or by

specifying the values either in the script or its arguments. For more information, see ["Workbook objects" on page 169](#).

Worksheet filtering

The Worksheet object's filter property now accepts only a Filter object definition. This replaces the previous behavior of allowing a Filter object and a filter expression that defines the part or reference part selection. This ensures the value specified for the filter is valid and can be verified.

After upgrading to RapidResponse 2016.2, you must review your scripts to verify they do not specify filter expressions for the filter property, otherwise your scripts will no longer function.

You can use the part, referencePart, and partType properties to define extended part filtering to a worksheet. This allows you to properly initialize workbooks that display one type of part, and replaces the usage of the filter expression specified for the filter property. This also allows you to use the same script to initialize regardless of the part type it displays. You can populate these properties from the #Worksheet script argument if you are calling the script from a workbook command, or by specifying the values either in the script or its arguments. For more information, see ["Worksheet objects" on page 197](#).

Method to support the VersionReclaimSweep command

RapidResponse 2016.2 introduces the VersionReclaimSweep command. RapidResponse scripting supports this command with the **executeVersionReclaimSweep** method.

This command is used to clean up the internal version structure used to support scenarios. The version reclaim process reduces the size of the RapidResponse database and frees system memory.

Part 2: RapidResponse Scripting

- [About RapidResponse scripts](#)
- [Developing scripts](#)
- [Running scripts](#)

CHAPTER 5: About RapidResponse scripts

Knowledge requirements for script authoring	54
About forms	55

RapidResponse scripts allow you to build custom applications using scripting language objects, functions, and methods to automate some RapidResponse processes. Scripts are executed on the RapidResponse System, and perform operations using RapidResponse resources and data. For example, you can create scripts to perform any of the following operations:

- Automating part of a simulation process.
- Creating historical scenarios or scenarios required by a process.
- Running automatic data modifications as part of a process.
- Retrieving worksheet data.
- Performing date arithmetic.
- Sending messages.
- Automating RapidResponse Server commands.
- Modifying data.

Most scenario management options are available through scripts. You can use a script to create, rename, update, commit, or delete a scenario. You can also access workbooks to run commands, retrieve worksheet data, or perform date calculations. A script can combine any of these operations. For example, you can calculate the difference between the current date and the last date a scenario was accessed, and then delete any scenarios that have not been accessed in a specified number of weeks.

If you are a system administrator, you can run RapidResponse Server commands to perform system administration tasks such as checking data integrity or restarting the server.

You can create scripts that are used in other scripts, that can be run by users, that run automatically on a schedule, that run from a workbook command, run from an overlying form, or that are run by a Web service client. Depending on how you intend the script to be used, you can share the script directly with other users, create a workbook that calls the script using a command, or include the script in another script.

When you create or edit a script that is used by a form, you might also be responsible for authoring the form. If you are creating a script that requires user input, it is recommended that you use a form as the user interface. For more information, see ["About forms" on page 55](#).

This documentation set (Help and Guide) is intended for users who have been granted permission to author and share scripts, and provides information about creating, running, and managing RapidResponse scripts. It also provides complete reference information for the objects and methods that can be used in scripts. Sample scripts and a community forum are also available to provide additional help.



Note: When you create scripts from a command that automatically modifies data in a shared scenario, users must have permission to edit data in shared scenarios.

Knowledge requirements for script authoring

RapidResponse script authors should be familiar with programming concepts such as defining variables, creating objects, calling methods, and testing code. Application programming experience is recommended.

RapidResponse scripting is implemented as an extension to JavaScript™ (ECMAScript). To author scripts, you must be familiar with JavaScript programming, including its syntax, data types, method calls, and error handling.

In addition, you must be familiar with your company's business processes and how they are implemented in RapidResponse. You can write scripts to automate these processes.

This documentation set does not provide information about how to program or how to write JavaScript.

For more information about JavaScript, go to <http://www.ecma-international.org/publications/standards/Ecma-262.htm>.

About forms

Forms are customizable user interfaces for scripts. Users specify values for the script arguments that are passed from the form to the script when the form is run.

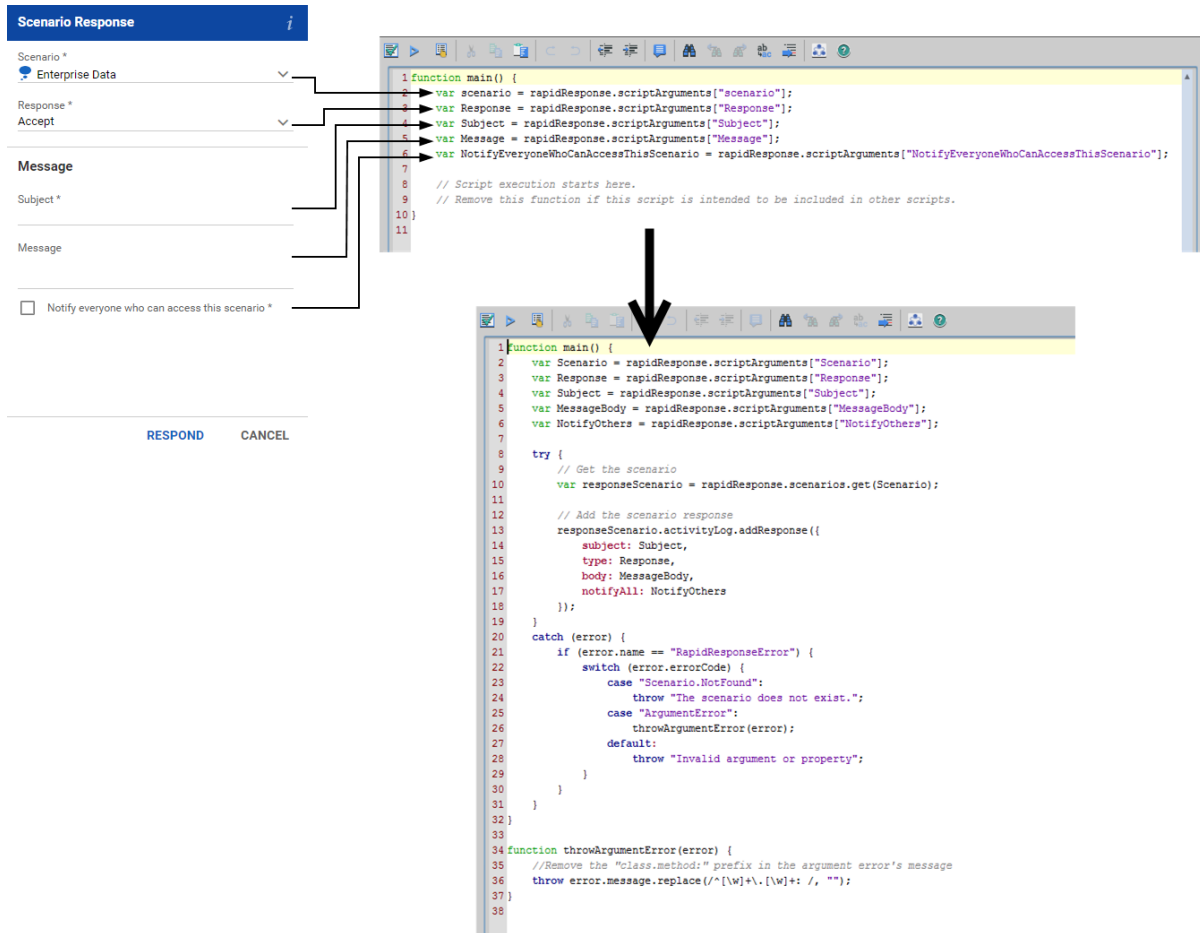
The diagram below illustrates the flow between a form and its underlying script.

1. The user views the form, types information and makes selections in the controls, and then clicks the action button "Respond". Any form-level validation errors, such as leaving a required control blank, must be resolved before the form can be run.
2. If the form has all its required controls defined with appropriate values, the form passes those values to the corresponding arguments in the underlying script.
3. The script executes the action or actions it has been authored for. This script manages scenarios and sends a message.
4. Optionally, the script can return an error or success message back to the form for the user to view.

Forms authors can generate forms based on existing scripts or build new forms and then generate script arguments mapped to controls on the form. The generated script only contains the arguments and does not constitute a functioning script.

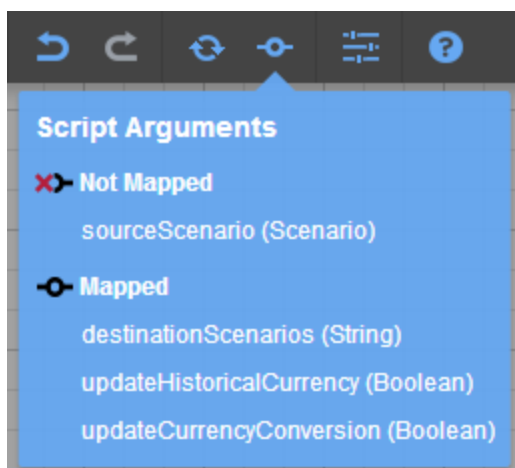
As a script author, you might be responsible for completing the generated script for a form by adding all the required body material, including validation, business logic, and error handling, to make the script functional.

The images below illustrate the generated script and the completed version of the script for the Scenario Response form.



You might also be responsible for editing existing scripts in response to changes made in corresponding forms based on the scripts.

If you also have form authoring permission, you can directly check for arguments that are not mapped from a form.



For scripts that require direct user input, it is recommended that a form be created based on the script. As seen in the images below, the form provides a more user-friendly interface and controls that require a value to pass to the script argument do not need to be separately identified. With forms, you can also embed help directly in the resource to support users.

The image displays two screenshots of the 'Run Script' dialog box. The left screenshot shows the 'Copy Event' script selected, with fields for Name, Type, Description, Site, Code, CopyAllPhases (set to Yes), Phase (set to All), AdjustmentType (set to All), and EventToCopy. The right screenshot shows the 'Copy Event' script selected, with a 'New event' section and a 'Name' field. A tooltip is visible over the 'New event' section, stating: 'Copies an existing event and creates a new record in the Event and Event Phase worksheets. You can choose to copy all event phases or just one, and which adjustment type of the phase to copy.'

Forms offer design flexibility, and can include informative text and headers, images, and read-only fields. You can include custom string values in the script that display as customized validation or feedback messages.

For more information about creating forms, see the [RapidResponse Resource Authoring Guide](#).

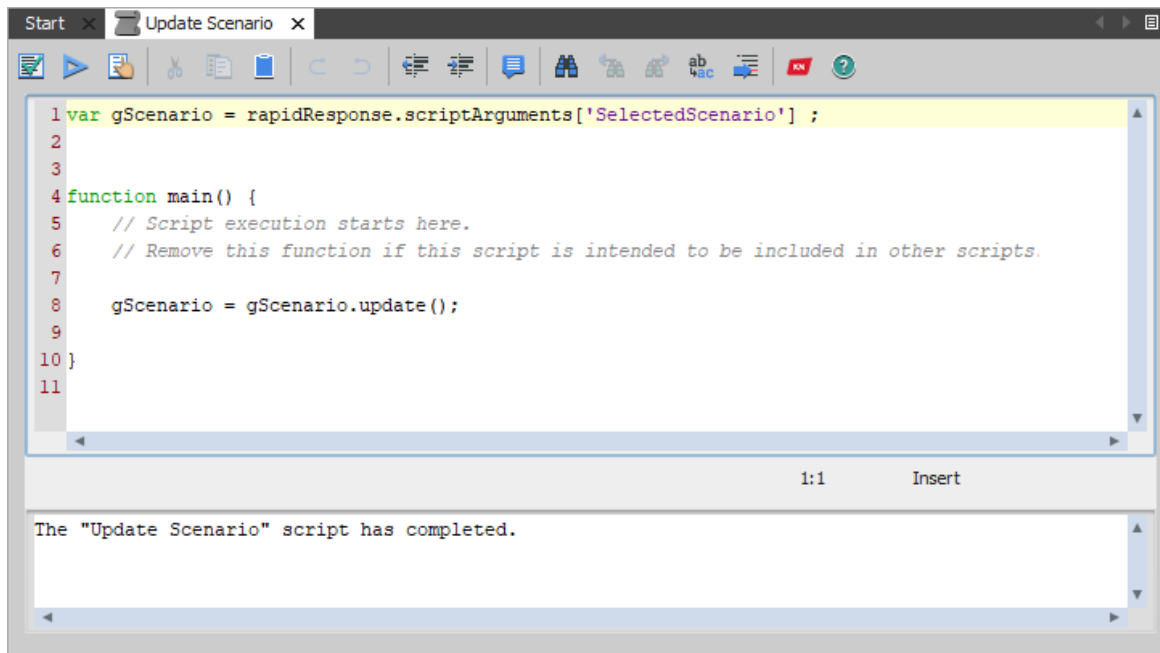
CHAPTER 6: Developing scripts

Create a script	62
Script versioning	63
Create script code	64
Define arguments for a script	68
Specify dependencies	73
Test a script	74
Upgrade a script	75
Calling other scripts	77
Adding help for scripts	79
Editing and managing scripts	82
Custom messages in forms	84
Notices	85
Control errors	87
Using scripts to communicate with external systems	88
Sample scripts available for RapidResponse	89
Create a new order using a form	91

Scripts are authored in the RapidResponse script authoring window, which provides you with the following.

- **Toolbar:** includes commonly performed actions.
- **Script code editor:** contains the script code.
- **Output console:** displays any messages returned by your script.

The script editor is shown in the following illustration.



► Script editor toolbar

The toolbar includes the following functions.

Item	Description	
	Validate Script	Validates the script syntax to find unmatched parentheses and invalid structure. This does not validate function calls, method calls, or declarations.
	Run Script	Validates function calls, method calls, and declarations, and runs the script.
	More Properties	Opens the script properties, where you can view, add, or modify arguments, dependencies, and help.
	Cut	Cuts the selected text.
	Copy	Copies the selected text.
	Paste	Inserts copied or cut text at the insertion point.
	Undo	Undoes the previous change.
	Redo	Repeats the previous change.
	Decrease Indent	Decreases the indentation of the selected lines.

Item	Description
 Increase Indent	Increases the indentation of the selected lines.
 Toggle Comment	Formats the current line as a comment.
 Find	Search for specific text in the script code editor or console.
 Find Previous	Jumps to the previous instance of the search string.
 Find Next	Jumps to the next instance of the search string.
 Replace	Replaces the specified string with another string.
 Go To Line	Jumps to the specified line number.
 Developer Forum	Opens the RapidResponse Developer Forum section of the Kinaxis Knowledge Network.
 Scripting Help	Opens the <i>RapidResponse Scripting Help</i> in an HTML format.

► Script code editor

The script code editor is the area where all the script code is written. Each line is numbered so if errors occur, you can find the line that is causing the error.

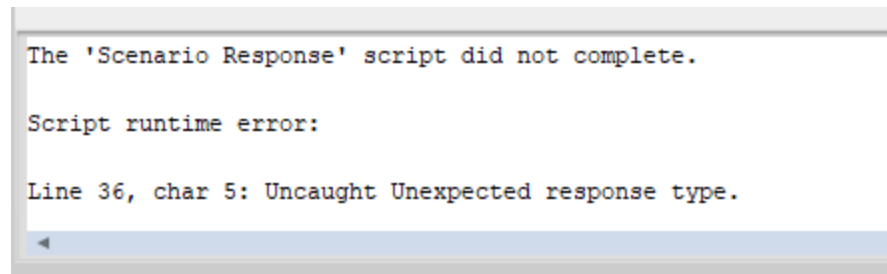
The text in the coding area is automatically color coded depending on context, as shown in the following table.

Text format	Description
<code>var</code>	Declaration (Green)
<code>return</code>	JavaScript keyword (Blue and bold)
<code>subject:</code>	JavaScript literal or property (Dark red)
<code>"scenario"</code>	String literal (Purple)
<code><i>Comment</i></code>	Comment (Gray and italic)

► Output console

After you run a script, the output console displays any debugging messages specified in the script and the script's return value. You can use this to test your script and to find where a script is failing. If the

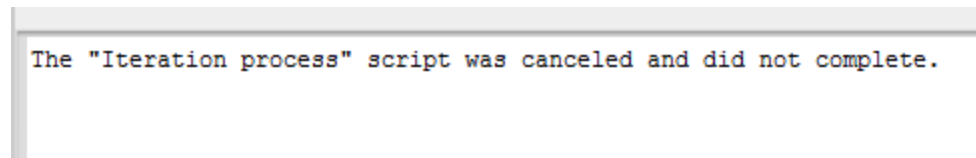
script does not complete, the error message is displayed in the console, as shown in the following illustration.



```
The 'Scenario Response' script did not complete.  
  
Script runtime error:  
  
Line 36, char 5: Uncaught Unexpected response type.
```

The output console displays the last 32 kilobytes of text output from the script. You can also view the output in the script log, which displays the last 5,120 characters of output. For more information, see ["Script logging" on page 97](#).

If you cancel a script, its debugging messages are not displayed in the console. The message shown in the following illustration is displayed instead.



```
The "Iteration process" script was canceled and did not complete.
```

In this case, you can view the messages in the script log. For more information, see ["Script logging" on page 97](#).

► Search the scripting help

If you require help about a script object or method, you can search for help for that object or method.

1. In either the code editor or console, select or place the insertion point in the text you want to search for.
2. Press **F1**.
3. In the **Search** window, select the topic you want to review.

Create a script

A script is composed of its code, arguments that provide data values, dependent resources that the script uses, and help for users. The dependencies can be workbooks or scripts. Dependent workbooks allow the script to run workbook commands or retrieve data from worksheets. Dependent scripts allow the script to use functions from or run the dependent script. To enable workflows to run the script, you can set the script to be available to workflows.

You should plan the actions performed by a script and the data required for those actions before you begin creating the script. The data required by the script is obtained by creating script arguments that take values or by running other scripts. You can also use functions from other scripts and run them as part of the script or take the output values of a workflow and use them in a script.

You can define the following components of a script:

- Arguments that pass values to the script.
- Dependencies that identify which resources are required to run the script.
- Help information for script users.

Scripts can support as many arguments and dependencies as you require, but each argument must use a unique name within the script.

► Create a script

1. On the **File** menu, point to **New**, and then click **Script**.
2. In the **New Script** dialog box, on the **General** tab, type a name for the script.
3. If the script will be used by a workflow, select the **Make available to workflows** checkbox.
4. Create the scripting code. For more information, see ["Create script code" on page 64](#).
5. Define arguments that pass input values into the script. For more information, see ["Define arguments for a script" on page 68](#).
6. Specify any scripts or workbooks that are required for the script. For more information, see ["Specify dependencies" on page 73](#).
7. Write help for script users and notes for other script authors. For more information, see ["Adding help for scripts" on page 79](#).
8. Test the script to ensure it functions as you expect. For more information, see ["Test a script" on page 74](#).



Note: You can perform steps 3 to 6 in any order. For example, you could create arguments before writing script code, or write code, define a dependency, and then resume writing code.



Tip: You can also create a script by clicking **New Resource**  on the RapidResponse toolbar.

Script versioning

If your company has enabled resource version control, each script you create can be added to the repository, which allows you to share it with other users, records revision notes, and revert changes to an earlier version if necessary.

For complete information about resource version control, see the *RapidResponse Author Guide*.

Create script code

Each RapidResponse scripting function and method you can call is defined in the RapidResponse object. This object defines each of the resource objects, such as scenarios and workbooks, as well as the script's arguments, the name of the user running the script, and any messages the script sends. A typical RapidResponse object method call looks like the following:

```
rapidResponse.resourceType.method();
```

Scripts you create must contain a global function named `main()`, which is used as the start of the script. Other functions and methods can be called from the `main()` function. The `main()` function is included by default when you create the script, and should not be removed, unless you are creating a script to be used as a library in another script. For more information, see ["Guidelines for creating scripts to be used by other scripts" on page 77](#).

If you specify a return value for the `main()` function, that is the value the script returns. Otherwise, the script does not return a value. The script's return value is displayed in a dialog box after the script completes, and is displayed if you run the script from the Explorer or from the Run Automation Task dialog box, or if you run a workbook command that runs the script. The return value is also visible in the script log. For more information, see ["Script logging" on page 97](#).

For information about the objects and methods you can call in scripts, see ["RapidResponse script object model" on page 103](#).

All object properties, unless otherwise indicated, cannot be modified by a script. Values for properties are specified when the object is retrieved, and cannot be changed.

If your company has access to a test system, you should do all script development and testing on that test system, and then move the script into your production system. This allows you to test the functionality of the script without fear of causing errors in production or disrupting your RapidResponse user community. For more information about test systems, see the [RapidResponse Administration Guide](#).

You can search for information about scripting objects and methods as you write your script code. For more information, see ["Search the scripting help" on page 62](#).

RapidResponse scripts run using the Google V8 scripting engine, and allow you to use any Javascript and ECMAScript features supported in Google V8 version 7.4. Although these features are supported, the sample code included in the guide focuses only on methods and objects defined in the `rapidResponse` object.

► Using resources as objects

Any resource you want to use in a script is accessed as an object. This allows you to call methods on the resource's object. All resources available to the script are contained in collections. A resource object can be retrieved from its collection by calling the collection's `get()` method and specifying the resource's name and scope. For example, to retrieve the private scenario named "My Scenario" from the scenarios collection, you could use the following.


```
scenarioObject = rapidResponse.scenarios.get({ name:"My Scenario",
scope:"Private" })
```

You must retrieve any existing resource you want to use. However, if your script creates resources, objects representing those resources are created along with the resource. In addition, if a scenario is passed in to the script through a scenario argument, you do not need to retrieve that scenario.

Resources used in method calls can be identified using variables or by including the object definition in the method call. For example, if the "My Scenario" scenario defined above is used in another method, you can identify it either using the `scenarioObject` variable, or by using the `{name: "My Scenario", scope: "Private"}` syntax.

Most resources have a `scope` property, which specifies whether the resource is private or public. A private scenario is available only to you, and a public resource is available to multiple users. Public resources can be shared with other users, but a public resource that is not shared with other users is always available to administrators. Objects that do not have a `scope` property are typically available to multiple users.

For more information about collections, see ["Resource collections" on page 110](#). For more information about retrieving resource objects, see ["get method" on page 112](#).



Note: If you are creating a script that modifies data in a scenario with a `scope` property of "Public", users must have permission to edit data in shared scenarios.

► Using script argument values

Script arguments are accessed within the script by referring to them by name. Each argument's name is used as an index into the array of arguments passed to the script.

For example, to set a variable to the value passed from the argument named `Argument1`, use the following syntax:

```
var argumentValue = rapidResponse.scriptArguments["Argument1"];
```

When using file arguments, you must use the `toString()` or `toBase64()` function to get the contents of the file or to convert an image file to a base64-encoded string the script can process, as shown in the following examples:

```
var fileContent = rapidResponse.scriptArguments
["fileArgumentName"].toString();

var profilePicture = rapidResponse.scriptArguments
["imageFileName"].toBase64();
```

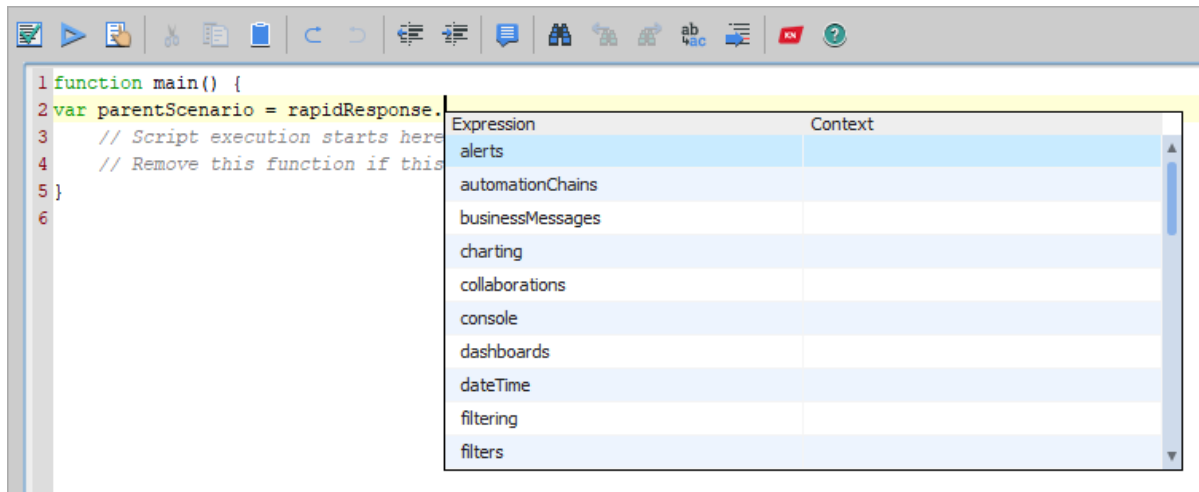
There is an exception if the file is not encoded as Unicode or UTF8.

For more information, see ["Define arguments for a script" on page 68](#).

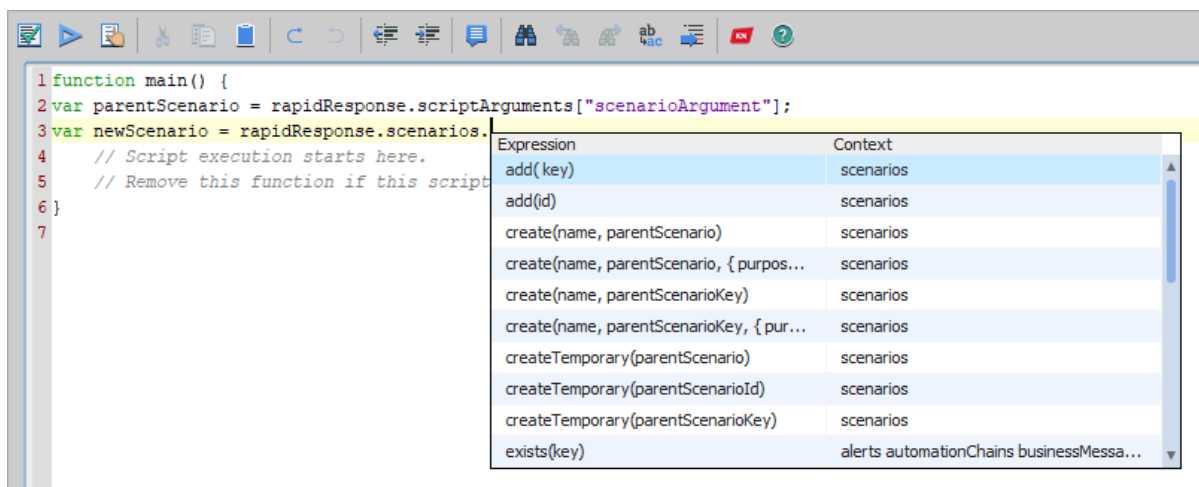
► Selecting methods and objects

As you type your script code, you can press **CTRL + SPACE** to view a list of available objects, properties, or methods. You can select an item from this list to insert it into your code. Each item shows the context it can be called from, which shows you the collections or object types the item is valid for.

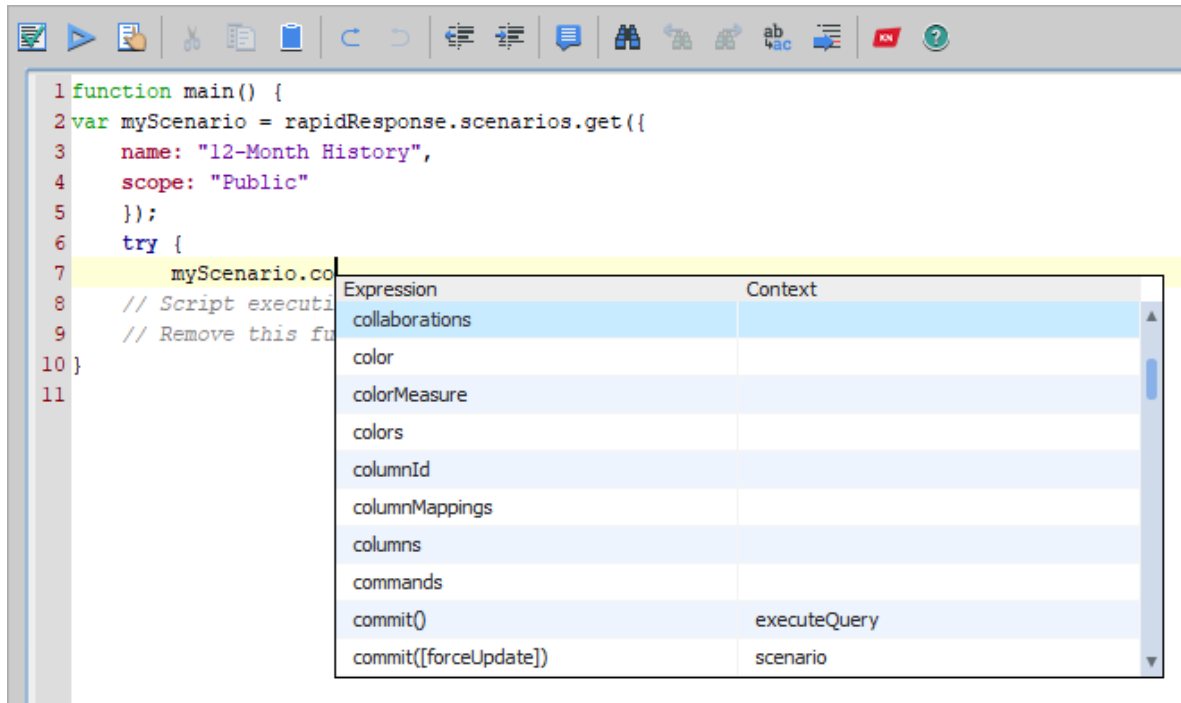
If you view the list for the `rapidResponse` object or a resource collection, the list is filtered to display the properties or methods defined by that object or collection. For example, the following illustration shows the properties of the `rapidResponse` object.



The following illustration shows methods available for the `scenarios` collection. Some of these methods are also valid for the `scripts` or `workbooks` collections.



If you view the list for a variable, the list is not automatically filtered by context. To help you find the item you want, you can search in the list of available items by typing in the line of code. The items displayed in the list contain the specified text in their names or parameters. For example, the following illustration shows items that contain the text "co".



Inserting a method using the list also inserts the method's parameters, which you can customize.

For information about the objects, properties, and methods you can use, see ["RapidResponse script object model" on page 103](#). For more information about collections and the methods that can be called from them, see ["Resource collections" on page 110](#).



Note: Some objects define properties or methods that are intended to be used only internally by Kinaxis. These are displayed in the list of items, but should not be used in your scripts.

► Error handling

You can handle errors by using a try/catch/finally structure, that catches the errors you want to check for. You should also include a throw clause to ensure any unexpected errors are caught.

The exception object that the RapidResponse object model throws has a name property with the value "RapidResponseError", and an errorCode property with values that for resource-related errors are defined using the convention ResourceType.ErrorCondition. For example, a scenario not found error can be caught using the following syntax in a catch block:

```
catch (error) {
    if (error.name == "RapidResponseError" && error.errorCode ==
        "Scenario.NotFound") {
        return ("The specified scenario does not exist.");
    }
    throw error;
}
```

When an error is caught, you can choose to report the error in the output console and continue running the script, to return an error message, or to terminate the script by throwing the error. You can also access a stack trace for the error. For more information about exceptions, see ["Exception object" on page 323](#).

► Adding comments

You can convert lines of code into comments by doing the following:

- Select a line of code, and then on the script authoring window toolbar, click **Toggle Comment** . An example of a commented line of code is shown in the following illustration.

```
//scenario.accessControlList.add("myUser", "Modify");
```



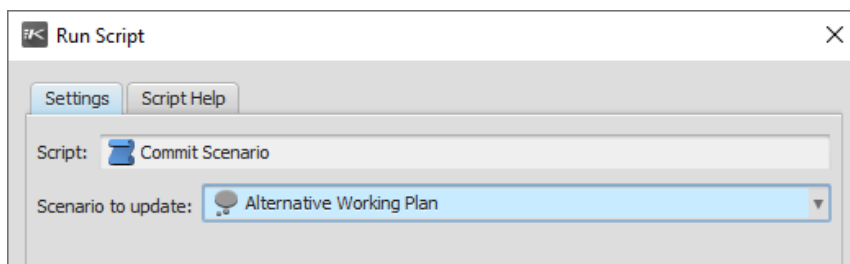
Notes:

- You can place the insertion point anywhere in a line of code before clicking **Toggle Comment**. The entire line is commented.
- If you click **Toggle Comment** when you have a commented line of code selected, that line is no longer commented.

Define arguments for a script

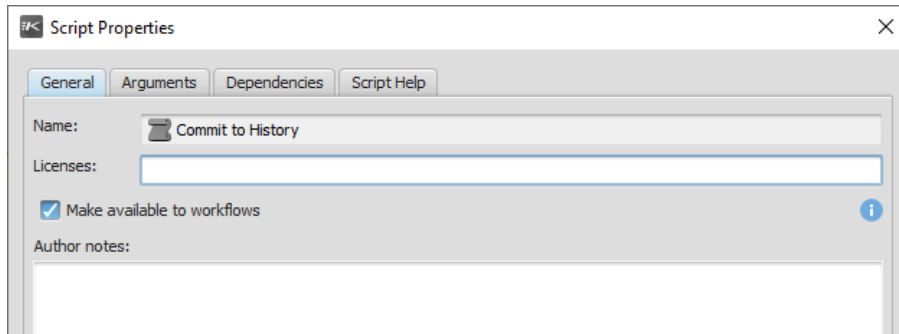
You can use arguments to pass values to a script. Arguments can be customized values such as a user specified name for a workbook or an automated order number.

When users run the script, they can specify values for each of the arguments defined in the script. If you have specified default values for the arguments, users can use those instead. An example of specifying an argument value is displayed in the following illustration.



If the script is used in a workbook command, the workbook's author can specify default values for the arguments. For more information, see the [RapidResponse Resource Authoring Guide](#).

If the script is used in a workflow, the script must be made available to workflows by selecting the **Make available to workflows** checkbox in the Script Properties dialog box.



For more information, see the [RapidResponse Resource Authoring Guide](#).

You can create the following types of arguments.

Type	Description
Boolean	Passes a JavaScript Boolean object into the script. You can use this type of argument in cases where only two options are available.
Date	Passes a JavaScript Date object into the script. You can use this type of argument to specify a minimum or maximum date to consider in worksheets.
File	Passes information from a file into the script. You can use this type of argument to integrate information contained externally in files into RapidResponse. The file used in the script must be encoded as either Unicode or UTF8. When running the script directly, you provide the path to any local file. When running the script through a scheduled task, the file must be located in a pre-configured RapidResponse file location. You configure file locations in the File Locations worksheet in the RapidResponse Administration workbook. To open this workbook, in the Administration pane, under System Settings, click Configuration. For more information, see the RapidResponse Administration Guide. If you are running the script from a Web service, you must present the file content as a base64-encoded string. For more information, see the RapidResponse Web Services Guide. A file argument can be used to set a profile picture for a user. To set an image as a profile picture, the image file object must be converted to a base64-encoded string. This is achieved using the <code>toBase64()</code> method. For example: <pre>var profilePictureString = rapidResponse.scriptArguments ["profilePicture"].toBase64();</pre>
Number	Passes a numeric value into the script. You can use this type of argument to specify any numerical data, such as a minimum value to use in a filter expression, or a number of calendar periods to add or subtract from a date.
Scenario	Passes a RapidResponse Scenario object into the script. You can use this type of argument to define a parent scenario for scenarios created by the script. This is the only RapidResponse resource that can be passed as an argument.
String	Passes text into the script. You can use this type of argument to specify a name for a scenario to create, or to specify a specific part to filter a worksheet using.

When you create a script that uses arguments, consider creating a form based on the script to provide a better user experience. For more information, see ["About forms" on page 55](#).

Arguments have the following properties:

- **Name:** The name used to refer to the argument in the script. For example, an argument named `parentScenario` would be referenced as `rapidResponse.scriptArguments["parentScenario"]`
- **Label:** The name displayed to users when they run the script. If you do not specify a value, the name is displayed to users.
- **Default value:** The value used if the user running the script does not pass a value for the argument. For scenario arguments, if the user does not have access to the scenario you specify as the default value, an 'undefined' value is passed instead. For a script that runs a script that has arguments, the default values for the arguments are used if you do not specify values to pass to that script. No default value can be set for file arguments unless the script is run as a scheduled task.
- **Description:** Information about the argument. This information can be displayed in the script help and the Run Script dialog box. For more information, see ["Running scripts" on page 95](#) or ["Create help for a script" on page 80](#).

For information about using arguments in the script, see ["Using script argument values" on page 65](#).

► Resource Context

For scripts that run from workbooks, forms, or workflows, additional arguments are available to determine the context the script was run with. This ensures that the values required by the script are passed from the workbook, form, or workflow to the script.

You can use the following context arguments to pass the resource settings to a script:

Name	Description
#Workbook	The workbook that the script runs from.
#Worksheet	The worksheet that the script was run with.
#WorksheetSelection	The values selected in the worksheet when the script ran.
#Form	The form that the script runs from.
#WorkbookContext	The workbook context passed from a form to the script.
#Workflow	The workflow that the script runs from.
#WorkflowExecutionId	The workflow ID in GUID format that the script was run from.

If you refer to the #Workbook, #Worksheet, or #WorksheetSelection arguments in your script, that script can be run only through a workbook command.

If you refer to the #Form or #WorkbookContext arguments in your script, that script can only be run through a form.

If you refer to #Workflow or #WorkflowExecutionID arguments in your script, that script can only be run through a workflow.

Attempting to run scripts with context arguments from the Explorer, Run Automation Task dialog box, or a scheduled task results in an error. For more information, see ["Worksheet arguments" on page 199](#).

► Create a Boolean argument

1. In the **New Script** or **Script Properties** dialog box, click the **Arguments** tab.
2. Click **New**, and then click **Boolean Argument**.
3. In the **New Boolean Argument** dialog box, in the **Name** box, type the name you will use to identify the argument.
4. In the **Default value** box, click **true** or **false** to specify the value to be used if a value is not specified.
5. In the **Label** box, type a name to be displayed to the user running the script.
6. In the **False display value** box, type the name to be displayed to the user for the **false** value.
7. In the **True display value** box, type the name to be displayed to the user for the **true** value.
8. In the **Description** box, type a description of the argument and what it does. This information can be displayed to users in the script help.

► Create a date argument

1. In the **New Script** or **Script Properties** dialog box, click the **Arguments** tab.
2. Click **New**, and then click **Date Argument**.
3. In the **New Date Argument** dialog box, in the **Name** box, type the name you will use to identify the argument.
4. In the **Default value** box, select the date to be used if no other date is specified.
5. In the **Label** box, type a name to be displayed to the user running the script.
6. In the **Description** box, type a description of the argument and what it does. This information can be displayed to users in the script help.

► Create a file argument

1. In the **New Script** or **Script Properties** dialog box, click the **Arguments** tab.
2. Click **New**, and then click **File Argument**.
3. In the **New File Argument** dialog box, in the **Name** box, type the name you will use to identify the argument.
4. In the **Label** box, type a name to be displayed to the user running the script or leave it as the default label **File name**.

5. In the **Description** box, type a description of the argument and what it does. This information can be displayed to users in the script help.

► **Create a number argument**

1. In the **New Script** or **Script Properties** dialog box, click the **Arguments** tab.
2. Click **New**, and then click **Number Argument**.
3. In the **New Number Argument** dialog box, in the **Name** box, type the name you will use to identify the argument.
4. In the **Default value** box, type a value to be used if no other value is specified.
5. In the **Label** box, type a name to be displayed to the user running the script.
6. In the **Description** box, type a description of the argument and what it does. This information can be displayed to users in the script help.

► **Create a scenario argument**

1. In the **New Script** or **Script Properties** dialog box, click the **Arguments** tab.
2. Click **New**, and then click **Scenario Argument**.
3. In the **New Scenario Argument** dialog box, in the **Name** box, type the name you will use to identify the argument.
4. In the **Default value** box, select the scenario to be used if no other scenario is specified.
5. In the **Label** box, type a name to be displayed to the user running the script.
6. In the **Description** box, type a description of the argument and what it does. This information can be displayed to users in the script help.

► **Create a string argument**

1. In the **New Script** or **Script Properties** dialog box, click the **Arguments** tab.
2. Click **New**, and then click **String Argument**.
3. In the **New String Argument** dialog box, in the **Name** box, type the name you will use to identify the argument.
4. In the **Default value** box, type a value to be used if no other value is specified.
5. In the **Label** box, type a name to be displayed to the user running the script.
6. In the **Description** box, type a description of the argument and what it does. This information can be displayed to users in the script help.
7. Check the **Script Log** box if you want to write the argument's value in the script log after the script has run. Consider not selecting this option for information that might be considered confidential. For example, if the string argument is taking a password, you might not want to enable this option.

► Copy an argument

1. In the **New Script** or **Script Properties** dialog box, click the **Arguments** tab.
2. Select the argument you want to copy, and then click **Copy**.
3. In the **New Argument** dialog box, type a name for the new argument.
4. Specify the default value for the new argument.
5. Type a label for the argument.
6. Type a description for the argument.

► Modify an argument

1. In the **New Script** or **Script Properties** dialog box, click the **Arguments** tab.
2. Select the argument you want to modify, and then click **Edit**.
3. Modify the name, default value, label, or description for the argument.

Specify dependencies

Scripts can depend on other resources for functionality. For example, you can call a function from another script or use a workbook to perform an automatic data modification. Specifying resources as dependencies ensures that the scripts or workbooks are always available to users you share the script with, and exports a copy of a dependent script or workbook if you export the script. You can define the following types of script dependencies:

- included scripts that act as libraries, providing functions and variables the script can use.
- referenced scripts that can be called.
- referenced workbooks that can run automatic data modifications.

You can use any of the functions defined in an included library script. You must ensure the functions and variables in your script do not have the same names as the functions and variables in the library script. For more information, see ["Notices" on page 85](#) and ["Call a function from an included script" on page 78](#).

Referenced scripts and workbooks can be used to modify data or perform additional tasks. Workbooks you reference can perform any workbook command or data modification defined in the workbook. Scripts you reference can run the script and provide the output to your script. For more information, see ["Call another script" on page 78](#).

You can also call scripts or run workbook commands from workbooks that are not specified as dependencies. However, if you share the script with other users, they might not have access to the required resource. For more information, see the [RapidResponse Resource Authoring Guide](#).

► **Include a script library**

1. In the **New Script** or **Script Properties** dialog box, click the **Dependencies** tab.
2. In the **Included script libraries** area, click **Add**.
3. In the **Select Scripts** dialog box, select each script you want to include, and then click **Add**.

► **Remove an included script**

1. In the **New Script** or **Script Properties** dialog box, click the **Dependencies** tab.
2. In the **Included script libraries** area, select the script you want to remove, and then click **Remove**.

► **Reference a script**

1. In the **New Script** or **Script Properties** dialog box, click the **Dependencies** tab.
2. In the **Referenced resources** area, click **Add**, and then click **Script**.
3. In the **Select Scripts** dialog box, select each script you want to include, and then click **Add**.

► **Reference a workbook**

1. In the **New Script** or **Script Properties** dialog box, click the **Dependencies** tab.
2. In the **Referenced resources** area, click **Add**, and then click **Workbook**.
3. In the **Select Workbook** dialog box, in the **Workbook** list, click the workbook you want to include.

► **Remove a reference**

1. In the **New Script** or **Script Properties** dialog box, click the **Dependencies** tab.
2. In the **Referenced resources** area, select the script or workbook you want to remove, and then click **Remove**.

► **View the properties of a dependency**

1. In the **New Script** or **Script Properties** dialog box, click the **Dependencies** tab.
2. In the **Included script libraries** or **Referenced resources** area, select the script or workbook you want to view properties of.
3. Click **Properties**.

Test a script

Before you deploy your script, you can test it by doing either of the following:

- Validating the script syntax.
- Running the script.

Validating the syntax finds any missing parentheses, mismatched parentheses, and malformed expressions. However, validating does not find incorrect function and method calls, variable definitions, or object usage. The result of validating the script is displayed in the output console.

Running the script executes the code, and either outputs the specified messages to the console, or reports any error conditions in the console. To view the error conditions, you should run the script from the script authoring window.

Messages are written to the console only if the script runs to completion. If you cancel the script, you can view the console output in the Automation Details and Log workbook. For more information, see ["Script logging" on page 97](#).

► **Validate a script**

1. In the **Explorer**, right-click the script you want to validate, and then click **Properties**.
2. On the script authoring window toolbar, click **Validate Script** .
3. Examine the output console.

► **Test a script by running it**

1. In the **Explorer**, right-click the script you want to test, and then click **Properties**.
2. On the script authoring window toolbar, click **Run Script** .
3. If the script returns a value, when the script completes, click **OK**.
4. Examine the output console.



Note: If you are testing a script as you are editing it, the script is automatically saved when you run it.

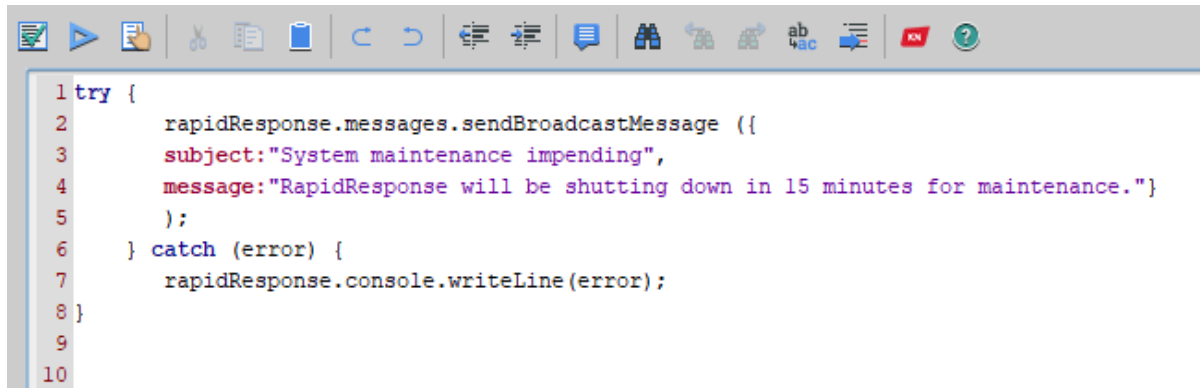


Tip: You can also run the script by pressing **F11**.

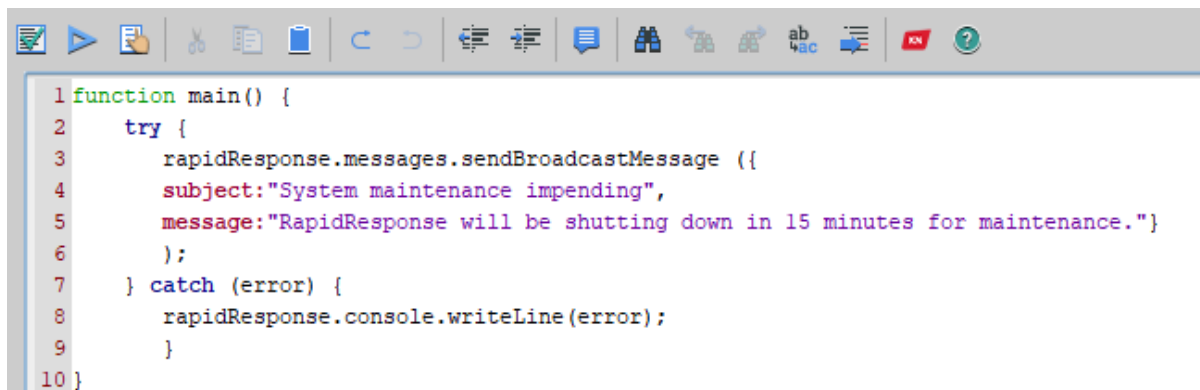
Upgrade a script

If you have upgraded to RapidResponse 11.2 or later, any scripts you have created prior to the upgrade continue to function. However, you can upgrade your scripts to take advantage of the RapidResponse 11.2 JavaScript runtime, which provides additional error reporting functionality, improves script execution performance, and supports JavaScript standards.

Upgrading a script requires changing the script's code and modifying the script's properties. You must add a global function named `main`, which is the script's start point, and is where the script begins executing. Typically, the `main` function includes variable declarations and can call other functions. If the script returns a value, that value must be returned by the `main` function. An example of adding a `main` function to a script is shown in the following illustrations.



```
1 try {
2     rapidResponse.messages.sendBroadcastMessage ({
3         subject:"System maintenance impending",
4         message:"RapidResponse will be shutting down in 15 minutes for maintenance."}
5     );
6 } catch (error) {
7     rapidResponse.console.writeLine(error);
8 }
9
10
```



```
1 function main() {
2     try {
3         rapidResponse.messages.sendBroadcastMessage ({
4             subject:"System maintenance impending",
5             message:"RapidResponse will be shutting down in 15 minutes for maintenance."}
6         );
7     } catch (error) {
8         rapidResponse.console.writeLine(error);
9     }
10 }
```

After adding the main function, you must also modify the script properties to change the script's runtime. If you add the global main function but do not change the script's runtime, your script runs but appears to not do anything. The RapidResponse 11.1 (and earlier) runtime does not execute the main() function, it only defines it.

Scripts created in RapidResponse 11.1 (and earlier) retain the option to use the other runtime. If you want to remove this option, you can export the script and then import a new copy of it. For more information, see the [RapidResponse Resource Authoring Guide](#).

► Upgrade a script

1. In the **Explorer**, open the properties of the script you want to upgrade.
2. At the beginning of the script, declare a new function named **main()**. For example,
function main() {
3. Locate the line where the main function should end, and then close the parentheses.
4. On the script editor toolbar, click **Properties** .
5. In the **Script Properties** dialog box, on the **General** tab, clear the **Run with RapidResponse 11.1 (and earlier) JavaScript runtime** checkbox.
6. Click **OK**.
7. Save your changes, and then test your script.

Calling other scripts

As you develop scripts, you might have access to a script that performs a function that you want to perform as part of another script. In this case, you can call that script or call a single function defined in that script, and use its return value in your script.

For more information, see the following.

- ["Guidelines for creating scripts to be used by other scripts" on page 77](#)
- ["Call a function from an included script" on page 78](#)
- ["Call another script" on page 78](#)

Guidelines for creating scripts to be used by other scripts

You can create a script that is intended to be used in other scripts. This can either be a script called by another script, or a script included in another script so its functions and global variables can be used. Depending on how the script is intended to be used, you must design and write the script specifically for this purpose.

If you are creating a library script that will be included in other scripts, you must ensure the global names (functions and variables) are unique. For example, you could use a prefix to indicate a function comes from the library script and to ensure the same name is not used in another script. You must also ensure each function is documented in the script help or author notes, including its required inputs and output type. This information is essential for other script authors to understand how to use the functions in the library script.

A library script should not contain a `main()` function, because it is intended only to provide functions for another script. When you create a library script, you must either delete or comment out the `main()` function declaration.

When the script runs, the library scripts run first. This ensures the library functions are available for the script to use.

For scripts that will be called by other scripts, you must ensure the arguments passed to the script and the data returned by the script are documented in the script help or author notes. If these scripts are intended only to be run by other scripts, you should leave the argument labels blank so the names are displayed in the help. Otherwise, you must ensure the argument names are documented.

Because these scripts run on their own and not as part of another script, the global names do not have to be unique. However, the called script uses the same runtime as the script that calls it. For example, if you call a script created in RapidResponse 11.2 from a script created in RapidResponse 11.1, the RapidResponse 11.1 and earlier JavaScript runtime is used.

Call a function from an included script

If you include a script in another script, you can access objects and functions defined in that script. Calling a function from an included script is identical to calling any other function, and requires the function name and input values. For example, you could create a function for sharing scenarios, which takes a scenario object and a group name as input, and then call that function from other scripts.

This function could be defined as follows

```
function share(scenario, groupName) { ... }
```

and called in another script as follows

```
var newScenario = rapidResponse.scenarios.create("Demand Spike",  
parentScenario);  
share(newScenario, "Buyers");
```

If the script's author has provided help, you can view information about the functions defined in the included script in that script's help or author notes. For more information, see ["View script help" on page 97](#).

► View help defined in an included script

1. On the script authoring window toolbar, click **Properties** .
2. Click the **Dependencies** tab.
3. In the **Included script libraries** list, select the script you want to view, and then click **Properties**.
4. In the **Script Properties** dialog box, do one or both of the following:
 - To view information specific to script authors, click the **General** tab and read the notes in the **Author Notes** box.
 - To view information about the script, click the **Help** tab.

Call another script

You can run other scripts from your script. Calling a script executes the code in that script.

To call a script, do the following:

- Retrieve the script you want the script to call using the [get\(\) method](#).
- Call the script using the [call\(\) method](#), including values for arguments.

You can provide values for any arguments the script takes. Arguments are specified as an object that defines the argument names and the values passed to them. If you do not provide a value for an argument, its default value is used. However, if you specify a value for an argument that does not exist, the script call fails. For more information about arguments, see ["Define arguments for a script" on page 68](#).

Every called script must be available to the user running the script, otherwise the script cannot run. To ensure users have access to all required scripts, you can specify the script as a dependency. If you define the script as a dependency, it is shared along with the script that calls it. For more information, see the [RapidResponse Resource Authoring Guide](#).

You can call multiple scripts, and each script you call can call other scripts. If any of the called scripts contain an error, every script fails.

► Call a script

1. Get the script object you want to call. For example,

```
var callScript = rapidResponse.scripts.get({name:"Delete Child",  
scope:"Public"});
```
2. If the script returns a value, declare a variable to hold that value.
3. Call the script using the script's `call()` function

```
callScript.call({scenario: myScenario});
```

or

```
var scriptOutput = callScript.call({scenario: myScenario});
```



Notes:

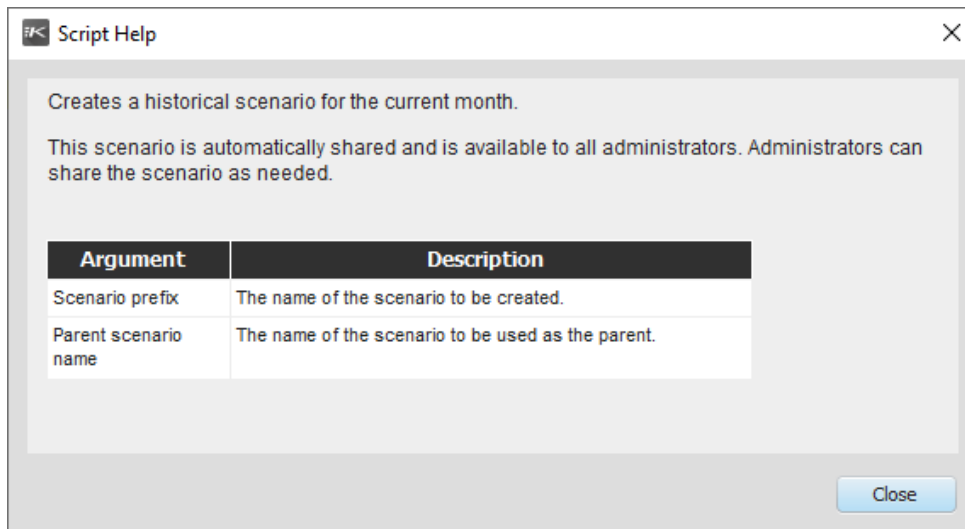
- This example assumes a scenario object named `myScenario` was created previously in the script.
- For more information, see ["call method" on page 136](#).

Adding help for scripts

You can add help to your scripts to provide information about what the script does. Depending on how you intend the script to be used, you can provide different information. For example, if the script is intended to be run by users, you can describe what the script does and how to set values for its arguments. If the script is intended to be a library script for other script authors to use, you can describe what each of the script's functions does and its required inputs.

You can also automatically include descriptions of the script's arguments. For information about adding script help, see ["Create help for a script" on page 80](#).

An example of script help is shown in the following illustration.

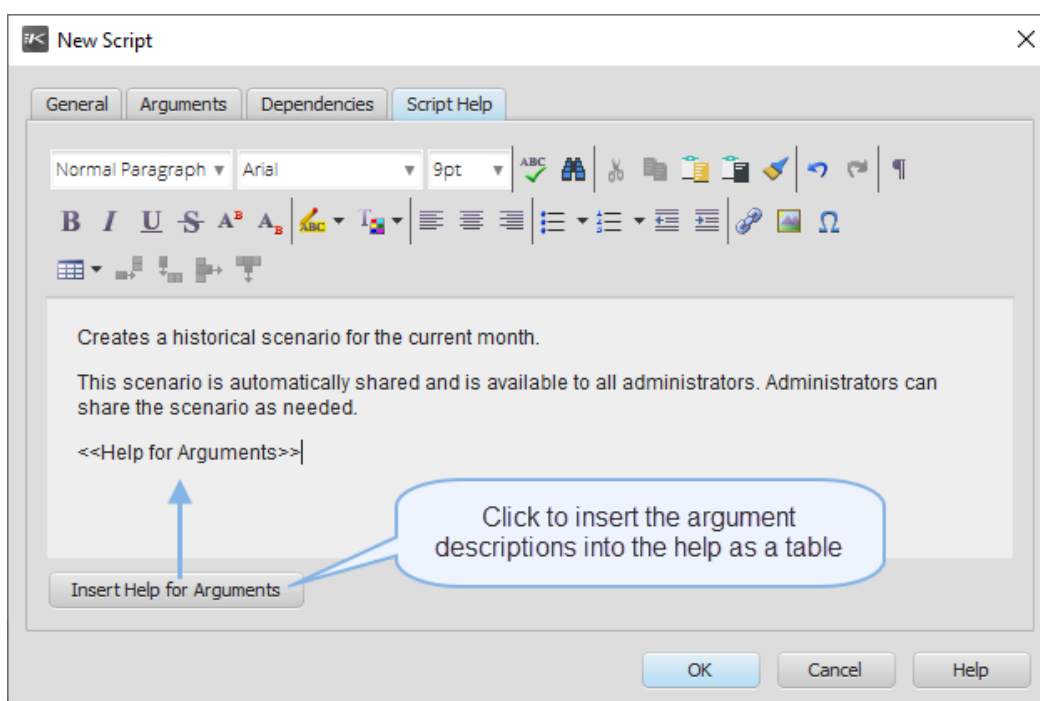


In addition to this help, you can also include notes specifically for other script authors. For more information, see ["Add author notes" on page 81](#).

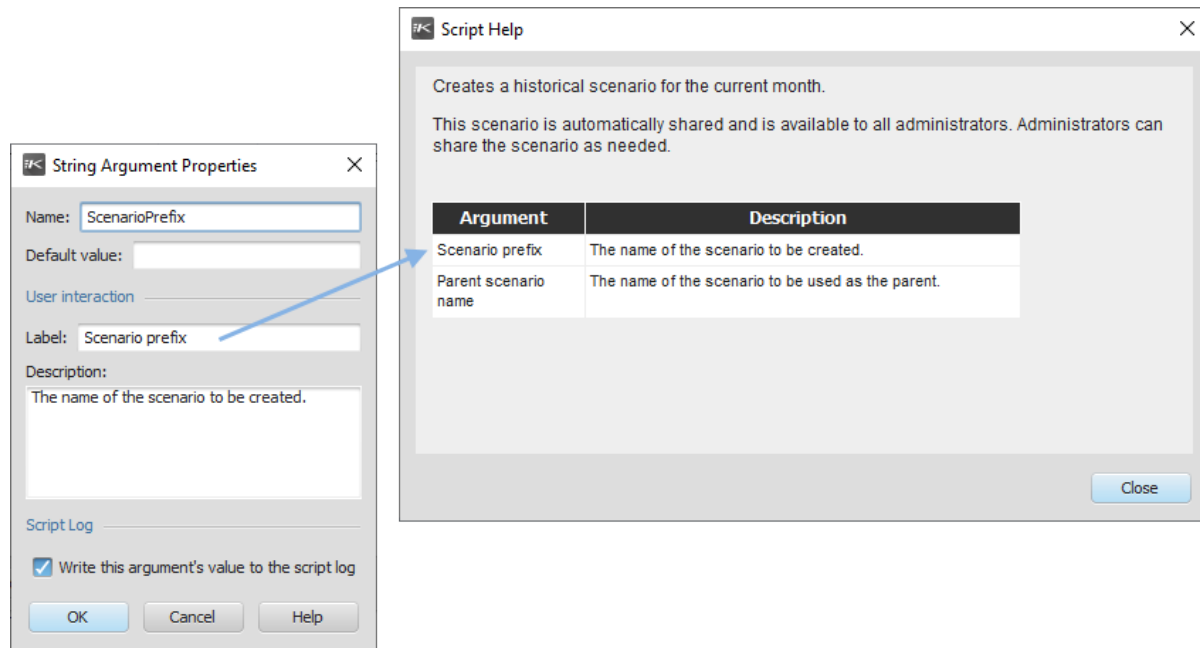
Create help for a script

Script help is defined in the script's properties. The help consists of text you enter, and an optional table of arguments, which can be automatically inserted. You can also insert graphics, links to external Web sites, or lists into the help. For complete information about the options for help authoring, see "Using the text editing tools" in the [RapidResponse Resource Authoring Guide](#).

An example of script help is shown in the following illustration.



When the script help is displayed, the <<Help for Arguments>> tag is replaced by a table showing descriptions for each argument. These descriptions are taken from the argument definition, as shown in the following illustration.



► Create script help

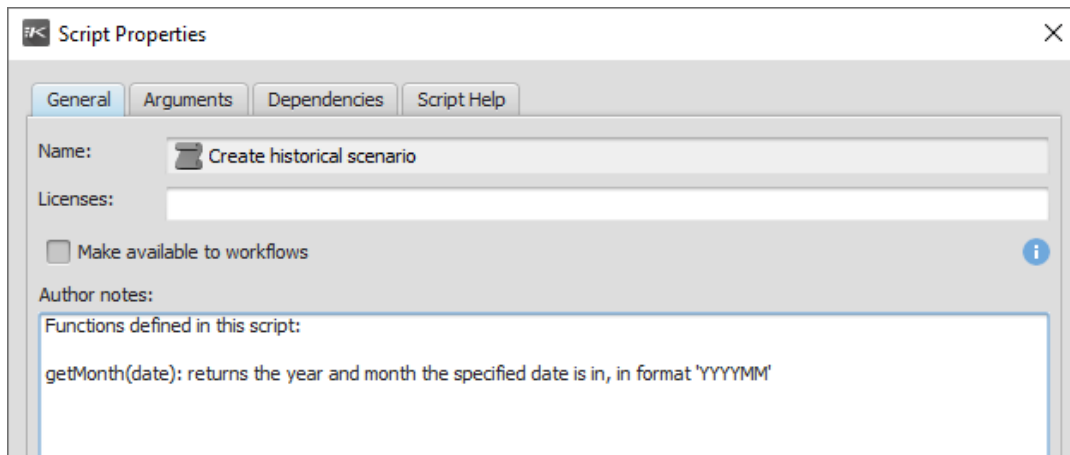
1. In the **New Script** or **Script Properties** dialog box, click the **Script Help** tab.
2. In the help authoring window, describe the script.
3. To include the table of argument descriptions, place the insertion point where you want the table to display, and then click **Insert Help for Arguments**.

Add author notes

In addition to the script help for users, you can include author notes, which provide information about what the script does or the functions it defines or calls. These notes are intended for other script authors, and are not displayed to users. For example, you can use author notes to:

- If the script returns a value, provide an explanation of the value returned and its data type.
- Provide revision history.
- Describe functions defined in the script, which could be called by other scripts.
- Describe dependencies.
- Provide plans for future improvement.
- Describe known issues.

An example of an author note is shown in the following illustration.



► Add an author note

1. In the **New Script** or **Script Properties** dialog box, click the **General** tab.
2. In the **Author Notes** box, type the information you want to provide for other script authors.

Editing and managing scripts

After creating a script, you might need to edit its code or its properties. For more information, see ["Edit a script" on page 82](#).

You can also rename, share, give, or delete scripts. For more information, see ["Managing scripts" on page 84](#).

Edit a script

You can modify the code or properties of any script you own. For example, you can define new arguments, modify existing arguments, add or remove dependencies, add or remove functions, and add or remove method calls and function calls.

If your company uses resource version control, you must check out any shared script you modify. For more information, see the [RapidResponse Resource Authoring Guide](#).

If you modify a shared script and add a dependency, that dependent workbook or script is automatically shared with the other users that have access to the script.

► Edit script code

1. In the **Explorer**, select the script you want to modify.
2. On the **Actions** menu, click **Properties**.
3. Modify the code.

4. Do one of the following:
 - To save the changes, on the **File** menu, click **Save Script**.
 - To save the changes as a new script, on the **File** menu, click **Save Script As**, and then specify a name for the new script.
5. Test the changes. For more information, see ["Test a script" on page 74](#).

► Edit arguments

1. In the **Explorer**, select the script you want to modify.
2. On the **Actions** menu, click **Properties**.
3. On the script authoring window toolbar, click **Properties** .
4. In the **Script Properties** dialog box, click the **Arguments** tab.
5. Select the argument you want to modify, and then click **Properties**.
6. Modify the argument's name, label, or default value. For more information, see ["Define arguments for a script" on page 68](#).
7. Update the code to refer to the new or modified arguments.
8. Do one of the following:
 - To save the changes, on the **File** menu, click **Save Script**.
 - To save the changes as a new script, on the **File** menu, click **Save Script As**, and then specify a name for the new script.
9. Test the changes. For more information, see ["Test a script" on page 74](#).

► Delete an argument

1. In the **Explorer**, select the script you want to modify.
2. On the **Actions** menu, click **Properties**.
3. On the script authoring window toolbar, click **Properties** .
4. In the **Script Properties** dialog box, click the **Arguments** tab.
5. Select the argument you want to delete, and then click **Delete**.
6. Update the code to remove references to the deleted argument.
7. Do one of the following:
 - To save the changes, on the **File** menu, click **Save Script**.
 - To save the changes as a new script, on the **File** menu, click **Save Script As**, and then specify a name for the new script.
8. Test the changes. For more information, see ["Test a script" on page 74](#).

► Add or remove dependencies

1. In the **Explorer**, select the script you want to modify.
2. On the **Actions** menu, click **Properties**.
3. On the script authoring window toolbar, click **Properties** .
4. In the **Script Properties** dialog box, click the **Dependencies** tab.
5. Add or remove included library scripts or referenced resources as discussed in ["Specify dependencies" on page 73](#).
6. Update the code to refer to the new dependent resources or remove the references to the deleted dependencies.
7. Do one of the following:
 - To save the changes, on the **File** menu, click **Save Script**.
 - To save the changes as a new script, on the **File** menu, click **Save Script As**, and then specify a name for the new script.
8. Test the changes. For more information, see ["Test a script" on page 74](#).



Tip: You can also edit a script by selecting it in the **Explorer**, and then pressing **ALT + ENTER**.

Managing scripts

You can perform the resource management operations on scripts, including copying, giving, sharing, importing, exporting, and deleting. For complete information about managing resources, see the [RapidResponse Resource Authoring Guide](#).

Custom messages in forms

You can create custom messages in the script to display on the form or on form controls. By providing specific information, custom messages provide more support for form users.

You can create two types of messages based on the scripting objects they use:

Notices

Notices can be success notices, error messages, warnings, general information, or questions. Each message type displays in a different color and style. Depending on how you define the notice, you can add predefined or custom buttons for users to interact with or you can set the notice to automatically close. For more information, see ["Notices" on page 85](#).

Control errors

Control errors display on a control in response to an invalid value. The message displays until the user corrects the error and runs the form again. All input controls, except for checkbox controls, can display a control error. For more information, see ["Control errors" on page 87](#).

Notices

When you create a notice with custom buttons or a notice prompt, you can also specify the type of notice that displays: success notices, error messages, warnings, general information, or questions. Each notice type features a different icon.



42 new orders were inserted

OK



The order couldn't be processed because the site is closed for maintenance

OK



This order modifies 3 records.

OK



This is the second order for customer BCI

OK



This due date is two days that the usual delivery for customer BCI. Do you want to continue?

NO

YES

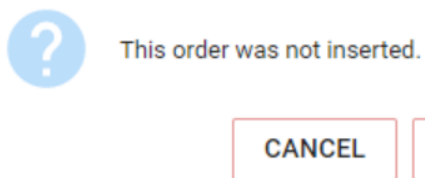
► Notices with predefined buttons

You can create a notice with an object method that displays a set of predefined buttons on the notice:

- OK and Cancel buttons
- Yes and No buttons
- Yes, No, and Cancel buttons

All of these notices display the question type style and color.

The code sample below outlines how you create a notice that displays OK and Cancel buttons. The default action for Cancel is to dismiss the script unless you specify otherwise.



```
var notice = Form.okCancelNotice("This order was not inserted.",
Form.runScriptAction("retryInsert"), Form.closeFormAction());

return Form.response ({
  notice:notice,
  data: {
    scenarioKey: {
      name: 'My Scenario',
      scope: 'Private'
    }
  }
});
```



Tip: You can create different types of messages with OK, Cancel, Yes, and No buttons by specifying those button labels in custom buttons.

► Notices with custom buttons

You can add notices with up to three custom buttons you define. You can specify the button label and action and what type of notice displays: success, error, warning, info, or question.

The code sample below outlines how to create a notice with two custom buttons: Continue and Cancel.



This order is a duplicate of order TR-908.
Do you want to submit this order?

CANCEL

CONTINUE

```
var notice = Form.notice ({
  message: "This order is a duplicate of order " + orderId +
    ". Do you want to submit this order?",
  type: "Warning",
  buttons: [
    Form.button("Continue", Form.runScriptAction("continue")),
    Form.button("Cancel", Form.dismissNoticeAction())
  ]
});
```

► Notice prompts

This type of message displays briefly and then automatically closes the form. No buttons display on a notice prompt. For example, you might create a prompt message to inform users about how many new orders were inserted.

The code sample below outlines how to create a notice prompt.



42 new orders were inserted

```
var notice = Form.autoCloseNotice ("42 new orders were inserted",
  "Info");
```

Control errors

You can create custom error messages on form controls that display after the form validation has completed and the form is run. Control errors are defined using `ControlMessage` objects in the script and passed back to the form in an `errorResponse`.

The code sample below outlines how to create a control error that displays on the form control mapped to the quantity argument, when the input value is not a multiple of 5.

Part Name  *

CS004 

Quantity  *

4

Quantity must be a multiple of 5.

```
var argumentErrors = [];  
if (quantity % 5 != 0)  
{  
    argumentErrors.push(Form.argumentError('quantity',  
        'Quantity must be a multiple of 5.'));  
}  
if(argumentErrors.length > 0)  
{  
    throw Form.errorResponse(  
        {  
            controlMessages: argumentErrors  
        });  
}
```

Using scripts to communicate with external systems

You can use a script to send notifications or data from RapidResponse to any external system you have access to. These communications require your RapidResponse administrator to define an endpoint for each system you need to communicate with. Endpoints define the location of the external system and include authentication, so you can make requests without requiring additional information about the endpoint. Each endpoint supports at least one HTTP method, which is defined by the external system, and defines what the endpoint does with the data or request you send it.

You can communicate with external systems to send notifications and messages about processes to external systems that are expecting data, and use the notification to trigger a process or indicate a process is ready. You can also send data to these systems using the endpoint, which requires you to define a payload. Endpoints called through scripts can accept payloads up to 50 MB.

As part of calling the endpoint, you can also pass values for custom and standard headers, and define parameters. You can use this to define connection parameters specific to the external system if required, or to change how the request is made. For example, if your external system requires a header that defines the source of the data, you can add that header and set its value to identify the data comes from RapidResponse.

If you pass data to the endpoint, you can define the data by constructing values in the script, or by creating a workbook or process that defines the data to be sent. Regardless of how you generate the data, it must be configured to contain the columns or fields the endpoint requires, in the format the

external system expects. For example, if the endpoint expects a JSON string with four fields, your script must define a string or object with four fields and convert them to a JSON string.

If you define the data to send to the endpoint using worksheets, you must configure the worksheet to return the data the external system requires, and you can use the script to retrieve that data and convert it to the format the endpoint requires. You can use the worksheet data rows and cells to get the values for each column in each row, and format them for the external system. For example, if your endpoint expects a JSON string, you can send worksheet data by constructing an array that contains the values from each cell in a worksheet row, and then converting the array to a JSON string.

You can also use these endpoints to retrieve data or notifications from external systems. These function similarly to the RapidResponse REST API endpoints, but allow you to coordinate operations using messages or bring in small amounts of data to support a process. For example, if you have defined a process that exports data from RapidResponse and provides it to an external system, you can use a script to check if the external system is done processing the data, and then bring data changes back in when the external process is complete. For more information about the RapidResponse REST API, see the [RapidResponse Web Services Guide](#).

To use these endpoint triggers in processes, you can create scripts that perform notifications and include them in an automation chain. For example, you can define an automation chain that runs a workbook command through a scheduled task, and then runs a script that notifies an external system that data in the workbook has been modified. For more information about automation chains, see the [RapidResponse User Guide \(Java Client\)](#)

For information about accessing endpoints and sending requests to external systems, see ["integrationPlatformService property" on page 423](#).

Sample scripts available for RapidResponse

You can access sample scripts that define some common practices. You can use these samples as a starting point for creating your own scripts. The following sample scripts are available.

- A script that creates a scenario. For more information, see ["Example: creating a scenario" on page 89](#).
- A script that maintains data in a set of historical scenarios. For more information, see ["Managing historical scenarios" on page 90](#).
- A script that modifies data values in a workbook. For more information, see ["Modify worksheet data" on page 91](#).

Example: creating a scenario

The **Sample: Create monthly scenario** script is available from the RapidResponse Developer Forum. This script creates a scenario using a string entered by the user and a date calculated by a function in the script, and then shares it.

This script has the following arguments:

Argument	Default value	Description
newScenarioName	"HistoricalScenario"	The string value that defines the first part of the scenario's name.
parentScenario	Enterprise Data	The new scenario's parent.

You can access the script by doing the following:

1. Sign in to the [Kinaxis Knowledge Network](#) and then access the [Files](#) section of the Developer Forum.
2. Click the link for the **Sample Scripts.zip** file.
3. Specify a location to save the file, and then click **OK**.
4. Open the **Sample Scripts.zip** file, and extract the sample scripts.
5. In RapidResponse, import the **Sample: Create Monthly Scenario.spt** file. For more information, see the [RapidResponse Resource Authoring Guide](#).

This script catches three specific exceptions that can be thrown when attempting to create a scenario. These exceptions return custom error messages. Other exceptions are caught and re-thrown, which terminates the script with the exception's error message.

Managing historical scenarios

The **Sample: Historical Scenario Management** script is available from the RapidResponse Developer Forum. This script maintains a set of historical scenarios. This script is intended to be run on a regular schedule, such as at the end of each month. By default, three months and four quarters of rolling historical data are maintained, but you can modify these values. Scenarios for periods outside of the ones maintained are deleted.

This script can optionally take an argument, which passes a date in. To use this value, you must create the argument, which is described in the script's comments. For more information, see "[Define arguments for a script](#)" on page 68.

The script also contains a function that deletes older scenarios based on their creation date. By default, this function call is commented, so you must remove the comment to call it .

The script creates a new historical scenario for the past month, and then determines which existing historical scenarios can be rolled over and which need to be deleted.

You can obtain this script by doing the following.

1. Sign in to the [Kinaxis Knowledge Network](#) and then access the [Files](#) section of the Developer Forum.
2. Click the link for the **Sample Scripts.zip** file.
3. Specify a location to save the file, and then click **OK**.
4. Open the **Sample Scripts.zip** file, and extract the sample scripts.

5. In RapidResponse, import the **Sample: Historical Scenario Management.spt** file. For more information, see the [RapidResponse Resource Authoring Guide](#).

This script defines functions that perform date calculations and manage scenarios.

Modify worksheet data

You can use a script to modify data in a worksheet. This requires you to have access to a workbook that contains data modification commands, which you then call from the script.

The workbook you use for modifying the data must contain worksheets for each of its data modifications. For more information, see the [RapidResponse Resource Authoring Guide](#).

Depending on the requirements of your data modification, you can define the worksheet's filter as a variable that you specify in the script. The example below uses a variable that defines a list of parts, and is used as the worksheet's filter expression.

The following code sample defines the script that runs the command on a temporary scenario, and then commits that scenario.

```
var workbookName = "workbookUpdate"; //Workbook Name
var partFilter = "Part.Name in ('Cruiser', 'Racer', 'Mountain')"; //
Filter to select parts
var commandName = "wkCmdUpdate";
var scenario = rapidResponse.scenarios.createTemporary({name:"Approved
Actions", scope:"Public"});

//Open the workbook
var workbook = rapidResponse.workbooks.get( {name: workbookName,
scope:"Public"},
{
    scenarios: [ { name: scenario, scope:"Private" } ],
    variables: [ { name:"filterValue", value: partFilter } ]
} );

//Execute the command
var workbookCommand = workbook.commands.get(commandName);
workbookCommand.execute();
scenario.commit();
```

Create a new order using a form

You can use a form and its underlying script to create new customer order records in a worksheet. Users will need access to the form and the workbook where the customer order records are modified.

The following code sample defines the underlying script that executes when a form to add a new order record is run. The script checks to see that the order quantity submitted is a multiple of five and if no order is submitted, displays an error message for users.

```
//The listed arguments are created by defining arguments on the
Arguments tab of the script properties dialog box.
```

```

function main() {
var scenario = rapidResponse.scriptArguments["scenario"];
var site = rapidResponse.scriptArguments['site'];
var customer = rapidResponse.scriptArguments['customer'];
var partName = rapidResponse.scriptArguments['partName'];
var quantity = rapidResponse.scriptArguments['quantity'];
var dueDate = rapidResponse.scriptArguments['dueDate'];
var orderNumber = rapidResponse.scriptArguments['orderNumber'];
var orderType = rapidResponse.scriptArguments['orderType'];
var highPriority = rapidResponse.scriptArguments['highPriority'];

//This script can only be run through a form.

var Form = rapidResponse.scriptArguments['#Form'];

// Allows user to request quantities only in multiples of 5. This
value can also be sourced from the lot size of the part being
requested.

var quantityMultiple = 5
var argumentErrors = [];

if (quantity % quantityMultiple != 0)
{
    throw Form.errorResponse ({
        controlMessages:[Form.argument.Error("quantity","Quantity must
be a multiple of " + quantityMultiple + ".")]
    });
}

//If no order number is specified in the form, a default number is
created by merging the customer name with Forms. Order.

if(!orderNumber || orderNumber.length == 0)
{
    orderNumber = 'Forms.Order' + customer;
}

```

```

    }

    //Shortcut for an IF statement, where if the order priority is high,
    then the value is specified as high, else it is specified as med.

    var orderPriority = highPriority ? "high" : "med";

    //Calling the workbook and specific worksheet to import values from
    the form into.

    var workbook = rapidResponse.workbooks.get({ name: 'Orders', scope:
    'Private'},
        {
            scenarios: [scenario],
            filter: {name: "All Items", scope:"Public"},
            siteGroup: {name: site, scope: "Public"},
            variables: [{name:"orderNumber", value: orderNumber},
            {name:"orderSite", value: site}]
        });

    var worksheet = workbook.worksheets.get("InsertOrder");

    //Cells aligned to column order and ID in the worksheet are created to
    pass values from the form into the workbook.

    var importData = [[customer, orderNumber, orderType, orderPriority,
    partName, site, quantity, dueDate]];
    var result = worksheet.importData(importData);

    //If no record is inserted, then a custom error message displays.

    if(result.inserted != 1) {
        var notice = null;

        notice = Form.okCancelNotice("Are you want to commit the order in
        this scenario?", Form.runScriptAction("ok"), Form.runScriptAction
        ("cancel"));

        return Form.response ({
            notice:notice,

            data: {
                scenarioKey: {
                    name: 'My Scenario',
                    scope: 'Private'
                }
            }
        });
    }

    worksheet = workbook.worksheets.get("GetFilteredOrder");

```

```

var worksheetData = worksheet.getData();
var numRows = worksheetData.rows.count;

if(worksheetData.status == "OK" && numRows > 0) {
    var defaultLineLength = "Forms.Line".length;
    var maxLineNum = 0;
    var availableDate;

    //Loop to insert data from the form into new records.
    for(var i = 0; i < numRows; i++) {
        var cells = worksheetData.rows[i].cells;
        var line = cells[2].value;
        var num = line.slice(defaultLineLength);
        if(num.length > 0) {
            var lineNum = +num;
            if(lineNum > maxLineNum) {
                maxLineNum = lineNum;
                availableDate = cells[10].value;
            }
            else if(!availableDate) {
                availableDate = cells[10].value;
            }
        }
        if(availableDate) {
            var notice = Form.autoCloseNotice("Order " + orderNumber +
            " will be available on " + availableDate);
            return Form.response({notice: notice});
        }
    }
}

```

CHAPTER 7: Running scripts

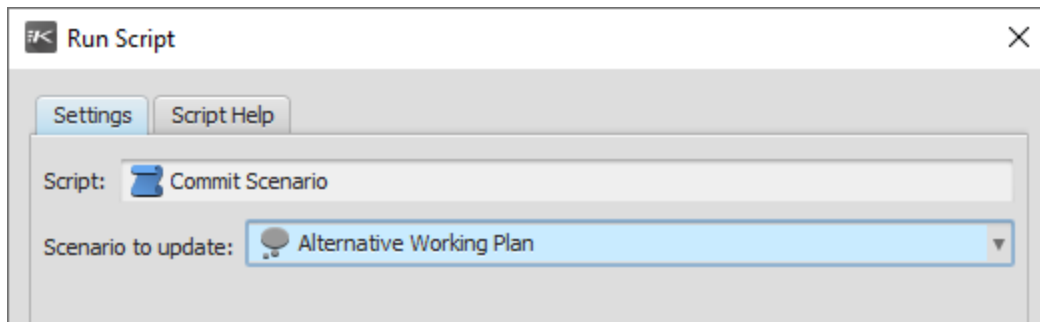
Schedule a script to run automatically	97
Script logging	97
Running scripts from Web services	99

A script you own or that has been shared with you can be run in any of the following ways:

- From the Run Automation Task dialog box.
- From the Explorer.
- Using a form. For more information, see ["About forms" on page 55](#).
- Using a workbook command. For more information, see the [RapidResponse Resource Authoring Guide](#).
- Using a scheduled task. For more information, see ["Schedule a script to run automatically" on page 97](#).
- By a Web service client, as discussed in ["Running scripts from Web services" on page 99](#).

For more information about sharing scripts with other users, see the [RapidResponse Resource Authoring Guide](#).

When you run a script, you can specify values for each of its arguments, as shown in the following illustration.



If you need more information about an argument, you can view the script's help. For more information about arguments, see ["Define arguments for a script" on page 68](#).

If you run a script from the Explorer, you are prompted to cancel the script after it has been running for 20 seconds. If you do not cancel the script, you must wait for it to complete. However, if you cancel the script, any operations it has performed are not reverted. Depending on the operations the script performs, canceling the script could leave your system in an unstable state.

If a user who is not a script author runs the script, or you schedule it or run a workbook command that calls it, only a RapidResponse administrator can cancel the script. For more information, see the [RapidResponse Administration Guide](#). For more information about running workbook commands that call scripts, see the [RapidResponse User Guide \(Java Client\)](#).

► Run a script

1. On the RapidResponse toolbar, click **Run Automation Task** .
2. In the **Run Automation Task** dialog box, select the script you want to run, and then click **OK**.
3. In the confirmation dialog box, click **Yes**.
4. When the script completes, click **OK**.



Tip: You can also run a script from the **Explorer** by right-clicking the script you want to run, and then clicking **Run Now**.

► Run a script with arguments

1. On the RapidResponse toolbar, click **Run Automation Task** .
2. In the **Run Automation Task** dialog box, select the script you want to run, and then click **OK**.
3. In the **Run Script** dialog box, on the **Settings** tab, specify a value for each listed argument.
4. Click **Run**.
5. When the script completes, click **OK**.



Note: The **Run Script** dialog box contains only the script arguments if the script does not have help defined. In this case, the **Settings** tab label is not displayed.



Tip: You can also run a script from the **Explorer** by right-clicking the script you want to run, and then clicking **Run Now**.

► View script help

- In the **Run Script** dialog box, click the **Script Help** tab.



Note: The **Script Help** tab is displayed only if the script has help defined.

Schedule a script to run automatically

You can create a scheduled task that runs a script at a specific time. You can schedule any script you have access to.

When you schedule a task, you must specify when it runs and the settings it runs with. For tasks that run scripts, this includes which script to run and values for each argument.

For complete information about scheduling operations, see the [RapidResponse User Guide \(Java Client\)](#).

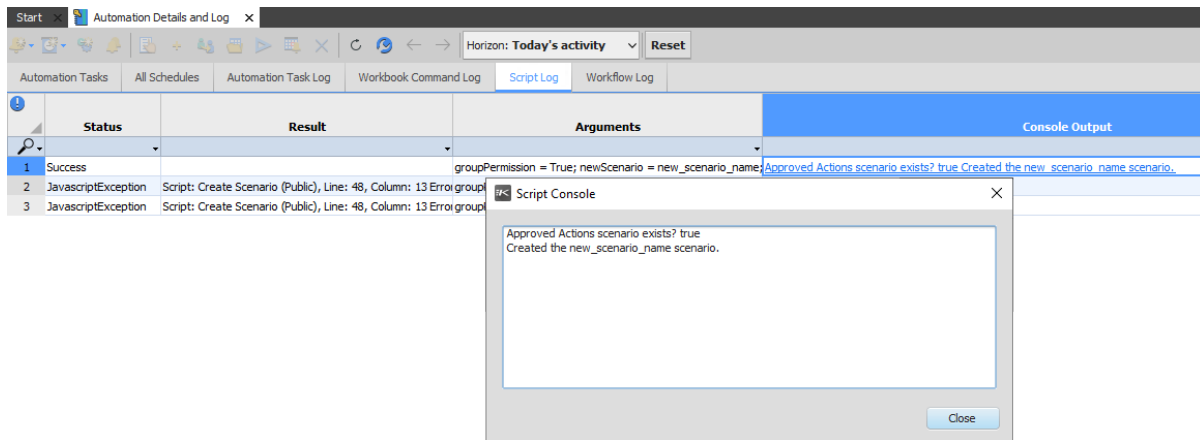
Script logging

When you run a script, its result is recorded in the Automation Details and Log workbook's Script Log worksheet. The Script Log contains details about the script, including when it ran, how long it took to run, the result of running it, and the script's console output, which includes error information if the script failed. You can use the Script Log to view your script history.

Scripts are logged after you run them from the script authoring window, the Run Automation Task dialog box, a workbook command, or a scheduled task. An example of script log entries is shown in the following illustration.

	Start	Finish	Run Time (seconds)	Script	Status	Result
1	07-29-21 4:58 PM	07-29-21 4:58 PM	0	Broadcast Maintenance	EntryPointNotFound	Script: Broadcast Maintenance (Private), Line: 1, Column: 1 ErrorCo
2	07-29-21 3:20 PM	07-29-21 3:20 PM	0	Copy User Account	Script.Argument.NotFound	Script: Copy User Account (Public), Line: 2, Column: 45 ErrorCode:
3	07-29-21 3:18 PM	07-29-21 3:18 PM	0	Reset User Account	Success	

For scripts that write output to the console, the messages are also reported in this worksheet. If you cancel the script, this worksheet is the only place you can view the messages. The messages are displayed in the worksheet, with a link to the complete console output, as shown in the following illustration.



For scripts that fail, you can view the line the script's exception was thrown from. This allows you to determine which line the exception originated from, regardless of whether you re-throw the exception or stop execution when the exception is caught. The line number of the exception is available in the Result and Console Output columns of the Script Log worksheet.

You can choose how much history is displayed in the Script Log. For example, you can view scripts that have run on the current day, the past week, past month, and so on.

► View the script log

1. On the **View** menu, click **Automation Details and Log**.
2. In the **Automation Details and Log** workbook, click the **Script Log** tab.



Note: For more information about the **Automation Details and Log** workbook, see the [RapidResponse User Guide \(Java Client\)](#).

► View console output

1. In the **Automation Details and Log** workbook, click the **Script Log** worksheet.
2. Click a link in the **Console Output** column.

► Specify how many days of log entries are displayed

1. In the **Automation Details and Log** workbook, click the **Script Log** worksheet.
2. In the **Horizon** list, click the number of days to display entries for.

► Refresh the script log

1. In the **Automation Details and Log** workbook, click the **Script Log** worksheet.
2. On the workbook toolbar, click **Refresh Worksheet** .

Running scripts from Web services

A RapidResponse Web service client can be used to run scripts. This requires a Web service user account to have access to the script and any additional resources the script requires, such as scenarios or filters. Web services allow you to run scripts over the Internet or your company's local intranet, and to run the script as part of an external process.

For more information about Web services, see the [RapidResponse Web Services Guide](#).

Part 3: Script reference

- [RapidResponse script object model](#)
- [Resource objects](#)
- [Script objects](#)
- [Scenario objects](#)
- [Workbook objects](#)
- [Worksheet objects](#)
- [TreemapQuery object](#)
- [FilterManager object](#)
- [Hierarchy objects](#)
- [SiteGroup object](#)
- [Alert objects](#)
- [ScheduledTask objects](#)
- [AutomationChain objects](#)
- [Schedule objects](#)
- [TaskFlow objects](#)
- [ProfileVariable objects](#)
- [ProcessingRule object](#)
- [Dashboard objects](#)
- [Widget objects](#)
- [Form objects](#)
- [Scorecard objects](#)
- [Collaboration objects](#)
- [Responsibility objects](#)
- [Workflow objects](#)
- [Query object](#)
- [ProcessTemplate objects](#)
- [charting property](#)
- [integrationPlatformService property](#)

- Link object
- dateTime property
- console property
- messages collection
- User objects
- Group object
- server property
- Exception object

CHAPTER 8: RapidResponse script object model

The rapidResponse object is defined as a JavaScript global variable and has objects and properties that are used to access all RapidResponse scripting functionality.

The rapidResponse object has the following properties, which define the input to the script, the output methods, and resources available to the script.

Property	Description
alerts	The collection of alerts available to the script. For more information, see " alerts collection " on page 281
automationChains	The collection of automation chains available to the script. For more information, see " automationChains collection " on page 301. This property was added in RapidResponse 2014.1.
charting	The charting methods used for displaying charts in crosstab worksheets and dashboards. For more information, see " charting property " on page 419
collaborations	The collection of collaborations available to the script. For more information, see " collaborations collection " on page 375.
console	The debugging console output, including methods for writing information to the console. For more information, see " console property " on page 445.

Property	Description
dashboards	The collection of dashboards available to the script. For more information, see "dashboards collection" on page 333 .
dateTime	The RapidResponse date and time constants and methods used for date calculations. For more information, see "dateTime property" on page 439 .
exceptions	The object for exceptions throw by the RapidResponse scripting object model. For more information, see Exception object
filters	The collection of filters available to the script. For more information, see "filters collection" on page 242 .
filtering	The filter creation methods used for creating filter expressions from the data settings applied to a workbook. For more information, see "FilterManager object" on page 253 .
forms	The collection of forms available to the script. For more information, see " forms collection" on page 351 .
groups	The collection of user groups. For more information, see "groups collection" on page 468 .
help	The RapidResponse help. For more information, see "HelpInformation object" on page 110 . This property was added in RapidResponse 2013.4.
hierarchies	The collection of hierarchies available to the script. For more information, see hierarchies collection .
integrationPlatformService	The RapidResponse Integration Platform endpoint, which is used for running tasks on the Integration Platform for RapidResponse powered by Talend or sending messages using the Real-time Integration Platform. This property is relevant only if your company uses the On-Demand RapidResponse service, and uses either the Integration Platform for RapidResponse powered by Talend or the Real-time Integration Platform. For more information, see the RapidResponse Data Integration Guide .
links	The collection of links that are defined for dashboards. For more information, see "links collection" on page 435 .
loggedInUser	The user running the script.

Property	Description
messages	Messages that can be sent to other users, groups, or email addresses. For more information, see "messages collection" on page 449 .
permissions	The set of permissions that determine script functionality. For more information, see "Permissions object" on page 464 . This property was added in RapidResponse 2013.4.
profileVariables	The collection of profile variables available to the script. For more information, see "profileVariables collection" on page 325 .
processingRule	The collection of processing rules or system settings in RapidResponse. For more information, see processingRules collection .
processTemplate	The collection of high level processes defined by process templates. For more information, see processTemplates collection .
queries	Methods for retrieving data records. For more information, see Query object .
resources	The collection of resources types available to the script. For more information, see Resource collections .
responsibilities	The collection of responsibilities available to the script. For more information, see responsibilities collection .
scenarios	The collection of scenarios available to the script. For more information, see "scenarios collection" on page 142 .
scheduledTasks	The collection of scheduled tasks available to the script. For more information, see "scheduledTasks collection" on page 291 .
schedules	The collection of schedules available to the script. For more information, see "schedules collection" on page 311 .
scriptArguments	Defines the input arguments passed to a script. Each argument's name is used as an index in the array of arguments passed to the script. For example, <code>rapidResponse.scriptArguments["Argument"]</code> For more information, see "Using script argument values" on page 65 .

Property	Description
scripts	The collection of scripts available to the script. For more information, see "scripts collection" on page 129 .
	The collection of scorecards available to the script. For more information, see Scorecards collection .
server	Server operations that system administrators can perform. For more information, see "server property" on page 475 .
siteGroups	The collection of sites and site filters available to the script. For more information, see "siteGroups collection" on page 273 .
taskflows	The collection of taskflows available to the script. For more information, see taskFlows collection .
treemapQueries	The collection of treemap data available to the script. For more information, see Treemap data collections .
users	The collection of RapidResponse users. For more information, see "users collection" on page 455 .
widgets	The collection of widgets available to the script. For more information, see "widgets collection" on page 343 .
workbooks	The collection of workbooks available to the script. For more information, see "workbooks collection" on page 172 .
workflows	The collection of workflows available to the script. For more information, see "workflows collection" on page 389 .
worksheets	The collection of worksheets in workbooks that are available to the script. For more information, see worksheets collection .

The resources that can be defined as objects and used in a script are defined as one of the following types.

Type	Description
Resource	The generic resource object, which defines common properties and methods that other resource objects can use. For more information, see "Resource objects" on page 109.
Alert	Defines alert properties and provides methods for managing and running alerts. For more information, see "Alert objects" on page 281.

Type	Description
Automation chain	Defines automation chain properties and provides methods for managing and running automation chains. For more information, see " AutomationChain objects " on page 301.
Principal	Defines users and groups. For more information, see " Principal objects " on page 123.
Profile variable	Defines profile variables and provides methods for retrieving and setting values in variables. For more information, see " ProfileVariable objects " on page 325.
Scenario	Defines scenario properties and methods, including methods for creating, updating, committing, or deleting scenarios. For more information, see " Scenario objects " on page 139.
Scheduled task	Defines scheduled task properties and provides methods for managing and running scheduled tasks. For more information, see " ScheduledTask objects " on page 291.
Scorecard	Defines scorecard properties and provides methods for managing scorecards. For more information, see Scorecard objects .
Script	Defines script properties and methods for calling another script. For more information, see " Script objects " on page 129.
Workflow	Defines workflow methods for managing and running workflows. For more information, see " Workflow objects " on page 389.
Workbook	Defines workbook properties and data modification commands. Includes methods for retrieving data from a workbook and running commands. For more information, see " Workbook objects " on page 169.
Worksheet	Defines worksheet properties and provides methods for retrieving worksheet data and performing data modifications. For more information, see " Worksheet objects " on page 197.

CHAPTER 9: Resource objects

HelpInformation object	110
Resource collections	110
Collection iteration functions	112
Resource object methods	116
accessControlList collection	121
Principal objects	123
Permission object	126

All resource types have common properties and methods, which are defined by the abstract Resource object. Each resource is defined by an object that implements the Resource object. All resource objects are contained in resource collections, which provide additional methods for managing resources. For more information, see ["Resource collections" on page 110](#).

Resource objects contain the following properties.

Property	Type	Description
name	String	The resource's name.
scope	String	Whether the resource is Private or Public.
id	String	The resource's identifier. This value is used to identify the resource within its resource collection.
owner	User	The user who owns the resource.
creator	String	The user who created the resource. Note: The creator property of a Scenario object is always undefined.

Property	Type	Description
creationDate	Date	When the resource was created.
lastModifiedDate	Date	The last date the resource was modified.
lastUsedDate	Date	The last date the resource was used.
usageCount	Number	The number of times the resource has been accessed.
accessControlList	Object	A collection of users and groups who have access to the resource. For more information, see "accessControlList collection" on page 121 .

HelpInformation object

The HelpInformation object provides access to the RapidResponse Help. This object contains the following properties.

Property	Type	Description
customHelpEnabled	Boolean	Whether custom help has been defined for your RapidResponse system.
customHelpLabel	String	The label displayed for your company's custom help.
customHelpUrl	String	The address your company's custom help is available from.
helpUrl	String	The RapidResponse Help address.
LOCAL_URL	String	The HTTP URL base for the local RapidResponse help. Help links use this base to locate the help systems.
ONDEMAND_URL	String	The HTTP URL base for the online RapidResponse help. Help links use this base to locate the help systems.



Note: This object was added in RapidResponse 2013.4.

Resource collections

The resources available to the script are stored in collections, with each type of resource stored in its own collection. For example, the `workbooks` collection contains all workbooks available to the user running the script.

To use a resource in the script, it must be retrieved from the collection, typically by referring to its resource key, which is defined as an object with the following properties from the [resource object](#).

Property	Type	Description
name	String	The resource's name.
scope	String	Specifies if the resource is Public or Private.

Resources in collections can also be retrieved using an identifier, or by an array index. For more information, see ["Collection indexing" on page 111](#).

Collections also support the standard JavaScript iteration functions. For more information, see ["Collection iteration functions" on page 112](#).

The following methods are defined for resource collections. Each resource type can implement these methods differently, and can require different parameters or throw different exceptions. For example, the `workbooks` collection has an optional parameter defined for its `get` method. In addition, some resource types do not implement all collection methods, which means you cannot use that method for that resource type. For example, the `scripts` collection does not implement the `create` method, so you cannot create a script using a script.

Method	Description
<code>get(key)</code>	Retrieves the resource object with the specified resource key.
<code>create(name)</code>	Creates a new resource with the specified name.
<code>remove(name)</code>	Deletes the specified resource.
<code>exists(resource)</code>	Determines whether the specified resource exists.

Collection indexing

Resource collections are indexed by the `id` property of the objects within the collection. These values are unique within the collection, and are typically used internally by RapidResponse to identify the resource object. You can retrieve or refer to an object by its identifier, if it is known. For example, the following retrieves a collection object using its `id` property.

```
collectionObject = collection[id];
```

where *collection* is a resource collection.

The collection can also be converted to a JavaScript array using the `toArray()` function. This allows you to manage the resource collection using a numeric index. For example, the following converts a resource collection to an array.

```
collectionArray = collection.toArray();
```

where *collection* is a resource collection.

Only resources present in the collection at the time the array is created are included. For example, if a script converts a collection to an array and then creates a new resource in the collection, the new resource is not included in the array.

You can use the array to enumerate all objects in the collection, or to iterate over the collection. For example, you can retrieve all scenarios in the `scenarios` collection using the following code.

```
var scenariosArray = rapidResponse.scenarios.toArray();
for (var i = 0, n = scenariosArray.length; i < n; ++i) {
    var scenario = scenariosArray[i];
}
```

Collection iteration functions

Collections also support the standard JavaScript iteration functions `forEach()` and `forEachId()`, which perform a specified function on all resources in the collection.

For example, you can iterate over the worksheets in a workbook and output each worksheet's name using the following:

```
var partPropertiesWb = rapidResponse.workbooks.get({name: 'Part
Properties', scope: 'Public' });

partPropertiesWb.worksheets.forEach(function(ws) {

    rapidResponse.console.writeLine(ws.name);

});
```

You can use these functions to perform operations on items in a collection without specifying the exact name or identifier for each item. For example, if you expect the items in the collection to change over time, these functions allow you to create a script that performs its function no matter how the collection is modified.

get method

Retrieves a resource object, which allows you to manipulate the resource in your script. Retrieving a resource allows you to use any of the methods defined by the resource type.

► Syntax

```
resource = rapidResponse.resourceCollection.get({name:"name",
scope:"Public or Private"});
```

where *resourceCollection* is the collection to delete the resource from.

Parameter	Type	Description
name	String	The name of the resource.
scope	String	Whether the resource is Public or Private. For more information, see "Using resources as objects" on page 64 .

► Returns

Type	Description
Object	A Resource object.

► Exceptions

The `get` method can throw the following exceptions.

Exception	Description
<code>ArgumentError</code>	A blank string or <code>null</code> value was passed to the method.
<code>ResourceType.NotFound</code>	No resource with the specified name exists.

The `ResourceType` depends on the type of resource you are retrieving. For example, if you are retrieving a scenario, the `NotFound` exception is `Scenario.NotFound`.

► Example

To retrieve a scenario object:

```
var myScenario = rapidResponse.scenarios.get({ name:"myScenario",
scope:"Private" });
```

exists method

Determines whether a resource exists. However, because `RapidResponse` supports multiple users working with the same resources, a user might delete the specified resource after this method runs. If you are using this method to test whether a resource exists, you should still catch a `NotFound` exception for the resource type you are using.

► Syntax

```
resourceExists = resourceCollection.exists({name: "Name", scope:
"Public or Private"});
```

where `resourceCollection` is the collection to delete the resource from.

Parameter	Type	Description
name	String	The name of the resource to test for existence.
scope	String	Whether the resource is Private or Public.

► Returns

Type	Description
Boolean	• true if the resource exists • false if the resource does not exist

► Exceptions

The `exists` method can throw the following exceptions.

Exception	Description
ArgumentError	One of the following has occurred: • The resource parameter is missing a name or scope value. • The scope value specified is not "Private" or "Public".

► Example

To determine whether a workbook exists:

```
var workbookExists = rapidResponse.workbooks.exists({name: "Order
Analysis", scope: "Public"});
```

create method

Creates a resource in the specified resource collection.

► Syntax

```
newResource = resourceCollection.create(name);
```

where *resourceCollection* is the collection you are creating the new resource in.

Parameter	Type	Description
name	String	The name of the resource you are creating.

► Returns

Type	Description
Object	A Resource object.

► Exceptions

The `create` method can throw the following exceptions.

Exception	Description
<code>ArgumentError</code>	The specified resource name is blank or <code>undefined</code> .
<code>ResourceType.NoPermission</code>	You do not have permission to create the resource.
<code>ResourceType.NameValidationError</code>	Either a resource with the same name already exists or the specified name is not valid for the resource type.
<code>NotImplemented</code>	A <code>create</code> method is not implemented for the resource type.

The `ResourceType` depends on the type of resource you are creating, For example, if you are creating a scenario, the `NoPermission` exception is `Scenario.NoPermission`.

remove method

Deletes a resource from the server, which also removes it from its resource collection. Deleted resources are no longer available to any user or process, and you can only delete resources you own or have permission to manage.

► Syntax

```
rapidResponse.resourceCollection.remove(resourceName);
```

where `resourceCollection` is the collection to delete the resource from.

Parameter	Type	Description
<code>resourceName</code>	String	The name of the resource you are deleting.

► Returns

No return value.

► Exceptions

The `remove` method can throw the following exceptions.

Exception	Description
<i>ResourceType.NoPermission</i>	You do not have permission to delete the specified scenario.
<i>ResourceType.NotFound</i>	The specified scenario does not exist.
<i>NotImplemented</i>	A <code>remove</code> method is not implemented for the resource type.

The *ResourceType* depends on the type of resource you are deleting. For example, if you are deleting a scenario, the *NotFound* exception is *Scenario.NotFound*.

► Example

To delete a script:

```
var deleteScript = rapidResponse.scripts.get({name:"Obsolete_Manage
Scenarios", scope:"Public"});
rapidResponse.scripts.remove({name:"Obsolete_Manage Scenarios",
scope:"Public"});
```

Resource object methods

The Resource object defines the following methods, which can be implemented by other resource objects. Not all resources implement all of these methods. For example, scenario objects do not implement the `copy` method, so you cannot copy a scenario using a script.

Method	Description
<code>give</code> <code>(newOwner)</code>	Changes the owner of a resource.
<code>makePublic()</code>	Changes the scope of a resource to "Public", which is the equivalent of sharing the resource. Public resources can be accessed by administrators and users and groups with access.
<code>rename</code> <code>(newName)</code>	Changes the name of a resource.
<code>copy</code> <code>(newName)</code>	Makes a copy of a resource with the specified name.

give method

Gives a resource to another user by changing the resource's owner. You can give any resource you own.

If the resource is shared with other users, you can still access it after giving it. However, if the resource is private, you cannot access it after you give it.

► Syntax

```
resource.give(newOwner)
```

where *resource* is a resource object.

Parameter	Type	Description
newOwner	String	The user to give the resource to. Note: You cannot give a resource to a group.

► Returns

No return value.

► Exceptions

The `give` method can throw the following exceptions.

Exception	Description
<i>ResourceType</i> .NotFound	The specified resource does not exist.
Principal.NotFound	The specified user does not exist.
<i>ResourceType</i> .NonUniqueName	The specified user already owns a resource with the same name as the one you are giving.
<i>ResourceType</i> .NoPermission	The resource does not allow the give operation or you do not have permission to give the resource.
NotImplemented	A <code>give</code> method is not implemented for the resource type.

The *ResourceType* depends on the type of resource you are giving. For example, if you are giving a scenario, the NonUniqueName exception is `Scenario.NonUniqueName`.

► Example

To give a scenario:

```
var myScenario = rapidResponse.scenarios.get({ name:"myScenario",  
scope:"Private" });  
myScenario.give("myUser");
```

makePublic method

Changes the scope of the specified resource from "Private" to "Public". This method does not specify other users or groups to share the resource with, so it is available to only you and administrators.

If you call this method on a variable that represents a resource, you must ensure you assign the returned public resource to the same variable. Otherwise, the variable retains the scope it was created with.

You can also share the resource with specific users and groups using the [accessControlList collection's add method](#).

Administrators can access any shared resource, and you can allow other users and groups to access a shared resource.

Making a resource public also adds it and any dependent resources to the resource repository. The script that calls this method is included in the repository notes, and the version is automatically saved as 0.1. For more information about the repository, see the [RapidResponse Resource Authoring Guide](#).

► Syntax

```
publicResource = resource.makePublic()
```

where *resource* is a Resource object.

► Returns

A Resource object that represents the same resource with its scope changed to "Public".

► Exceptions

The `makePublic` method can throw the following exceptions

Exception	Description
<i>ResourceType.NotFound</i>	The specified resource does not exist.
<i>ResourceType.NonUniqueName</i>	A shared resource with the same name as the one you are sharing already exists.
<i>ResourceType.NoPermission</i>	You do not have permission to share the resource.
<i>NotImplemented</i>	A <code>makePublic</code> method is not implemented for the resource type.

The *ResourceType* depends on the type of resource you are sharing. For example, if you are sharing a scenario, the *NonUniqueName* exception is `Scenario.NonUniqueName`

► Example

To share a scenario:

```
var myScenario = rapidResponse.scenarios.get({ name:"myScenario",  
scope:"Private" });  
myScenario = myScenario.makePublic();
```

rename method

Changes the name of a resource. You can rename only private resources that you own.

► Syntax

```
resource.rename (newName)
```

where *resource* is a resource object.

Parameter	Type	Description
newName	String	The new name for the resource.

► Returns

No return value.

► Exceptions

The `rename` method can throw the following exceptions.

Exception	Description
<code>ArgumentError</code>	A blank, invalid string, string or <code>null</code> value was specified for the <code>newName</code> parameter.
<code>ResourceType.NotFound</code>	The specified resource does not exist.
<code>ResourceType.NoPermission</code>	The resource cannot be renamed.
<code>ResourceType.NameValidationError</code>	Either a resource with the same name already exists or the specified name is not valid for the resource type.
<code>ResourceType.RenamePublicError</code>	The resource is shared and cannot be renamed.
<code>NotImplemented</code>	A <code>rename</code> method is not implemented for the resource type.

The *ResourceType* depends on the type of resource you are renaming, For example, if you are renaming a scenario, the `NonUniqueName` exception is `Scenario.NonUniqueName`

► Example

To rename a scenario:

```
var myScenario = rapidResponse.scenarios.get({ name:"myScenario",  
scope:"Private" });  
myScenario.rename("myOtherScenario");
```

copy method

Creates a new private resource by copying an existing resource. If you copy a shared resource, the copy is private.

If the resource you are copying has dependencies, the new resource has the same dependencies.

► Syntax

```
resource.copy(newName)
```

where *resource* is a resource object.

Parameter	Type	Description
<i>newName</i>	String	The name for the new resource.

► Returns

No return value.

► Exceptions

The `copy` method can throw the following exceptions.

Exception	Description
<code>ArgumentError</code>	A blank string, invalid string, or <code>null</code> value was specified for the <code>newName</code> parameter.
<code>ResourceType.NotFound</code>	The specified resource does not exist.
<code>ResourceType.NameValidationError</code>	Either a resource with the same name already exists or the specified name is not valid for the resource type.
<code>ResourceType.NoPermission</code>	The resource cannot be copied.
<code>NotImplemented</code>	A <code>copy</code> method is not implemented for the resource type.

The `ResourceType` depends on the type of resource you are copying, For example, if you are copying a script, the `NonUniqueName` exception is `Script.NonUniqueName`

► Example

To copy a script:

```
var myScript = rapidResponse.scripts.get({ name:"myScript",  
scope:"Private" });  
myScript.copy("myOtherScript");
```

accessControlList collection

Access to a resource can be granted or denied by adding or removing users from the accessControlList collection for that resource object. Each resource object has an accessControlList collection, which contains objects with the following properties.

Property	Type	Description
principal	Principal	The user or group with permission to access the resource.
level	String	How the user or group can access the resource. This reports only how the resource was directly shared with the user or group, and does not include any resources assigned through group memberships. For more information about groups, see "GroupCollectionObject object" on page 470 This property can contain one of the following values: • Full: The user or group has administrative permissions for the resource. • Modify: The user or group can modify data in the resource. • View: The user or group can only view data in the resource.

The accessControlList collection defines the following methods, which are available to all resource types unless indicated otherwise.

Method	Description
add(principal, level)	Shares the resource with the specified user or group with the specified access level by adding that user or group to the collection.
remove(principal)	Remove access for the specified user or group by removing that user or group from the collection.

remove method

Removes resource access from the specified user or group. If you remove access from all users or groups, the resource remains shared and can be accessed by administrators.

► Syntax

```
resource.accessControlList.remove(principal);
```

where *resource* is a resource object.

Parameter	Type	Description
principal	String	The user or group to remove resource access from.

► Returns

No return value.

► Exceptions

The `remove` method can throw the following exceptions.

Exception	Description
<code>ResourceType.NoPermission</code>	You do not have permission to modify the access control.
<code>Principal.NotFound</code>	The specified user or group does not exist.

The `ResourceType` depends on the type of resource you are removing access from, For example, if you are removing access to a scenario, the `NoPermission` exception is `Scenario.NoPermission`.

► Example

To remove access from a scenario:

```
var myScenario = rapidResponse.scenarios.get({ name:"myScenario",
scope:"Public" });
myScenario.accessControlList.remove("myUser");
```

add method

Shares a resource or scenario by adding the specified user or group to the list of users and groups that have access to the resource. Sharing a resource using this method makes the resource public (or, if it's a scenario, shared), so you can call this method instead of calling the `makePublic()` method. For more information, see "[makePublic method](#)" on page 117.

► Syntax

```
resource.accessControlList.add(principal, level);
```

where *resource* is a resource object.

Parameter	Type	Description
<code>principal</code>	String	The user or group to share the resource or scenario with.
<code>level</code>	String	Determines how the user or group can access the resource or scenario. Can be "View" or "Modify". This value is required for all resource types, but the permission level is used only for scenarios.

► Returns

No return value.

► Exceptions

The `add` method can throw the following exceptions.

Exception	Description
<code>ArgumentError</code>	An invalid or null value was specified for a parameter.
<code>AccessControl.InvalidPermissionLevel</code>	A value other than "View" or "Modify" was specified for the level parameter.
<code>Principal.NotFound</code>	The specified user or group does not exist.
<code>ResourceType.NonUniqueName</code>	A shared resource or scenario with the same name as the one you are sharing already exists.
<code>ResourceType.NoPermission</code>	You do not have permission to share the resource.

The `ResourceType` depends on the type of resource you are sharing. If you are sharing a scenario, the `NonUniqueName` exception is `Scenario.NonUniqueName`.

► Example

To share a scenario:

```
var myScenario = rapidResponse.scenarios.get({ name:"myScenario",
scope:"Private" });
myScenario.accessControlList.add("myUser", "Modify");
myScenario.accessControlList.add("mySecondUser", "View");
```

Principal objects

Principal objects represent users and groups. The Principal object has the following properties.

Property	Type	Description
<code>isUser</code>	Boolean	Determines whether the principal is a user. For more information, see "User objects" on page 453 .
<code>isGroup</code>	Boolean	Determines whether the principal is a group. For more information, see "Group object" on page 467 .
<code>displayName</code>	String	The name displayed for the user or group.
<code>permissions</code>	Object	A collection of Permission objects for the user or group. This property was added in RapidResponse 2014.4.

The `permissions` property refers to the `PermissionCollection` object, which contains the following methods.

Method	Description
allow(permissionId)	Adds a permission to a user or group.
deny(permissionId)	Removes a permission from a user or group.
hasPermission(permissionId)	Determines if a user or user group has the specified permission.

Users and groups are also defined by User objects and Group objects, and are contained in collections. For more information, see "[User objects](#)" on page 453 and "[Group object](#)" on page 467.

allow method

The allow method adds a permission to a user or group.

If the principal inherits the permission or if an ancestor group denies the permission, the permission is not added. If the principle is denied the permission directly, the denial is removed and the permission is added.

► Syntax

```
principal.permissions.allow(permissionId);
```

Parameter	Type	Description
permissionId	String	The ID of the permission being added. Click here for a list of permission IDs.

► Returns

Type	Description
Object	A PermissionCollection object.

No return value.

► Exceptions

Exception	Description
Permission.NotFound	The permission ID does not exist.
Permission.NoPermission	You do not have permission to add permissions.
Permission.NotAllowed	The specified permission cannot be granted to the user.

► Examples

This example allows the user with the user ID "jsmith" to author forms.

```
rapidResponse.users["jsmith"].permissions.allow("AuthorForms");
```

This example allows the user with the user ID "jsmith" to author and share forms.

```
rapidResponse.users["jsmith"].permissions.allow(["AuthorForms",  
"ShareForms"]);
```

This example allows the Buyers group to author workbooks.

```
rapidResponse.groups["Buyers"].permissions.allow("AuthorWorkbooks");
```



Notes:

- You can use this method to assign data administration (DataAdministration) or user administration (UserAdministration) permissions only if you are a system administrator.
- This method was added in RapidResponse 2014.4.

deny method

The deny method removes a permission from a user or group.

The permission is removed if the principal has the permission directly or through group membership. If the principal does not have the permission already, no change is made.

► Syntax

```
principal.permissions.deny(permissionId);
```

Parameter	Type	Description
permissionId	String	The ID of the permission being removed. Click here for a list of permission IDs.

► Returns

Type	Description
Object	A PermissionCollection object.

► Exceptions

Exception	Description
Permission.NotFound	The permission ID does not exist.
Permission.NoPermission	You do not have permission to deny the permission.

► Example

This example denies the user with the user ID "jsmith" the author forms permission.

```
rapidResponse.users["jsmith"].permissions.deny("AuthorForms");
```



Note: This method was added in RapidResponse 2014.4.

hasPermission method

The `hasPermission` method determines if a user or group has the specified permission.

► Syntax

```
principal.permissions.hasPermission(permissionId);
```

Parameter	Type	Description
permissionId	String	The ID of the permission. Click here for a list of permission IDs.

► Returns

Type	Description
Boolean	• true if the user or group has the permission • false if the user or group does not have the permission

► Exceptions

Exception	Description
Permission.NotFound	The permission ID does not exist.

► Example

This example determines if the Buyers group has permission to author workbooks.

```
rapidResponse.groups["Buyers"].permissions.hasPermission("AuthorWorkbooks");
```



Note: This method was added in RapidResponse 2014.4.

Permission object

The Permission object defines whether a user or user group has a permission, is denied a permission, or has inherited a permission. The Permission object has the following properties.

Property	Type	Description
id	String	The permission's identifier. Click here to view the list of valid permission IDs.
hasPermission	Boolean	Whether the user or group has the specified permission.
isDenied	Boolean	Whether the user or group is denied the permission.
isInherited	Boolean	Whether the user or group has inherited the permission or the permission denial.

The following is a list of all permission IDs that can be used with the allow, deny, and hasPermission methods.

- AuthorScenarios
- ShareScenarios
- ShareVisibleScenarios
- ManageAllScenarios
- AuthorAllFilters
- AuthorStaticOnlyFilters
- ShareFilters
- AuthorAlerts
- ShareAlerts
- AuthorWorkbooks
- AuthorLibraryWorkbooks
- RetrievePredefinedResources
- ViewWorkbookProperties
- OverrideRecordLimit
- EditInSharedScenarios
- WarnBeforeSharedScenarioEdit
- PasswordNeverExpires
- PasswordChangeDisabled
- SystemAdministration
- UserAdministration
- DataAdministration
- CanManageIntegrations
- ShareWidgets
- AuthorProcessTemplates
- ShareProcessTemplates
- AuthorForms
- ShareForms
- AuthorResponsibilityRoles
- ShareResponsibilityRole
- CanImportIntoWorkbook
- CanImportIntoScenario
- AuthorDataModification
- AuthorScheduledTasks (scheduleAutomation)
- ShareScheduledTasks
- AuthorCompositeTasks
- ShareCompositeTasks
- AccessEventLog
- ViewDataModel
- IsStartPageAccessible
- ManageS&OPProcess
- ManageS&OPSubProcess
- AccessBusinessMessageLog
- ShareWorkbooks
- AuthorScorecards
- ShareScorecards
- AuthorTaskFlows
- ShareTaskFlows
- AuthorHierarchies
- ShareHierarchies
- ShareAddIns
- ConfigureInsertDefinitions
- ModifyMacrosAndProfileVariables
- EditMetricTargetValues
- MessageCenter
- SendLinks
- UseWebServices
- AuthorScripts
- ShareScripts
- AuthorDashboards
- ShareDashboards
- AuthorWidgets



Notes:

- You must be a system administrator to assign any of the administration permissions to a user or group.
- This object was added in RapidResponse 2014.4.

CHAPTER 10: Script objects

scripts collection	129
Script object methods	132

Script objects allow you to use methods from other scripts or call other scripts. Script objects available to the script are contained in the `scripts` collection. For more information, see "[scripts collection](#)" on page 129.

Script objects contain the same [properties as Resource objects](#).

For more information about using functions from other scripts, see "[Call a function from an included script](#)" on page 78.

scripts collection

The `scripts` collection defines all scripts available to the user running the script. Any script retrieved or called by the script must be present in this collection.

The `scripts` collection implements the following [resource collection](#) methods.

Method	Description
get(key)	Retrieves an instance of a script object.
remove(script)	Deletes the specified script.
exists(script)	Determines whether a script with the specified name exists in the collection.

get method

Retrieves a script object, which allows you to manipulate the script in your script. Retrieving a script allows you to use any of the script methods.

► Syntax

```
aScript = rapidResponse.scripts.get({name:"name", scope:"Public or Private"});
```

Parameter	Type	Description
name	String	The name of the script.
scope	String	Whether the script is Public or Private. For more information, see "Using resources as objects" on page 64.

► Returns

Type	Description
Object	A script object

► Exceptions

The `get` method can throw the following exceptions.

Exception	Description
ArgumentError	A blank string or <code>null</code> value was passed to the method.
Script.NotFound	No script with the specified name exists.

► Example

```
var myScript = rapidResponse.scripts.get({ name:"myScript",  
scope:"Private" });
```

exists method

Determines whether a script exists. However, because RapidResponse supports multiple users working with the same resources, a user might delete the specified script after this method runs. If you are using this method to test whether a script exists, you should still catch a `NotFound` exception for the script you are using.

► Syntax

```
scriptExists = rapidResponse.scripts.exists({name: "Name", scope: "Public or Private"});
```

Parameter	Type	Description
name	String	The name of the script to test for existence.
scope	String	Whether the script is Private or Public.

► Returns

Type	Description
Boolean	• true if the script exists • false if the script does not exist

► Exceptions

The `exists` method can throw the following exceptions.

Exception	Description
ArgumentError	One of the following has occurred: • The resource parameter is missing a name or scope value. • The scope value specified is not "Private" or "Public".

► Example

```
var scriptExists = rapidResponse.scripts.exists({name: "Order Analysis", scope: "Public"});
```

remove method

Deletes a script from the server, which also removes it from the `scripts` collection. Deleted scripts are no longer available to any user or process, and you can only delete scripts you own or have permission to manage.

► Syntax

```
rapidResponse.scripts.remove( {name: 'scriptName', scope: 'Public or Private'} );
```

Parameter	Type	Description
scriptName	Object	The script you are deleting. This can be specified as either a Script object or by using the {name, scope} syntax.

► Returns

No return value.

► Exceptions

The `remove` method can throw the following exceptions.

Exception	Description
Script.NoPermission	You do not have permission to delete the specified script.
Script.NotFound	The specified script does not exist.

► Example

```
rapidResponse.scripts.remove({name:"Obsolete_Manage Scenarios",
scope:"Public"});
```

Script object methods

Script objects implement the following [resource object](#) methods, as well as the [access control](#) methods.

Method	Description
give (newOwner)	Changes the owner of a script.
makePublic()	Changes the scope of a script to "Public", which is the equivalent of sharing the script. Public scripts can be accessed by administrators and users or groups who have been given access to the script. You can also share a script with a particular user or group using the accessControlList collection's add method .
rename (newName)	Changes the name of a script.
copy (newName)	Makes a copy of a script with the specified name.

Script objects have the following method.

Method	Description
call(args)	Runs a script with the specified argument values.

give method

Gives a script to another user by changing the script's owner. You can give any script you own.

If the script is shared with other users, you can still access it after giving it. However, if the script is private, you cannot access it after you give it.

► Syntax

```
script.give(newOwner);
```

where *script* is a script object.

Parameter	Type	Description
newOwner	String	The user to give the script to. Note: You cannot give a script to a group.

► Returns

No return value.

► Exceptions

The `give` method can throw the following exceptions.

Exception	Description
Script.NotFound	The specified script does not exist.
Principal.NotFound	The specified user does not exist.
Script.NonUniqueName	The specified user already owns a script with the same name as the one you are giving.
Script.NoPermission	You do not have permission to give the script.

► Example

```
var myScript = rapidResponse.scripts.get({ name:"myScript",
scope:"Private" });
myScript.give("myUser");
```

makePublic method

Changes the scope of the specified script from "Private" to "Public", which shares the script and allows other users to access it. This method does not specify other users or groups to share the script with, so it is available to only you and administrators.

If you call this method on a variable that represents a script, you must ensure you assign the returned public script to the same variable. Otherwise, the variable retains the scope it was created with.

You can also share the script with specific users and groups using the [accessControlList collection's add method](#).

Administrators can access any shared script, and you can allow other users and groups to access a shared script.

► Syntax

```
publicScript = script.makePublic();
```

where *script* is a script object.

► Returns

A Script object that represents the same script with its scope changed to "Public".

► Exceptions

The `makePublic` method can throw the following exceptions

Exception	Description
Script.NotFound	The specified script does not exist.
Script.NonUniqueName	A shared script with the same name as the one you are sharing already exists.
Script.NoPermission	You do not have permission to share the script.

► Example

```
var myScript = rapidResponse.scripts.get({ name:"myScript",  
scope:"Private" });  
myScript = myScript.makePublic();
```

rename method

Changes the name of a script. You can rename only private scripts that you own.

► Syntax

```
script.rename(newName);
```

where *script* is a script object.

Parameter	Type	Description
newName	String	The new name for the script.

► Returns

No return value.

► Exceptions

The `rename` method can throw the following exceptions.

Exception	Description
ArgumentError	A blank string, invalid string, or <code>null</code> value was specified for the <code>newName</code> parameter.
Script.NotFound	The specified script does not exist.
Script.NameValidationError	Either a script with the same name already exists or the specified name is not valid.
Script.NoPermission	The script cannot be renamed or you do not have permission to rename it.
Script.RenamePublicError	The script is shared and cannot be renamed.

► Example

```
var myScript = rapidResponse.scripts.get({ name:"myScript",
scope:"Private" });
myScript.rename("myOtherScript");
```

copy method

Creates a new private script by copying an existing script. If you copy a shared script, the copy is private.

If the script you are copying has dependencies, the new script has the same dependencies.

► Syntax

```
script.copy(newName);
```

where *script* is a script object.

Parameter	Type	Description
newName	String	The name for the new resource.

► Returns

No return value.

► Exceptions

The `copy` method can throw the following exceptions.

Exception	Description
<code>ArgumentError</code>	A blank string, invalid string, or <code>null</code> value was specified for the <code>newName</code> parameter.
<code>Script.NotFound</code>	The specified script does not exist.
<code>Script.NameValidationError</code>	Either a script with the same name already exists or the specified name is not valid.
<code>Script.NoPermission</code>	The script cannot be copied or you do not have permission to copy it.

► Example

```
var myScript = rapidResponse.scripts.get({ name:"myScript",  
scope:"Private" });  
myScript.copy("myOtherScript");
```

call method

Calls a script, which runs the script and, if applicable, returns a value that you can use in your script. If the called script contains unhandled errors, your script stops running and the line number where the error occurred is reported in the output console.

► Syntax

```
scriptObject.call(args);
```

Parameter	Type	Description
<code>args</code>	Object	An optional set of arguments to be passed to the script being called. Each argument must be specified using the <code>name: value</code> syntax. Any argument that is not specified uses the default value defined in the script. For more information about arguments, see "Define arguments for a script" on page 68 .

► Returns

The return value of the called script.

► Exceptions

The `call` method can throw the following exceptions.

Exception	Description
ArgumentError	A specified argument does not exist in the script being called, or a value provided for an argument is the wrong data type.
Script.IncludeDependency.IsSelf	An included script includes this script.
Script.IncludeDependency.NotFound	An included dependent script does not exist or you do not have access to it.
Script.Dependency.NotFound	A dependency required by the called script does not exist or you do not have access to it.
Script.CompilationError	The called script or one of its dependencies contains syntax errors.
Script.CallDepthExceeded	More than 10 nested script calls are made by calling the script.

► Example

```
var scriptOutput = myScript.call({ quantity: 20 });
```


CHAPTER 11: Scenario objects

ActivityLog objects	141
scenarios collection	142
createWorkflowScenario method	149
Obtaining scenario sequence numbers	150
Scenario object methods	153
cdcInstances collection	164
CDCInstance objects	165

Scenario objects allow you to compare data or store historical data. Scenario objects are contained in the `scenarios` collection. For more information, see ["scenarios collection" on page 142](#).

Scenario objects have the same [properties as Resource objects](#), as well as the following properties.

Property	Type	Description
accessPermission	String	How the scenario can be accessed. This value can be one of the following: • View: You can view data in the scenario. • Modify: You can view or modify data in the scenario. • Full: You own the scenario, and can view or modify data, and modify the scenario's properties.
activityLog	Object	Used to record activities performed on the scenario. For more information, see "ActivityLog objects" on page 141 . This property was added in RapidResponse 2014.1.

Property	Type	Description
cdcInstances	Object	The Change Data Capture (CDC) instances that monitor this scenario for data changes. This is a collection of CDCInstance objects, and can be empty if the scenario is not monitored. For more information, see "CDCInstance objects" on page 165. This property was added in RapidResponse 2014.1.
childScenarios	Object	Scenarios that are children of this scenario. This property is a collection of Scenario objects, and can be empty if the scenario has no children. Child scenarios can be accessed using the scenario collection methods described in "scenarios collection" on page 142.
isAutoUpdate	Boolean	Whether the scenario automatically updates when its parent is modified. This property can be modified using the setProperties method.
isPermanent	Boolean	Whether the scenario is permanent. This property can be modified using the setProperties method.
isUpToDate	Boolean	Whether the scenario is up to date or needs to be updated.
lastUpdatedDate	Date	When the scenario was last updated. If the scenario has never been updated (has a null date), this property returns "Thu Jan 01 1970 00:00:00 GMT-0500 (Eastern Standard Time)".
lastCommittedDate	Date	When changes in the scenario were last committed to its parent.
parentScenario	Object	The scenario's parent.
perspective	Object	The perspective applied to the scenario.
purpose	String	The scenario's purpose. This property can be modified using the setProperties method.
scopedName	String	The scope and name of the scenario. The scope identifies if the scenario is shared or private.
status	String	The scenario's status. This property can contain one of the following values: • InProgress: The scenario's course of action is ongoing. • Suspended: The scenario's course of action is paused, and might be resumed later. • Canceled: The scenario's course of action has been canceled. • Completed: The scenario's course of action has been completed.
suppressEditWarning	Boolean	Whether to suppress or display a warning message to users when they edit a shared scenario.

Property	Type	Description
type	String	The scenario's type. This property can contain one of the following values: - Normal: A simulation scenario, which is visible in the Scenarios pane. These scenarios are managed by users. Most scenarios defined in your system are normal scenarios. - Temporary: A scenario created as part of a process. These scenarios are removed when the process completes and are not visible in the Scenarios pane. Temporary scenarios are always private and cannot be shared. - ChangeDataCapture: A CDC instance, which monitors a scenario for changes. These scenarios are used only for change data capture, and cannot be managed or modified using the scenario collection or resource object methods. For more information, see the RapidResponse Web Services Guide. - ProcessOrchestration: A process orchestration scenario, which is used to store process calendar and activity changes within a RapidResponse process instance. These scenarios are not visible in the Scenarios pane. For more information about processes, see the RapidResponse Resource Authoring Guide.
usersWithOpenQueries	Object	The users who have open queries on the scenario. This property contains a user collection. For more information, see " users collection " on page 455.

The Perspective object has the following properties.

Property	Type	Description
name	String	The name of the perspective applied to the scenario.
isInherited	Boolean	Whether the scenario inherits the perspective from its parent.

ActivityLog objects

The ActivityLog object that is associated with a scenario is used to record the activity performed on the scenario. It contains the following properties.

Property	Type	Description
subject	String	The title of the activity log entry.
type	String	The type of activity being logged.
body	String	Description of the changes performed in the activity.
notifyAll	Boolean	Whether all people with access to the scenario are notified.

The ActivityLog object contains the following method.

Method	Description
addResponse(message)	Adds a message to the scenario's activity log.



Note: This object was added in RapidResponse 2014.1.

scenarios collection

The `scenarios` collection contains all scenarios available to the user running the script. Any scenario retrieved or used by the script must be present in this collection.

The `scenarios` collection has the following properties:

Property	Type	Description
<code>processOrchestrationScenario</code>	Object	The scenario that is used for process orchestration activities.
<code>systemRootScenario</code>	Object	The root scenario, typically named Enterprise Data. If you do not have access to this scenario, this value is <code>undefined</code> .

The `scenarios` collection implements the following [resource collection](#) methods.

Method	Description
create(name, parent, options)	Creates a scenario with the specified name and creation options.
remove(name)	Deletes the specified scenario.
get(key)	Retrieves a scenario object.
exists(scenario)	Determines whether the specified scenario exists.

The `scenarios` collection also implements the following method.

Method	Description
createTemporary(parent)	Creates a temporary scenario as a child of the specified parent. The temporary scenario's name is automatically generated.

get method

Retrieves a scenario object, which allows you to manipulate the scenario in your script. Retrieving a scenario allows you to use any of the scenario methods.

► Syntax

```
aScenario = rapidResponse.scenarios.get({name:"name", scope:"Public or Private"});
```

Parameter	Type	Description
name	String	The name of the scenario.
scope	String	Whether the scenario is Public or Private. For more information, see "Using resources as objects" on page 64 .



Note: The term "public" is used for shared scenarios in scripting because, in much earlier versions of RapidResponse, scenarios were referred to as private and public, rather than private and shared. "Public" is retained in reference to scenario scope to avoid breaking users' scripts.

This method can also be called using the following syntax:

```
aScenario = rapidResponse.scenarios.get({SequenceNumber});
```

where SequenceNumber is the unique number that identifies the scenario. Each scenario has a sequence number, which is unique across your RapidResponse environment and increments every time a scenario is created. For more information about scenario sequence numbers, see ["Obtaining scenario sequence numbers" on page 150](#).

► Returns

Type	Description
Object	A scenario object

► Exceptions

The `get` method can throw the following exceptions.

Exception	Description
ArgumentError	A blank string or <code>null</code> value was passed to the method.
Scenario.NotFound	No scenario with the specified name exists.

► Example

```
var myScenario = rapidResponse.scenarios.get({ name:"myScenario",  
scope:"Private" });
```

exists method

Determines whether a scenario exists. However, because RapidResponse supports multiple users working with the same resources, a user might delete the specified scenario after this method runs. If you are using this method to test whether a scenario exists, you should still catch a `NotFound` exception.

► Syntax

```
scenarioExists = rapidResponse.scenarios.exists({name: "Name", scope: "Public or Private"});
```

Parameter	Type	Description
name	String	The name of the scenario to test for existence.
scope	String	Whether the scenario is Private or Public.



Note: The term "public" is used for shared scenarios in scripting because, in much earlier versions of RapidResponse, scenarios were referred to as private and public, rather than private and shared. "Public" is retained in reference to scenario scope to avoid breaking users' scripts.

This method can also be called using the following syntax:

```
scenarioExists = rapidResponse.scenarios.exists({SequenceNumber});
```

where `SequenceNumber` is the unique number that identifies the scenario. Each scenario has a sequence number, which is unique across your RapidResponse environment and increments every time a scenario is created. For more information about scenario sequence numbers, see ["Obtaining scenario sequence numbers" on page 150](#).

► Returns

Type	Description
Boolean	• true if the scenario exists • false if the scenario does not exist

► Exceptions

The `exists` method can throw the following exceptions.

Exception	Description
ArgumentError	One of the following has occurred: - The resource parameter is missing a name or scope value. - The scope value specified is not "Private" or "Public".

► Example

```
var scenarioExists = rapidResponse.scenarios.exists({name:
"myScenario", scope: "Private"});
```

create method

This method is used to create a scenario. You must specify the scenario this scenario is based on.

By default, this creates a private scenario. However, some of the options you can specify for the new scenario are not applicable to a private scenario. If you use these settings, the scenario is automatically created as a shared scenario.

► Syntax

```
rapidResponse.scenarios.create (name, parentScenario,
{creationOptions});
```

Parameter	Type	Description
name	String	The name of the scenario you are creating.
parentScenario	Object	A Scenario object that represents the new scenario's parent.
creationOptions	Object	Optional. Additional options for creating the scenario. This parameter can contain the following attributes: - autoUpdate: Boolean. Whether the scenario updates automatically when its parent is modified. If this property is set to <code>true</code>, the scenario is automatically shared. - perspective: String. The perspective applied to the scenario. - permanent: Boolean. Whether the scenario is permanent. If this property is set to <code>true</code>, the scenario is automatically shared. - purpose: String. The scenario's purpose. The autoUpdate, permanent, and purpose options can also be modified using a script. For more information, see "setProperty method" on page 163 .

The parentScenario can also be identified using its sequence number, as described in ["get method" on page 142](#).

► Returns

No return value.

► Exceptions

The `create` method can throw the following exceptions.

Exception	Description
<code>ArgumentError</code>	The specified scenario name is not valid, or no parent scenario is specified.
<code>Scenario.NoPermission</code>	You do not have permission to create scenarios.
<code>Scenario.NameValidationError</code>	Either a scenario with the same name already exists or the specified name is not valid.
<code>Scenario.NotFound</code>	The specified parent scenario does not exist.
<code>Perspective.NotFound</code>	The specified perspectiveName does not exist.
<code>Scenario.ReservedName</code>	The specified newScenarioName is a reserved scenario.
<code>Scenario.TooManyScenarios</code>	The limit of scenarios has been reached, and would be exceeded if this scenario is created.

► Example

This example creates a private scenario based on the Enterprise Data scenario.

```
var parent = { name:"Enterprise Data", scope:"Public" };
rapidResponse.scenarios.create("myScenario", parent);
```

This example creates a scenario that is updated automatically and contains a purpose. Because the automatic update setting can apply only to a shared scenario, the new scenario is shared.

```
var parent = { name:"Enterprise Data", scope:"Public" };
rapidResponse.scenarios.create("myScenario", parent, {purpose: " A new
scenario for simulation", autoUpdate: true});
```

This example creates a private scenario that has the Capable perspective applied.

```
var parent = { name:"Enterprise Data", scope:"Public" };
rapidResponse.scenarios.create("myScenario", parent, {perspective:
"Capable"});
```

This example creates a shared, permanent scenario and shares it with the Buyers group.

```
var parent = { name:"Enterprise Data", scope:"Public" };
```

```
var newScenario = rapidResponse.scenarios.create("myScenario", parent,
{permanent:true});

newScenario.accessControlList.add("Buyers");
```

remove method

Deletes a scenario. You can delete only scenarios that you own, and you must own all the scenario's children. In addition, you cannot delete a scenario if it is in use, such as in a workbook.

The remove method uses an optional Options object, which defines properties that determine how the scenario is handled during the remove operation. The Options object contains the following properties.

Property	Type	Description
failIfScenarioHasChildren	Boolean	Whether the remove fails if the scenario being removed has children. Default value: false
failIfScenarioHasOpenQueries	Boolean	Whether the remove fails if there are queries running on the scenario. Default value: true

► Syntax

```
rapidResponse.scenarios.remove(scenario, [options])
```

Parameter	Type	Description
scenario	Object	The scenario that will be deleted. This can be specified as either an instance of a scenario obtained from calling the get() method or a scenario object using the syntax {name, scope}.
options	Object	An optional Options object that defines how the scenario removal is handled.

This method can also be called using the following syntax:

```
rapidResponse.scenarios.remove({SequenceNumber})
```

where SequenceNumber is the unique number that identifies the scenario. Each scenario has a sequence number, which is unique across your RapidResponse environment and increments every time a scenario is created. For more information about scenario sequence numbers, see ["Obtaining scenario sequence numbers" on page 150](#).

► Returns

No return value.

► Exceptions

The remove method can throw the following exceptions.

Exception	Description
Scenario.NoPermission	Either you do not have permission to delete the specified scenario or the scenario is a change data capture scenario and cannot be removed.
Scenario.NotFound	The specified scenario does not exist.
Scenario.HasOpenQueries	The scenario has queries running on it, or is being used in a workbook or scorecard.
Scenario.HasChildren	The scenario is the parent of one or more child scenarios.
Scenario.Delete.NotOwnerOfAllChildren	You do not own all of this scenario's children.

► Example

This example removes a scenario.

```
rapidResponse.scenarios.remove({ name:"myScenario", scope:"Private"
});
```

This example removes a scenario, but fails if the scenario has children.

```
rapidResponse.scenarios.remove({ name:"myScenario", scope:"Private" },
{failIfScenarioHasChildren: true});
```

This example removes a scenario, even if queries are running on it, but fails if the scenario has children.

```
rapidResponse.scenarios.remove({ name:"myScenario", scope:"Private" },
{
    failIfScenarioHasOpenQueries: false,
    failIfScenarioHasChildren: true
});
```



Note: This method was modified in RapidResponse 2014.1.

createTemporary method

Creates a temporary scenario. Temporary scenarios are created with an automatically-generated name, and cannot be shared.

► Syntax

```
rapidResponse.scenarios.createTemporary(parent);
```

Parameter	Type	Description
parent	Object	The temporary scenario's parent. This can also be identified using its sequence number, as described in "get method" on page 142 .

► Returns

A Scenario object with its `type` property set to `Temporary`.

► Exceptions

The `createTemporary` method can throw the following exceptions.

Exception	Description
<code>ArgumentError</code>	The specified scenario name is not valid, or no parent scenario is specified.
<code>Scenario.NoPermission</code>	You do not have permission to create scenarios.
<code>Scenario.NotFound</code>	The specified parent scenario does not exist.
<code>Scenario.TooManyScenarios</code>	The limit of scenarios has been reached, and would be exceeded if this scenario is created.

► Example

```
var scenario = rapidResponse.scenarios.createTemporary({
  name: "Enterprise Data", scope: "Public"});
```

createWorkflowScenario method

This method is used to create a scenario within a workflow. It gets an scenario, for example the Process Orchestration scenario, and using the workflow execution ID it runs a script to create a new child scenario.

► Syntax

```
rapidResponse.scenarios.createWorkflowScenario (workflowExecutionId,
  parentScenario [, perspective]);
```

Parameter	Type	Description
workflowExecutionId	String	The unique ID for a workflow instance.
parentScenario	String	The parent scenario of the scenario created by this method.
perspective	String	The perspective applied to the scenario. This parameter is optional.



Note: This method can create an unlimited number of scenarios using the same `workflowExecutionId`, however only a maximum of 20 scenarios can exist in a workflow. It is recommended that the workflow manage these scenarios by committing and/or deleting them accordingly to keep within the 20 scenario limit.

► Returns

A new child scenario of the specified parent scenario. This child scenario is private in scope and hidden.

► Exceptions

The `createWorkflowScenario` method can throw the following exceptions.

Exception	Description
<code>ArgumentError</code>	The specified workflow instance identifier is not valid, or the parent scenario is not specified.
<code>Scenario.NoPermission</code>	You do not have permission to create scenarios.
<code>Scenario.NotFound</code>	The specified parent scenario does not exist.
<code>Scenario.TooManyScenarios</code>	The limit of 20 scenarios has been reached in the workflow, and would be exceeded if this scenario is created.

► Example

```
var workingScenario = rapidResponse.scenarios.createWorkflowScenario
(workflowExecutionId, parentScenario);
return {
    workingScenario : workingScenario.scopedName,
}
```

Obtaining scenario sequence numbers

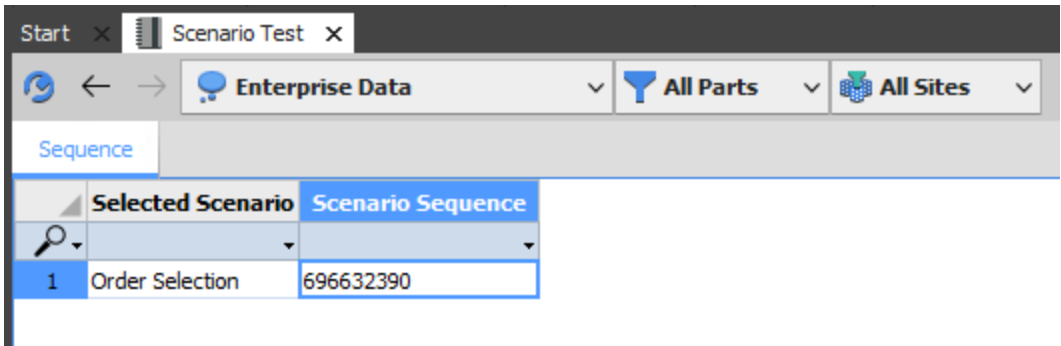
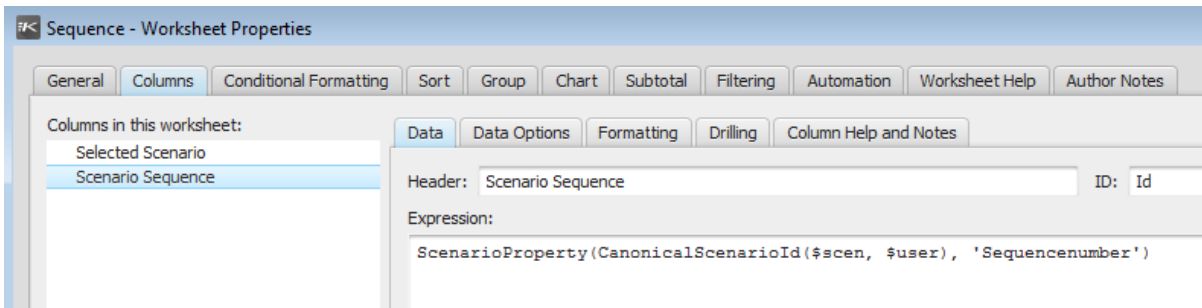
A scenario's sequence number can be used in place of its name and scope or its identifier for identifying scenarios in methods. Each scenario has a sequence number, which is a numerical identifier that is unique across your RapidResponse environment. The sequence is incremented each time a scenario is created, and, if you are an administrator, allows you to manage scenarios you do not own. For example, you can automate a process that updates or deletes scenarios that belong to other users.

The sequence number is associated with the scenario, but is not part of the `Scenario` object. Therefore, if you want to use sequence numbers, you must read the value from a worksheet that has been configured to display the sequence number for a specified scenario.

The RapidResponse query language includes functions to access scenario properties that are not otherwise exposed. The worksheet must use these functions to return the sequence number:

- CanonicalScenarioId: Takes the scenario and its owner, and returns an identifier for the scenario.
- ScenarioProperty: Takes the canonical scenario identifier and returns a String value that can report the scenario's name, type, owner, or sequence number.

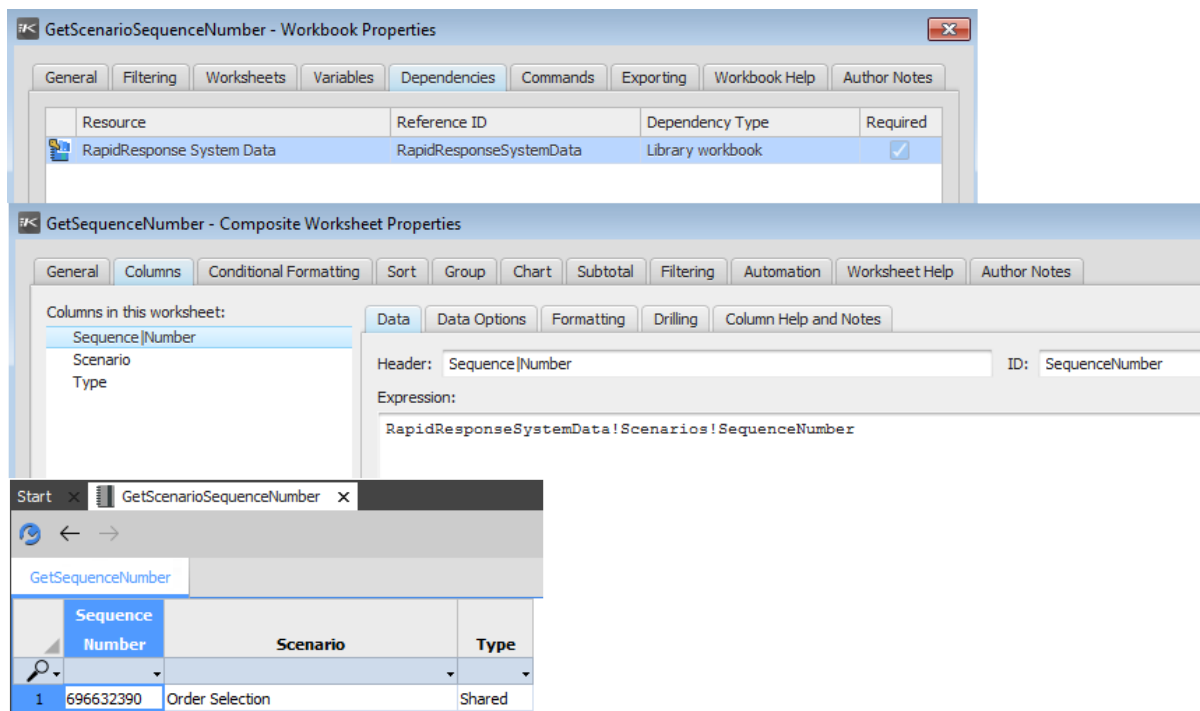
An example of using these values is shown in the following illustration.



If you have access to the RapidResponse System Data workbook, you can also get the sequence number from the Scenarios worksheet.

RapidResponse System Data							
Scenarios							
Scenarios - Hierarchical							
Users							
Groups							
Automation Task Log							
Workbook Command Log							
Data Modification Data							
Scenario		Data Age (Days)	Update Method	Permanent	Monitored	Perspective	Sequence Number
1	Order Selection	0	Manual	N	N		696632390

You can also create a workbook that uses the RapidResponse System Data workbook as a reference, and create a worksheet that contains the Sequence Number column.



Regardless of how you access it, you can read the data from the worksheet column, and then pass that value to the scenario collection method you want to use. For example, using the GetScenarioSequenceNumber workbook shown above, you can get the worksheet data, read the value from the SequenceNumber column, and then use the sequence number to retrieve the scenario.


```

1 function main() {
2   var scenarioName = rapidResponse.scriptArguments["scenarioName"];
3   var scenarioScope = rapidResponse.scriptArguments["scenarioScope"];
4
5   var workbook = rapidResponse.workbooks.get( {name: "GetScenarioSequenceNumber", scope: "Private"}, {
6     variables: [
7       {name: "ScenarioName", value: scenarioName},
8       {name: "ScenarioScope", value: scenarioScope} // Private | Shared
9     ]
10  });
11
12  var worksheet = workbook.worksheets.get("GetSequenceNumber");
13  var data = worksheet.getData();
14  var sequenceNumber = -1;
15
16  if (data.status == "OK" && data.rows.count > 0) {
17    sequenceNumber = data.rows[0].cells.get("SequenceNumber").value;
18  } else {
19    rapidResponse.console.writeLine("Scenario not found");
20  }
21  rapidResponse.console.writeLine("Sequence Number: " + sequenceNumber);
22
23  var scenario = rapidResponse.scenarios.get(sequenceNumber);
24  return ("Retrieved scenario " + scenario.name + ", scope " + scenario.scope + " and sequence number " + sequenceNumber);
25 }

```

The "Scenario Sequence" script has completed.
Sequence Number: 2046082575

Retrieved scenario Promotions, scope Private and sequence number 2046082575

Scenario object methods

Scenario objects implement the following [Resource object](#) methods, as well as the [access control](#) methods.

Method	Description
give (newOwner)	Changes the owner of a scenario.
makePublic()	Changes the scope of a scenario to "Public", which is the equivalent of sharing the scenario. Shared scenarios can be accessed by administrators and users and groups with access. You can also share a scenario with a particular user or group using the accessControlList collection's add method. Note: The term "public" is used for shared scenarios in scripting because, in much earlier versions of RapidResponse, scenarios were referred to as private and public, rather than private and shared. "Public" is retained in reference to scenario scope to avoid breaking users' scripts.
rename (newName)	Changes the name of a private scenario.

Scenario objects also have the following methods.

Method	Description
update()	Updates the scenario with changes from its parent. If the scenario automatically updates, you do not have to call this method.
reset()	Discards all changes in a scenario, and sets its data to be identical to its parent.
commit()	Commits changes in the scenario to its parent.
setProperties(props)	Specifies values for scenario properties.

give method

Gives a scenario to another user by changing the scenario's owner. You can give any scenario you own.

After you give the scenario, you can no longer access it. If you require access to the scenario after you give it, you must share it with yourself before calling this method. For more information, see ["accessControlList collection" on page 121](#).



Note: If you give a shared scenario using a script, the give operation functions differently than giving a scenario through the **Scenarios** pane, which can share the scenario with you before giving it to the new owner. For more information, see the [RapidResponse User Guide \(Java Client\)](#).

► Syntax

```
scenario.give(newOwner);
```

where *scenario* is a Scenario object.

Parameter	Type	Description
newOwner	String	The user to give the scenario to. Note: You cannot give a scenario to a group.

► Returns

No return value.

► Exceptions

The `give` method can throw the following exceptions.

Exception	Description
Scenario.NotFound	The specified scenario does not exist.
Principal.NotFound	The specified user does not exist.
Scenario.NoPermission	Either you do not have permission to give the scenario, or the scenario is a temporary or change data capture scenario and cannot be given.
Scenario.NonUniqueName	The new owner already owns a private scenario with the same name.

► Example

```
var myScenario = rapidResponse.scenarios.get({ name:"myScenario",
scope:"Private" });
myScenario.give("myUser");
```

This example creates a scenario, shares it with the user running the script, and then gives it to another user.

```
var giveScenario = rapidResponse.scenarios.create("Action Proposal",
{name:"Approved Actions", scope:"Public"}, {purpose: "A new scenario
for simulating a course of action."});
giveScenario.makePublic();
giveScenario.accessControlList.add(rapidResponse.loggedInUser,
"View");
giveScenario.give("myUser");
```

makePublic method

Changes the scope of the specified scenario from "Private" to "Public", which shares the scenario and allows other users to access it. This method does not specify other users or groups to share the scenario with, so it is available to only you and administrators.

If you call this method on a variable that represents a scenario, you must ensure you assign the returned shared scenario to the same variable. Otherwise, the variable retains the scope it was created with.

You can also share the scenario with specific users and groups using the [accessControlList collection's add method](#).

Administrators can access any shared scenario, and you can allow other users and groups to access a shared scenario.

► Syntax

```
publicScenario = scenario.makePublic();
```

where *scenario* is a scenario object.

► Returns

A Scenario object that represents the same scenario with its scope changed to "Public".

► Exceptions

The `makePublic` method can throw the following exceptions

Exception	Description
Scenario.NotFound	The specified scenario does not exist.
Scenario.NoPermission	Either you do not have permission to share the scenario, or the scenario is a temporary or change data capture scenario and cannot be shared.
Scenario.NonUniqueName	There is already a shared scenario with the same name.

► Example

```
var myScenario = rapidResponse.scenarios.get({ name:"myScenario",
scope:"Private" });
myScenario = myScenario.makePublic();
```



Note: The term "public" is used for shared scenarios in scripting because, in much earlier versions of RapidResponse, scenarios were referred to as private and public, rather than private and shared. "Public" is retained in reference to scenario scope to avoid breaking users' scripts.

rename method

Changes the name of a scenario. You can rename private and shared scenarios that you own. This differs from the RapidResponse client where you can only rename private scenarios that you own.

► Syntax

```
scenario.rename(newName, failIfScenarioHasOpenQueries);
```

where *scenario* is a scenario object.

Parameter	Type	Description
newName	String	The new name for the resource.
failIfScenarioHasOpenQueries	Boolean	An optional parameter that defines whether the rename operation fails if queries are running on the scenario.

► Returns

No return value.

► Exceptions

The `rename` method can throw the following exceptions.

Exception	Description
<code>ArgumentError</code>	A blank string, invalid string, or <code>null</code> value was specified for the <code>newName</code> parameter.
<code>Scenario.NotFound</code>	The specified scenario does not exist.
<code>Scenario.NameValidationError</code>	Either a scenario with the same name already exists or the specified name is not valid.
<code>Scenario.NoPermission</code>	Either you do not have permission to rename the scenario or the scenario is a temporary or change data capture scenario and cannot be renamed.
<code>Scenario.HasOpenQueries</code>	The scenario has queries running on it, or is being used in a workbook or scorecard.

► Example

This example renames a scenario.

```
var myScenario = rapidResponse.scenarios.get({ name:"myScenario",
scope:"Private" });
myScenario.rename("myOtherScenario");
```

This example renames a scenario, but fails if any queries are running on that scenario

```
var myScenario = rapidResponse.scenarios.get({ name:"myScenario",
scope:"Private" });
myScenario.rename("myOtherScenario", true);
```



Note: This method was modified in RapidResponse 2014.1.

update method

Updates a scenario with data changes from its parent.

► Syntax

```
updateScenario = updateScenario.update();
```

where `updateScenario` is a scenario object.

► Returns

Type	Description
Object	The updated scenario object

► Exceptions

The `update` method can throw the following exceptions.

Exception	Description
<code>Scenario.NoPermission</code>	Either you do not have permission to update the specified scenario or the scenario is a change data capture scenario and cannot be updated.
<code>Scenario.NotFound</code>	The specified scenario does not exist.
<code>Scenario.NonRootOnly</code>	The specified is the root scenario, which cannot be updated.
<code>Scenario.NeedsReset</code>	The scenario's parent has been reset, so the scenario being updated must be reset as well. For more information, see "reset method" on page 158 .

► Example

```
var myScenario = rapidResponse.scenarios.get({ name:"myScenario",
scope:"Private" });
myScenario = myScenario.update();
```

reset method

Resets a scenario, making its data the same as its parent.

► Syntax

```
resetScenario.reset();
```

where `resetScenario` is a scenario object.

► Returns

No return value.

► Exceptions

The `reset` method can throw the following exceptions.

Exception	Description
<code>Scenario.NoPermission</code>	Either you do not have permission to reset the specified scenario or the scenario is a change data capture scenario and cannot be reset.
<code>Scenario.NotFound</code>	The specified scenario does not exist.
<code>Scenario.NonRootOnly</code>	The specified scenario is the root scenario, which cannot be reset.

► Example

```
var myScenario = rapidResponse.scenarios.get({ name:"myScenario",
scope:"Private" });
myScenario.reset();
```

commit method

Commits data changes from a scenario into its parent. You can commit any scenario you own, as long as you own or have permission to modify its parent scenario and the scenario is up to date. If the scenario you are committing has children, you can specify how those scenarios are handled. For more information about committing scenarios, see the [RapidResponse User Guide \(Java Client\)](#).

The commit method uses an optional Options object, which contains properties that determine how the commit is handled, and how child scenarios are managed. The Options object contains the following properties.

Property	Type	Description
failIfScenarioHasChildren	Boolean	Whether the commit fails if the scenario being committed has children. This property is considered if the <code>keepScenarioAfterCommit</code> property is <code>false</code> . Default value: <code>false</code> .
failIfScenarioHasOpenQueries	Boolean	Whether the commit fails if the scenario has queries running on it. Default value: <code>true</code> .
failOnUpdateMergeConflicts	Boolean	Whether commit fails if the data brought in by a forced update conflicts with changes in the scenario. This property is considered if the <code>forceUpdate</code> property is <code>true</code> . If this property is <code>false</code> , the scenario is committed and the conflicts affect the data. For more information, see the RapidResponse User Guide (Java Client) . Default value: <code>false</code>

Property	Type	Description
<code>forceUpdate</code>	Boolean	Whether the scenario is updated before the commit. Default value: <code>true</code>
<code>keepScenarioAfterCommit</code>	Boolean	Whether the scenario is deleted after the commit. If this property is <code>false</code> , the scenario is removed and its children are deleted. Default value: <code>false</code>

► Syntax

```
committedScenario = committedScenario.commit( {Options} );
```

where `committedScenario` is a `Scenario` object.

Parameter	Type	Description
<code>Options</code>	Object	An optional <code>Options</code> object, which defines how the commit is handled.

Prior to RapidResponse 2014.1, the `commit` method accepted an optional `forceUpdate` parameter, which is identical to the `forceUpdate` property of the `Options` object. This syntax is still accepted.

► Returns

Type	Description
Object	The committed scenario object

► Exceptions

The `commit` method can throw the following exceptions.

Exception	Description
<code>Scenario.NoPermission</code>	Either you do not have permission to commit the specified scenario or the scenario is a change data capture scenario and cannot be committed.
<code>Scenario.NotFound</code>	The specified scenario does not exist.
<code>Scenario.NeedsUpdate</code>	The scenario is out of date with its parent and needs to be updated before it can be committed. This exception is not thrown if the <code>Options.forceUpdate</code> parameter is set to <code>true</code> .
<code>Scenario.NeedsReset</code>	The scenario needs to be reset before it can be committed.

Exception	Description
Scenario.NonRootOnly	The specified scenario is the root scenario, which cannot be committed.
Scenario.HasOpenQueries	The scenario has queries running on it, or is being used in a workbook or scorecard.
Scenario.HasChildren	The scenario is the parent of one or more child scenarios.

► Example

The following example commits a scenario and forces it to update.

```
var myScenario = rapidResponse.scenarios.get({ name:"myScenarioChild",
scope:"Private" });
myScenario = myScenario.commit({forceUpdate: true});
```

The following example commits a scenario and forces it to update, but fails if the scenario has children.

```
var myScenario = rapidResponse.scenarios.get({ name:"myScenarioChild",
scope:"Private" });
myScenario = myScenario.commit({
    forceUpdate: true,
    keepScenarioAfterCommit: false,
    failIfScenarioHasChildren: true
});
```

The following example commits a scenario, using the syntax prior to RapidResponse 2014.1.

```
var myScenario = rapidResponse.scenarios.get({ name:"myScenarioChild",
scope:"Private" });
myScenario = myScenario.commit(true);
```



Note: This method was modified in RapidResponse 2014.1.

addResponse method

Adds a message to the scenario's activity log.

► Syntax

```
Scenario.activityLog.addResponse( {subject, type, body, notifyAll} );
```

where *Scenario* is a Scenario object.

Parameter	Type	Description
subject	String	The subject of the response message, which is displayed in the activity log. This parameter is required.
type	String	The type of activity being logged. This can be one of the following values: • Note: The response contains information about the activity performed. • Accept: For collaboration, indicates the proposed changes can proceed. • Reject: For collaboration, indicates the proposed changes cannot proceed. This parameter is optional. If no value is provided, "Note" is used.
body	String	Describes the activity performed. This parameter is optional. If no value is provided, a blank value is used.
notifyAll	Boolean	Indicates whether all users with access to the scenario are notified about the changes. This parameter is optional. If no value is provided, <code>false</code> is used.

► Return value

No return value.

► Exceptions

The `addResponse` method does not throw exceptions.

► Example

This example notifies all users of a change accepted in the Enterprise Data scenario.

```
rapidResponse.scenarios.systemRootScenario.activityLog.addResponse( {
    subject: "Accept Change",
    type: "Accept",
    body: "All changes accepted.",
    notifyAll: true
} );
```

This example records a note about the changes performed by a workbook command.

```
var modificationScenario = rapidResponse.scriptArguments["Scenario"];
var workbookWithCommand = rapidResponse.workbooks.get({name:"Order
Quantity Comparison", scope:"Public"},
{
    scenarios: modificationScenario,
    filter: {name: "All Parts", scope:"Public"},
    siteGroup: {name: "HQ", scope: "Public"},
    variables: [
        {name: "showLate", value: true},
        {name: "onDate", value: rapidResponse.dateTime.TODAY},
    ]
});
```

```
var command = workbookWithCommand.commands.get("QtyDividedBy2");  
modificationScenario.activityLog.addResponse({subject:"Command  
executed", body:("Executed the " + command.name + " command.")});
```



Note: This method was added in RapidResponse 2014.1.

setProperty method

Specifies values for scenario properties. This method allows you to modify the specified property values after retrieving a Scenario object.

If you specify values for multiple properties in a single call, the method succeeds only if all properties are set successfully. If any property cannot be set, the method fails and no values are changed.

► Syntax

```
scenario.setProperty( {permanent, autoUpdate, purpose, status} );
```

where `scenario` is a Scenario object.

Parameter	Type	Description
permanent	Boolean	Whether the scenario is permanent. Permanent scenarios cannot be deleted and are not removed when they are committed.
autoUpdate	Boolean	Whether the scenario automatically updates when its parent is modified.
purpose	String	A description of what the scenario is used for.
status	String	The scenario's status. This is typically used for scenarios that contain collaboration exercises, and can be one of the following values: - InProgress: The scenario's course of action is ongoing. - Suspended: The scenario's course of action is paused, and might be resumed later. - Canceled: The scenario's course of action has been canceled. - Completed: The scenario's course of action has been completed.

► Return value

No return value.

► Exceptions

The `setProperty` method can throw the following exceptions.

Exception	Description
ArgumentException	A null value was passed to a parameter.
Scenario.ParentNotPermanent	The scenario's parent is not permanent and a True value was passed to the permanent parameter.

► Examples

This example designates a scenario's course of action as complete, and disables automatic updates. This ensures the data is not modified as it is reviewed before being committed.

```
var myScenario = rapidResponse.scenarios.get({name: "Demand Balance",
scope: "Public"} );
myScenario.setProperties(
{
    autoUpdate: false,
    status: "Completed"
} );
```

This example designates a scenario as permanent, and updates its purpose to indicate when it was made permanent.

```
var scenarioModify = rapidResponse.scriptArguments["scenario"];
if (scenarioModify.parentScenario.isPermanent) {
    scenarioModify.setProperties( {
        permanent: true,
        purpose: scenarioModify.purpose + "\nMade permanent for
analysis on " + rapidResponse.dateTime.NOW
    } );
}
else {
    return ("The parent of the " + scenarioModify.name + " scenario
is not permanent, and cannot be made permanent.");
}
```



Note: This method was added in RapidResponse 2014.1.

cdcInstances collection

All change data capture (CDC) instances defined on a [scenario](#) are contained in the cdcInstances collection. Each instance is available to a single user.

The cdcInstances collection implements the following method.

Method	Description
get(instance)	Retrieves the specified CDC instance.



Note: This collection was added in RapidResponse 2014.1.

get method

Retrieves a change data capture (CDC) instance, which you can use to capture and send data changes to an external system.

► Syntax

```
inst = scenario.cdcInstances.get(instanceID);
```

where `scenario` is a `Scenario` object.

Argument	Type	Description
instanceID	String	The identifier of the CDC instance you want to retrieve.

► Returns

A `CDCInstance` object.

► Exceptions

The `get` method can throw the following exceptions.

Exception	Description
<code>ArgumentError</code>	A blank string or <code>null</code> value was passed to the method.
<code>CDCInstance.NotFound</code>	No CDC instance with the specified identifier exists.

► Example

```
var myInstance = dataScenario.cdcInstances.get("OrderCapture");
```



Note: This method was added in RapidResponse 2014.1.

CDCInstance objects

The `CDCInstance` object defines a Change Data Capture (CDC) instance that monitors a scenario for data changes. The instance maintains the changes made in the monitored scenario, and allows you to retrieve the changes for synchronizing data changes between systems. All CDC instances you have access to are contained in the `cdcInstances` collection. For more information, see ["cdcInstances collection" on page 164](#).

The CDCInstance object has the following property.

Property	Type	Description
name	String	The name of the CDC instance.

The CDCInstance object contains the following methods.

Method	Description
captureChanges()	Retrieves all changes made in the scenario since the instance was last cleared.
clearChanges()	Clears all captured changes from the instance, which ensures changes are not captured multiple times.
resetChanges()	Resets the instance, which clears all changes whether they have been captured or not.

For more information about CDC, see the [RapidResponse Web Services Guide](#).



Note: This object was added in RapidResponse 2014.1.

captureChanges method

The captureChanges method retrieves all changes from a change data capture (CDC) instance.

► Syntax

```
instance.captureChanges();
```

where *instance* is a CDCInstance object.

► Returns

No return value.

► Exceptions

The captureChanges method can throw the following exception

Exception	Description
ArgumentError	An argument was passed to the method.
CDCInstance.NotFound	No CDC instance with the specified identifier exists.

► Example

```
var dataInstance = dataScenario.cdcInstances.get("myInstance");  
dataInstance.captureChanges();
```



Note: This method was added in RapidResponse 2014.1.

clearChanges method

Clears all captured changes from a change data capture (CDC) instance. Calling this method ensures the changes that have been captured are not captured again, preventing double reporting.

Changes made in the scenario between the time when the changes were captured and when they were cleared are captured the next time the `captureChanges` method is called. For more information, see ["captureChanges method" on page 166](#).

► Syntax

```
instance.clearChanges();
```

where *instance* is a CDCInstance object.

► Returns

No return value.

► Exceptions

The `clearChanges` method can throw the following exception.

Exception	Description
ArgumentError	An argument was passed to the method.
CDCInstance.NotFound	No CDC instance with the specified identifier exists.

► Example

```
var dataInstance = dataScenario.cdcInstances.get("myInstance");
dataInstance.captureChanges();
dataInstance.clearChanges();
```



Note: This method was added in RapidResponse 2014.1.

resetChanges method

Clears all changes from a change data capture (CDC) instance and resets the instance so only changes made after the reset are captured. This method should typically be called when a course of action has been reverted, so those changes are not captured. For more information about capturing changes, see ["captureChanges method" on page 166](#).

► Syntax

```
instance.resetChanges();
```

where *instance* is a CDCInstance object.

► Returns

No return value.

► Exceptions

The `resetChanges` method can throw the following exceptions.

Exception	Description
ArgumentError	An argument was passed to the method.
CDCInstance.NotFound	No CDC instance with the specified identifier exists.

► Example

```
var dataInstance = dataScenario.cdcInstances.get("myInstance");  
dataInstance.resetChanges();
```



Note: This method was added in RapidResponse 2014.1.

CHAPTER 12: Workbook objects

WorkbookPreferences objects	172
workbooks collection	172
Workbook object methods	177
dependencies collection	185
variables collection	186
commands collection	189
Command object	191

Workbook objects can be used to run commands, and contain worksheets that can be used to retrieve data. Workbook objects are contained in the `workbooks` collection. For more information, see ["workbooks collection" on page 172](#).

Workbook objects have the same [properties as Resource objects](#), as well as the following properties.

Property	Type	Description
commands	Object	A collection of commands defined in the workbook. Only workbook commands that modify data are included, commands that run scripts are ignored. This collection is empty if the workbook has no commands. For more information, see "commands collection" on page 189 .
dependencies	Object	A collection of workbooks that this workbook depends on for functionality. This collection is empty if the workbook does not have dependencies. For more information, see "dependencies collection" on page 185 .

Property	Type	Description
variables	Object	A collection of workbook variables defined in the workbook. This collection is empty if the workbook has no variables. For more information, see "variables collection" on page 186 .
worksheets	Object	A collection of worksheets in the workbook. Each worksheet is an instance of a Worksheet object. For more information, see "Worksheet objects" on page 197 and "worksheets collection" on page 200 .

The Workbook object also uses a WorkbookInitialization object, which defines the resources used to define the data returned by the worksheet queries. The properties of this object determine the resources used for the entire workbook. However, you can also specify resources individually for each worksheet. If you do not specify values for each worksheet, the values for the workbook are used. The WorkbookInitialization object has the following properties.

Property	Type	Description
scenarios	Object	The scenario or scenarios that data is returned from. For more information, see "Scenario objects" on page 139 . This property is used for all workbooks except for change data capture (CDC) workbooks. For CDC workbooks, this property is undefined and the <code>cdcInstance</code> property is used instead.
filter	Object	The filter used with the workbook, identified as either a Filter object or by using the <code>{name, scope}</code> syntax.
siteGroup	Object	The site or site filter used with the workbook, identified as either a SiteGroup object or by using the <code>{name, scope}</code> syntax. Individual sites are always public.
hierarchies	Object	The hierarchy or hierarchies applied to the workbook, and the values selected within them. This value is an array of hierarchies.
variables	Object	An array of variables defined in the workbook. Values must be provided for each variable defined in this property. For list variables, you must specify the list entry's display value. The values specified in this property when the workbook is retrieved are the only values that can be set for the variables. If you want to modify a variable's value, you must retrieve another copy of the workbook. For more information, see "get method" on page 172 .
currency	String	The currency used to display Money values in the workbook. This must be a currency code that has been defined in your RapidResponse instance, such as USD or EUR.
unitOfMeasure	String	The units used to display Quantity values in the workbook. This must be a measure that has been defined in your RapidResponse instance, such as gallons or cartons.
model	String	The model used with the workbook.
pool	String	The pool used with the workbook.

Property	Type	Description
partType	String	The part type selected in the workbook. This value can be "Parts" or Reference Parts".
part	String	The part selected in the workbook. This value is relevant if the partType value is "Parts".
referencePart	String	The reference part selected in the workbook. This value is relevant if the partType value is "Reference Parts".
constraint	String	The active constraint displayed in the workbook.
workCenter	String	The work center displayed in the workbook.
processInstance	String	The process this workbook is used in.
cdcInstance	Object	The change data capture (CDC) instance that monitors the workbook for changes. This property is undefined for workbooks that are not used for CDC. For CDC workbooks, this property is used instead of the <code>scenario</code> property. For more information, see "CDCInstance objects" on page 165 .

Values for the WorkbookInitialization object's properties can be specified as the workbook's filter settings when you retrieve a workbook from the [workbooks](#) collection. These values cannot be modified after retrieving the workbook. For more information, see ["get method" on page 172](#).

The `hierarchies` property refers to a WorkbookHierarchyInitialization object, which has the following properties.

Property	Type	Description
hierarchy	Object	The hierarchy used, identified using the <code>{name, scope}</code> syntax.
path	Object	The values selected in the hierarchy. This property includes the names of the hierarchy values and optionally, values selected in levels underneath the current level.

The `path` property is an object that has the following properties.

Property	Type	Description
name	String	The name of the hierarchy value selected.
childPath	Object	The values selected in the level below the current level. The <code>childPath</code> property is a path object, so each <code>childPath</code> can have its own <code>childPath</code> , until the last level of the hierarchy is reached. The last level of the hierarchy has a blank value for its <code>childPath</code> .

The `variables` property of the workbook initialization object refers to an object that has the following properties.

Property	Type	Description
name	String	The name of the variable.
value	String	The value contained in the variable. For list variables, this must refer to the list item's Display value.

For more information about workbook variables, see the [RapidResponse Resource Authoring Guide](#).

A Workbook object can be used to generate a report of worksheet data, which is stored in a ReportOutput object. The ReportOutput object does not contain properties, and can be included in a message. For more information, see "[messages collection](#)" on page 449.

WorkbookPreferences objects

The WorkbookPreferences object contains your personal resource preferences. This object contains no properties or methods, but can be used to pass your preferences from a dashboard to a workbook, or from a worksheet to a filter manager object. For more information, see "[FilterManager object](#)" on page 253.

workbooks collection

The `workbooks` collection contains all workbooks available to the user running the script. Any workbook retrieved or used by the script must be present in this collection

The `workbooks` collection implements the following [resource collection](#) methods.

Method	Description
get(key, options)	Retrieves the specified workbook using the specified filtering options.
exists(workbook)	Determines whether the specified workbook exists.
remove(workbook)	Deletes the specified workbook, removing it from the collection.

get method

Retrieves a workbook object, which allows you to manipulate the workbook in your script. You can use workbooks to retrieve worksheet data and run automatic data modifications.

► Syntax

```
rapidResponse.workbooks.get( {name:"Name", scope:"Public or Private"},
{initialization} );
```

Parameter	Type	Description
name	String	Identifies the workbook to retrieve.
scope	String	Whether the workbook is Private or Public. For more information, see "Using resources as objects" on page 64 .
initialization	Object	An optional set of options that determine the data displayed in the workbook. The properties you can specify are contained in the WorkbookInitialization object of the Workbook object. For information about the properties you can specify, see "Workbook objects" on page 169 .

You can also use the workbook's identifier within the `workbooks` collection instead of the `{name, scope}` syntax. This is typically used for retrieving workbooks in a resource iteration function, such as the `forEach` or `forEachId` functions. For more information, see ["Resource collections" on page 110](#).

► Returns

Type	Description
Object	A workbook object

► Exceptions

The `get` method can throw the following exceptions.

Exception	Description
Workbook.NotFound	The specified workbook does not exist.
Scenario.NotFound	The specified scenario does not exist.
Filter.NotFound	The specified filter does not exist.
SiteGroup.NotFound	The specified site or site filter does not exist.
Model.NotFound	The specified model does not exist.
Pool.NotFound	The specified pool does not exist.
Variable.NotFound	The specified variable does not exist.
Hierarchy.NotFound	The specified hierarchy does not exist.
CDCInstance.NotFound	The specified change data capture instance does not exist.

► Example

Retrieving a workbook.

```
var myWorkbook = rapidResponse.workbooks.get({ name:"My Workbook",
scope:"Public" },
```

```

{
  scenarios: [{ name:"myScenario", scope:"Private" }],
  filter: {name: "Buy Parts", scope:"Public"},
  siteGroup: {name: "DC-Asia", scope: "Public"},
  variables: [
    {name: "showLate", value: true},
    {name: "onDate", value: rapidResponse.dateTime.TODAY},
    {name: "worksheetFilterExpression", value:
      "Order.Site.Value = 'HQ'" },
    {name: "maxQuantity", value: 4000},
    {name: "orderType", value: "= All ="}
  ],
  hierarchies: [{
    hierarchy: {name:"Customer", scope:"Public"},
    path: [{
      name:"C",
      childPath: [
        {name:"Computer Mart"}
      ]
    }]
  }],
  currency:"USD"
});

```

Retrieving a workbook that displays data for a specific model of a part, with monetary values displayed in Euros.

```

var myWorkbook = rapidResponse.workbooks.get({ name:"My Workbook",
scope:"Public" },
{
  scenarios: [{ name:"myScenario", scope:"Private" }],
  filter: {name: "All Parts", scope:"Public"},
  siteGroup: {name: "HQ", scope: "Public"},
  part:"Cruiser",
  model:"Standard",

  currency:"EUR"
});

```

Retrieving a change data capture workbook.

```

var cdcWorkbook = rapidResponse.workbooks.get({name: "Order Capture",
scope: "Public"},
{
  cdcInstance: "PurchaseOrderMonitor",
  filter: {name: "Make Parts", scope: "Public"},
  siteGroup: {name: "HQ", scope:"Public"}
} );

```

Retrieving a workbook that displays part or reference part data, depending on what is specified in the worksheet the script is called from.

```
var currentScenario = rapidResponse.scriptArguments["Scenario"];
var currentWorksheet = rapidResponse.scriptArguments["#Worksheet"];
var currentWorkbook = rapidResponse.workbooks.get
(currentWorksheet.parentWorkbook,{
    scenarios: [currentScenario],
    filter: currentWorksheet.filter,
    siteGroup: {name:"All Sites", scope:"Public"},
    partType: currentWorksheet.partType,
    part: currentWorksheet.part,
    referencePart: currentWorksheet.referencePart
} );
```

Retrieving a workbook that uses a filter expression in a variable for its worksheets. For the `worksheetFilterExpression` variable, the worksheet's filter expression must use the EVAL function to evaluate the expression contained in the variable. For more information, see the [RapidResponse Resource Authoring Guide](#).

```
var myWorkbook = rapidResponse.workbooks.get({ name:"Orders",
scope:"Public" },
{
    scenarios: [{ name:"myScenario", scope:"Private" }],
    filter: {name: "Make Parts", scope:"Public"},
    siteGroup: {name: "Company Sites", scope: "Public"},
    variables: [{name: "worksheetFilterExpression", value:
        "Order.Site.Value = 'HQ'" }]
});
```

Retrieving multiple workbooks using the same data settings.

```
var workbookSettings = {
    scenarios: [{ name:"myScenario", scope:"Private" }],
    filter: {name: "Buy Parts", scope:"Public"},
    siteGroup: {name: "DC-Asia", scope: "Public"},
    pool: "Unpooled",
    variables: [
        {name: "showLate", value: true},
        {name: "onDate", value: rapidResponse.dateTime.TODAY}
    ],
    currency:"USD"
};
var myWorkbook = rapidResponse.workbooks.get({ name:"My Workbook",
scope:"Public" }, workbookSettings);
var myOtherWorkbook = rapidResponse.workbooks.get({ name: "Orders
Workbook", scope:"Private" }, workbookSettings);
```

exists method

Determines whether a workbook exists. However, because RapidResponse supports multiple users working with the same resources, a user might delete the specified workbook after this method runs. If you are using this method to test whether a workbook exists, you should still catch a `NotFound` exception for the workbook you are using.

► Syntax

```
workbookExists = rapidResponse.workbooks.exists({name: "Name", scope: "Public or Private"});
```

Parameter	Type	Description
name	String	The name of the workbook to test for existence.
scope	String	Whether the workbook is Private or Public.

► Returns

Type	Description
Boolean	• true if the workbook exists • false if the workbook does not exist

► Exceptions

The `exists` method can throw the following exceptions.

Exception	Description
ArgumentError	One of the following has occurred: • The parameter is missing a name or scope value. • The scope value specified is not "Private" or "Public".

► Example

```
var workbookExists = rapidResponse.workbooks.exists({name: "Order Analysis", scope: "Public"});
```

remove method

Deletes a workbook from the server, which also removes it from the `workbooks` collection. Deleted workbooks are no longer available to any user or process, and you can only delete workbooks you own

or have permission to manage.

► Syntax

```
rapidResponse.workbooks.remove( name );
```

Parameter	Type	Description
name	Object	The workbook you are deleting. This can be specified as either a Workbook object or by using the {name, scope} syntax.

► Returns

No return value.

► Exceptions

The `remove` method can throw the following exceptions.

Exception	Description
Workbook.NoPermission	You do not have permission to delete the specified workbook.
Workbook.NotFound	The specified workbook does not exist.

► Example

```
rapidResponse.workbooks.remove({name:"Planned Inventory",  
scope:"Public"});
```



Note: This method was added in RapidResponse 2014.1.

Workbook object methods

The Workbook object implements the following [resource object](#) methods.

Method	Description
give (newOwner)	Gives the workbook to the specified user.
makePublic()	Changes the scope of a workbook to "Public", which is the equivalent of sharing the workbook. Public workbooks can be accessed by administrators and users and groups with access. You can also share a workbook with a particular user or group using the accessControlList collection's add method .

Method	Description
rename (newName)	Changes the workbook's name.
copy (newName)	Creates a new workbook by copying an existing workbook.

The Workbook object has the following method.

Method	Description
generateReport(worksheet, format, options)	Creates a report of data in the workbook, using the options you specify.

give method

Gives a workbook to the specified user.

► Syntax

```
workbook.give(newOwner) ;
```

where *workbook* is a Workbook object.

Parameter	Type	Description
<i>newOwner</i>	String	The user to give the workbook to. Note: You cannot give a workbook to a group.

► Return value

No return value.

► Exceptions

The *give* method can throw the following exceptions.

Exception	Description
Workbook.NotFound	The specified workbook does not exist.
Principal.NotFound	The specified user does not exist.
Workbook.NonUniqueName	The specified user already owns a workbook with the same name as the one you are giving.
Workbook.NoPermission	You do not have permission to give the workbook.

► Example

```
var myWorkbook = rapidResponse.workbooks.get({ name:"Purchase
Comparisons", scope:"Private" });
myWorkbook.give("myUser");
```



Note: This method was added in RapidResponse 2014.1.

makePublic method

Changes the scope of the specified workbook from "Private" to "Public", which shares the workbook and allows other users to access it. This method does not specify other users or groups to share the workbook with, so it is available to only you and administrators.

If you call this method on a variable that represents a workbook, you must ensure you assign the returned public workbook to the same variable. Otherwise, the variable retains the scope it was created with.

You can also share the workbook with specific users and groups using the [accessControlList collection's add method](#).

Administrators can access any shared workbook, and you can allow other users and groups to access a shared workbook.

► Syntax

```
publicWorkbook = workbook.makePublic();
```

where *workbook* is a Workbook object.

► Return value

A Workbook object that represents the same workbook with its scope changed to "Public".

► Exceptions

The `makePublic` method can throw the following exceptions.

Exception	Description
Workbook.NotFound	The specified workbook does not exist.
Workbook.NonUniqueName	A shared workbook with the same name as the one you are sharing already exists.
Workbook.NoPermission	You do not have permission to share the workbook.

► Example

```
var myWorkbook = rapidResponse.workbooks.get({ name:"Demand
Comparison", scope:"Private" });
```

```
myWorkbook = myWorkbook.makePublic();
```



Note: This method was added in RapidResponse 2014.1.

rename method

Changes the name of a workbook.

► Syntax

```
workbook.rename(newName);
```

where *workbook* is a *Workbook* object.

Parameter	Type	Description
newName	String	The new name for the filter.

► Return value

No return value.

► Exceptions

The `rename` method can throw the following exceptions.

Exception	Description
ArgumentError	A blank string, invalid string, or <code>null</code> value was specified for the <code>newName</code> parameter.
Workbook.NotFound	The specified workbook does not exist.
Workbook.NameValidationError	Either a workbook with the same name already exists or the specified name is not valid.
Workbook.NoPermission	The workbook cannot be renamed or you do not have permission to rename it.
Workbook.RenamePublicError	The workbook is shared and cannot be renamed.

► Example

```
var myWorkbook = rapidResponse.filters.get({ name:"Inventory  
Analysis", scope:"Private" });  
myWorkbook.rename("Inventory for Buyers");  
myWorkbook.accessControlList.add("Buyers", "View");
```



Note: This method was added in RapidResponse 2014.1.

copy method

Creates a new workbook by copying an existing workbook. If you copy a public workbook, the copy is private.

► Syntax

```
workbook.copy(newName);
```

where *workbook* is a Workbook object.

Parameter	Type	Description
newName	String	The name for the new workbook.

► Return value

No return value.

► Exceptions

The `copy` method can throw the following exceptions.

Exception	Description
ArgumentError	A blank string, invalid string, or <code>null</code> value was specified for the <code>newName</code> parameter.
Workbook.NotFound	The specified workbook does not exist.
Workbook.NameValidationError	Either a workbook with the same name already exists or the specified name is not valid.
Workbook.NoPermission	The workbook cannot be copied or you do not have permission to copy it.

► Example

```
var myWorkbook = rapidResponse.workbooks.get({ name:"Inventory for  
Buyers", scope:"Public" });  
myWorkbook.copy("Excess Inventory");  
myWorkbook.accessControlList.add("Production Planners", "View");
```



Note: This method was added in RapidResponse 2014.1.

generateReport method

Creates a report of the data contained in a worksheet. You can specify the file format used to create the report, and options for formatting the data.

► Syntax

```
report = workbook.generateReport( worksheet, format, {encoding,  
formatHiddenDuplicates, includeColumnHeaders, includeFormatting} );
```

where *workbook* is a Workbook object.

Parameter	Type	Description
worksheet	String	The worksheet identifier of the worksheet to create the report from. For XLSX reports, you can generate the report for all worksheets in the workbook by specifying an asterisk (*) instead of the worksheet.
format	String	The file format of the report. This can be one of the following values: • XLSX: A Microsoft Excel file (versions 2007-2019), which can be viewed in Microsoft Excel. • PDF: A PDF file, which can be viewed in any PDF reader, such as Adobe Reader. • HTML: A web page, which can be viewed in a web browser. • XML: An XML file, which can be viewed in a web browser or word processor. • TAB: A tab-delimited tab file, which can be viewed in a word processor. • excelTemplate: If the workbook uses a report template, exports the data defined by that template. This option must be used with a single worksheet.

Parameter	Type	Description
encoding	String	The character encoding used for the report. This can be one of the following values: - ANSI: plain text non-Unicode encoding. The exported file will use the locale setting of RapidResponse. - Unicode: a 16-bit character encoding system that supports the processing and display of the major texts in the world. Use Unicode encoding for data that will be used in a multi-lingual setting. - UnicodeBigEndian: a sequence of bytes is stored with the most significant value first (a word is stored big-end first). This encoding is usually supported on computers that do not use an x86 CPU (Intel and AMD), such as the older generation Macintosh computers which used a Motorola CPU. - UTF8: an 8-bit version of Unicode that can be used with transmission media that supports only 8 bits of data within one byte. Use the UTF-8 version of Unicode for data that will be used in a multi-lingual setting, and on different operating systems. It is also important to use UTF-8 for files exported in XML format. This parameter does not apply to Microsoft Excel reports. This parameter is optional. If a value is not provided but is relevant for a format, the following values are used: - HTML: UTF8 - XML: UTF8 - TAB: UTF8
formatHiddenDuplicates	String	Defines how hidden duplicate values in the worksheet are shown in the report. This can be one of the following values: - Show: Displays all duplicate values. - Hide: Hides duplicate values in the report. - Clear: Deletes duplicate values from the report. This parameter is optional. If a value is not provided, the following values are used for each format: - XLSX: Hide - PDF: Hide - HTML: Hide - XML: Show - TAB: Show

Parameter	Type	Description
includeColumnHeaders	Boolean	Indicates whether column headers are included in the report. This parameter is optional. If a value is not provided but is relevant for a format, the following values are used: • XLSX: True • HTML: True • XML: True • TAB: False
includeFormatting	Boolean	Indicates whether the worksheet's formats (colors, text formatting, fonts, and so on) are preserved in the report. This parameter applies to Microsoft Excel reports. This parameter is optional. If a value is not provided but is relevant, the "True" value is used. following values are used:

► Return value

A [ReportOutput](#) object.

► Exceptions

The `generateReport` method can throw the following exceptions.

Exception	Description
ArgumentException	A null value was passed for a parameter or a required parameter is missing.
Workbook.Report.WorksheetsWithErrors	A worksheet included in the report contains errors.
Workbook.Report.ExceededExcelLimit	The worksheet results in more than 1048576 rows for an XLSX report.

► Example

This example generates a single worksheet report as an HTML file.

```
var myWorkbook = rapidResponse.workbooks.get({ name:"Orders",
scope:"Public" },
{
    scenarios: [{ name:"myScenario", scope:"Private" }],
    filter: {name: "Make Parts", scope:"Public"},
    siteGroup: {name: "Company Sites", scope: "Public"}
});
output = myWorkbook.generateReport( "OrderSummary", "HTML",
{
    encoding : "UTF8",
    formatHiddenDuplicates: "Hide",
    includeColumnHeaders: false,
});
```


This example generates a report of all worksheets in the workbook as an XLSX file and then sends it to the Production Planners group.

```
var myWorkbook = rapidResponse.workbooks.get({ name:"Orders",
scope:"Public" },
{
    scenarios: [{ name:"myScenario", scope:"Private" }],
    filter: {name: "Make Parts", scope:"Public"},
    siteGroup: {name: "Company Sites", scope: "Public"}
});
output = myWorkbook.generateReport( "*", "XLSX",
{
    formatHiddenDuplicates: "Show",
    includeColumnHeaders: true,
    includeFormatting: true
});
rapidResponse.messages.sendMessage( {to:"Production Planners",
subject: "New Report", attachments:[ {fileName: "Orders.xlsx",
content: output} ]});
```

This example generates a report using the workbook's report template and sends it to the Production Planners group.

```
var myWorkbook = rapidResponse.workbooks.get({ name:"Orders",
scope:"Public" },
{
    scenarios: [{ name:"myScenario", scope:"Private" }],
    filter: {name: "Make Parts", scope:"Public"},
    siteGroup: {name: "Company Sites", scope: "Public"}
});

var output = myWorkbook.generateReport( "OrderSummary", "XLSX");

rapidResponse.messages.sendMessage( {to:"Production Planners",
subject: "New Report", attachments:[ {fileName: "Orders.xlsx",
content: output} ]});
```



Note: This method was added in RapidResponse 2014.1.

dependencies collection

The `dependencies` collection contains all workbooks that a workbook depends on, represented as [WorkbookDependency](#) objects.

The `dependencies` collection implements only the collection iteration functions. For more information, see ["Resource collections" on page 110](#).

WorkbookDependency objects

WorkbookDependency objects define workbooks that another workbook depends on for functionality. The WorkbookDependency object has the following properties.

Property	Type	Description
dependentId	String	The dependency identifier.
dependentType	String	The type of dependency.
variableMappings	Object	A collection of WorkbookDependencyVariableMapping objects that define how the variables in the dependent workbook map to the variables in the workbook it depends on.
workbookKey	Object	The Workbook object the dependency exists for.

The WorkbookDependencyVariableMapping object has the following properties.

Property	Type	Description
targetId	String	The identifier of the variable in the target workbook.
sourceId	String	The identifier of the variable in the source workbook.

variables collection

The `variables` collection contains all workbook variables defined in a workbook, represented as [WorkbookVariable](#) objects.

The `variables` collection implements only the collection iteration functions. For more information, see ["Resource collections" on page 110](#).

WorkbookVariable objects

The WorkbookVariable object represents a variable defined in a workbook. The WorkbookVariable object contains the following properties.

Property	Type	Description
isVisible	Boolean	Whether the variable is available to users. Variables that are not visible cannot be modified by users.
description	String	The variable's description.
label	String	The label displayed to users.

Property	Type	Description
name	String	The variable's name.
defaultType	String	The default value for the variable. This property must be one of the following: • DefaultForType: The variable's default value is the default for the type of variable. For more information about the default values for variables, see the RapidResponse Resource Authoring Guide. • None: The variable does not have a default value. • First: For list variables, the default value is the first value in the list. • Value: The specified value is the default for the variable.
isLabelOnToolbar	Boolean	Whether the variable's label is displayed on the workbook toolbar.
defaultValue	String	The value used for the variable if no other value is specified. This property is required only if the variable takes a default value.
isOnToolbar	Boolean	Whether the variable is displayed on the workbook toolbar.
type	String	The variable type.
quantityValue	Object	For quantity variables, contains a VariableValueQuantity object that defines the value contained in the variable.
booleanValue	Object	For Boolean variables, contains a VariableValueBoolean object that defines the value contained in the variable.
listValue	Object	For list variables, contains a VariableValueList object that defines the list values contained in the variable.

VariableValueQuantity object

The VariableValueQuantity object defines the minimum and maximum values for a quantity variable. This object has the following properties.

Property	Type	Description
min	Number	The minimum value for the variable.
minSpecified	Number	The minimum value specified in the variable.
max	Number	The maximum value for the variable.
maxSpecified	Number	The maximum value specified for the variable.

VariableValueBoolean object

The VariableValueBoolean object defines the true and false values in a Boolean variable. This object contains the following properties.

Property	Type	Description
falseDataValue	String	The data value used when the variable is set to false.
falseDisplayValue	String	The value displayed in resources when the variable is set to false.
trueDataValue	String	The data value used when the variable is set to true.
trueDisplayValue	String	The value displayed in resources when the variable is set to true.

VariableValueList object

The VariableValueList object defines the values included in a list variable. List variables define data values that are used in expressions and display values that users see. All processes that use list variables must specify the display value.

This object contains the following properties.

Property	Type	Description
fixed	Object	A VariableValueListFixed object that defines the fixed values in the list.
query	Object	A VariableValueListQuery object that defines the query-generated values in the list.

The VariableValueListFixed object contains the following properties.

Property	Type	Description
fixedValues	Object	A VariableValueListFixedFixedValue object that contains the fixed values used in the list.
sortFixedValues	Boolean	Whether the fixed values are sorted.
useFixedValues	Boolean	Whether the list uses fixed values.

The VariableValueListFixedFixedValue object contains the following properties.

Property	Type	Description
dataValue	String	The fixed value's data value, which is used in expressions.
displayValue	String	The fixed value's display value, which is displayed in the toolbar control or Data Settings dialog box.

The VariableValueListQuery object contains the following property.

Property	Type	Description
useQueryValues	Boolean	Whether the list uses query-generated values.

NameValuePair object

The NameValuePair object defines the value stored in a variable. Prior to RapidResponse 11.2, this object was used in the [ControlSettings](#) object. This object contains the following properties.

Property	Type	Description
name	String	The variable name.
value	String	The value provided for the variable.

commands collection

The `commands` collection contains the commands defined in a workbook, which are represented as [Command](#) objects.

The `commands` collection implements the following methods.

Method	Description
get(command)	Retrieves a Command object from the <code>commands</code> collection.
exists(command)	Determines whether the specified workbook command exists.



Note: This collection was modified in RapidResponse 2014.1.

get method

Retrieves a workbook command.

► Syntax

```
var command = workbook.commands.get (commandName) ;
```

where *workbook* is a Workbook object.

Parameter	Type	Description
commandName	String	The name of the command you want to retrieve.

► Returns

Type	Description
Object	A Workbook command object

► Exceptions

The `get` method can throw the following exceptions.

Exception	Description
<code>Workbook.Command.NotFound</code>	The specified workbook command does not exist.

► Example

```
var workbookWithCommand = rapidResponse.workbooks.get({name:"Order
Quantity Comparison", scope:"Public"},
{
  scenarios: [{ name:"myScenario", scope:"Private" }],
  filter: {name: "All Parts", scope:"Public"},
  siteGroup: {name: "HQ", scope: "Public"},
  variables: [
    {name: "showLate", value: true},
    {name: "onDate", value: rapidResponse.dateTime.TODAY},
  ]
});
var command = workbookWithCommand.commands.get("QtyDividedBy2");
```

exists method

Determines whether a command exists in a workbook. However, because `RapidResponse` supports multiple users working with the same resources, a user might delete the specified command after this method runs. If you are using this method to test whether a command exists, you should still catch a `NotFound` exception for the command you are using.

► Syntax

```
exist = workbook.commands.exists(name);
```

where *workbook* is a `Workbook` object.

Parameter	Type	Description
<code>name</code>	<code>String</code>	The name of the command to test for existence.

► Returns

Type	Description
Boolean	• true if the command exists • false if the command does not exist

► Exceptions

The `exists` method can throw the following exceptions.

Exception	Description
<code>ArgumentError</code>	The parameter is missing a <code>name</code> value.

► Example

```
var commandExists = workbook.commands.exists({name: "QtyDividedBy2"});
```

Tests if a command exists, and if so, runs it.

```
var workbookWithCommand = rapidResponse.workbooks.get({name: "Order  
Quantity Comparison", scope: "Public"},  
{  
  scenarios: [{ name: "myScenario", scope: "Private" }],  
  filter: {name: "All Parts", scope: "Public"},  
  siteGroup: {name: "HQ", scope: "Public"},  
  variables: [  
    {name: "showLate", value: true},  
    {name: "onDate", value: rapidResponse.dateTime.TODAY},  
  ]  
});  
var commandExists = workbookWithCommand.commands.exists({name:  
"QtyDividedBy2"});  
if (commandExists) {  
  var command = workbookWithCommand.commands.get("QtyDividedBy2");  
  command.execute();  
}
```

Command object

The Command object defines a command in a workbook, which can either modify data in the workbook or run a script. All commands defined in a workbook are contained in the `commands` collection.

The Command object has the following properties.

Property	Type	Description
name	String	The name of the workbook command.
type	String	The type of command. This can be one of the following: • DataUpdate: Performs a data modification using the actions defined in worksheets. • Script: Runs a script. • OpenForm: Opens a form. This property was modified in RapidResponse 2014.4.

DataUpdateCommand object

The DataUpdateCommand object defines a workbook command that inserts, modifies, deletes, or updates records in worksheets. The DataUpdateCommand object has the following properties.

Property	Type	Description
actions	Object	A collection of Action objects that define the data modifications performed by the command.
name	String	The name of the workbook command.
type	String	The type of command. This value is DataUpdate.

The Action object represents one action the command performs, and has the following property.

Property	Type	Description
index	Number	The index of the action in the array.

The DataUpdateCommand object has the following method.

Method	Description
execute()	Runs the workbook command.



Note: This object was added in RapidResponse 2014.2.

ScriptCommand object

The ScriptCommand object defines a workbook command that executes a script. The ScriptCommand object has the following properties.

Property	Type	Description
name	String	The name of the workbook command.
type	String	The type of command. This value is Script.

The ScriptCommand object has the following method.

Method	Description
execute()	Runs the workbook command.



Note: This object was added in RapidResponse 2014.2.

OpenFormCommand object

The OpenFormCommand object defines a command in a workbook that opens a form. The OpenFormCommand object has the following properties.

Property	Type	Description
name	String	The name of the form command.
type	String	The OpenForm command.

execute method

Runs a workbook command, which performs the data modifications defined in or runs the script specified in the command.

The `execute` method can also be called on the actions contained with a data modification command, which performs the action specified.

The actions specified in a workbook command perform the data modifications defined in worksheets in the workbook. For more information, see "[dataModification](#)" on page 198.

► Syntax

```
workbookCommand.execute();
```

```
workbookCommand.execute( [function(action)]{result = action.execute();});
```

```
result = scriptCommand.execute();
```

where *workbookCommand* is a workbook command object that performs a data modification, and *scriptCommand* is a workbook command object that runs a script.

Parameter	Type	Description
function (action)	Function	For data modification commands, an optional function that is executed on each action in the command.



Note: For users to successfully run the workbook command in a shared scenario, they must have permission to edit data in shared scenarios.

► Returns

For data modification commands, an object that contains a summary of all actions performed by the command.

Each action within the command returns an object with the following properties.

Property	Type	Description
deleted	Number	The number of records deleted by the modification.
inserted	Number	The number of records inserted by the modification.
modified	Number	The number of records modified by the modification.

The properties populated by this object depend on the type of the data modification. For example, a data modification of type Delete populates only the `deleted` property. An Update data modification can populate all three of the properties, depending on the actions defined in the data modification. The values for a property not populated by an action are 'undefined'.

For script commands, the script's return value. If the script does not return a value, this method does not return a value.

► Exceptions

The `execute` method can throw the following exceptions.

Exception	Description
<code>Workbook.DataUpdateError</code>	One of the actions performed by the command failed.
<code>Scenario.ReadOnly</code>	The scenario the command modifies is view-only.
<code>Workbook.Command.Scenario.NotSpecified</code>	A scenario was not specified when the workbook object was retrieved. The command requires a scenario to run.

► Example

To run a workbook command:

```
var workbookWithCommand = rapidResponse.workbooks.get({name:"Order
Quantity Comparison", scope:"Public"},
```

```

{
    scenarios: [{ name:"myScenario", scope:"Private" }],
    filter: {name: "All Parts", scope:"Public"},
    siteGroup: {name: "HQ", scope: "Public"},
    variables: [
        {name: "showLate", value: true},
        {name: "onDate", value: rapidResponse.dateTime.TODAY},
    ]
});
var command = workbookWithCommand.commands.get("QtyDividedBy2");
command.execute();

```

To execute only one action defined in a workbook command and return the number of records modified:

```

var workbookWithCommand = rapidResponse.workbooks.get({name:"Order
Quantity Comparison", scope:"Public"},
{
    scenarios: [{ name:"myScenario", scope:"Private" }],
    filter: {name: "All Parts", scope:"Public"},
    siteGroup: {name: "HQ", scope: "Public"},
    variables: [
        {name: "showLate", value: true},
        {name: "onDate", value: rapidResponse.dateTime.TODAY},
    ]
});
var command = workbookWithCommand.commands.get("QtyDividedBy2");
command.execute( function(action) {
    // Only execute the 2nd action in this command
    if (action.index == 1) {
        var result = action.execute();
    }
} );
return result.modified;

```

To execute a workbook command only if a private scenario is passed to the script:

```

var scenario = rapidResponse.scriptArguments['scenario'];
if (scenario.scope == 'Public'){
    return ("You must select a private scenario");
}

var workbookWithCommand = rapidResponse.workbooks.get({name:"Order
Quantity Comparison", scope:"Public"},
{
    scenarios: [scenario],
    filter: {name: "All Parts", scope:"Public"},
    siteGroup: {name: "HQ", scope: "Public"},
});
var command = workbookWithCommand.commands.get("QtyDividedBy2");
command.execute();

```



Note: This method was modified in RapidResponse 2014.1.

CHAPTER 13: Worksheet objects

CrosstabWorksheet object	199
Worksheet arguments	199
worksheets collection	200
Worksheet object methods	202
convertToJSON method	210
columns collection	216
Worksheet data collections	217
Worksheet data methods	221
WorksheetDataModification object	224
Worksheet bucketing objects	225
WorksheetColumnDrillToDetail object	231
DrillTarget object	233

Worksheet objects define the worksheets in a workbook, and can be used to perform data modifications or retrieve data. Worksheet objects are contained in the worksheets collection. For more information, see "[worksheets collection](#)" on page 200.

Worksheet objects have the following properties.

Property	Type	Description
id	String	The worksheet's identifier. This value must be unique within a workbook.
name	String	The worksheet's name.
displayName	String	The name displayed in the workbook for the worksheet.

Property	Type	Description
scenarios	Object	The scenario or scenarios selected in the worksheet. This property can be a scenario object or an array of Scenario objects. For more information, see "Scenario objects" on page 139 .
filter	Object	The filter applied to the worksheet, identified as either a Filter object or by using the {name, scope} syntax.
siteGroup	Object	The site or site filter the worksheet displays data for, identified as either a SiteGroup object or by using the {name, scope} syntax. Individual sites are always Public.
model	String	The model the worksheet displays.
pool	String	The pool the worksheet displays.
partType	String	The part type selected in the worksheet. This value can be "Parts" or Reference Parts".
part	String	The part selected in the worksheet. This value is relevant if the partType value is "Parts".
referencePart	String	The reference part selected in the worksheet. This value is relevant if the partType value is "Reference Parts".
hierarchies	Object	The hierarchy or hierarchies applied to the worksheet, and the values selected within them. For more information about how hierarchies are defined in scripts, see "Workbook objects" on page 169 .
variables	Object	An array of variables used by the worksheet, and the values specified for them. For more information about how variables are specified, see "variables" on page 170 .
columns	Object	The columns in the worksheet. For more information, see "WorksheetColumn object" on page 216 .
sortedColumns	Object	An array of worksheet columns, ordered according to the sort order of the columns in the worksheet.
parentWorkbook	Object	The workbook that contains the worksheet.
isHidden	Boolean	Whether the worksheet is hidden.
canSort	Boolean	Whether the worksheet columns can be sorted.
dataModification	Object	The type of data modification defined in the worksheet. For more information, see "WorksheetDataModification object" on page 224 .
worksheetData	Object	The data rows and cells contained in the worksheet. For more information, see "Worksheet data objects" on page 219 .

Property	Type	Description
importData	Object	Data that is imported into the worksheet. For more information, see "Worksheet data import objects" on page 220 .
crosstabWorksheet	Object	The crosstab settings for this worksheet. For more information, see "CrosstabWorksheet object" on page 199 .
autobucketInfo	Object	The automatically-generated bucket settings for a crosstab worksheet. For more information, see "AutobucketInfo object" on page 230 .
bucketSettings	Object	Settings and options specified for the worksheet's buckets. For more information, see "Worksheet bucketing objects" on page 225 .

CrosstabWorksheet object

The CrosstabWorksheet object defines settings specific to crosstab worksheets. This object contains the following properties.

Property	Type	Description
rawBucketValues	String	An array of values representing the worksheet's bucket values.
bucketColumnId	String	The identifier of the column used to define the buckets.
bucketDataType	String	The type of data the worksheet is bucketed by
index	Number	The worksheet's index in the worksheets collection.
id	String	The worksheet's identifier.

Worksheet arguments

For scripts that run from workbook commands, additional arguments are automatically used to define the workbook (#Workbook), worksheet (#Worksheet), and selected data (#WorksheetSelection) the script runs on. For information about these arguments, see ["Define arguments for a script" on page 68](#).

The #WorksheetSelection argument defines an object that contains the rows and cells that the worksheet user has selected. The selection made in the worksheet determines a set of data that you can use with the worksheet methods. The #WorksheetSelection argument contains the following properties.

Property	Description
start	Defines the first (top left) cell of the selection. This is set by specifying the following: - rowIndex: The row the selected cell is in. This value is set in relation to the first row in the worksheet. - cellIndex: The selected cell. This value is set in relation to the first cell in the rows.
end	Defines the last (bottom right) cell of the selection. This is set by specifying the following: - rowIndex: The row the selected cell is in. This value is set in relation to the first row in the worksheet. - cellIndex: The selected cell. This value is set in relation to the first cell in the rows.

These properties define the start and end cells of the selection. The `rowIndex` and `cellIndex` values are taken from the data rows and cells from the worksheet. You can use the start and end properties as the input to the [slice\(\)](#) method to limit the data analysis to only the worksheet selection.

worksheets collection

The `worksheets` collection contains worksheet objects representing each worksheet in a workbook. Each workbook contains a `worksheets` collection.

The `worksheets` collection implements the following [resource collection](#) methods.

Method	Description
get(name)	Retrieves the specified worksheet.
exists(worksheet)	Determines whether the specified worksheet exists.

get method

Retrieves a worksheet object, which allows you to manipulate the worksheet in your script. Worksheets can be used to run data modifications and retrieve data.

► Syntax

```
workbook.worksheets.get(worksheetId);
```

where *workbook* is a workbook object.

Parameter	Type	Description
worksheetId	String	The identifier of the worksheet you want to retrieve.

► Returns

Type	Description
Object	A worksheet object

► Exceptions

The `get` method can throw the following exceptions.

Exception	Description
Worksheet.NotFound	The specified worksheet does not exist in the workbook.

► Example

```
var dataModWorkbook = rapidResponse.workbooks.get( {name: "Data  
Modification", scope: "Private"}, {  
    scenarios: [{name: "myScenario", scope: "Private"}],  
    filter: {name: "Make Parts", scope: "Public"},  
    siteGroup: {name: "HQ", scope: "Public"}  
});  
var worksheet = dataModWorkbook.worksheets.get("ScheduledReceipt");
```

exists method

Determines whether a worksheet exists. However, because RapidResponse supports multiple users working with the same resources, a user might delete the specified worksheet after this method runs. If you are using this method to test whether a worksheet exists, you should still catch a `NotFound` exception for the worksheet you are using.

► Syntax

```
worksheetExists = workbook.worksheets.exists(worksheetId);
```

Parameter	Type	Description
worksheetId	String	The identifier of the worksheet you want to test for existence.

► Returns

Type	Description
Boolean	• true if the worksheet exists • false if the worksheet does not exist

► Exceptions

The `exists` method can throw the following exceptions.

Exception	Description
<code>ArgumentError</code>	The parameter is missing a <code>name</code> value.

► Example

```
var worksheetExists = workbook.worksheets.exists("OrderAnalysis");
```

Test if a worksheet exists, and if so, retrieve its data.

```
var dataWorkbook = rapidResponse.workbooks.get( {name: "Data  
Modification", scope: "Private"}, {  
    scenarios: [{name: "myScenario", scope: "Private"}],  
    filter: {name: "Make Parts", scope: "Public"},  
    siteGroup: {name: "HQ", scope: "Public"}  
});  
if (dataWorkbook.worksheets.exists("DataSummary")) {  
    var dataWorksheet = dataWorkbook.worksheets.get("DataSummary");  
    var allRows = dataWorksheet.getData();  
}
```

Worksheet object methods

Worksheet objects have the following methods.

Method	Description
<code>getAvailableHierarchies()</code>	Retrieves the valid hierarchies that can be applied to the worksheet.
<code>getChart([height, width])</code>	Retrieves a chart from the worksheet, with the specified dimensions.
<code>getChartDefinition()</code>	Retrieves the properties of a chart.
<code>getData()</code>	Retrieves the data rows and cells from the worksheet. For more information about rows and cells, see "Worksheet data objects" on page 219 .
<code>getFilterManager()</code>	Retrieves the filter manager object that contains the worksheet's filter settings.
<code>getPossibleControlValues (type, settings)</code>	Displays the valid settings for the specified type of control.
<code>importData(data)</code>	Imports the specified data. For more information about the data to import, see "Worksheet data import objects" on page 220 .
<code>convertToJSON(format, startRow, pageSize)</code>	Converts worksheet data to a JSON object that can be sent to other services and systems.

The WorksheetDataModification object has the following method.

Method	Description
execute()	Performs the data modification defined in the worksheet.

getAvailableHierarchies method

Retrieves the hierarchies that can be applied to the worksheet.

► Syntax

```
hierarchies = worksheet.getAvailableHierarchies();
```

where *worksheet* is a Worksheet object.

► Returns

An array of String values.

► Exceptions

The `getAvailableHierarchies` method does not throw exceptions.

► Example

```
var myWorkbook = rapidResponse.workbooks.get(myWorkbook);  
var myWorksheet = myWorkbook.worksheets.get("Monthly Totals");  
var hierarchies = myWorksheet.getAvailableHierarchies();
```

getChart method

Retrieves a chart from a worksheet.

► Syntax

```
worksheet.getChart([height, width]);
```

where *worksheet* is a Worksheet object.

Parameter	Type	Description
height	Number	The height of the chart, in pixels. This value must be a positive value less than or equal to 2880.
width	Number	The width of the chart, in pixels. This value must be a positive value less than or equal to 2880.



Note: If you are working with a mobile device with the screen in portrait layout, the maximum values for the height and width parameters are reversed.

► Returns

A [JsChart](#) object representing the chart.

► Exceptions

The `getChart` method can throw the following exception.

Exception	Description
ArgumentException	One of the following has occurred. • A parameter is missing or a null value was passed for a parameter. • The values passed for the height or width parameters are either not numbers, are negative, or are greater than the maximum size for the parameter.

► Example

```
var myWorkbook = rapidResponse.workbooks.get({name:"Periodic Revenue Review", scope:"Public"},
{
    scenarios: [{ name:"myScenario", scope:"Private" }],
    filter: {name: "Buy Parts", scope:"Public"},
    siteGroup: {name: "DC-Asia", scope: "Public"}
});
var myWorksheet = myWorkbook.worksheets.get("MonthlyTotals");
var chart = myWorksheet.getChart(800, 1400);
```

getChartDefinition method

Retrieves a chart definition object.

► Syntax

```
chartDefinition = worksheet.getChartDefinition();
```

where *worksheet* is a Worksheet object.

► Returns

A chart definition object.

► Exceptions

The `getChartDefinition` method does not throw exceptions.

► Example

```
var myWorkbook = rapidResponse.workbooks.get({name:"Periodic Revenue Review", scope:"Public"},
{
    scenarios: [{ name:"myScenario", scope:"Private" }],
    filter: {name: "Buy Parts", scope:"Public"},
    siteGroup: {name: "DC-Asia", scope: "Public"}
});
var myWorksheet = myWorkbook.worksheets.get("MonthlyTotals");
var chartDef = myWorksheet.getChartDefinition();
```

getData method

Gets the data from a worksheet.

► Syntax

```
worksheetData = worksheet.getData();
```

where *worksheet* is a worksheet object.

► Returns

Type	Description
Object	The worksheet data object that contains the worksheet's data row collection.

► Exceptions

The `getData` method can throw the following exceptions.

Exception	Description
Scenario.NotFound	The scenario specified for the worksheet does not exist.
Filter.NotFound	The filter specified for the worksheet does not exist.
SiteGroup.NotFound	The site or site filter specified for the worksheet does not exist.
Model.NotFound	The model specified for the worksheet does not exist.
Pool.NotFound	The pool specified for the worksheet does not exist.
Hierarchy.NotFound	A hierarchy specified for the worksheet does not exist.
Variable.NotFound	A variable specified for the worksheet does not exist.

► Example

This example gets data for make parts from the HQ site.

```
var dataWorkbook = rapidResponse.workbooks.get( {name: "Supply
Report", scope: "Public"},
{
    scenarios: [{name: "myScenario", scope: "Private"}],
    filter: {name: "Make Parts", scope: "Public"},
    siteGroup: {name: "HQ", scope: "Public"}
});
var worksheet = dataWorkbook.worksheets.get("ScheduledReceipt");
var data = worksheet.getData();
```

getFilterManager method

Retrieves the FilterManager object that defines the worksheet's filter settings.

► Syntax

```
worksheetFilter = worksheet.getFilterManager();
```

where *worksheet* is a Worksheet object.

► Returns

A [FilterManager](#) object.

► Exceptions

The `getFilterManager` method does not throw exceptions.

► Example

```
var myWorkbook = rapidResponse.workbooks.get({name:"Periodic Revenue
Review", scope:"Public"},
{
    scenarios: [{ name:"myScenario", scope:"Private" }],
    filter: {name: "Buy Parts", scope:"Public"},
    siteGroup: {name: "DC-Asia", scope: "Public"}
});
var myWorksheet = myWorkbook.worksheets.get("MonthlyTotals");
var filterSetting = myWorksheet.getFilterManager();
```

getPossibleControlValues method

Returns all valid options for a worksheet control setting, based on the data settings that are valid for a widget based on the worksheet.

► Syntax

```
worksheet.getPossibleControlValues(type, controlSettings);
```

where *worksheet* is a Worksheet object.

Parameter	Type	Description
type	String	The type of control setting the valid values are retrieved for. This can be one of the following: • scenario • filter • siteGroup • part • model • pool • project • referencePart • workCenter • constraint
controlSettings	Object	The ControlSettings object that defines the data displayed in the worksheet.

► Returns

An array of objects that are valid for the control values.

► Exceptions

The `getPossibleControlValues` method can throw the following exceptions.

Exception	Description
ArgumentException	One of the following has occurred. • A parameter value is missing or a null value was specified. • The value specified for the <code>type</code> parameter is not a valid control type. • The value specified for the <code>controlSettings</code> parameter is not a <code>ControlSettings</code> object

► Example

This example lists scenarios that can be used with the worksheet the Monthly Revenue widget is based on.

```
var widget = rapidResponse.widgets.get({name:"Monthly Revenue",
scope:"Public"});
var widgetWorkbook = rapidResponse.workbooks.get
(widget.settings.workbookKey);
var widgetWorksheet = widgetWorkbook.worksheets.get
(widget.settings.worksheetId);
var settings = widgetWorksheet.getPossibleControlValues("scenario",
widget.settings.controlSettings);
```

```
var names = "";
for (var i=0; i<settings.length; i++){
    names = names + settings[i].name+ "\n";
}
return names;
```

getTreemapQuery method

This method retrieves the treemap defined on a worksheet.

This method was added in RapidResponse 2014.2.

► Syntax

```
treemap = worksheet.getTreemapQuery();
```

where *worksheet* is a Worksheet object.

► Returns

A [TreemapQuery](#) object.

► Exceptions

The `getTreemapQuery` method can throw the following exceptions.

Exception	Description
ArgumentException	An argument was provided for the method.
Worksheet.NotFound	The worksheet does not exist.

► Example

```
var dataWorkbook = rapidResponse.workbooks.get( {name: "Data Display",
scope: "Private"}, {
    scenarios: [{name: "myScenario", scope: "Private"}],
    filter: {name: "Make Parts", scope:"Public"},
    siteGroup: {name: "HQ", scope: "Public"}
});
var worksheet = dataWorkbook.worksheets.get("SupplyMap");
var treemap = worksheet.getTreemapQuery();
```

importData method

Imports data into the specified worksheet. This method requires different parameters depending on the type of worksheet you are importing data into.

The worksheet specified must be designed for allowing data imports. For more information, see the [RapidResponse Resource Authoring Guide](#).

► Syntax

```
worksheet.importData(importData, forceUpdate);
```

where *worksheet* is the worksheet object to import data into.

Parameter	Type	Description
importData	Object	The importData object that defines the data to import into the worksheet. Depending on the worksheet type, this object must contain the following: - bucketing: defines the worksheet bucket dates and bucketing calendar. Each bucket date in the worksheet must be included in this parameter. Bucket dates can be specified as an array of Dates. This value is required only for crosstab worksheets. - data: defines the data values imported into each worksheet column or bucket. This is represented as an array of arrays, with each array representing one row of worksheet data. This can also be an array of worksheetDataRow objects.
forceUpdate	Boolean	Optionally, whether the scenario is updated before the import operation begins.

► Return value

The `importData` method returns an `importResults` object, which has the following properties.

Property	Type	Description
inserted	Number	The number of records inserted by the import operation.
modified	Number	The number of records modified by the import operation.
deleted	Number	The number of records deleted by the import operation.

► Exceptions

The `importData` method can throw the following exceptions.

Exception	Description
ArgumentError	A null value, incorrect data type for a column, or an invalid date was specified.

► Examples

Example of importing data into a crosstab worksheet. This example uses a crosstab worksheet with four rows, two dimension columns, and five monthly buckets. The script changes the value in the third bucket to 6000 for each row. All other values are unchanged.

```
var worksheetImport = workbook.worksheets.get  
("ImportCrosstabWorksheet");  
var worksheetData = worksheetImport.getData();  
var result = [];
```

```

var rowResultArray = [];
for(var i = 0; i < 4; i++ ) { //Copy worksheet rows into array
    result = [];
    for(var j = 0; j < 7; j++ ) {
        result[j] = worksheetData.rows[i].cells[j].value;
    }
    result[4] = 6000; //Third bucket contains 6000 for each row
    rowResultArray[i] = result;
}
var buckets = [new Date(2012, 5, 1), new Date(2012, 6, 2), new Date(
    2012, 7, 1),
    new Date(2012, 8, 3), new Date(2012, 9, 1)]; //Monthly buckets for
June to October
var importData = {
    bucketing: {
        buckets: buckets, calendarName: "Month"
    },
    data: rowResultArray
};
var importResult = worksheet.importData(importData);

```

Example of importing data into a vertical worksheet, and updating the scenario before importing the data.

```

var worksheetImport = workbook.worksheets.get("ImportWorksheet");
var importData = [
    ["HNGH-3k3", 44, "Past", "PCW-1295", "Asia"],
    ["UCND-32k", 16, "Future", "PCW-1295", "Asia"]];
var importResult = worksheetImport.importData(importData, true);

```

convertToJSON method

Converts a worksheet's data to a JSON string that can be sent to external systems and services. You can specify the formatting applied to this string to make it fit the required input for the system you are sending it to. Typically, you would use the return value from this method as one of the inputs to an endpoint, as described in ["invokeEndpoint method" on page 429](#).

Depending on your requirements, you can format each row of data as an object, an array, or a concatenated string with a delimiter between values. You can also optionally include a name for the row entries, if your system requires it.

► Syntax

```
worksheetData.convertToJSON(startRow, pageSize, format)
```

where *worksheetData* is a *WorksheetData* object.

Argument	Data Type	Description
startRow	Number	The first worksheet row to include in the JSON output.
pageSize	Number	The number of rows to include in the output.
format	Object	Defines the formatting applied to the JSON string, including the name for the object that represents the data, how to format the rows, and a delimiter character for separating values, if necessary.

The object that defines the formatting has the following properties:

Property	Description
rowsObjectName	The name for the entire data object in the JSON output. This is used to identify the data in the endpoint call and should be set according to the expected input for the service or system you send data to. The data is output as an array of values formatted according to the settings specified in the other properties of this object. Default value: "Rows"
includeRowName	Whether each row is represented as a separate object in the output. If false, the row values are presented as an array of values. Default value: true
rowName	A name to identify each row of data in the output. This value is ignored if <code>includeRowName</code> is false. Default value: "Values"
rowOutputFormat	Specifies how the worksheet rows are represented. Regardless of your choice, a value is included for each column in the worksheet. This must be one of the following: <code>AsArray</code>: Defines each row as an array of column values. <code>AsDelimitedString</code>: Defines each row as a single String value, with column values separated by a configurable delimiter character. <code>AsObject</code>: Defines each row as an object consisting the column values in name:value pairs. Default value: <code>AsArray</code>
delimiter	Defines the delimiter character used to separate column values if <code>rowOutputFormat</code> is set to <code>AsDelimitedString</code>. This must be a single character, or the equivalent to a single character. For example, if you specify <code>"\t"</code> as the delimiter, a tab character is inserted between the column values. Any instances of the delimiter character in the column values are automatically escaped in the output. Default value: <code>","</code>

► Return value

Returns a JSON object that contains the worksheet values formatted according to the rules you specified.

► Exceptions

The `convertToJSON` method can throw the following

Exception	Description
<code>ArgumentError</code>	An argument is missing, or an invalid value was presented for the <code>rowOutputFormat</code> or <code>delimiter</code> arguments.

► Examples

The following example retrieves the first five records from a worksheet that contains part data (name, site, primary part source, and description) and converts it into a JSON string with no row identifier and column values represented as entries in an array.

```
var workbook = rapidResponse.workbooks.get({name:"Part
Data",scope:"Private"},{
    scenarios:[{name:"Enterprise Data",scope:"Public"}],
    filter:{name:"All Parts", scope:"Public"},
    siteGroup:{name:"Dayton", scope:"Public"}
});
var sheet = workbook.worksheets.get("Part");
return sheet.getData().convertToJSON(0, 5, {
    rowsObjectName:"AllRows",
    includeRowName: false,
    rowOutputFormat:"AsArray",
});
```

This returns the following JSON string:

```
{"AllRows":[
  ["C-FRAME-STD","Dayton","Yield150","Cruiser Frame"],
  ["FR-0101","Dayton","Yield277","Main Frame Subassembly"],
  ["FR-0101A","Dayton","Yield287","Racer Main Frame Sub, Steel"],
  ["FR-0101B","Dayton","Yield292","Racer Main Frame Sub,
  Aluminum"],
  ["FR-0101C","Dayton","Yield297","Racer Main Frame Sub,
  Titanium"]
]}
```

In this example, the rows are an array with each row represented as an array of values.

If this example included the row names, each row would be represented as an object consisting of the row label and the array in a name:value pair. For example:

```
{"Values":["C-FRAME-STD","Dayton","Yield150","Cruiser Frame"]}
```

The following example returns the same set of worksheet records as the previous example, with each row represented as an object.

```
var workbook = rapidResponse.workbooks.get({name:"Part
Data",scope:"Private"},{
  scenarios:[{name:"Enterprise Data", scope:"Public"}],
  filter:{name:"All Parts", scope:"Public"},
  siteGroup:{name:"Dayton",scope:"Public"}
});
var sheet = workbook.worksheets.get("Part");
return sheet.getData().convertToJSON(0, 5, {
  rowName:"Row",
  rowOutputFormat:"AsObject"
});
```

This returns the following JSON string:

```
{"Rows":[
  {"Row":{"Name":"C-FRAME-STD","Site":"Dayton","PrimarySource":"Yield150","Description":"Cruiser Frame"}},
  {"Row":{"Name":"FR-0101","Site":"Dayton","PrimarySource":"Yield277","Description":"Main Frame Subassembly"}},
  {"Row":{"Name":"FR-0101A","Site":"Dayton","PrimarySource":"Yield287","Description":"Racer Main Frame Sub, Steel"}},
  {"Row":{"Name":"FR-0101B","Site":"Dayton","PrimarySource":"Yield292","Description":"Racer Main Frame Sub, Aluminum"}},
  {"Row":{"Name":"FR-0101C","Site":"Dayton","PrimarySource":"Yield297","Description":"Racer Main Frame Sub, Titanium"}}
]}
```

In this example, the rows are an array of objects, with each row's data represented as an object with the column identifier and value in name:value pairs.

If this example did not include the row names, each row would be represented as an object in an array. For example:

```
{ "Rows": [{ "Name": "C-FRAME-STD", "Site": "Dayton", "PrimarySource": "Yield150", "Description": "Cruiser Frame" }, ...
```

The following example returns the same set of worksheet records, with the rows represented as a single string and values delimited by commas.

```
var workbook = rapidResponse.workbooks.get({name:"Part Data",scope:"Private"},{
    scenarios:[{name:"Enterprise Data", scope:"Public"}],
    filter:{name:"All Parts", scope:"Public"},
    siteGroup:{name:"Dayton",scope:"Public"}
});
var sheet = workbook.worksheets.get("Part");
return sheet.getData().convertToJSON(0, 5, {
    rowsObjectName:"AllRows",
    includeRowName:false,
    rowOutputFormat:"AsDelimitedString"
});
```

This returns the following JSON string:

```
{ "AllRows": ["C-FRAME-STD,Dayton,Yield150,Cruiser Frame","FR-0101,Dayton,Yield277,Main Frame Subassembly","FR-0101A,Dayton,Yield287,Racer Main Frame Sub\\, Steel","FR-0101B,Dayton,Yield292,Racer Main Frame Sub\\, Aluminum","FR-0101C,Dayton,Yield297,Racer Main Frame Sub\\, Titanium"] }
```

In this example, the rows are an array of values, with each row represented as a single string value in the array. Commas in the column values are escaped so they are represented properly in the results.

If this example included the row names, each row would be represented as an object consisting of the row label and the delimited string in a name:value pair. For example,

```
{ "Values": "FR-0101A,Dayton,Yield287,Racer Main Frame Sub\\, Steel" }
```

The following example returns the same set of worksheet records as the previous example, using default values for the row names and pipe (|) delimited string instead.

```
var workbook = rapidResponse.workbooks.get({name:"Part Data",scope:"Private"},{
    scenarios:[{name:"Enterprise Data",scope:"Public"}],
    filter:{name:"All Parts", scope:"Public"},
    siteGroup:{name:"Dayton", scope:"Public"}
});
var sheet = workbook.worksheets.get("Part");
return sheet.getData().convertToJSON(0, 5, {
    rowOutputFormat:"AsDelimitedString",
```

```

        delimiter:"|"
    }
);

```

This returns the following JSON string:

```

{"Rows": [
  {"Values": "C-FRAME-STD|Dayton|Yield150|Cruiser Frame"},
  {"Values": "FR-0101|Dayton|Yield277|Main Frame Subassembly"},
  {"Values": "FR-0101A|Dayton|Yield287|Racer Main Frame Sub, Steel"},
  {"Values": "FR-0101B|Dayton|Yield292|Racer Main Frame Sub, Aluminum"},
  {"Values": "FR-0101C|Dayton|Yield297|Racer Main Frame Sub, Titanium"}
]}

```

The following example returns the same set of worksheet records as the previous examples, using the default formatting values.

```

var workbook = rapidResponse.workbooks.get({name:"Part Data",scope:"Private"},{
  scenarios:[{name:"Enterprise Data",scope:"Public"}],
  filter:{name:"All Parts", scope:"Public"},
  siteGroup:{name:"Dayton", scope:"Public"}
});
var sheet = workbook.worksheets.get("Part");
return sheet.getData().convertToJSON(0,5);

```

This returns the following JSON string:

```

{"Rows": [
  {"Values": ["C-FRAME-STD", "Dayton", "Yield150", "Cruiser Frame"]},
  {"Values": ["FR-0101", "Dayton", "Yield277", "Main Frame Subassembly"]},
  {"Values": ["FR-0101A", "Dayton", "Yield287", "Racer Main Frame Sub, Steel"]},
  {"Values": ["FR-0101B", "Dayton", "Yield292", "Racer Main Frame Sub, Aluminum"]},
  {"Values": ["FR-0101C", "Dayton", "Yield297", "Racer Main Frame Sub, Titanium"]}
]}

```

Using only the default formatting creates a JSON string that matches the format required by the RapidResponse bulk data endpoints to read data. For more information, see the [RapidResponse Web Services Guide](#).



Note: This method was added in Service Update 2111.

columns collection

The columns collection contains all columns defined in a worksheet, represented as [WorksheetColumn](#) objects.

The columns collection implements only the collection iteration functions. For more information, see "[Resource collections](#)" on page 110.

WorksheetColumn object

The WorksheetColumn object defines a column in the worksheet. Columns are collected in the `columns` collection.

The WorksheetColumn object has the following properties.

Property	Type	Description
columnId	String	The column identifier.
drillToDetails	Object	A collection of WorksheetColumnDrillToDetail objects that represent the drill to details links defined for the column.
groupRule	String	The grouping rule applied to the column.
header	String	The column header displayed in the worksheet.
hidden	Boolean	Whether the column is hidden.
hideZeroValues	Boolean	Whether zero values are hidden if the column contains a quantity value.
imageMap	Object	A collection of WorksheetColumnImageMapping objects that represent the images displayed in the column.
width	Integer	The column's width in pixels.

imageMap collection

The imageMap collection contains all image mappings defined for a column, represented as [WorksheetColumnImageMapping](#) objects.

The imageMap collection has the following property.

Property	Type	Description
count	Integer	The number of image mappings in the collection.

The `imageMap` collection implements only the collection iteration functions. For more information, see ["Resource collections" on page 110](#).

WorksheetColumnImageMapping object

The `WorksheetColumnImageMapping` object defines images that can display in a column. These image mappings are contained in the [imageMap](#) collection.

The `WorksheetColumnImageMapping` object contains the following properties.

Property	Type	Description
<code>dataValue</code>	String	The value that an image is mapped for.
<code>image</code>	String	A base64-encoded String that represents an image file. This string can be displayed as an image in an HTML document.
<code>imageName</code>	String	The image name as displayed in the worksheet properties.
<code>index</code>	Integer	The image mapping's index in the collection.
<code>toolTip</code>	String	The tooltip displayed when a user pauses the mouse pointer over the image.

Worksheet data collections

Worksheet data is contained in collections of worksheet data rows and cells.

The `rows` collection contains the worksheet's data rows, which are represented as an index of [WorksheetDataRow](#) objects. The `rows` collection has the following properties.

Property	Type	Description
<code>count</code>	Number	The total number of rows in the array.

The cells within each row are contained in the `cells` collection, which contains an index of [WorksheetDataCell](#) objects. These objects contain the data values in each worksheet column for a specific row.

Worksheet row and cell objects

Worksheet rows and cells are represented by the `WorksheetRow` and `WorksheetCell` objects. The `WorksheetRow` object contains the following properties.

Property	Type	Description
isPivoted	Boolean	Whether the row is a crosstab row.
recordId	String	The identifier for the record in the row.
number	Number	The row number.
pivotedColumnIndex	Number	For a crosstab row, the index of the crosstab row.
cells	Object	A collection of WorksheetCell objects.
pivotedColumnId	String	The identifier for a crosstab row.
type	String	The column type.

The WorksheetCell object contains the following properties.

Property	Type	Description
formattedValue	String	The value in the cell with all formatting applied.
isTransposedHeader	Boolean	Whether the cell contains a crosstab row header.
width	Number	The width of the cell, in pixels.
columnId	String	The column identifier.
isCrosstab	Boolean	Whether the cell is a crosstab data cell.
isPivotedandBucketed	Boolean	Whether the cell is used to define a crosstab worksheet's date buckets.
value	String	The data value in the cell.
dataAlignment	String	How the cell data is aligned.
dataType	String	The type of data in the cell.
style	Object	A WorkbookStyle object that defines the font formatting and color for the cell.

The WorkbookStyle object contains the following properties.

Property	Type	Description
fontStyle	String	The style applied to the font in a worksheet cell.
foregroundColor	String	The color applied to the font in a cell.
backgroundColor	String	The color displayed in the background of a cell.

Worksheet data objects

Worksheet data returned by a script is represented as a `WorksheetData` object, which contains the rows of data from the worksheet, and the individual data cells in each row. The rows are represented by the `rows` collection, which contains `WorksheetDataRow` objects, and the cells by the `cells` collection, which contains `WorksheetDataCell` objects. For information about the `rows` and `cells` collections, see ["Worksheet data collections" on page 217](#).

The `WorksheetData` object has the following properties.

Property	Type	Description
<code>rows</code>	Object	The worksheet data row collection representing the rows in the worksheet. Each row in the collection contains values for each cell in that row. The rows and cells are represented by <code>WorksheetDataRow</code> and <code>WorksheetDataCell</code> objects. For more information, see "Worksheet data collections" on page 217 and "Worksheet data objects" on page 219 .
<code>status</code>	String	Describes how the data was retrieved. This can be one of the following values: • <code>OK</code>: The data was retrieved successfully. • <code>MemoryLimitReached</code>: Data could not be retrieved because the data set consumed too much memory. • <code>Cancelled</code>: Data retrieval was cancelled or interrupted.

The `WorksheetDataRow` object has the following properties.

Property	Type	Description
<code>index</code>	Number	A row's position in the collection. The index starts at zero.
<code>number</code>	Number	The row number of a row in the collection. The row numbers start at one.

Property	Type	Description
type	String	The type of data in the row. This can be one of the following: • Normal: The data from a vertical (tabular) worksheet. Each cell in the row can have a different data type, depending on the columns in the worksheet. • Subtotal: The data from a subtotal or grand total row in a vertical worksheet. Each cell contains the sum, average, maximum, or minimum of all values in the column. • TimeBasedSubtotal: The sum or average of values over a period of time. • Crosstab: The data from a crosstab (horizontal) worksheet. Each cell in the row represents data values for a single column over a period of time.
cells	Object	A collection of worksheet data cell objects representing the data in each cell of the row.

The WorksheetDataCell object has the following properties.

Property	Type	Description
index	Number	A cell's position in the array. The index starts at zero.
value	Number String Date Boolean	The data in the cell. The data type of the <code>value</code> property depends on the data in the worksheet cell.

Worksheet data import objects

Worksheet data imported into a RapidResponse worksheet is represented using an ImportData object. Depending on the type of worksheet you are importing into, different properties of this object are required.

Property	Type	Description
bucketing	Object	The crosstab worksheet date buckets that data is imported into.
data	Object	An array of data values imported into the worksheet.

The bucketing property refers to the Bucketing object, which has the following properties.

Property	Type	Description
buckets	Object	An array of dates representing the worksheet's bucket settings. You can create an array of Date objects by specifying the year, month, and day for each bucket. Months begin with zero and days begin with one. For example, to define a bucket for January 2, 2012, specify <code>new Date(2012, 0, 2)</code>. For more information about specifying buckets, see "importData method" on page 208.
calendar	String	The worksheet's bucketing calendar.

Worksheet data methods

The `WorksheetData` object has the following methods.

Method	Description
close()	Closes the worksheet query.

The `rows` collection has the following method.

Method	Description
slice(start [, end])	Creates a new worksheet data row collection that contains a subset of another row collection, specified by the start and optional end parameters.

The `cells` collection has the following method.

Method	Description
get(columnId)	Retrieves the value in the column with the specified column identifier. You can use this method to retrieve data from a specific column regardless of its position in the worksheet.

Worksheet data values can be retrieved using the worksheet's [getData\(\)](#) method and then specified by row and cell index, or by row index and column identifier. For example, you can use the following code to determine the value in the fourth cell of the fifth row of a worksheet's data.

```
var worksheet = workbookName.worksheets.get("worksheetName");
var myValue = worksheet.getData().rows[4].cells[3].value;
```

get method

Retrieves a column from the worksheet data `cells` collection. The specified column identifier maps to the column's index in the collection.

► Syntax

```
column = worksheetRow.cells.get(columnId);
```

where *worksheetRow* is a [WorksheetDataRow](#) object.

Argument	Type	Description
columnId	String	The identifier for the column you want to retrieve.

► Returns

A [WorksheetDataCell](#) object.

► Exceptions

The `get` method can throw the following exceptions.

Exception	Description
ArgumentError	Either a blank or null value was specified for the <code>columnId</code> argument, or more than one argument was provided.
Workbook.Worksheet.Column.NotFound	The specified column does not exist in the worksheet.

► Example

```
var row = worksheet.getData().rows[0];
worksheet.columns.forEachId(
    function (columnId) {
        rapidResponse.console.writeLine("column: " + columnId + ",
            value: " + row.cells.get(columnId).value );
    }
);
```

slice method

Creates a new worksheet data row collection based on an existing collection. The new collection contains a subset of the rows in the original collection.

► Syntax

```
rowCollection = worksheetDataRows.slice(start[, end]);
```

where *worksheetDataRows* is a row collection.

Parameter	Type	Description
start	Number	The first row index to include in the new row collection. This value can be set relative to the end of the collection by using a negative value. For example, if <code>start</code> is <code>-20</code> , the new collection begins with the 20th row from the end of the original collection.
end	Number	Optionally, the last row index to include in the new array. Rows up to, but not including, this row are added to the new data row collection. This value can be set relative to the end of the collection by using a negative value. For example, if <code>end</code> is <code>-10</code> , the new collection ends with the 10th row from the end of the original collection. If this value is not specified, the last row in the original collection is used.

► Returns

Type	Description
Object	A worksheet data row collection

► Exceptions

The `slice` method does not throw exceptions.

► Example

```
var allRows = worksheet.getData().rows;
var firstHalf = allRows.slice(0, allRows.count / 2);
```

close method

Closes a worksheet query after it has finished running. A closed query is removed from memory and cannot be referenced, so this method should be the last one called when processing worksheet data.

► Syntax

```
worksheetData.close()
```

where `worksheetData` is a worksheet data row collection object.

► Returns

No return value.

► Exceptions

The `close` method does not throw exceptions.

► Example

```
var worksheetData = worksheet.getData().rows;
worksheetData.close();
```

WorksheetDataModification object

The WorksheetDataModification object defines the automatic data modification defined in the worksheet, and has the following property.

Property	Type	Description
type	String	The type of modification performed. This value can be "Insert", "Modify", "Delete", or "Update".

The WorksheetDataModification object has the following method

Method	Description
execute()	Performs the worksheet's data modification.

execute method

Runs the data modification defined in the specified worksheet.

► Syntax

```
worksheet.dataModification.execute( [scenario] );
```

where *worksheet* is a worksheet object and *dataModification* is the data modification property from that worksheet.

Parameter	Type	Description
scenario	Object	For insert and update modifications, optionally specifies the scenario the source data is taken from. If no scenario is specified, the scenario specified in the workbook's initialization object is used. This can be specified as a variable or a scenario object using the { name : , scope : } syntax.

► Returns

For reporting purposes, this method returns an object with the following properties.

Property	Type	Description
deleted	Number	The number of records deleted by the modification.
inserted	Number	The number of records inserted by the modification.

Property	Type	Description
modified	Number	The number of records modified by the modification.

The properties populated by this object depend on the type of the data modification. For example, a data modification of type Delete populates only the `deleted` property. An Update data modification can populate all three of the properties.

The data is modified regardless of whether you report the number of records affected.

► Exceptions

The `execute` method can throw the following exceptions.

Exception	Description
Worksheet.DataUpdateError	An error occurred when modifying the data.

► Example

To run a worksheet's data modification:

```
var worksheet = workbook_DataModification.worksheets.get(
  "ScheduledReceipt");
worksheet.dataModification.execute();
```

To report the number of records deleted by a worksheet's data modification:

```
var nDeleted = 0;
var worksheet = workbook_DataModification.worksheets.get(
  "ScheduledReceipt");
nDeleted = worksheet.dataModification.execute().deleted;
```

To update records from another scenario:

```
var worksheet = workbook_DataModification.worksheets.get("Move to
Stock");
worksheet.dataModification.execute( {name:"Supply update",
scope:"Private"} );
```

Worksheet bucketing objects

The `BucketSettings` object defines the buckets applied to a worksheet. This object has the following properties.

Property	Type	Description
type	String	The type of bucketing the worksheet uses. This value can be either "Basic" or "Advanced".
settings	Object	A BasicBucketSettings or AdvancedBucketSettings object that defines the number of buckets and bucketing calendar used in the worksheet.

BasicBucketSettings object

For a worksheet that uses basic bucketing, the BasicBucketSettings object defines the bucket calendar and number of buckets displayed in the worksheet. This object contains the following properties.

Property	Type	Description
bucketStartDate	Object	A BucketStartDate object that defines the worksheet's first bucket added before the worksheet's anchor date. This property was added in RapidResponse 2013.4.
bucketStartDateEnabled	Boolean	Whether the start date is displayed. This property was added in RapidResponse 2013.4.
bucketStartDateModifiable	Boolean	Whether worksheet users can modify the start date. This property was added in RapidResponse 2013.4.
intervals	Object	A BucketIntervals object that defines the buckets in the worksheet.
lockBucketing	Boolean	Whether worksheet users can modify the bucket settings. This property was added in RapidResponse 2013.4.
options	Object	A BucketOptions object that defines the bucket options used in the worksheet.

AdvancedBucketSettings object

For a worksheet that uses advanced bucket settings, the AdvancedBucketSettings object defines the bucket settings used in the worksheet. This object contains the following properties.

Property	Type	Description
anchorDate	Object	A BucketAnchorDate object that defines the bucket that contains the anchor date.
calendarToDateSubtotals	Object	An array of BucketCalendarToDate objects that define the calendar-to-date subtotals defined for the buckets, if present. This array can contain a maximum of two entries. This property was added in RapidResponse 2014.2.
includePastBucket	Boolean	Whether the Past bucket is included, which contains all values earlier than the earliest bucket in the worksheet.
includeFutureBucket	Boolean	Whether the Future bucket is included, which contains all values later than the latest bucket in the worksheet.
intervals	Object	A BucketIntervals object that defines the buckets in the worksheet.
lockBucketing	Boolean	Whether worksheet users can modify the bucket settings. This property was added in RapidResponse 2013.4

BucketInterval object

The BucketInterval object defines the intervals used in a bucketed worksheet. This object contains the following properties.

Property	Type	Description
bucketCalendar	String	The calendar used for bucketing the worksheet.
numberOfBuckets	Number	How many buckets of this size are displayed in the worksheet.
direction	String	Whether the buckets are defined before or after the anchor date.
useSubtotalCalendar1	Boolean	Whether the first subtotal calendar is displayed in the worksheet.
useSubtotalCalendar2	Boolean	Whether the second subtotal calendar is displayed in the worksheet.
useSubtotalCalendar3	Boolean	Whether the third subtotal calendar is displayed in the worksheet.
useSubtotalCalendar4	Boolean	Whether the fourth subtotal calendar is displayed in the worksheet.

BucketOption object

The BucketOption object defines the properties of a bucket. This object contains the following properties.

Property	Type	Description
isDefault	Boolean	Whether this set of bucketing options are the default bucket size and number of buckets for the worksheet.
numberOfBuckets	Number	The number of buckets displayed in the worksheet.
bucketCalendar	String	The calendar used to define the buckets.
subtotalCalendar	String	The calendar used to define subtotals.
index	Number	The bucket option's index.
id	String	Identifies the set of bucket options.

BucketAnchorDate object

The BucketAnchorDate object defines the worksheet's anchor date, which is the point that divides planning data from historical data. This object contains the following properties.

Property	Type	Description
type	String	The type of anchor date the worksheet uses. This can be one of the following values: • Planning: The planning date is used to define the anchor date. • Today: The current date is used as the anchor date. • Current: The beginning of a current calendar period is used as the anchor date.
offset	Object	A BucketDateOffset object that defines the date offset for the anchor date.
currentCalendar	String	The calendar used to define the anchor date
rawDate	Date	The anchor date.
index	Number	The anchor date bucket's index
id	String	The anchor date bucket's identifier.

BucketDateOffset object

The BucketDateOffset object defines the date and date offset used for defining a worksheet's anchor date or start date relative to another date. This object contains the following properties.

Property	Type	Description
adjustment	Number	The number of periods to adjust the date by.
calendar	String	The calendar used to adjust the date.
index	Number	The index of the date cell.
id	String	The identifier of the date cell.



Note: This object was renamed in RapidResponse 2013.4, and was named BucketAnchorDateOffset in RapidResponse 2013.2 and earlier.

BucketStartDate object

The BucketStartDate object defines the start date of basic bucketing displayed before the worksheet's anchor date. The BucketStartDate contains the following properties.

Property	Type	Description
calendar	String	The calendar used to define the start date bucket.
offset	Object	The BucketDateOffset object that defines how the date is offset.
type	String	The type of anchor date the worksheet uses. This can be one of the following values: • Planning: The planning date is used to define the anchor date. • Today: The current date is used as the anchor date. • Current: The beginning of a current calendar period is used as the anchor date.



Note: This object was added in RapidResponse 2013.4.

BucketCalendarToDate object

The BucketCalendarToDate object describes the calendar-to-date subtotals contained in the worksheet bucketing. This object contains the following properties.

Property	Type	Description
balanceBucketLabel	String	The label of the bucket that displays the subtotal for the remainder of the year.
bucketLabel	String	The label of the bucket that contains the calendar-to-date subtotal.
calendar	String	The calendar that defines the to-date subtotal.

Property	Type	Description
origin	String	The date that the calendar-to-date subtotal is calculated to. This can be one of the following values: • Today • Anchor Date • Planning Date
originCalendar	String	The calendar that the origin value is in. The calendar-to-date subtotal is calculated up to this period.



Note: This object was added in RapidResponse 2014.2.

AutobucketInfo object

The AutobucketInfo object defines the automatically-generated bucket settings for a crosstab worksheet. This object contains the following properties.

Property	Type	Description
includeFutureBucket	Boolean	Whether the worksheet includes the Future bucket.
includePastBucket	Boolean	Whether the worksheet includes the Past bucket.
buckets	Object	The DateBucket objects that define the worksheet's buckets.
autobucketColumnId	String	The column identifier of the column that is used to define the worksheet buckets.
allowPartialTimeSubtotals	Boolean	Whether the subtotals displayed in the worksheet can contain partial buckets.
index	Number	The index of the bucket column.
id	String	The identifier of the bucket column.

DateBucket object

The DateBucket object defines the beginning and end dates of a bucket.

Property	Type	Description
end	Date	The date the bucket ends on.
start	Date	The date the bucket begins on.

Property	Type	Description
index	Number	The index of the date bucket.
id	String	The identifier of the date bucket.

WorksheetColumnDrillToDetail object

The WorksheetColumnDrillToDetail object defines links between worksheets or between worksheets and forms. This object contains the following properties.

Property	Type	Description
condition	Object	A WorksheetColumnDrillToDetailCondition object that defines how the drill to details link in the worksheet works.
customLabel	String	The label displayed in the workbook link menu.
drillTarget	String	The worksheet or form object the drill to details link opens. For
labelSource	String	Where the label comes from.
columnMappings	Object	A collection of DrillToDetailColumnMapping objects that define how the columns in the source worksheet are mapped to the detail worksheet. This property was modified in RapidResponse 2014.2.
dependencyId	String	The identifier of the workbook dependency this drill to details link represents.
bucketVariableMappings	Object	A collection of WorksheetColumnDrillToDetailBucketVariableMapping objects that define how date buckets in the source worksheet map to a date column in the detail worksheet.
isTargetWorksheetTransformation	Boolean	Whether the target detail worksheet is a transformation worksheet.
isTargetWorksheetTreemap	Boolean	Whether the target detail worksheet is a treemap worksheet.
label	String	The label displayed for the drill to details link.

The WorksheetColumnDrillToDetailCondition object contains the following properties.

Property	Type	Description
value	String	The column data value that defines the link.
columnId	String	The worksheet column the link is defined in.

columnMappings collection

The columnMappings collection contains all drill to details links defined in a worksheet, represented as [DrillToDetailColumnMapping](#) objects.

The columnMappings collection implements only the collection iteration functions. For more information, see "[Resource collections](#)" on [page 110](#).



Note: This collection was added in RapidResponse 2014.2.

DrillToDetailColumnMapping object

The DrillToDetailColumnMapping object defines the values mapped from one column to another by a drill to details link in a worksheet. The drill to details links for a column are contained in the [drillToDetailColumnMappings](#) collection.

The DrillToDetailColumnMapping object has the following properties.

Property	Type	Description
index	Number	The column mapping's position within the collection.
sourceId	String	The column or variable the value is taken from.
targetId	String	The column or variable the value is mapped to.



Note: This object was added in RapidResponse 2014.2 and replaced the [WorksheetColumnDrillToDetailColumnMapping](#) object.

WorksheetColumnDrillToDetailBucketVariableMappings object

The WorksheetColumnDrillToDetailBucketVariableMappings object defines the mapping between a date bucket in a source worksheet and the date range in a detail worksheet. The WorksheetColumnDrillToDetailBucketVariableMappings object has the following properties.

Property	Type	Description
bucketStartDate	Date	The first date in the bucket.
bucketEndDate	Date	The last date in the bucket.

WorksheetColumnDrillToDetailColumnMapping object

The WorksheetColumnDrillToDetailColumnMapping object defines how a column in a source worksheet maps to a column in a detail worksheet. The value in the source column is used as a column search on the detail worksheet. The WorksheetColumnDrillToDetailColumnMapping object has the following properties.

Property	Type	Description
sourceId	String	The column identifier of the column in the source worksheet.
targetId	String	The column identifier of the column in the detail worksheet.



Note: This object was replaced by the [DrillToDetailColumnMapping](#) object in RapidResponse 2014.2.

DrillTarget object

The DrillTarget object defines the target of a drill to details link. This object contains the following properties.

Property	Type	Description
id	String	The worksheet or form that is the target of the drill.
type	String	The type of drill. This value is DrilltoWorksheet or OpenForm.

CHAPTER 14: TreemapQuery object

TreemapQueryColorMeasure object	236
TreemapQueryDimension object	236
dimensions collection	236
Treemap data objects	237
Treemap data collections	238
TreemapDrillToDetail object	238
Treemap methods	239

The TreemapQuery object defines the underlying data that implements a treemap. This includes the data displayed in the treemap and the dimensions and measures the treemap uses. The TreemapQuery object contains the following properties.

Property	Type	Description
defaults	Object	The default values for the treemap dimensions, defined as an object with the {dimension:value} syntax
dimensions	Object	A collection of TreemapQueryDimension objects that define the dimensions used to define the treemap.
measures	Object	A TreemapQueryColorMeasure object that defines the size and color measures that define the treemap.
supportsHierarchies	Boolean	Whether the treemap can use hierarchy values in its dimensions.



Note: This object was added in RapidResponse 2014.2.

TreemapQueryColorMeasure object

The TreemapQueryColorMeasure object defines the colors used in the treemap's color measure. The TreemapQueryColorMeasure object has the following properties.

Property	Type	Description
colors	String	A list of colors used in the treemap.
name	String	The name of the color measure.



Note: This object was added in RapidResponse 2014.2.

TreemapQueryDimension object

The TreemapQueryDimension object defines the dimensions that determine the data displayed in the treemap, including how the data is divided and the size of each data region. These objects are contained in the [dimensions](#) collection.

The TreemapQueryDimension object has the following properties.

Property	Type	Description
header	String	The name of the dimension.
index	Number	The dimension's index within the collection. The array starts with zero.



Note: This object was added in RapidResponse 2014.2.

dimensions collection

The dimensions collection contains the [TreemapQueryDimension](#) objects that define the dimensions of a treemap.

The dimensions collection implements only the collection iteration functions. For more information, see ["Resource collections" on page 110](#).



Note: This collection was added in RapidResponse 2014.2.

Treemap data objects

The data displayed in a treemap is defined by rows and cells of data from a worksheet that underlies the treemap.

Treemap data returned by a script is represented as a `TreemapQueryData` object, which contains the rows of data from the treemap, and the individual data cells in each row. The rows are represented by the `rows` collection, which contains `TreemapQueryRow` objects, and the cells by the `cells` collection, which contains `TreemapQueryCell` objects. For information about the `rows` and `cells` collections, see ["Treemap data collections" on page 238](#).

The `TreemapQueryData` object has the following properties.

Property	Type	Description
hasDimension1	Boolean	Whether the treemap has a value defined for its first dimension.
hasDimension2	Boolean	Whether the treemap has a value defined for its second dimension.
rows	Object	The treemap data row collection representing the rows in the treemap. Each row in the collection contains values for each cell in that row. The rows and cells are represented by <code>TreemapQueryRow</code> and <code>TreemapQueryCell</code> objects.
summary	Object	A <code>TreemapQuerySummaryData</code> object that represents an array of data summarized in the treemap.

The `TreemapQuerySummaryData` object contains an array of objects with the following properties.

Property	Type	Description
name	String	The dimension value the summary is generated for.
values	String	An array of values aggregated to generate the summary value for the dimension value.

The `TreemapQueryRow` object has the following properties.

Property	Type	Description
index	Number	A row's position in the collection. The index starts at zero.
cells	Object	A collection of treemap data cell objects representing the data in each cell of the row.

The `TreemapQueryCell` object has the following properties.

Property	Type	Description
formattedValue	String	The data in the cell, with formatting applied.
index	Number	A cell's position in the array. The index starts at zero.
value	Number String Date Boolean	The raw value in the cell. The data type of the <code>value</code> property depends on the data in the cell.

Treemap data collections

Treemap data is contained in collections of treemap data rows and cells.

The `rows` collection contains the treemap's data rows, which are represented as an index of `TreemapQueryRow` objects. The `rows` and `cells` collection both have the following property.

Property	Type	Description
count	Number	The total number of rows or cells in the collection.

The cells within each row are contained in the `cells` collection, which contains an index of [TreemapQueryCell](#) objects. These objects contain the data values in each treemap column for a specific row.

The rows and cells collections both implement only the collection iteration functions. For more information, see ["Resource collections" on page 110](#).



Note: These collections were added in RapidResponse 2014.2.

TreemapDrillToDetail object

The `TreemapDrillToDetail` object defines drill to details links from the treemap to another worksheet or workbook. The `TreemapDrillToDetail` object has the following properties.

Property	Type	Description
columnMappings	Object	A collection of <code>TreemapDrillToDetailMapping</code> objects that define the column mappings for the drill to details link.
dependencyId	String	The dependency identifier for the workbook the drill to details link opens.

Property	Type	Description
isTargetWorksheetTransformation	Boolean	Indicates whether the target of the drill to details link is a transformation worksheet.
isTargetWorksheetTreemap	Boolean	Indicates whether the target of the drill to details link is another treemap.
worksheetId	String	The identifier of the worksheet the drill to details link opens.



Note: This object was added in RapidResponse 2014.2.

columnMappings collection

The columnMappings collection contains all [TreemapDrillToDetailMapping](#) objects defined for a treemap.

This collection implements only the collection iteration functions. For more information, see "[Resource collections](#)" on page 110.



Note: This collection was added in RapidResponse 2014.2.

TreemapDrillToDetailMapping object

The TreemapDrillToDetailMapping object defines the drill to details links in a treemap worksheet. These objects are contained in the [columnMappings](#) collection.

The TreemapDrillToDetailMapping object has the following property.

Property	Type	Description
index	Number	The drill to details mapping's position in the collection.



Note: This object was added in RapidResponse 2014.2.

Treemap methods

The TreemapQuery object contains the following method.

Method	Description
getData(options)	Retrieves the data the treemap contains.

getData method

This method returns the data used to create a treemap.

This method was added in RapidResponse 2014.2.

► Syntax

```
data = treemap.getData(options);
```

where *treemap* is a *TreemapQuery* object.

Argument	Type	Description
options	Object	Optionally, defines the dimensions used to define the treemap. This is an object specified as follows: {dimension1: value, dimension2: value}

► Returns

A [TreemapQueryData](#) object that contains the rows and cells of data that are used to create the treemap.

► Exceptions

The `getData` method can throw the following exception.

Exception	Description
ArgumentError	Either the dimensions are missing or an extra argument has been provided.

► Example

```
var dataWorkbook = rapidResponse.workbooks.get( {name: "Data Display",
scope: "Private"}, {
    scenarios: [{name: "myScenario", scope: "Private"}],
    filter: {name: "Make Parts", scope: "Public"},
    siteGroup: {name: "HQ", scope: "Public"}
});
var worksheet = dataWorkbook.worksheets.get("SupplyMap");
var treemap = worksheet.getTreemapQuery();
var treeData = treemap.getData({dimension1: "Site",
dimension2: "Supplier"} );
```


CHAPTER 15: Filter objects

filters collection	242
Filter object methods	245
CompositeFilterData object	249
FilterManagerFactory object	250
FilterManager object	253
CompositeFilterManager object	262

Filter objects define values or expressions that are used to limit the records displayed in a worksheet. All filters available to the user running the script are contained in the [filters](#) collection.

Filter objects have the same [properties as Resource objects](#), as well as the following property.

Property	Type	Description
associatedTable	Object	A Table object that represents the table the filter is compatible with.

The Table object has the following property.

Property	Type	Description
name	String	The table name.



Note: This object was modified in RapidResponse 2014.1.

filters collection

The filters collection contains all [Filter](#) objects the user running the script has access to. The filters collection implements the following resource collection methods.

Method	Description
get({name, scope})	Retrieves the specified filter.
exists(filter)	Determines whether the specified filter exists in the collection.
remove(filter)	Deletes the specified filter.

The filters collection also implements the collection indexing and iteration functions. For more information, see "[Collection indexing](#)" on page 111.

get method

Retrieves a Filter object.

► Syntax

```
filter = rapidResponse.filters.get({name, scope});
```

Parameter	Type	Description
name	String	The filter's name.
scope	String	Whether the filter is Public or Private.

► Returns

A Filter object.

► Exceptions

The `get` method can throw the following exceptions.

Exception	Description
Filter.NotFound	The specified filter does not exist or the user does not have access to it.
ArgumentError	One of the following has occurred: • The parameter is missing a name or scope value. • The scope value specified is not "Private" or "Public".

► Example

```
var filt = rapidResponse.filters.get({name:"All Parts",
scope:"Public"});
```

exists method

Determines whether a filter exists. However, because RapidResponse supports multiple users working with the same resources, a user might delete the specified filter after this method runs. If you are using this method to test whether a filter exists, you should still catch a `NotFound` exception for the filter you are using.

► Syntax

```
filterExists = rapidResponse.filters.exists({name: "Name", scope:
"Public or Private"});
```

Parameter	Type	Description
name	String	The name of the filter to test for existence.
scope	String	Whether the filter is Private or Public.

► Returns

Type	Description
Boolean	• true if the filter exists • false if the filter does not exist

► Exceptions

The `exists` method can throw the following exception.

Exception	Description
ArgumentError	One of the following has occurred: • The parameter is missing a name or scope value. • The scope value specified is not "Private" or "Public".

► Example

```
var filterExists = rapidResponse.filters.exists({name: "Buy or
Transfer Parts", scope: "Public"});
```

Test for the existence of filters, and either retrieve a workbook using the first one a user has access to or use a filter expression if the user has access to none of the specified filters.

```
var filterCondition;
if(rapidResponse.filters.exists({name:"All Parts", scope:"Public"})) {
    filterCondition = {name:"All Parts", scope:"Public"};
} else if(rapidResponse.filters.exists({name:"Buy Parts",
scope:"Public"})){
    filterCondition = {name:"Buy Parts", scope:"Public"};
} else {
    var allFilters = rapidResponse.filters.toArray();
    filterCondition = allFilters[0];
}

var workbook = rapidResponse.workbooks.get( {name:"Order List",
scope:"Public"}, {
    filter: filterCondition,
    scenarios: [{name:"Approved Actions", scope:"Public"}],
    siteGroup: {name:"All Sites", scope:"Public"}
});
```

remove method

Deletes a filter from the server, which also removes it from the `filters` collection. Deleted filters are no longer available to any user or process, and you can only delete filters you own or have permission to manage.

► Syntax

```
rapidResponse.filters.remove(filter);
```

Parameter	Type	Description
filter	Object	The filter to be deleted. This can be specified as a Filter object or using the {name, scope} syntax.

► Return value

No return value.

► Exceptions

The `remove` method can throw the following exceptions.

Exception	Description
Filter.NoPermission	You do not have permission to delete the specified script.
Filter.NotFound	The specified script does not exist.

► Example

```
rapidResponse.filters.remove({name:"Obsolete Parts", scope:"Public"});
```



Note: This method was added in RapidResponse 2014.1.

Filter object methods

The Filter object implements the following methods.

Method	Description
give (newOwner)	Gives the filter to the specified user.
makePublic()	Changes the scope of a filter to "Public", which is the equivalent of sharing the filter. Public filters can be accessed by administrators and users and groups with access. You can also share a filter with a particular user or group using the accessControlList collection's add method .
rename (newName)	Changes a filter's name.
copy (newName)	Creates a new filter by copying an existing filter.

give method

Gives a filter to the specified user.

► Syntax

```
filter.give(newOwner);
```

where *filter* is a Filter object.

Parameter	Type	Description
newOwner	String	The user to give the filter to. Note: You cannot give a filter to a group.

► Return value

No return value.

► Exceptions

The `give` method can throw the following exceptions.

Exception	Description
<code>Filter.NotFound</code>	The specified filter does not exist.
<code>Principal.NotFound</code>	The specified user does not exist.
<code>Filter.NonUniqueName</code>	The specified user already owns a filter with the same name as the one you are giving.
<code>Filter.NoPermission</code>	You do not have permission to give the filter.

► Example

```
var myFilter = rapidResponse.filters.get({ name:"Buyer Parts",
scope:"Private" });
myFilter.give("myUser");
```



Note: This method was added in RapidResponse 2014.1.

makePublic method

Changes the scope of the specified filter from "Private" to "Public", which shares the filter and allows other users to access it. This method does not specify other users or groups to share the filter with, so it is available to only you and administrators.

If you call this method on a variable that represents a filter, you must ensure you assign the returned public filter to the same variable. Otherwise, the variable retains the scope it was created with.

You can also share the filter with specific users and groups using the [accessControlList collection's add method](#).

Administrators can access any shared filter, and you can allow other users and groups to access a shared filter.

► Syntax

```
publicFilter = filter.makePublic();
```

where *filter* is a Filter object.

► Return value

A Filter object that represents the same filter with its scope changed to "Public".

► Exceptions

The `makePublic` method can throw the following exceptions.

Exception	Description
Filter.NotFound	The specified filter does not exist.
Filter.NonUniqueName	A shared filter with the same name as the one you are sharing already exists.
Filter.NoPermission	You do not have permission to share the filter.

► Example

```
var myFilter = rapidResponse.filters.get({ name:"Parts With Demand",
scope:"Private" });
myFilter = myFilter.makePublic();
```



Note: This method was added in RapidResponse 2014.1.

rename method

Changes the name of a filter.

► Syntax

```
filter.rename(newName);
```

where *filter* is a Filter object.

Parameter	Type	Description
newName	String	The new name for the filter.

► Return value

A Filter object with the new name.

► Exceptions

The `rename` method can throw the following exceptions.

Exception	Description
ArgumentError	A blank string, invalid string, or <code>null</code> value was specified for the <code>newName</code> parameter.
Filter.NotFound	The specified filter does not exist.
Filter.NameValidationError	Either a filter with the same name already exists or the specified name is not valid.
Filter.NoPermission	The filter cannot be renamed or you do not have permission to rename it.
Filter.RenamePublicError	The filter is shared and cannot be renamed.

► Example

```
var myFilter = rapidResponse.filters.get({ name:"Too Much Excess",
scope:"Private" });
myFilter = myFilter.rename("Excesses");
myFilter.makePublic();
```



Note: This method was added in RapidResponse 2014.1.

copy method

Creates a new filter by copying an existing filter. If you copy a public filter, the copy is private.

► Syntax

```
filter.copy(newName);
```

where *filter* is a Filter object.

Parameter	Type	Description
newName	String	The name for the new filter.

► Return value

No return value.

► Exceptions

The `copy` method can throw the following exceptions.

Exception	Description
ArgumentError	A blank string, invalid string, or <code>null</code> value was specified for the <code>newName</code> parameter.
Filter.NotFound	The specified filter does not exist.
Filter.NameValidationError	Either a filter with the same name already exists or the specified name is not valid.
Filter.NoPermission	The filter cannot be copied or you do not have permission to copy it.

► Example

```
var myFilter = rapidResponse.filters.get({ name:"Excesses",
scope:"Public" });
myFilter.copy("Excess Inventory");
```



Note: This method was added in RapidResponse 2014.1.

CompositeFilterData object

The CompositeFilterData object defines the data settings that comprise a [CompositeFilterManager](#) object. This object contains the following objects.

Property	Type	Description
filterItemGroups	Object	The set of FilterItemGroup objects used in the CompositeFilterManager object.
scenarioGroups	Object	The set of ScenarioGroup objects used in the CompositeFilterManager object.
hierarchyGroups	Object	The set of HierarchyGroup objects used in the CompositeFilterManager object.
bucketSettingsGroups	Object	The set of BucketSettingsGroup objects used in the CompositeFilterManager object.

FilterItemGroup object

The FilterItemGroup defines the filter items used in a FilterManager object. This object has the following properties.

Property	Type	Description
filterItemDisplayValue	String	The value displayed for the filter item.
filterItemTypeId	String	The data type of the filter item.
filterItemValue	String	The value of the filter item.
filterManagerIds	Object	The set of FilterManager objects that use this filter item.
isValid	Boolean	Whether this filter item is valid for the specified worksheet. This property was added in RapidResponse 2013.4.

ScenarioGroup object

The ScenarioGroup object defines the scenarios used in a FilterManager object. This object contains the following properties.

Property	Type	Description
scenarios	Object	A set of scenarios included in the scenario group.
filterManagerIds	Object	A set of FilterManager objects that use this scenario group.

HierarchyGroup object

The HierarchyGroup object defines the hierarchies used in a FilterManager object. This object contains the following properties.

Property	Type	Description
hierarchies	Object	The set of hierarchies included in the hierarchy group.
filterManagerIds	Object	The set of FilterManager objects that use the hierarchy group.

BucketSettingsGroup object

The BucketSettingsGroup object defines the bucket settings used in a FilterManager object. This object contains the following properties.

Property	Type	Description
bucketSettings	Object	The set of bucket settings used in the bucket settings group.
filterManagerIds	Object	The set of FilterManager objects that use this bucket settings group.

FilterManagerFactory object

The FilterManagerFactory object defines methods used for constructing filtering settings for workbooks. The FilterManagerFactory object contains the following methods.

Method	Description
createFilterManager(workbook, worksheet, {settings})	Creates a FilterManager object using the specified workbook, worksheet, and optional display settings.
createCompositeFilterManager(filterManagerIds)	Creates a CompositeFilterManager object using a set of FilterManager objects.

createFilterManager method

Creates a FilterManager object using a specific workbook, worksheet, and display settings.

► Syntax

```
filter = rapidResponse.filtering.createFilterManager(workbook, worksheet, {controlSettings, bucketSettings, preferences});
```

Argument	Type	Description
workbook	Object	A Workbook object
worksheet	Object	A Worksheet object from the specified workbook.
controlSettings	Object	A ControlSettings object that defines the data in the worksheet.
bucketSettings	Object	An optional BucketSettings object that defines the bucketing used in the worksheet.
preferences	Object	An optional WorkbookPreferences object that defines the user settings for the workbook.

► Returns

A [FilterManager](#) object.

► Exceptions

The `createFilterManager` method can throw the following exceptions.

Exception	Description
ArgumentError	One of the following has occurred: - A null value or no value was passed to the workbook or worksheet argument. - The value passed to an argument was the incorrect data type. - Too many arguments were passed to the method.

► Example

```
var controls = {
  scenarios: [{ name:"myScenario", scope:"Private" }],
  filter: {name: "Buy Parts", scope:"Public"},
  siteGroup: {name: "DC-Asia", scope: "Public"},
  part: "Racer"
};
var myWorkbook = rapidResponse.workbooks.get({ name:"My Workbook",
scope:"Public" }, controls);
var myWorksheet = myWorkbook.worksheets.get("AllOrders");
var myFilter = rapidResponse.filtering.createFilterManager(myWorkbook,
myWorksheet, {controlSettings: controls});
```

createCompositeFilterManager method

Creates a composite filter setting.

► Syntax

```
compFilter = rapidResponse.filtering.createCompositeFilterManager  
(filterManagerIDs);
```

Argument	Type	Description
filterManagerIDs	Object	An array of FilterManager object identifiers representing the filter settings you want to combine into a composite filter setting.

► Returns

A [CompositeFilterManager](#) object.

► Exceptions

The `createCompositeFilterManager` method can throw the following exceptions.

Exception	Description
ArgumentError	One of the following has occurred: • A null or blank value was passed to the <code>filterManagerIDs</code> argument. • The value passed to the <code>filterManagerIDs</code> argument was the incorrect data type. • Too many arguments were passed to the method.

► Example

```
var myWorkbook = rapidResponse.workbooks.get({ name:"My Workbook",  
scope:"Public" },  
{  
    scenarios: [{ name:"myScenario", scope:"Private" }],  
    filter: {name: "Buy Parts", scope:"Public"},  
    siteGroup: {name: "DC-Asia", scope: "Public"}  
});  
var myWorksheet = myWorkbook.worksheets.get("AllOrders");  
var secondWorkbook = rapidResponse.workbooks.get({ name:"Order  
Summary", scope:"Public" },  
{  
    scenarios: [{ name:"myScenario", scope:"Private" }],  
    filter: {name: "Buy Parts", scope:"Public"},  
    siteGroup: {name: "DC-Asia", scope: "Public"},  
    variables: [  
        {name:"DateAdjust", value:"1"},  
        {name:"DateAdjustPeriod", value:"Month"}  
    ]  
});  
var secondWorksheet = secondWorkbook.worksheets.get  
("LastMonthOrders");  
  
var filter = rapidResponse.filtering.createFilterManager(myWorkbook,  
myWorksheet, { controlSettings: {
```

```

        scenarios: [{ name:"myScenario", scope:"Private" }],
        filter: {name: "Buy Parts", scope:"Public"},
        siteGroup: {name: "DC-Asia", scope: "Public"}
    });
var secondFilter = rapidResponse.filtering.createFilterManager
(secondWorkbook, secondWorksheet, { controlSettings: {
    scenarios: [{ name:"myScenario", scope:"Private" }],
    filter: {name: "Buy Parts", scope:"Public"},
    siteGroup: {name: "DC-Asia", scope: "Public"},
    variables: [
        {name:"DateAdjust", value:"1"},
        {name:"DateAdjustPeriod", value:"Month"}
    ]
}});

var compFilter = rapidResponse.filtering.createCompositeFilterManager
([filter.id, secondFilter.id]);

```

FilterManager object

The FilterManager object defines the filtering settings applied to a worksheet. The FilterManager object has the following property.

Property	Type	Description
id	String	Identifies the worksheet's filter settings.

The FilterManager object implements the following methods.

Method	Description
getWorkbookBaseTable()	Retrieves the table that filters must be compatible with to be applied to the worksheet.
getUsesSelectedFilter()	Retrieves whether the worksheet uses the currently selected filter.
getMultiScenarioCount()	Retrieves the number of scenarios displayed in the worksheet.
getWorkbookVariables()	Retrieves the workbook variables used in the worksheet.
getBucketSettings()	Retrieves the worksheet's bucket settings.
getFilterItems()	Retrieves the filter items from a worksheet a filter manager is defined on.
getFilterValues()	Retrieves the values defined in the selected filter.
getHierarchyNode()	Retrieves the hierarchy value selected in the worksheet.
isMultiScenario()	Determines whether the worksheet displays data from multiple scenarios.
validate()	Determines whether the filter is valid.

The FilterManager object also uses a FilterManagerFilterItem object, which displays a list of filter items specified by type. Both the selected filter item and all available filter items of that type display in the list.

Property	Type	Description
type	String	The type of filter item.
availableValues	String	A list of FilterManagerFilterItemValue objects.
level	Number	Level of indentation specified for the filter item in the Settings pane. For example, an item at level 3 is a child of an item at level 2.
isValid	Boolean	Identifies if the filter item is valid.
isSelectable	Boolean	Identifies if the filter item can be selected.
displayValue	String	The display value (label) for the filter time. This value displays in the Settings pane.
dataValue	Object	By default, the display value. Specifically for scenario, filter, and site filter items, it is an instance of the resource object.

The filterManager object also uses a filterManagerFilterItemValue object which displays the list of available filter items.

Property	Type	Description
level	Number	Level of indentation specified for the filter item in the Settings pane. For example, an item at level 3 is a child of an item at level 2.
isValid	Boolean	Identifies if the filter item is valid.
isSelectable	Boolean	Identifies if the filter item can be selected.
displayValue	String	The display value (label) for the filter time. This value displays in the Settings pane.
dataValue	Object	By default, the display value. Specifically for scenario, filter, and site filter items, it is an instance of the resource object.

getWorkbookBaseTable method

Retrieves the table compatibility for a workbook a filter manager object is based on.

► Syntax

```
table = filterManager.getWorkbookBaseTable();
```

where *filterManager* is a [FilterManager](#) object.

► Returns

A String value.

► Exceptions

The `getWorkbookBaseTable` method can throw the following exception.

Exception	Description
<code>ArgumentError</code>	An argument was passed to the method.

► Example

```
var controls = {
  scenarios: [{ name:"myScenario", scope:"Private" }],
  filter: {name: "Buy Parts", scope:"Public"},
  siteGroup: {name: "DC-Asia", scope: "Public"}
};
var myWorkbook = rapidResponse.workbooks.get({ name:"My Workbook",
scope:"Public" }, controls);
var myWorksheet = myWorkbook.worksheets.get("AllOrders");

var myFilter = rapidResponse.filtering.createFilterManager(myWorkbook,
myWorksheet, { controlSettings: controls});
var baseTable = myFilter.getWorkbookBaseTable();
```

This example conditionally runs a script depending on the workbook's filter compatibility, using the same `FilterManager` object defined in the previous example.

```
var baseTable = myFilter.getWorkbookBaseTable();
if (baseTable == "IndependentDemand") {
  var toRun = rapidResponse.scripts.get({name:"Demand Adjustment",
scope:"Public"});
  toRun.execute();
}
```

getUsesSelectedFilter method

Retrieves whether the user's selected filter can be used with the worksheet defined in a `FilterManager` object.

► Syntax

```
useFilter = filterManager.getUsesSelectedFilter();
```

where *filterManager* is a [FilterManager](#) object.

► Returns

A Boolean value.

► Exceptions

The `getUsesSelectedFilter` method can throw the following exception.

Exception	Description
<code>ArgumentError</code>	An argument was passed to the method.

► Example

```
var myFilter = rapidResponse.filtering.createFilterManager(myWorkbook,
myWorksheet, {controlSettings: {
    scenarios: [{ name:"myScenario", scope:"Private" }],
    filter: {name: "Buy Parts", scope:"Public"},
    siteGroup: {name: "DC-Asia", scope: "Public"}
}});
if (myFilter.getUsesSelectedFilter()) {
    return("Selected filter is compatible");
}
```

getMultiScenarioCount method

Retrieves the number of scenarios used in the workbook a filter manager object is based on.

► Syntax

```
numberScenarios = filterManager.getMultiScenarioCount();
```

where *filterManager* is a [FilterManager](#) object.

► Returns

A Number value.

► Exceptions

The `getMultiScenarioCount` method can throw the following exception.

Exception	Description
<code>ArgumentError</code>	An argument was passed to the method.

► Example

```
var myFilter = rapidResponse.filtering.createFilterManager(myWorkbook,
myWorksheet, {controlSettings: {
    scenarios: [{ name:"myScenario", scope:"Private" }],
    filter: {name: "Buy Parts", scope:"Public"},
    siteGroup: {name: "DC-Asia", scope: "Public"}
}});
var scenarioCount = myFilter.getMultiScenarioCount();
```

getWorkbookVariables method

Retrieves the workbook variables in the workbook the filter manager is defined on.

► Syntax

```
variableSet = filterManager.getWorkbookVariables();
```

where *filterManager* is a [FilterManager](#) object.

► Returns

An array of [WorkbookVariable](#) objects.

► Exceptions

The `getWorkbookVariables` method can throw the following exception.

Exception	Description
ArgumentError	An argument was passed to the method.

► Example

```
var myFilter = rapidResponse.filtering.createFilterManager(myWorkbook,
myWorksheet, {controlSettings: {
  scenarios: [{ name:"myScenario", scope:"Private" }],
  filter: {name: "Buy Parts", scope:"Public"},
  siteGroup: {name: "DC-Asia", scope: "Public"}
}});
var workbookVariable = myFilter.getWorkbookVariables();
```

getBucketSettings method

Retrieves the bucket settings from a worksheet a filter manager is defined on.

► Syntax

```
buckets = filterManager.getBucketSettings();
```

where *filterManager* is a [FilterManager](#) object.

► Returns

A [BucketSettings](#) object.

► Exceptions

The `getBucketSettings` method can throw the following exception.

Exception	Description
ArgumentError	An argument was passed to the method.

► Example

```
var myFilter = rapidResponse.filtering.createFilterManager(myWorkbook,
myWorksheet, {controlSettings: {
    scenarios: [{ name:"myScenario", scope:"Private" }],
    filter: {name: "Buy Parts", scope:"Public"},
    siteGroup: {name: "DC-Asia", scope: "Public"}
}});
var worksheetBuckets = myFilter.getBucketSettings();
```

getFilterItems method

Retrieves the filter items from a worksheet a filter manager is defined on.

► Syntax

```
filterManager.getFilterItems(type);
```

where *filterManager* is a [FilterManager](#) object.

Parameter	Type	Description
<i>type</i>	String	Optionally, the type of filter item to display. If not specified, all items are returned.

► Returns

A set of FilterManagerFilterItem objects.

► Exceptions

The `getFilterValues` method can throw the following exceptions.

Exception	Description
ArgumentError	The value specified for the <code>type</code> parameter is not a valid control type.

► Example

```
var myFilter = rapidResponse.filtering.createFilterManager(myWorkbook,
myWorksheet, {controlSettings: {
    scenarios: [{ name:"myScenario", scope:"Private" }],
    filter: {name: "Buy Parts", scope:"Public"},
    siteGroup: {name: "DC-Asia", scope: "Public"}
}});
var filterItems = myFilter.getFilterItems(type = "scenario");
```



Note: This method was added in RapidResponse 2016.2

getFilterValues method

Retrieves the values from a filter item.

► Syntax

```
values = filterManager.getFilterValues(type, variableName);
```

where *filterManager* is a [FilterManager](#) object.

Parameter	Type	Description
type	String	The type of filter item to retrieve. If you specify "variable" for the type, you must also specify the variable name.
variableName	String	The name of the variable to retrieve values from. This parameter is required only if you specified "variable" for the <code>type</code> parameter. This method should be used with variables that can contain multiple values, such as list or filter variables.

► Returns

A set of String values.

► Exceptions

the `getFilterValues` method can throw the following exceptions.

Exception	Description
ArgumentError	The value specified for the <code>type</code> parameter is not a valid control type, or a variable name was not provided for the "variable" type.

► Example

This example retrieves all scenarios available to be used in a worksheet.

```
var myFilter = rapidResponse.filtering.createFilterManager(myWorkbook,
myWorksheet, {controlSettings: {
    scenarios: [{ name:"myScenario", scope:"Private" }],
    filter: {name: "Buy Parts", scope:"Public"},
    siteGroup: {name: "DC-Asia", scope: "Public"}
}});
var filterValues = myFilter.getFilterValues("scenario");
```

This example retrieves all variable values available to a workbook list variable named "orderProcess".

```
var myFilter = rapidResponse.filtering.createFilterManager(myWorkbook,
myWorksheet, {controlSettings: {
    scenarios: [{ name:"myScenario", scope:"Private" }],
    filter: {name: "Buy Parts", scope:"Public"},
    siteGroup: {name: "DC-Asia", scope: "Public"}
}});
var filterValues = myFilter.getFilterValues("variable",
"orderProcess");
```

getHierarchyNode method

Retrieves the selected hierarchy value.

► Syntax

```
selectedLevel = filterManager.getHierarchyNode(hierarchyKey,
hierarchyPath);
```

where *filterManager* is a [FilterManager](#) object.

Parameter	Type	Description
hierarchyKey	Object	The hierarchy used in the FilterManager.
hierarchyPath	Object	The path to the selected value in the hierarchy.

► Returns

A String value.

► Exceptions

The `getHierarchyNode` method can throw the following exception.

Exception	Description
ArgumentError	A parameter value is missing or a null value was specified.

► Example

```
var myFilter = rapidResponse.filtering.createFilterManager(myWorkbook,
myWorksheet, {controlSettings: {
    scenarios: [{ name:"myScenario", scope:"Private" }],
    filter: {name: "Buy Parts", scope:"Public"},
    siteGroup: {name: "DC-Asia", scope: "Public"}
}});
var level = myFilter.getHierarchyNode(hierarchyKey:{name:"Customer",
scope:"Public"}, hierarchyPath:[{
    name:"C",
```

```
        childPath: [
            {name:"Computer Mart"}
        ]
    });
```

isMultiScenario method

Retrieves whether the workbook the filter manager object is based on is multi-scenario.

► Syntax

```
isMulti = filterManager.isMultiScenario();
```

where *filterManager* is a *FilterManager* object.

► Returns

A Boolean value.

► Exceptions

The `isMultiScenario` method can throw the following exception.

Exception	Description
ArgumentError	An argument was passed to the method.

► Example

```
var myFilter = rapidResponse.filtering.createFilterManager(myWorkbook,
myWorksheet, {controlSettings: {
    scenarios: [{ name:"myScenario", scope:"Private" }],
    filter: {name: "Buy Parts", scope:"Public"},
    siteGroup: {name: "DC-Asia", scope: "Public"}
}});
var scenarios;
if (myFilter.isMultiScenario()) {
    scenarios = myFilter.getMultiScenarioCount();
}
```

validate method

Determines if a *FilterManager* object is valid for a worksheet.

► Syntax

```
validFilter = FilterManager.validate();
```

where *FilterManager* is a *FilterManager* object.

► Returns

A Boolean value.

► Exceptions

The `validate` method can throw the following exceptions.

Exception	Description
<code>ArgumentError</code>	A parameter was provided.

► Example

```
var myFilter = rapidResponse.filtering.createFilterManager(myWorkbook,
myWorksheet, {controlSettings: {
    scenarios: [{ name:"myScenario", scope:"Private" }],
    filter: {name: "Buy Parts", scope:"Public"},
    siteGroup: {name: "DC-Asia", scope: "Public"}
}});
var validFilter = myFilter.validate();
```



Note: This method was added in RapidResponse 2013.4.

CompositeFilterManager object

The `CompositeFilterManager` object defines the filtering settings applied to a composite worksheet. The `CompositeFilterManager` object contains the following method.

Method	Description
<code>getData()</code>	Retrieves the data defined by the composite filter settings.

getData method

Retrieves the data defined by a `CompositeFilterManager` object.

► Syntax

```
filterData = compositeFilter.getData();
```

where *compositeFilter* is a [CompositeFilterManager](#) object.

► Returns

A [CompositeFilterData](#) object.

► Exceptions

The `getData` method can throw the following exception.

Exception	Description
<code>ArgumentError</code>	An argument was passed to the method.

► Example

```
var compFilter =  
rapidResponse.FilterManagerFactory.createCompositeFilterManager  
([filter.id, secondFilter.id]);  
var filterData = compFilter.getData();
```


CHAPTER 16: Hierarchy objects

Hierarchy objects define values that can be applied to workbooks to display data at different levels of detail. These values are arranged in a hierarchical structure, where each level of the hierarchy is related to the level above it.

Hierarchy objects have the same [properties as Resource objects](#).

hierarchies collection

The hierarchies collection contains all [Hierarchy](#) objects the user running the script has access to. The hierarchies collection implements the following resource collection methods.

Method	Description
get({name, scope})	Retrieves the specified hierarchy.
exists(hierarchy)	Determines whether the specified hierarchy exists in the collection.

The hierarchies collection also implements the collection indexing and iteration functions. For more information, see "[Collection indexing](#)" on page 111.

get method

Retrieves a Hierarchy object.

► Syntax

```
hierarchy = rapidResponse.hierarchies.get({name, scope});
```

Parameter	Type	Description
name	String	The hierarchy's name.
scope	String	Whether the hierarchy is Public or Private.

► Returns

A Hierarchy object.

► Exceptions

The `get` method can throw the following exceptions.

Exception	Description
Hierarchy.NotFound	The specified hierarchy does not exist or the user does not have access to it.
ArgumentError	One of the following has occurred: - The parameter is missing a name or scope value. - The scope value specified is not "Private" or "Public".

► Example

```
var hier = rapidResponse.hierarchies.get({name:"Region",
scope:"Public"});
```

exists method

Determines whether a hierarchy exists. However, because RapidResponse supports multiple users working with the same resources, a user might delete the specified hierarchy after this method runs. If you are using this method to test whether a hierarchy exists, you should still catch a `NotFound` exception for the hierarchy you are using.

► Syntax

```
exists = rapidResponse.hierarchies.exists({name: "Name", scope:
"Public or Private"});
```

Parameter	Type	Description
name	String	The name of the hierarchy to test for existence.
scope	String	Whether the hierarchy is Private or Public.

► Returns

Type	Description
Boolean	• true if the hierarchy exists • false if the hierarchy does not exist

► Exceptions

The `exists` method can throw the following exception.

Exception	Description
ArgumentError	One of the following has occurred: • The parameter is missing a name or scope value. • The scope value specified is not "Private" or "Public".

► Example

```
var hierarchyExists = rapidResponse.hierarchies.exists({name: "Product
Family", scope: "Public"});

if (!hierarchyExists) {
    return "Hierarchy does not exist.";
}
```

Hierarchy object methods

The Hierarchy object implements the following methods.

Method	Description
give (newOwner)	Gives the hierarchy to the specified user.
makePublic()	Changes the scope of a hierarchy to "Public", which is the equivalent of sharing the hierarchy. Public hierarchies can be accessed by administrators and users and groups with access. You can also share a hierarchy with a particular user or group using the accessControlList collection's add method .
rename (newName)	Changes a hierarchy's name.
copy (newName)	Creates a new hierarchy by copying an existing hierarchy.

give method

Gives a hierarchy to the specified user.

► Syntax

```
hierarchy.give(newOwner);
```

where *hierarchy* is a Hierarchy object.

Parameter	Type	Description
newOwner	String	The user to give the hierarchy to. Note: You cannot give a hierarchy to a group.

► Return value

No return value.

► Exceptions

The `give` method can throw the following exceptions.

Exception	Description
<code>Hierarchy.NotFound</code>	The specified hierarchy does not exist.
<code>Principal.NotFound</code>	The specified user does not exist.
<code>Hierarchy.NonUniqueName</code>	The specified user already owns a hierarchy with the same name as the one you are giving.
<code>Hierarchy.NoPermission</code>	You do not have permission to give the hierarchy.

► Example

```
var myHierarchy = rapidResponse.hierarchies.get({ name:"Manufacturing  
Regions", scope:"Private" });  
myHierarchy.give("myUser");
```



Note: This method was added in RapidResponse 2016.2 Service Update 15.

makePublic method

Changes the scope of the specified hierarchy from "Private" to "Public", which shares the hierarchy and allows other users to access it. This method does not specify other users or groups to share the hierarchy with, so it is available to only you and administrators.

If you call this method on a variable that represents a hierarchy, you must ensure you assign the returned public hierarchy to the same variable. Otherwise, the variable retains the scope it was created with.

You can also share the hierarchy with specific users and groups using the [accessControlList collection's add method](#).

Administrators can access any shared hierarchy, and you can allow other users and groups to access a shared hierarchy.

► Syntax

```
publicHierarchy = hierarchy.makePublic();
```

where *hierarchy* is a Hierarchy object.

► Return value

A Hierarchy object that represents the same hierarchy with its scope changed to "Public".

► Exceptions

The `makePublic` method can throw the following exceptions.

Exception	Description
Hierarchy.NotFound	The specified hierarchy does not exist.
Hierarchy.NonUniqueName	A shared hierarchy with the same name as the one you are sharing already exists.
Hierarchy.NoPermission	You do not have permission to share the hierarchy.

► Example

```
var myHierarchy = rapidResponse.hierarchies.get({ name:"Manufacturing  
Region", scope:"Private" });  
myHierarchy = myHierarchy.makePublic();
```

rename method

Changes the name of a hierarchy.

► Syntax

```
hierarchy.rename(newName);
```

where *hierarchy* is a Hierarchy object.

Parameter	Type	Description
newName	String	The new name for the hierarchy.

► Return value

A Hierarchy object with the new name.

► Exceptions

The `rename` method can throw the following exceptions.

Exception	Description
ArgumentError	A blank string, invalid string, or <code>null</code> value was specified for the <code>newName</code> parameter.
Hierarchy.NotFound	The specified hierarchy does not exist.
Hierarchy.NameValidationError	Either a hierarchy with the same name already exists or the specified name is not valid.
Hierarchy.NoPermission	The hierarchy cannot be renamed or you do not have permission to rename it.
Hierarchy.RenamePublicError	The hierarchy is shared and cannot be renamed.

► Example

```
var myHierarchy = rapidResponse.hierarchies.get({ name:"Customer
Region", scope:"Private" });
myHierarchy = myHierarchy.rename("Customers by Region");
myHierarchy.makePublic();
```

copy method

Creates a new hierarchy by copying an existing hierarchy. If you copy a public hierarchy, the copy is private.

► Syntax

```
hierarchy.copy(newName);
```

where *hierarchy* is a Hierarchy object.

Parameter	Type	Description
<code>newName</code>	String	The name for the new hierarchy.

► Return value

No return value.

► Exceptions

The `copy` method can throw the following exceptions.

Exception	Description
ArgumentError	A blank string, invalid string, or <code>null</code> value was specified for the <code>newName</code> parameter.
Hierarchy.NotFound	The specified hierarchy does not exist.
Hierarchy.NameValidationError	Either a hierarchy with the same name already exists or the specified name is not valid.
Hierarchy.NoPermission	The hierarchy cannot be copied or you do not have permission to copy it.

► Example

```
var myHierarchy = rapidResponse.hierarchies.get({ name:"Customers by
Region", scope:"Public" });
myHierarchy.copy("Regions");
```


CHAPTER 17: SiteGroup object

siteGroups collection	273
SiteGroup object methods	276

The SiteGroup object defines a site or site filter, and is used to identify the site or sites that a resource uses. SiteGroup objects are contained in the [siteGroups](#) collection.

SiteGroup objects have the same [properties as Resource objects](#). In addition, the siteGroup object also has the following property:

Property	Type	Description
imageId	String	Identifies the type of icon used for a site or site filter. For sites: physical or logical For site filters: private or shared.

For the SiteGroup object's `scope` property, individual sites always have a "Public" scope. The methods defined by the SiteGroup object or collection typically apply only to site filters.



Note: This object was added in RapidResponse 2014.1.

siteGroups collection

The siteGroups collection contains all sites and site filters available to the user running the script. The siteGroups collection implements the following resource collection methods.

Method	Description
get (key)	Retrieves a site or site filter.
exists (siteGroup)	Determines whether the specified site or site filter exists in the collection.
remove (siteGroup)	Deletes the specified site filter.

The siteGroups collection also implements the [resource enumeration](#) functions.

get method

Retrieves an instance of a site or site filter.

► Syntax

```
rapidResponse.siteGroups.get( {name, scope} );
```

Parameter	Type	Description
name	String	The name of the site or site filter.
scope	String	Whether the siteGroup is public or private. Sites must always be public. Site filters can be public or private.

► Return value

A SiteGroup object.

► Exceptions

The get method can throw the following exceptions.

Exception	Description
ArgumentError	A null value was passed to a parameter.
SiteGroup.NotFound	The specified site or site filter does not exist or you do not have access to it.

► Example

```
var siteFilter = rapidResponse.siteGroups.get( {name:"Company Sites",
scope: "Public" } );
var workbook = rapidResponse.workbooks.get ( {name: "Orders", scope:
"Public" }, {
    siteGroup: siteFilter,
    filter: {name: "All Parts", scope: "Public" }
} );
```



Note: This method was added in RapidResponse 2014.1.

exists method

Determines whether a specific site or site filter exists in the collection. However, because RapidResponse supports multiple users working with the same resources, a user might delete the specified site or site filter after this method runs. If you are using this method to test whether a site or site filter exists, you should still catch a `NotFound` exception.

► Syntax

```
rapidResponse.siteGroups.exists({name: "Name", scope:"Public or Private"});
```

Parameter	Type	Description
name	String	The name of the site or site filter.
scope	String	Whether the site or site filter is public or private.

► Return value

Type	Description
Boolean	• true if the site or site filter exists • false if the site or site filter does not exist

► Exceptions

The `exists` method can throw the following exception.

Exception	Description
ArgumentError	One of the following has occurred: • The resource parameter is missing a name or scope value. • The scope value specified is not "Private" or "Public".

► Example

```
var siteExists = rapidResponse.siteGroups.exists({name: "North America", scope: "Public"});
```



Note: This method was added in RapidResponse 2014.1.

remove method

Deletes a site filter from the server, which also removes it from the `siteGroups` collection. Deleted site filters are no longer available to any user or process, and you can only delete site filters you own. This method applies only to site filters, and cannot delete sites.

► Syntax

```
rapidResponse.siteGroups.remove(siteGroup);
```

Parameter	Type	Description
siteGroup	Object	The site filter to delete. This can be specified as a SiteGroup object or using the {name, scope} syntax.

► Return value

No return value.

► Exceptions

The `remove` method can throw the following exceptions.

Exception	Description
SiteGroup.NoPermission	You do not have permission to delete the specified site filter.
SiteGroup.NotFound	The specified site filter does not exist.

► Example

```
rapidResponse.siteGroups.remove({name:"Contract Sites",  
scope:"Public"});
```



Note: This method was added in RapidResponse 2014.1.

SiteGroup object methods

The SiteGroup object implement the following [Resource object](#) methods.

Method	Description
give (newOwner)	Gives the site filter to another user.
makePublic()	Changes the scope of a site filter to "Public", which is the equivalent of sharing the site filter. Public site filters can be accessed by any user or group with access to at least one site in the filter.
rename (newName)	Changes the site filter's name.
copy (newName)	Creates a new site filter by copying an existing one.

give method

Gives a site filter to another user.

► Syntax

```
siteGroup.give(newOwner);
```

where *siteGroup* is a SiteGroup object.

Parameter	Type	Description
newOwner	String	The user you are giving the site filter to. Note: You cannot give a site filter to a group.

► Return value

No return value.

► Exceptions

The `give` method can throw the following exceptions.

Exception	Description
SiteGroup.NotFound	The specified site filter does not exist.
Principal.NotFound	The specified user does not exist.
SiteGroup.NoPermission	You do not have permission to give the site filter.

► Example

```
var mySites = rapidResponse.siteGroups.get({ name:"North America",
scope:"Public" });
```

```
mySites.give("myUser");
```



Note: This method was added in RapidResponse 2014.1.

makePublic method

Makes a private site filter public. A public site filter is available to any user who has access to at least one site contained in the site filter.

If you call this method on a variable that represents a site filter, you must ensure you assign the returned public site filter to the same variable. Otherwise, the variable retains the scope it was created with.

► Syntax

```
publicSiteGroup = siteGroup.makePublic();
```

where *siteGroup* is a SiteGroup object.

► Return value

A SiteGroup object that represents the same site filter with its scope changed to "Public"

► Exceptions

The `makePublic` method can throw the following exceptions.

Exception	Description
SiteGroup.NotFound	The specified site filter does not exist.
SiteGroup.NoPermission	You do not have permission to share the site filter.

► Example

```
var mySiteFilter = rapidResponse.siteGroups.get({ name:"North-West  
Region", scope:"Private" });  
mySiteFilter = mySiteFilter.makePublic();
```



Note: This method was added in RapidResponse 2014.1.

rename method

Changes the name of a site filter.

► Syntax

```
siteGroup.rename(newName);
```

where *siteGroup* is a SiteGroup object.

Parameter	Type	Description
newName	String	The new name for the site filter.

► Return value

No return value.

► Exceptions

The `rename` method can throw the following exceptions.

Exception	Description
ArgumentError	A blank string, invalid string, or <code>null</code> value was specified for the <code>newName</code> parameter.
SiteGroup.NotFound	The specified site filter does not exist.
SiteGroup.NameValidationError	Either a site filter with the same name already exists or the specified name is not valid.
SiteGroup.NoPermission	You do not have permission to rename the site filter.
SiteGroup.RenamePublicError	The site filter is shared and cannot be renamed.

► Example

```
var mySiteFilter = rapidResponse.siteGroups.get({ name:"North  
America", scope:"Private" });  
mySiteFilter.rename("Northeast Region");
```



Note: This method was added in RapidResponse 2014.1.

copy method

Creates a new site filter by copying an existing site filter.

► Syntax

```
siteGroup.copy(newName);
```

where *siteGroup* is a SiteGroup object.

Parameter	Type	Description
newName	String	The name of the new site filter.

► Return value

No return value.

► Exceptions

The `copy` method can throw the following exceptions.

Exception	Description
<code>ArgumentError</code>	A blank string, invalid string, or <code>null</code> value was specified for the <code>newName</code> parameter.
<code>SiteGroup.NotFound</code>	The specified site filter does not exist.
<code>SiteGroup.NameValidationError</code>	Either a site filter with the same name already exists or the specified name is not valid.
<code>SiteGroup.NoPermission</code>	The site filter cannot be copied or you do not have permission to copy it.

► Example

This example copies a site filter.

```
var mySites = rapidResponse.siteGroups.get({ name:"Northeast Region",
scope:"Private" });
mySites.copy("Northern Sites");
```

This example copies a site filter, and then gives the copy to a user who will modify it.

```
var mySites = rapidResponse.siteGroups.get({ name:"Northeast Region",
scope:"Public" });
mySites.copy("Northern Sites");
var siteGive = rapidResponse.siteGroups.get( {name:"Northern Sites",
scope:"Private"} );
siteGive.give("siteManagement");
```



Note: This method was added in RapidResponse 2014.1.

CHAPTER 18: Alert objects

alerts collection	281
Alert object methods	284

Alert objects define alerts, which can be used to monitor data and create reports. Alert objects are contained in the `alerts` collection. For more information, see ["alerts collection" on page 281](#).

Alerts have the same [properties as Resource objects](#).

alerts collection

The `alerts` collection contains all alerts available to the user running the script. Any alert retrieved or run by the script must be present in this collection.

The `alerts` collection implements the following resource collection methods.

Method	Description
get(key)	Retrieves the alert object with the specified resource key.
remove(alert)	Deletes the specified alert.
exists(alert)	Determines whether the specified alert exists.

get method

Retrieves an instance of an alert object.

► Syntax

```
rapidResponse.alerts.get({name, scope});
```

Parameter	Type	Description
name	String	The name of the alert to retrieve.
scope	String	Whether the alert is Public or Private.

► Returns

An alert object.

► Exceptions

The `get` method can throw the following exceptions.

Exception	Description
ArgumentError	A blank String or null value was passed to the method.
Alert.NoPermission	You do not have permission to create alerts.
Alert.NotFound	The specified alert does not exist.

► Example

```
var myAlert = rapidResponse.alerts.get({name: "newOrderAlert", scope: "Private"});
```

exists method

Determines whether an alert exists. However, because RapidResponse supports multiple users working with the same resources, a user might delete the specified alert after this method runs. If you are using this method to test whether an alert exists, you should still catch a `NotFound` exception for the alert you are using.

► Syntax

```
alertExists = rapidResponse.alerts.exists({name: "Name", scope: "Public or Private"});
```

Parameter	Type	Description
name	String	The name of the alert to test for existence.
scope	String	Whether the alert is Private or Public.

► Returns

Type	Description
Boolean	• true if the alert exists • false if the alert does not exist

► Exceptions

The `exists` method can throw the following exceptions.

Exception	Description
ArgumentError	One of the following has occurred: • The parameter is missing a name or scope value. • The scope value specified is not "Private" or "Public".

► Example

```
var alertExists = rapidResponse.alerts.exists({name: "New Orders",
scope: "Public"});
```

This example checks for the existence of an alert, and then if it exists, retrieves and runs it.

```
var alertExists = rapidResponse.alerts.exists({name: "New Orders",
scope: "Public"});
if (alertExists){
    var alertRun = rapidResponse.alerts.get({name: "Order Sequence",
scope: "Private"});
    alertRun.runAsync();
} else {
    return ("Alert does not exist.");
}
```

remove method

Deletes an alert from the server, which also removes it from the `alerts` collection. Deleted alerts are no longer available to any user or process, and you can only delete alerts you own or have permission to manage.

► Syntax

```
rapidResponse.alerts.remove(alert);
```

Parameter	Type	Description
alert	Object	The alert that will be deleted. This can be specified as either an instance of an alert obtained from calling the <code>get()</code> method or an alert object using the syntax <code>{name, scope}</code>

► Returns

No return value.

► Exceptions

The `remove` method can throw the following exceptions.

Exception	Description
Alert.NoPermission	You do not have permission to delete the specified alert.
Alert.NotFound	The specified alert does not exist.

► Example

```
rapidResponse.alerts.remove({name: "newAndModifiedOrderAlert", scope: "Private"});
```

Alert object methods

The Alert object implements the following [resource object](#) methods, as well as the [access control](#) methods.

Method	Description
give (newOwner)	Changes the owner of an alert.
makePublic()	Changes the scope of an alert to "Public", which is the equivalent of sharing the resource. Public resources can be accessed by administrators and users and groups with access.
rename (newName)	Changes the name of an alert.
copy (newName)	Makes a copy of an alert with the specified name.

The Alert object also implements the following method.

Method	Description
runAsync()	Runs the alert. The alert runs asynchronously.

give method

Gives an alert to another user. The recipient must have permission to author alerts.

► Syntax

```
alert.give(owner);
```

where *alert* is an alert object.

Parameter	Type	Description
owner	String	The user you are giving the alert to. Note: You cannot give an alert to a group.

► Returns

No return value.

► Exceptions

The *give* method can throw the following exceptions.

Exception	Description
Alert.NotFound	The specified alert does not exist.
Principal.NotFound	The specified user does not exist.
Alert.NoPermission	You do not have permission to give the alert.

► Example

```
var myAlert = rapidResponse.alerts.get({ name:"newOrderAlert",
scope:"Private" });
myAlert.give("myUser");
```

makePublic method

Changes the scope of the specified alert from "Private" to "Public", which shares the alert and allows other users to access it. This method does not specify other users or groups to share the alert with, so it is available to only you and administrators.

If you call this method on a variable that represents an alert, you must ensure you assign the returned public alert to the same variable. Otherwise, the variable retains the scope it was created with.

You can also share the alert with specific users and groups using the

Administrators can access any shared alert, and you can allow other users and groups to access a shared alert.

► Syntax

```
publicAlert = alert.makePublic();
```

where *alert* is an alert object.

► Returns

An Alert object that represents the same alert with its scope changed to "Public".

► Exceptions

The `makePublic` method can throw the following exceptions

Exception	Description
Alert.NotFound	The specified alert does not exist.
Alert.NoPermission	You do not have permission to share the alert.

► Example

```
var myAlert = rapidResponse.alerts.get({ name:"newOrderAlerts", scope:"Private"
});
myAlert = myAlert.makePublic();
```

rename method

Changes the name of an alert. You can rename only private alerts that you own.

► Syntax

```
alert.rename(newName);
```

where *alert* is an alert object.

Parameter	Type	Description
newName	String	The new name for the alert.

► Returns

No return value.

► Exceptions

The `rename` method can throw the following exceptions.

Exception	Description
<code>ArgumentError</code>	A blank string, invalid string, or <code>null</code> value was specified for the <code>newName</code> parameter.
<code>Alert.NotFound</code>	The specified alert does not exist.
<code>Alert.NameValidationError</code>	Either an alert with the same name already exists or the specified name is not valid.
<code>Alert.NoPermission</code>	You do not have permission to rename the alert.
<code>Alert.RenamePublicError</code>	The alert is shared and cannot be renamed.

► Example

```
var myAlert = rapidResponse.alerts.get({ name:"newOrderAlert",
scope:"Private" });
myAlert.rename("orderUpdateAlert");
```

copy method

Creates a new private alert by copying an existing alert. If you copy a shared alert, the copy is private.

► Syntax

```
alert.copy(newName);
```

where *alert* is an alert object.

Parameter	Type	Description
newName	String	The name for the new alert.

► Returns

No return value.

► Exceptions

The `copy` method can throw the following exceptions.

Exception	Description
<code>ArgumentError</code>	A blank string, invalid string, or <code>null</code> value was specified for the <code>newName</code> parameter.
<code>Alert.NotFound</code>	The specified alert does not exist.
<code>Alert.NameValidationError</code>	Either an alert with the same name already exists or the specified name is not valid.
<code>Alert.NoPermission</code>	The alert cannot be copied or you do not have permission to copy it.

► Example

```
var myAlert = rapidResponse.alerts.get({ name:"newOrderAlert",
scope:"Private" });
myAlert.copy("newAndModifiedOrderAlert");
```

runAsync method

Runs an alert. When a script runs an alert, the alert runs asynchronously, and the script cannot wait for the alert to finish.

► Syntax

```
alert.runAsync();
```

where *alert* is an alert object.

► Returns

No return value.

► Exceptions

The `runAsync` method can throw the following exceptions.

Exception	Description
<code>Alert.AlreadyRunning</code>	The alert is already running with the same data settings.
<code>Alert.LockedOut</code>	The alert is locked.
<code>Alert.NotFound</code>	The specified alert does not exist.
<code>Alert.ScenarioDoesNotExist</code>	The scenario specified in the alert's settings does not exist.

► **Example**

```
var myAlert = rapidResponse.alerts.get({name: "newOrderAlert", scope:
"Private"});
myAlert.runAsync();
```


CHAPTER 19: ScheduledTask objects

scheduledTasks collection	291
ScheduledTask object methods	294

ScheduledTask objects define scheduled tasks, which can be used to run workbook data modifications or scripts, or to publish data to a subscriber. Scheduled task objects are contained in the `scheduledTasks` collection. For more information, see "[scheduledTasks collection](#)" on page 291.

Scheduled tasks have the same [properties as Resource objects](#).

scheduledTasks collection

The `scheduledTasks` collection contains all scheduled tasks available to the user running the script. Any scheduled task retrieved or run by the script must be present in this collection.

The `scheduledTasks` collection implements the following resource collection methods.

Method	Description
get(key)	Retrieves the scheduled task object with the specified resource key.
remove(scheduledTask)	Deletes the specified scheduled task.
exists(scheduledTask)	Determines whether the specified scheduled task exists.

get method

Retrieves an instance of a scheduled task object.

► Syntax

```
rapidResponse.scheduledTasks.get({name, scope});
```

Parameter	Type	Description
name	String	The name of the scheduled task to retrieve.
scope	String	Whether the scheduled task is Public or Private.

► Returns

A scheduled task object.

► Exceptions

The `get` method can throw the following exceptions.

Exception	Description
ArgumentError	A blank String or null value was passed to the method.
ScheduledTask.NoPermission	You do not have permission to create scheduled tasks.
ScheduledTask.NotFound	The specified scheduled task does not exist.

► Example

```
var myTask = rapidResponse.scheduledTasks.get({name: "orderUpdateTask", scope: "Private"});
```

exists method

Determines whether a scheduled task exists. However, because RapidResponse supports multiple users working with the same resources, a user might delete the specified scheduled task after this method runs. If you are using this method to test whether a scheduled task exists, you should still catch a `NotFound` exception for the scheduled task you are using.

► Syntax

```
taskExists = rapidResponse.scheduledTasks.exists({name: "Name", scope: "Public or Private"});
```

Parameter	Type	Description
name	String	The name of the scheduled task to test for existence.
scope	String	Whether the scheduled task is Private or Public.

► Returns

Type	Description
Boolean	• true if the scheduled task exists • false if the scheduled task does not exist

► Exceptions

The `exists` method can throw the following exceptions.

Exception	Description
ArgumentError	One of the following has occurred: • The parameter is missing a name or scope value. • The scope value specified is not "Private" or "Public".

► Example

```
var taskExists = rapidResponse.scheduledTasks.exists({name:
"orderUpdateTask", scope: "Private"});
```

This example checks for the existence of a scheduled task, and then if it exists, retrieves and runs it.

```
var scheduledTaskExists = rapidResponse.scheduledTasks.exists({name:
"orderUpdatetask", scope: "Private"});
if (scheduledTaskExists){
    var schedule = rapidResponse.scheduledTasks.get({name:
"orderUpdatetask", scope: "Private"});
    schedule.runAsync();
} else {
    return ("Scheduled task does not exist.");
}
```

remove method

Deletes a scheduled task from the server, which also removes it from the `scheduledTasks` collection. Deleted scheduled tasks are no longer available to any user or process, and you can only delete scheduled tasks you own or have permission to manage.

► Syntax

```
rapidResponse.scheduledTasks.remove(task);
```

Parameter	Type	Description
task	Object	The scheduled task that will be deleted. This can be specified as either an instance of a scheduled task obtained from calling the <code>get()</code> method or a scheduled task object using the syntax <code>{name, scope}</code>

► Returns

No return value.

► Exceptions

The `remove` method can throw the following exceptions.

Exception	Description
<code>ScheduledTask.NoPermission</code>	You do not have permission to delete the specified scheduled task.
<code>ScheduledTask.NotFound</code>	The specified scheduled task does not exist.

► Example

```
rapidResponse.scheduledTasks.remove({name: "createAndModifyOrders",
scope: "Private"});
```

ScheduledTask object methods

The `ScheduledTask` object implements the following [resource object](#) methods, as well as the [access control](#) methods.

Method	Description
give (newOwner)	Changes the owner of a scheduled task.
makePublic()	Changes the scope of a scheduled task to "Public", which is the equivalent of sharing the task. Public resources can be accessed by administrators and users and groups with access.
rename (newName)	Changes the name of a scheduled task.
copy (newName)	Makes a copy of a scheduled task with the specified name.

The `ScheduledTask` object also implements the following method.

Method	Description
runAsync()	Runs the scheduled task. The task runs asynchronously.

give method

Gives a scheduled task to another user. The recipient must have permission to author scheduled tasks.

► Syntax

```
scheduledTask.give(owner);
```

where *scheduledTask* is a scheduled task object.

Parameter	Type	Description
owner	String	The user you are giving the scheduled task to. Note: You cannot give a scheduled task to a group.

► Returns

No return value.

► Exceptions

The *give* method can throw the following exceptions.

Exception	Description
ScheduledTask.NotFound	The specified scheduled task does not exist.
Principal.NotFound	The specified user does not exist.
ScheduledTask.NoPermission	You do not have permission to give the scheduled task.

► Example

```
var myTask = rapidResponse.scheduledTasks.get({
name:"orderUpdateTask", scope:"Private" });
myTask.give("myUser");
```

makePublic method

Changes the scope of the specified scheduled task from "Private" to "Public", which shares the scheduled task and allows other users to access it. This method does not specify other users or groups to share the scheduled task with, so it is available to only you and administrators.

If you call this method on a variable that represents a scheduled task, you must ensure you assign the returned public scheduled task to the same variable. Otherwise, the variable retains the scope it was created with.

You can also share the scheduled task with specific users and groups using the [accessControlList collection's add method](#).

Administrators can access any shared scheduled task, and you can allow other users and groups to access a shared scheduled task.

► Syntax

```
publicScheduledTask = scheduledTask.makePublic();
```

where *scheduledTask* is a scheduled task object.

► Returns

A ScheduledTask object that represents the same scheduled task with its scope changed to "Public".

► Exceptions

The `makePublic` method can throw the following exceptions

Exception	Description
ScheduledTask.NotFound	The specified scheduled task does not exist.
ScheduledTask.NoPermission	You do not have permission to share the scheduled task.

► Example

```
var myTask = rapidResponse.scheduledTasks.get({  
  name:"orderUpdateTask", scope:"Private" });  
myTask = myTask.makePublic();
```

rename method

Changes the name of a scheduled task. You can rename only private scheduled tasks that you own.

► Syntax

```
scheduledTask.rename(newName);
```

where *scheduledTask* is a scheduled task object.

Parameter	Type	Description
newName	String	The new name for the scheduled task.

► Returns

No return value.

► Exceptions

The `rename` method can throw the following exceptions.

Exception	Description
<code>ArgumentError</code>	A blank string, invalid string, or <code>null</code> value was specified for the <code>newName</code> parameter.
<code>ScheduledTask.NotFound</code>	The specified scheduled task does not exist.
<code>ScheduledTask.NameValidationError</code>	Either a scheduled task with the same name already exists or the specified name is not valid.
<code>ScheduledTask.NoPermission</code>	You do not have permission to rename the scheduled task.
<code>ScheduledTask.RenamePublicError</code>	The scheduled task is shared and cannot be renamed.

► Example

```
var myTask = rapidResponse.scheduledTasks.get({
  name:"orderUpdateTask", scope:"Private" });
myTask.rename("Update Orders");
```

copy method

Creates a new private scheduled task by copying an existing scheduled task. If you copy a shared scheduled task, the copy is private.

► Syntax

```
scheduledTask.copy(newName);
```

where *scheduledTask* is a scheduled task object.

Parameter	Type	Description
<code>newName</code>	String	The name for the new scheduled task.

► Returns

No return value.

► Exceptions

The `copy` method can throw the following exceptions.

Exception	Description
ArgumentError	A blank string, invalid string, or <code>null</code> value was specified for the <code>newName</code> parameter.
ScheduledTask.NotFound	The specified scheduled task does not exist.
ScheduledTask.NameValidationError	Either a scheduled task with the same name already exists or the specified name is not valid.
ScheduledTask.NoPermission	The scheduled task cannot be copied or you do not have permission to copy it.

► Example

```
var myTask = rapidResponse.scheduledTasks.get({
name:"orderUpdateTask", scope:"Private" });
myTask.copy("newAndModified");
```

runAsync method

Runs a scheduled task. When a script runs a scheduled task, the task runs asynchronously, and the script cannot wait for the task to finish or use any data returned by the task.

► Syntax

```
scheduledTask.runAsync();
```

where *scheduledTask* is a scheduled task object.

► Returns

No return value.

► Exceptions

The `runAsync` method can throw the following exceptions.

Exception	Description
ScheduledTask.AlreadyRunning	The scheduled task is already running and cannot be started again until it finishes.
ScheduledTask.LockedOut	The scheduled task is locked.
ScheduledTask.NotFound	The specified scheduled task does not exist.
ScheduledTask.ScenarioDoesNotExist	The scenario specified in the scheduled task's data settings does not exist. Note: This exception can only occur if the specified scheduled task modifies data.

► **Example**

```
var myTask = rapidResponse.scheduledTasks.get({name:
"orderUpdateTask", scope: "Private"});
myTask.runAsync();
```


CHAPTER 20: AutomationChain objects

automationChains collection	301
AutomationChain object methods	304

The AutomationChain object defines an automation chain, which is used to run a sequence of other automation tasks. All automation chains you have access to are contained in the `automationChains` collection. For more information, see "[automationChains collection](#)" on page 301.

AutomationChain objects have the same [properties as Resource objects](#).



Note: This object was added in RapidResponse 2014.1.

automationChains collection

The `automationChains` collection contains all [automation chains](#) available to the user running the script. Any automation chain retrieved or run by the script must be present in this collection.

The `automationChains` collection implements the following resource collection methods.

Method	Description
get(key)	Retrieves the automation chain object with the specified resource key.
remove(automationChain)	Deletes the specified automation chain.
exists(automationChain)	Determines whether the specified automation chain exists.



Note: This collection was added in RapidResponse 2014.1.

remove method

Deletes an automation chain from the server, which also removes it from the `automationChains` collection. Deleted automation chains are no longer available to any user or process, and you can only delete automation chains you own or have permission to manage.

► Syntax

```
rapidResponse.automationChains.remove(automationChain);
```

Parameter	Type	Description
automationChain	Object	The automation chain that will be deleted. This can be specified as either an instance of an automation chain obtained from calling the <code>get()</code> method or an automation chain object using the syntax <code>{name, scope}</code>

► Returns

No return value.

► Exceptions

The `remove` method can throw the following exceptions.

Exception	Description
AutomationChain.NoPermission	You do not have permission to delete the specified automation chain.
AutomationChain.NotFound	The specified automation chain does not exist.

► Example

```
rapidResponse.automationChains.remove({name: "Operation Sequence",  
scope: "Private"});
```



Note: This method was added in RapidResponse 2014.1.

get method

Retrieves an instance of an automation chain object.

► Syntax

```
rapidResponse.automationChains.get({name, scope});
```

Parameter	Type	Description
name	String	The name of the automation chain to retrieve.
scope	String	Whether the automation chain is Public or Private.

► Returns

An automation chain object.

► Exceptions

The `get` method can throw the following exceptions.

Exception	Description
ArgumentError	A blank String or null value was passed to the method.
AutomationChain.NoPermission	You do not have permission to create automation chains.
AutomationChain.NotFound	The specified automation chain does not exist.

► Example

```
var myChain = rapidResponse.automationChains.get({name: "Order
Sequence", scope: "Private"});
```



Note: This method was added in RapidResponse 2014.1.

exists method

Determines whether an automation chain exists. However, because RapidResponse supports multiple users working with the same resources, a user might delete the specified automation chain after this method runs. If you are using this method to test whether an automation chain exists, you should still catch a `NotFound` exception for the automation chain you are using.

► Syntax

```
autoChainExists = rapidResponse.automationChains.exists({name: "Name",
scope: "Public or Private"});
```

Parameter	Type	Description
name	String	The name of the automation chain to test for existence.
scope	String	Whether the automation chain is Private or Public.

► Returns

Type	Description
Boolean	• true if the automation chain exists • false if the automation chain does not exist

► Exceptions

The `exists` method can throw the following exceptions.

Exception	Description
ArgumentError	One of the following has occurred: • The parameter is missing a name or scope value. • The scope value specified is not "Private" or "Public".

► Example

```
var autoChainExists = rapidResponse.automationChains.exists({name:
"Order Sequence", scope: "Private"});
```

This example checks if an automation chain exists and if so, runs it.

```
var autoChainExists = rapidResponse.automationChains.exists({name:
"Order Sequence", scope: "Private"});
if (autoChainExists){
    var automation = rapidResponse.automationChains.get({name:
    "Order Sequence", scope: "Private"});
    automation.runAsync();
} else {
    return ("Automation chain does not exist.");
}
```



Note: This method was added in RapidResponse 2014.1.

AutomationChain object methods

The AutomationChain object implements the following [resource object](#) methods, as well as the [access control](#) methods.

Method	Description
give (newOwner)	Changes the owner of an automation chain.
makePublic()	Changes the scope of an automation chain to "Public", which is the equivalent of sharing the task. Public resources can be accessed by administrators and users and groups with access.
rename (newName)	Changes the name of an automation chain.
copy (newName)	Makes a copy of an automation chain with the specified name.

The AutomationChain object also implements the following method.

Method	Description
runAsync()	Runs the automation chain. The task runs asynchronously.

give method

Gives an automation chain to another user. The recipient must have permission to author automation chains.

► Syntax

```
automationChain.give(owner);
```

where *automationChain* is an automation chain object.

Parameter	Type	Description
owner	String	The user you are giving the automation chain to. Note: You cannot give an automation chain to a group.

► Returns

No return value.

► Exceptions

The `give` method can throw the following exceptions.

Exception	Description
AutomationChain.NotFound	The specified automation chain does not exist.
Principal.NotFound	The specified user does not exist.
AutomationChain.NoPermission	You do not have permission to give the automation chain.

► Example

```
var myChain = rapidResponse.automationChains.get({name: "Order
Sequence", scope: "Private"});
myChain.give("myUser");
```



Note: This method was added in RapidResponse 2014.1.

makePublic method

Changes the scope of the specified automation chain from "Private" to "Public", which shares the automation chain and allows other users to access it. This method does not specify other users or groups to share the automation chain with, so it is available to only you and administrators.

If you call this method on a variable that represents an automation chain, you must ensure you assign the returned public automation chain to the same variable. Otherwise, the variable retains the scope it was created with.

You can also share the automation chain with specific users and groups using the [accessControlList collection's add method](#).

Administrators can access any shared automation chain, and you can allow other users and groups to access a shared automation chain.

► Syntax

```
publicAutomationChain = automationChain.makePublic();
```

where *automationChain* is an automation chain object.

► Returns

An AutomationChain object that represents the same automation chain with its scope changed to "Public".

► Exceptions

The `makePublic` method can throw the following exceptions

Exception	Description
AutomationChain.NotFound	The specified automation chain does not exist.
AutomationChain.NoPermission	You do not have permission to share the automation chain.

► Example

```
var myChain = rapidResponse.automationChains.get({name: "Order
Sequence", scope: "Private"});
myChain = myChain.makePublic();
```



Note: This method was added in RapidResponse 2014.1.

rename method

Changes the name of an automation chain. You can rename only private automation chains that you own.

► Syntax

```
automationChain.rename(newName);
```

where *automationChain* is as automation chain object.

Parameter	Type	Description
newName	String	The new name for the automation chain.

► Returns

No return value.

► Exceptions

The `rename` method can throw the following exceptions.

Exception	Description
ArgumentError	A blank string, invalid string, or <code>null</code> value was specified for the <code>newName</code> parameter.
AutomationChain.NotFound	The specified automation chain does not exist.
AutomationChain.NameValidationError	Either an automation chain with the same name already exists or the specified name is not valid.

Exception	Description
AutomationChain.NoPermission	You do not have permission to rename the automation chain.
AutomationChain.RenamePublicError	The automation chain is shared and cannot be renamed.

► Example

```
var myChain = rapidResponse.automationChains.get({name: "Order
Sequence", scope: "Private"});
myChain.rename("Update Orders");
```



Note: This method was added in RapidResponse 2014.1.

copy method

Creates a new private automation chain by copying an existing automation chain. If you copy a shared automation chain, the copy is private.

► Syntax

```
automationChain.copy(newName);
```

where *automationChain* is an automation chain object.

Parameter	Type	Description
newName	String	The name for the new automation chain.

► Returns

No return value.

► Exceptions

The `copy` method can throw the following exceptions.

Exception	Description
ArgumentError	A blank string, invalid string, or <code>null</code> value was specified for the <code>newName</code> parameter.
AutomationChain.NotFound	The specified automation chain does not exist.
AutomationChain.NameValidationError	Either an automation chain with the same name already exists or the specified name is not valid.
AutomationChain.NoPermission	The automation chain cannot be copied or you do not have permission to copy it.

► Example

```
var myChain = rapidResponse.automationChains.get({name: "Order  
Sequence", scope: "Private"});  
myChain.copy("Modified Order Sequence");
```



Note: This method was added in RapidResponse 2014.1.

runAsync method

Runs an automation chain. When a script runs an automation chain, the automation chain runs asynchronously, and the script cannot wait for the automation chain to finish or use any data returned by steps in the automation chain.

► Syntax

```
automationChain.runAsync();
```

where *automationChain* is an automation chain object.

► Returns

No return value.

► Exceptions

The `runAsync` method can throw the following exceptions.

Exception	Description
CompositeTask.AlreadyRunning	The automation chain is already running and cannot start again until it has finished.
CompositeTask.LockedOut	The automation chain is locked or at least one of the resources it depends on is locked.
CompositeTask.NotFound	The specified automation chain does not exist.

► Example

```
var myChain = rapidResponse.automationChains.get({name: "Order  
Sequence", scope: "Private"});  
myChain.runAsync();
```



Note: This method was added in RapidResponse 2014.1.

CHAPTER 21: Schedule objects

schedules collection	311
Schedule object methods	313

Schedule objects define schedules that can be used to specify when an automation task runs. All tasks that use the same schedule will run at the same time. A schedule can specify a specific time to run the automation task, or to run the task on a data change or data update.

You cannot use a script to create or manage schedules, and the Schedule object does not implement any of the [Resource object](#) properties or methods.

schedules collection

The `schedules` collection contains all schedules you have access to. This includes private schedules you have created and all shared schedules.

The `schedules` collection implements the following methods.

Method	Description
get(name)	Retrieves a schedule from the collection
exists(name)	Tests whether a schedule exists

get method

Retrieves a schedule from the `schedules` collection.

► Syntax

```
rapidResponse.schedules.get( {name, scope} );
```

Parameter	Type	Description
name	String	The name of the schedule to retrieve.
scope	String	Whether the schedule is Public or Private.

► Returns

A Schedule object.

► Exceptions

The `get` method can throw the following exceptions.

Exception	Description
ArgumentError	One of the following has occurred: • The parameter is missing a name or scope value. • The scope value specified is not "Private" or "Public".
Schedule.NotFound	The specified schedule does not exist.

► Example

```
var mySchedule = rapidResponse.schedules.get({name: "Daily Tasks",  
scope: "Private"});
```

exists method

Determines whether a schedule exists. However, because RapidResponse supports multiple users working with the same resources, a user might delete the specified schedule after this method runs. If you are using this method to test whether a schedule exists, you should still catch a `NotFound` exception for the schedule you are using.

► Syntax

```
scheduleExists = rapidResponse.schedules.exists( {name, scope} );
```

Parameter	Type	Description
name	String	The name of the schedule to test for existence.
scope	String	Whether the schedule is Private or Public.

► Returns

Type	Description
Boolean	• true if the schedule exists • false if the schedule does not exist

► Exceptions

The `exists` method can throw the following exceptions.

Exception	Description
ArgumentError	One of the following has occurred: • The parameter is missing a name or scope value. • The scope value specified is not "Private" or "Public".

► Example

```
var scheduleExists = rapidResponse.schedules.exists({name: "Daily  
Tasks", scope: "Private"});
```

Schedule object methods

The Schedule object implements the following method.

Method	Description
runAutomationTasks()	Runs all automation tasks that uses the schedule

runAutomationTasks method

Runs all automation tasks associated with a schedule. This runs every task regardless of the schedule, and is equivalent to retrieving each task and calling its `runAsync()` method. Only automation tasks you have access to are run by this method.

You must be a system administrator to run this method.

► Syntax

```
Schedule.runAutomationTasks();
```

where *Schedule* is a Schedule object.

► Returns

No return value.

► Exceptions

The `runAutomationTasks` method throws the following exceptions

Exception	Description
<code>Schedule.NoPermission</code>	The user running the script is not a system administrator and does not have permission to run this method.
<code>Schedule.NotFound</code>	The specified schedule does not exist.

► Example

```
var mySchedule = rapidResponse.schedules.get({name: "Daily Tasks",
scope: "Private"});
mySchedule.runAutomationTasks();
```

CHAPTER 22: TaskFlow objects

TaskFlow objects define steps to follow to perform a task, and allow users to open other resources as required.

TaskFlow objects have the same [properties as Resource objects](#).

taskFlows collection

The taskFlows collection contains all [TaskFlow](#) objects the user running the script has access to. The taskFlows collection implements the following resource collection methods.

Method	Description
get({name, scope})	Retrieves the specified task flow.
exists(taskFlow)	Determines whether the specified task flow exists in the collection.

The task flows collection also implements the collection indexing and iteration functions. For more information, see "[Collection indexing](#)" on page 111.

get method

Retrieves a TaskFlow object.

► Syntax

```
taskflow= rapidResponse.taskFlows.get({name, scope});
```

Parameter	Type	Description
name	String	The task flow's name.
scope	String	Whether the task flow is Public or Private.

► Returns

A TaskFlow object.

► Exceptions

The `get` method can throw the following exceptions.

Exception	Description
TaskFlow.NotFound	The specified task flow does not exist or the user does not have access to it.
ArgumentError	One of the following has occurred: - The parameter is missing a name or scope value. - The scope value specified is not "Private" or "Public".

► Example

```
var tasks = rapidResponse.taskFlows.get({name:"Resolve Safety Stock
Issues", scope:"Public"});
```

exists method

Determines whether a task flow exists. However, because RapidResponse supports multiple users working with the same resources, a user might delete the specified task flow after this method runs. If you are using this method to test whether a task flow exists, you should still catch a NotFound exception for the task flow you are using.

► Syntax

```
exists = rapidResponse.taskFlows.exists({name: "Name", scope: "Public
or Private"});
```

Parameter	Type	Description
name	String	The name of the task flow to test for existence.
scope	String	Whether the task flow is Private or Public.

► Returns

Type	Description
Boolean	• true if the task flow exists • false if the task flow does not exist

► Exceptions

The `exists` method can throw the following exception.

Exception	Description
<code>ArgumentError</code>	One of the following has occurred: • The parameter is missing a name or scope value. • The scope value specified is not "Private" or "Public".

► Example

```
var taskFlowExists = rapidResponse.taskFlows.exists({name: "Resolve  
Safety Stock Issues", scope: "Public"});  
  
if (!taskFlowExists) {  
    return "Task flow does not exist."  
}
```

TaskFlow object methods

The `TaskFlow` object implements the following methods.

Method	Description
give (newOwner)	Gives the task flow to the specified user.
makePublic()	Changes the scope of a task flow to "Public", which is the equivalent of sharing the task flow. Public task flows can be accessed by administrators and users and groups with access. You can also share a task flow with a particular user or group using the accessControlList collection's add method .
rename (newName)	Changes a task flow's name.
copy (newName)	Creates a new task flow by copying an existing task flow.

give method

Gives a task flow to the specified user.

► Syntax

```
taskFlow.give(newOwner);
```

where *taskFlow* is a TaskFlow object.

Parameter	Type	Description
newOwner	String	The user to give the task flow to. Note: You cannot give a task flow to a group.

► Return value

No return value.

► Exceptions

The `give` method can throw the following exceptions.

Exception	Description
TaskFlow.NotFound	The specified task flow does not exist.
Principal.NotFound	The specified user does not exist.
TaskFlow.NonUniqueName	The specified user already owns a task flow with the same name as the one you are giving.
TaskFlow.NoPermission	You do not have permission to give the task flow.

► Example

```
var myTaskFlow = rapidResponse.taskFlows.get({ name:"Updating  
Manufacturing Regions", scope:"Private" });  
myTaskFlow.give("myUser");
```

makePublic method

Changes the scope of the specified task flow from "Private" to "Public", which shares the task flow and allows other users to access it. This method does not specify other users or groups to share the task flow with, so it is available to only you and administrators.

If you call this method on a variable that represents a task flow, you must ensure you assign the returned public task flow to the same variable. Otherwise, the variable retains the scope it was created with.

You can also share the task flow with specific users and groups using the [accessControlList collection's add method](#).

Administrators can access any shared task flow, and you can allow other users and groups to access a shared task flow.

► Syntax

```
publicTaskFlow = taskFlow.makePublic();
```

where *taskFlow* is a TaskFlow object.

► Return value

A TaskFlow object that represents the same task flow with its scope changed to "Public".

► Exceptions

The `makePublic` method can throw the following exceptions.

Exception	Description
TaskFlow.NotFound	The specified task flow does not exist.
TaskFlow.NonUniqueName	A shared task flow with the same name as the one you are sharing already exists.
TaskFlow.NoPermission	You do not have permission to share the task flow.

► Example

```
var myTaskFlow = rapidResponse.taskFlows.get({ name:"Updating  
Manufacturing Regions", scope:"Private" });  
myTaskFlow = myTaskFlow.makePublic();
```

rename method

Changes the name of a task flow.

► Syntax

```
taskFlow.rename(newName);
```

where *taskFlow* is a TaskFlow object.

Parameter	Type	Description
newName	String	The new name for the task flow.

► Return value

A TaskFlow object with the new name.

► Exceptions

The `rename` method can throw the following exceptions.

Exception	Description
<code>ArgumentError</code>	A blank string, invalid string, or <code>null</code> value was specified for the <code>newName</code> parameter.
<code>TaskFlow.NotFound</code>	The specified task flow does not exist.
<code>TaskFlow.NameValidationError</code>	Either a task flow with the same name already exists or the specified name is not valid.
<code>TaskFlow.NoPermission</code>	The task flow cannot be renamed or you do not have permission to rename it.
<code>TaskFlow.RenamePublicError</code>	The task flow is shared and cannot be renamed.

► Example

```
var myTaskFlow = rapidResponse.taskFlows.get({ name:"New Process",
scope:"Private" });
myTaskFlow = myTaskFlow.rename("Forecast Disaggregation Process");
myTaskFlow.makePublic();
```

copy method

Creates a new task flow by copying an existing task flow. If you copy a public task flow, the copy is private.

► Syntax

```
taskFlow.copy(newName);
```

where *taskFlow* is a `TaskFlow` object.

Parameter	Type	Description
<code>newName</code>	<code>String</code>	The name for the new task flow.

► Return value

No return value.

► Exceptions

The `copy` method can throw the following exceptions.

Exception	Description
ArgumentError	A blank string, invalid string, or <code>null</code> value was specified for the <code>newName</code> parameter.
TaskFlow.NotFound	The specified task flow does not exist.
TaskFlow.NameValidationError	Either a task flow with the same name already exists or the specified name is not valid.
TaskFlow.NoPermission	The task flow cannot be copied or you do not have permission to copy it.

► Example

```
var myTaskFlow = rapidResponse.taskFlows.get({ name:"Forecast
Disaggregation Process", scope:"Public" });
myTaskFlow.copy("New Forecast");
```


CHAPTER 23: Exception object

Exceptions thrown by the RapidResponse scripting object model use an Exception object, which contains the following properties.

Property	Description
name	The exception name. This value is "RapidResponseError".
errorCode	The error that occurred.
message	Information about the error. This information can be reported in the output console to help you debug the script.
stack	A stack trace of the error. This property is available only if the script uses the RapidResponse 11.2 runtime.

The following `errorCode` values are possible for any method call.

errorCode	Description
ArgumentError	Either too many arguments are provided for a method, the values supplied are the wrong type, or a <code>null</code> , <code>undefined</code> , or blank String value has been provided.
UnexpectedError	A RapidResponse error occurred during the method call.
PropertyNotWritable	A method attempted to modify a read-only property.

For each method called on resources, exceptions can be thrown with `errorCode` values that are based on the object or collection a function is called on. For each of the following exceptions, the *ResourceType* is the type of resource used in the call.

errorCode	Description
<i>ResourceType.NotFound</i>	The specified resource does not exist or you do not have access to it.
<i>ResourceType.NonUniqueName</i>	A resource of the same type with the same name already exists.
<i>ResourceType.NoPermission</i>	You do not have permission to perform the function on the resource.
<i>NotImplemented</i>	A specified method is not implemented for the resource type.

Each resource type can throw exceptions that are unique to that resource.

Resource	errorCode	Description
Scenario	Scenario.ReservedName	The name specified for a scenario is reserved internally by RapidResponse.
	Scenario.TooManyScenarios	A new scenario cannot be created because all scenario version numbers are in use.
	Scenario.HasOpenQueries	A scenario operation cannot be performed because queries are running on the scenario data.
	Scenario.Delete.NotOwnerOfAllChildren	You cannot delete a scenario because you do not own all of its children.
	Scenario.NonRootOnly	The function cannot be called on the root scenario.
	Scenario.NeedsReset	The scenario must be reset before the function can run.
	Scenario.ReadOnly	A scenario used in a workbook command call is read-only.
Script	Script.IncludeDependency.IsSelf	A called script includes your script.
	Script.CompilationError	A called script contains syntax errors.
	Script.CallDepthExceeded	Calling a script results in more than 10 nested script calls.
Workbook	Workbook.DataUpdateError	An action performed by a workbook command failed.
Worksheet	Worksheet.DataUpdateError	The data modification performed by the worksheet failed.

CHAPTER 24: ProfileVariable objects

profileVariables collection	325
--	------------

A ProfileVariable object defines a variable, which can contain a persisted value that can be used in successive script executions. For example, you can store the script's last run date in a profile variable, and use that value in the script to determine which operations of a command to run. Profile variable objects are contained in the `profileVariables` collection. For more information, see ["profileVariables collection" on page 325](#).

Profile variable objects have the following properties.

Property	Type	Description
name	String	The variable's name.
value	String	The value contained in the profile variable.

profileVariables collection

The `profileVariables` collection contains all [profile variables](#) available to the user running the script. Any profile variable retrieved or used in the script must be in this collection.

The `profileVariables` collection has the following methods.

Method	Description
get(name)	Retrieves a profile variable object with the specified name.
set(name, value)	Saves the specified value in the specified profile variable. If the variable does not exist, it is created.
setConstantVariable(name, value, principal)	Saves the specified value in a Constant profile variable for a specific user or group. If the variable does not exist, it is created.

set method

Sets the value of a profile variable object, or creates the variable if it does not exist.

► Syntax

```
rapidResponse.profileVariables.set(name, value);
```

Parameter	Type	Description
name	String	The name of the profile variable to set or create. Note: Profile variable names should only contain letters, numbers, and underscores (spaces are not allowed). Additionally, the name should start with a letter (must not start with an underscore).
value	String	The value to store in the profile variable.

► Returns

The ProfileVariable object that was created or set.

► Exceptions

The `set` method can throw the following exception.

Exception	Description
ArgumentError	A blank string or <code>null</code> value was passed to the method.

► Example

```
var runTime = rapidResponse.profileVariables.set("lastRun",
rapidResponse.dateTime.NOW);
```

setConstantVariable method

Sets the value of a Constant profile variable, or creates the variable if it does not exist. Only Constant variables can be created or modified by this method, other profile variable types can only have values specified for the user running the script using the `set` method. See ["set method" on page 326](#).

For more information about profile variable types, see the [RapidResponse Resource Authoring Guide](#).

► Syntax

```
rapidResponse.profileVariables.setConstantVariable(name, value, principal);
```

Parameter	Type	Description
name	String	The name of the profile variable to set or create
value	String	The value to store in the profile variable.
principal	String	The user or group the profile variable belongs to. The user running the script must have permission to modify macros and profile variables to set or create profile variables for other users and groups.

► Returns

The ProfileVariable object that was created or set.

► Exceptions

The `setConstantVariable` method can throw the following exception.

Exception	Description
ArgumentError	A blank string or <code>null</code> value was passed to the method.
ProfileVariable.TypeNotSupported	The specified variable is not a Constant variable.
Principal.NotFound	The specified user or group does not exist.
AccessControl.NoPermission	You do not have permission to modify profile variables.

► Example

```
var newVar = rapidResponse.profileVariables.setConstantVariable("FilterLevel", 1, "Production Planners");
```

get method

Retrieves an instance of a profile variable as an object.

► Syntax

```
rapidResponse.profileVariables.get(name, principal);
```

Parameter	Type	Description
name	String	The name of the profile variable to retrieve.
principal	String	Optionally, the user or group the profile variable is defined for. If not provided, a variable belonging to the user running the script is retrieved. The user running the script must have permission to modify macros and profile variables to retrieve profile variables for other users and groups.

► Returns

The ProfileVariable object with the specified name.

► Exceptions

The `get` method can throw the following exceptions.

Exception	Description
ArgumentError	A blank string or <code>null</code> value was passed to the method.
ProfileVariable.NotFound	No profile variable with the specified name exists.
AccessControl.NoPermission	You do not have permission to modify macros and profile variables for other users and groups.
Principal.NotFound	The specified user or group does not exist.

► Example

The following example retrieves a profile variable for the user running the script.

```
var profile = rapidResponse.profileVariables.get("lastRun");
```

The following example retrieves a profile variable for the Buyers group.

```
var profile = rapidResponse.profileVariables.get("SupplierName", "Buyers");
```


CHAPTER 25: ProcessingRule object

processingRules collection	329
---	------------

A ProcessingRule object defines a processing rule or system setting in RapidResponse. For example, the value of the Client_AllowUserProfilePictures processing rule determines whether users' profile pictures are displayed in RapidResponse. ProcessingRule objects are contained in the ["processingRules collection" on page 329](#).

ProcessingRule objects have the following properties.

Property	Type	Description
id	String	The ID of the processing rule.
value	String	The value or setting of the processing rule.



Note: This object was added in RapidResponse 2015.3.

processingRules collection

The `processingRules` collection contains all [processing rules](#) available to the user running the script. Any processing rule used in the script must be in this collection.

The `processingRules` collection has the following method.

Method	Description
get(id)	Retrieves the value of the processing rule with the specified ID.



Note: This collection was added in RapidResponse 2015.3.

get method

Retrieves the value of a processing rule.

► Syntax

```
rapidResponse.processingRules.get(id);
```

Parameter	Type	Description
id	String	The ID of the processing rule the value is being retrieved for.

► Returns

The value of the specified processing rule.

► Exceptions

The `get` method can throw the following exceptions.

Exception	Description
ArgumentError	A blank string or <code>null</code> value was passed to the method.

► Example

This example determines whether profile pictures are turned on in the Java client

```
rapidResponse.processingRules.get("Client_
AllowUserProfilePictures").value;
```



Note: This method was added in RapidResponse 2015.3.

CHAPTER 26: Dashboard objects

widgetReferences collection	332
dashboards collection	333
Dashboard object methods	334

The Dashboard object defines dashboards, which display information from widgets. All dashboards available to the user running the script are contained in the dashboards collection.

In addition to the [resource object](#) properties, dashboard objects have the following property.

Property	Type	Description
backgroundColor	String	The dashboard's background, as an RGB color code.
preferences	String	Your personal viewing preferences for the dashboard.
showDataSettingsDialogOnOpen	Boolean	Whether the Data Settings dialog box is displayed when the dashboard is opened.
widgetReferences	Object	A collection of WidgetReference objects that defines the widgets used in this dashboard.
widgetStyleSettings	Object	A WidgetStyleSettings object that defines the layout for each widget.

The WidgetStyleSettings object contains the following properties.

Property	Type	Description
showTitle	Boolean	Whether the widgets' titles are displayed in the dashboard
showBorder	Boolean	Whether the widgets display borders

Property	Type	Description
paddingTop	Number	The number of pixels inserted above each widget.
paddingLeft	Number	The number of pixels inserted to the left of each widget.
paddingBottom	Number	The number of pixels inserted below each widget.
paddingRight	Number	The number of pixels inserted to the right of each widget.

widgetReferences collection

The `widgetReferences` collection contains all widgets included in a dashboard, represented as [WidgetReference](#) objects.

The `widgetReferences` collection implements only the collection iteration functions. For more information, see ["Resource collections" on page 110](#).

WidgetReference objects

A `WidgetReference` object represents a widget used in a dashboard. The `WidgetReference` object has the following properties.

Property	Type	Description
widgetKey	Object	The widget's identifier in the <code>widgets</code> collection. For more information, see "widgets collection" on page 343 .
referenceType	String	How the widget is used in the dashboard.
controlSettings	Object	A ControlSettings object that defines the data display settings for the widget.
bucketSettings	Object	A BucketSettings object that defines the buckets in the widget's worksheet.
workbookPreferences	Object	A WorkbookPreferences object that defines the user settings for the widget's workbook.
title	String	The widget's title in the dashboard.
height	Number	The widget's height in pixels.
layoutId	String	The widget's layout.

For more information about widgets, see ["Widget objects" on page 341](#).

dashboards collection

The `dashboards` collection contains all dashboards available to the user running the script. Any dashboard retrieved or used by the script must be present in this collection.

The dashboards collection implements the following resource collection methods.

Method	Description
get (key)	Retrieves a dashboard object
exists (dashboard)	Determines whether the specified dashboard exists in the collection

get method

Retrieves an instance of a dashboard object, which allows the dashboard to be used in other methods and processes.

► Syntax

```
Dashboard = rapidResponse.dashboards.get({name, scope});
```

Parameter	Description
name	The name of the dashboard to retrieve.
scope	Whether the dashboard is private or public.

► Returns

A dashboard object.

► Exceptions

The `get` method can throw the following exceptions.

Exception	Description
<code>ArgumentError</code>	A blank or null value was passed for a parameter.
<code>Dashboard.NotFound</code>	The specified dashboard does not exist or you do not have access to it.

► Example

```
var dash = rapidResponse.dashboards.get({name: 'SCM Dashboard', scope: 'Public'});
```

exists method

Determines whether a specific dashboard exists. However, because RapidResponse supports multiple users working with the same resources, a user might delete the specified dashboard after this method runs. If you are using this method to test whether a dashboard exists, you should still catch a `NotFound` exception.

► Syntax

```
dashboardExists = rapidResponse.dashboards.exists({name: "Name",  
scope: "Public or Private"});
```

Parameter	Type	Description
name	String	The name of the resource to test for existence.
scope	String	Whether the resource is Private or Public.

► Returns

Type	Description
Boolean	• true if the dashboard exists • false if the dashboard does not exist

► Exceptions

The exists method can throw the following exceptions.

Exception	Description
ArgumentError	One of the following has occurred: • The resource parameter is missing a name or scope value. • The scope value specified is not "Private" or "Public".

► Example

```
var dashboardExists = rapidResponse.dashboards.exists({name: "SCM  
Dashboard", scope: "Public"});
```

Dashboard object methods

The Dashboard object implements the following methods.

Method	Description
give(newOwner)	Gives the dashboard to the specified user.
makePublic()	Changes the scope of a dashboard to "Public", which is the equivalent of sharing the dashboard. Public dashboards can be accessed by administrators and users and groups with access. You can also share a dashboard with a particular user or group using the accessControlList collection's add method .
rename(newName)	Changes a dashboard's name.
copy(newName)	Creates a new dashboard by copying an existing dashboard.
getLayout()	Retrieves the dashboard layout.
createEmbeddedLinkUrl()	Creates a link to the current view of the dashboard.
createLink	Creates a link to the current view of a dashboard in a collaboration.

give method

Gives a dashboard to the specified user.

► Syntax

```
dashboard.give(newOwner) ;
```

where *dashboard* is a Dashboard object.

Parameter	Type	Description
newOwner	String	The user to give the dashboard to. Note: You cannot give a dashboard to a group.

► Return value

No return value.

► Exceptions

The `give` method can throw the following exceptions.

Exception	Description
Dashboard.NotFound	The specified dashboard does not exist.
Principal.NotFound	The specified user does not exist.

Exception	Description
Dashboard.NonUniqueName	The specified user already owns a dashboard with the same name as the one you are giving.
Dashboard.NoPermission	You do not have permission to give the dashboard.

► Example

```
var myDashboard = rapidResponse.dashboards.get({ name:"Order
Satisfaction", scope:"Private" });
myDashboard.give("myUser");
```

makePublic method

Changes the scope of the specified dashboard from "Private" to "Public", which shares the dashboard and allows other users to access it. This method does not specify other users or groups to share the dashboard with, so it is available to only you and administrators.

If you call this method on a variable that represents a dashboard, you must ensure you assign the returned public dashboard to the same variable. Otherwise, the variable retains the scope it was created with.

You can also share the dashboard with specific users and groups using the [accessControllist collection's add method](#).

Administrators can access any shared dashboard, and you can allow other users and groups to access a shared dashboard.

► Syntax

```
publicDashboard = dashboard.makePublic();
```

where *dashboard* is a Dashboard object.

► Return value

A Dashboard object that represents the same dashboard with its scope changed to "Public".

► Exceptions

The `makePublic` method can throw the following exceptions.

Exception	Description
Dashboard.NotFound	The specified dashboard does not exist.
Dashboard.NonUniqueName	A shared dashboard with the same name as the one you are sharing already exists.
Dashboard.NoPermission	You do not have permission to share the dashboard.

► Example

```
var myDashboard = rapidResponse.dashboards.get({ name:"Order
Satisfaction", scope:"Private" });
myDashboard = myDashboard.makePublic();
```

rename method

Changes the name of a dashboard.

► Syntax

```
dashboard.rename(newName);
```

where *dashboard* is a Dashboard object.

Parameter	Type	Description
newName	String	The new name for the dashboard.

► Return value

A Dashboard object with the new name.

► Exceptions

The `rename` method can throw the following exceptions.

Exception	Description
ArgumentError	A blank string, invalid string, or <code>null</code> value was specified for the <code>newName</code> parameter.
Dashboard.NotFound	The specified dashboard does not exist.
Dashboard.NameValidationError	Either a dashboard with the same name already exists or the specified name is not valid.
Dashboard.NoPermission	The dashboard cannot be renamed or you do not have permission to rename it.
Dashboard.RenamePublicError	The dashboard is shared and cannot be renamed.

► Example

```
var myDashboard = rapidResponse.dashboards.get({ name:"Revenue
Breakdown", scope:"Private" });
myDashboard = myDashboard.rename("Revenue Analysis");
myDashboard.makePublic();
```

copy method

Creates a new dashboard by copying an existing dashboard. If you copy a public dashboard, the copy is private.

► Syntax

```
dashboard.copy(newName);
```

where *dashboard* is a Dashboard object.

Parameter	Type	Description
newName	String	The name for the new dashboard.

► Return value

No return value.

► Exceptions

The `copy` method can throw the following exceptions.

Exception	Description
ArgumentError	A blank string, invalid string, or <code>null</code> value was specified for the <code>newName</code> parameter.
Dashboard.NotFound	The specified dashboard does not exist.
Dashboard.NameValidationError	Either a dashboard with the same name already exists or the specified name is not valid.
Dashboard.NoPermission	The dashboard cannot be copied or you do not have permission to copy it.

► Example

```
var myDashboard = rapidResponse.dashboards.get({ name:"Order  
Satisfaction", scope:"Public" });  
myDashboard.copy("Order Fulfillment");
```

getLayout method

Retrieves the layout of a dashboard.

► Syntax

```
dashboardLayout = dashboard.getLayout();
```

where *dashboard* is a Dashboard object.

► Returns

A dashboard layout.

► Exceptions

The `getLayout` method does not throw exceptions.

► Example

```
var dash = rapidResponse.dashboards.get({name: 'SCM Dashboard', scope: 'Public'});  
var layout = dash.getLayout();
```

createEmbeddedLinkUrl method

Creates a link to the current view of the dashboard, including the widgets displayed and the scenario, filter, site, and other data display settings specified. This link can be sent to users and groups.

When you create a link, it is added to the [links collection](#).

► Syntax

```
link = Dashboard.createEmbeddedLinkUrl();
```

where *Dashboard* is a dashboard object.

► Returns

A String value.

► Exceptions

The `createEmbeddedLinkUrl` method does not throw exceptions.

► Example

```
var dash = rapidResponse.dashboards.get({name: "My Dashboard", scope: "Public"});  
var link = dash.createEmbeddedLinkUrl();
```

createLink method

Creates a link in a collaboration to the current view of a dashboard, including the widgets displayed and with the scenario, filter, site, and other data display settings specified.

When you create a link, it is added to the [links collection](#).

► Syntax

```
link = Dashboard.createLink();
```

where *Dashboard* is a dashboard object.

► **Returns**

A String value.

► **Exceptions**

The `createLink` method does not throw exceptions.

► **Example**

```
var dash = rapidResponse.dashboards.get({name: "My Dashboard", scope:
"Public"});
var link = dash.createLink();
```



Note: This method was added in RapidResponse 2016.2

CHAPTER 27: Widget objects

ControlSettings objects	342
widgets collection	343
Widget object methods	345

Widget objects are used to display data in dashboards. All widgets available to the script are contained in the widgets collection.

In addition to the resource object properties, the Widget object has the following properties.

Property	Type	Description
settings	Object	A WidgetWorksheetSettings object that defines the data used to display data in the widget.
type	String	The type of widget.
title	String	The widget's title as displayed in the dashboard.
help	String	The help defined for the widget.

The WidgetWorksheetSettings object contains the following properties.

Property	Type	Description
workbookKey	Object	The workbook the widget is contained in.
displayType	String	How the widget is displayed
worksheetId	String	The worksheet identifier for the worksheet the widget is defined in.
controlSettings	Object	A ControlSettings object that defines the data display settings for the widget.

ControlSettings objects

The ControlSettings object contains the data display settings used to view data in a widget. The ControlSettings object has the following properties.

Property	Type	Description
hierarchies	Object	The HierarchySetting object that defines the hierarchy used to display data in the widget.
siteGroup	Object	The site or site filter used in the widget.
filter	Object	The filter object used in the widget.
pool	String	The pool used to display data in the widget.
processInstance	String	The process this widget is used in.
constraint	String	The active constraint displayed in the widget.
referencePart	String	The reference part used in the widget.
scenarios	Object	The Scenario object or objects displayed in the widget.
variables	Object	A collection of NameValueDisplayValue objects that define the variables and values used in the widget. For list variables, these must be the display values that correspond to list items.
model	String	The model displayed in the widget.
project	String	The project displayed in the widget.
workCenter	String	The work center displayed in the widget.
part	String	The part displayed in the widget.

The HierarchySetting object contains the same properties as the [WorkbookHierarchyInitialization](#) object.

The NameValueDisplayValue object contains the following properties.

Property	Type	Description
name	String	The variable name.
value	String	The value provided for the variable.
displayValue	String	The value displayed for the specified value. This value should be used for all processes that include the variable.

widgets collection

The `widgets` collection contains all widgets available to the user running the script. Any widget retrieved or used by the script must be present in this collection.

The widgets collection implements the following resource collection methods.

Method	Description
get (key)	Retrieves a widget object
exists (widget)	Determines whether the specified widget exists in the collection

exists method

Determines if a specific widget exists. However, because RapidResponse supports multiple users working with the same resources, a user might delete the specified widget after this method runs. If you are using this method to test whether a widget exists, you should still catch a `NotFound` exception.

► Syntax

```
widgetExists = rapidResponse.widgets.exists({name: "Name", scope: "Public or Private"});
```

Parameter	Type	Description
name	String	The name of the widget to test for existence.
scope	String	Whether the widget is Private or Public.

► Returns

Type	Description
Boolean	• true if the widget exists • false if the widget does not exist

► Exceptions

The `exists` method can throw the following exceptions.

Exception	Description
ArgumentError	One of the following has occurred: - The resource parameter is missing a name or scope value. - The scope value specified is not "Private" or "Public".

► Example

```
var widgetExists = rapidResponse.widgets.exists({name: "Gross Margin",
scope: "Public"});
```

get method

Retrieves an instance of a widget object.

► Syntax

```
widgetObject = rapidResponse.widgets.get({name:'Name', scope:'Public
or Private'});
```

Parameter	Description
name	The name of the widget to retrieve.
scope	Whether the widget is private or public.

► Returns

A widget object

► Exceptions

The `get` method can throw the following exceptions.

Exception	Description
ArgumentError	A blank or null value was passed for a parameter.
Widget.NotFound	The specified widget does not exist or you do not have access to it.

► Example

```
var marginWidget = rapidResponse.widgets.get({name: 'Gross Margin',
scope: 'Public'});
```


Widget object methods

The Widget object implements the following methods.

Method	Description
give(newOwner)	Gives the widget to the specified user.
makePublic()	Changes the scope of a widget to "Public", which is the equivalent of sharing the widget. Public widgets can be accessed by administrators and users and groups with access. You can also share a widget with a particular user or group using the accessControlList collection's add method .
rename(newName)	Changes a widget's name.
copy(newName)	Creates a new widget by copying an existing widget.
getAttachments()	Retrieves the widget's attachments.

give method

Gives a widget to the specified user.

► Syntax

```
widget.give(newOwner);
```

where *widget* is a Widget object.

Parameter	Type	Description
newOwner	String	The user to give the widget to. Note: You cannot give a widget to a group.

► Return value

No return value.

► Exceptions

The `give` method can throw the following exceptions.

Exception	Description
Widget.NotFound	The specified widget does not exist.
Principal.NotFound	The specified user does not exist.
Widget.NonUniqueName	The specified user already owns a widget with the same name as the one you are giving.
Widget.NoPermission	You do not have permission to give the widget.

► Example

```
var myWidget = rapidResponse.widgets.get({ name:"Order Cost Chart",
scope:"Private" });
myWidget.give("myUser");
```

makePublic method

Changes the scope of the specified widget from "Private" to "Public", which shares the widget and allows other users to access it. This method does not specify other users or groups to share the widget with, so it is available to only you and administrators.

If you call this method on a variable that represents a widget, you must ensure you assign the returned public widget to the same variable. Otherwise, the variable retains the scope it was created with.

You can also share the widget with specific users and groups using the [accessControlList collection's add method](#).

Administrators can access any shared widget, and you can allow other users and groups to access a shared widget.

► Syntax

```
publicWidget = widget.makePublic();
```

where *widget* is a Widget object.

► Return value

A Widget object that represents the same widget with its scope changed to "Public".

► Exceptions

The `makePublic` method can throw the following exceptions.

Exception	Description
Widget.NotFound	The specified widget does not exist.
Widget.NonUniqueName	A shared widget with the same name as the one you are sharing already exists.
Widget.NoPermission	You do not have permission to share the widget.

► Example

```
var myWidget = rapidResponse.widgets.get({ name:"Order Cost Chart",
scope:"Private" });
myWidget = myWidget.makePublic();
```

rename method

Changes the name of a widget.

► Syntax

```
widget.rename(newName);
```

where *widget* is a Widget object.

Parameter	Type	Description
newName	String	The new name for the widget.

► Return value

A Widget object with the new name.

► Exceptions

The `rename` method can throw the following exceptions.

Exception	Description
ArgumentError	A blank string, invalid string, or <code>null</code> value was specified for the <code>newName</code> parameter.
Widget.NotFound	The specified widget does not exist.
Widget.NameValidationError	Either a widget with the same name already exists or the specified name is not valid.
Widget.NoPermission	The widget cannot be renamed or you do not have permission to rename it.
Widget.RenamePublicError	The widget is shared and cannot be renamed.

► Example

```
var myWidget = rapidResponse.widgets.get({ name:"Late Revenue",
scope:"Private" });
myWidget = myWidget.rename("Late Revenue Chart");
myWidget.makePublic();
```

copy method

Creates a new widget by copying an existing widget. If you copy a public widget, the copy is private.

► Syntax

```
widget.copy(newName);
```

where *widget* is a Widget object.

Parameter	Type	Description
newName	String	The name for the new widget.

► Return value

No return value.

► Exceptions

The `copy` method can throw the following exceptions.

Exception	Description
ArgumentError	A blank string, invalid string, or <code>null</code> value was specified for the <code>newName</code> parameter.
Widget.NotFound	The specified widget does not exist.
Widget.NameValidationError	Either a widget with the same name already exists or the specified name is not valid.
Widget.NoPermission	The widget cannot be copied or you do not have permission to copy it.

► Example

```
var myWidget = rapidResponse.widgets.get({ name:"Order Cost Chart",
scope:"Public" });
myWidget.copy("Order Costs");
```

getAttachments method

Retrieves images and linked files that are defined in a text widget. Only images that have been uploaded and attached, not those from the image library, are returned by this method.

► Syntax

```
widget.getAttachments();
```

where *widget* is a Widget object.

► Return value

An object that contains linked images and files, with the following properties.

Property	Description
name	The linked file's name.
value	The base64-encoded representation of the linked file.

► Exceptions

The `getAttachments` method does not throw exceptions.

► Example

```
var myWidget = rapidResponse.widgets.get( {name: "Generic Help",  
scope: "Public"} );  
var attached = myWidget.getAttachments();
```



Note: This method was added in RapidResponse 2013.4.

CHAPTER 28: Form objects

forms collection	351
Form object methods	354
formDependencies collection	363
Action objects	363
closeFormAction method	365
Response object	365
ControlMessage object	366
Notice object	366
ErrorResponse object	366

The Form object defines a form. A form is a customized user interface for input values that executes an underlying script when run. Form objects are contained in the [forms](#) collection.

Form objects have the same [properties as Resource objects](#) .



Note: This object was added in RapidResponse 2014.4.

forms collection

The forms collection contains all forms available to the user running the script. The forms collection implements the following resource collection methods.

Method	Description
get (key)	Retrieves a form.
exists (form)	Determines whether the specified form exists in the collection.
remove (form)	Deletes the specified form.

The form collection also implements the [resource enumeration](#) functions.



Note: This collection was added in RapidResponse 2014.4.

get method

Retrieves an instance of a form.

► Syntax

```
rapidResponse.forms.get( {name, scope} );
```

Parameter	Type	Description
name	String	The name of the form.
scope	String	Whether the form is public or private.

► Return value

A Form object.

► Exceptions

The `get` method can throw the following exceptions.

Exception	Description
ArgumentError	A null value was passed to a parameter.
Form.NotFound	The specified form does not exist or you do not have access to it.

► Example

```
var form = rapidResponse.forms.get( {name:"Start Activity",
scope: "Public" } );
```

exists method

Determines whether a specific form exists in the collection. However, because `RapidResponse` supports multiple users working with the same resources, a user might delete the specified form after this method runs. If you are using this method to test whether a form exists, you should still catch a `NotFound` exception.

► Syntax

```
rapidResponse.forms.exists({name: "Name", scope:"Public or Private"});
```

Parameter	Type	Description
name	String	The name of the form.
scope	String	Whether the form is public or private.

► Return value

Type	Description
Boolean	• true if the form exists • false if the form does not exist

► Exceptions

The `exists` method can throw the following exception.

Exception	Description
<code>ArgumentError</code>	One of the following has occurred: • The resource parameter is missing a name or scope value. • The scope value specified is not "Private" or "Public".

► Example

```
var formExists = rapidResponse.forms.exists({name: "Finish Activity",  
scope: "Public"});
```

remove method

Deletes a form from the server, which also removes it from the `forms` collection. Deleted forms are no longer available to any user or process, and you can only delete forms you own or have permission to manage.

► Syntax

```
rapidResponse.forms.remove(form);
```

Parameter	Type	Description
form	Object	The form to delete. This can be specified as a Form object or using the {name, scope} syntax.

► Return value

No return value.

► Exceptions

The `remove` method can throw the following exceptions.

Exception	Description
Form.NoPermission	You do not have permission to delete the specified form.
Form.NotFound	The specified form does not exist.

► Example

```
rapidResponse.forms.remove({name:"Start Activity", scope:"Public"});
```

Form object methods

The Form object implements the following [Resource object](#) methods.

Method	Description
give (newOwner)	Gives the form to another user.
makePublic()	Changes the scope of a form to "Public", which is the equivalent of sharing the form. Public forms can be accessed by administrators and users and groups with access. You can also share a forms with a particular user or group using the accessControllist collection's add method .
rename (newName)	Changes the form's name.
copy (newName)	Creates a new form by copying an existing one.

The Form object also implements the following methods.

Method	Description
<code>okCancelNotice(message, okAction, cancelAction)</code>	Creates a notice with OK and Cancel buttons.
<code>yesNoNotice(message, yesAction, noAction)</code>	Creates a notice with Yes and No buttons.
<code>yesNoCancelNotice(message, yesAction, noAction), cancelAction</code>	Creates a notice with Yes, No, and Cancel buttons.
<code>autoCloseNotice(message)</code>	Creates a notice that briefly displays and then closes automatically.

give method

Gives a form to another user.

► Syntax

```
form.give(newOwner);
```

where *form* is a Form object.

Parameter	Type	Description
<code>newOwner</code>	String	The user you are giving the form to. Note: You cannot give a form to a group.

► Return value

No return value.

► Exceptions

The `give` method can throw the following exceptions.

Exception	Description
<code>form.NotFound</code>	The specified form does not exist.
<code>Principal.NotFound</code>	The specified user does not exist.
<code>form.NoPermission</code>	You do not have permission to give the form.

► Example

```
var form = rapidResponse.forms.get({ name:"Start Activity",
scope:"Public" });
form.give("myUser");
```

makePublic method

Changes the scope of the specified form from "Private" to "Public", which shares the form and allows other users to access it. This method does not specify other users or groups to share the form with, so it is available to only you and administrators.

If you call this method on a variable that represents a form, you must ensure you assign the returned public form to the same variable. Otherwise, the variable retains the scope it was created with.

You can also share the form with specific users and groups using the [accessControlList collection's add method](#).

Administrators can access any shared form, and you can allow other users and groups to access a shared form.

► Syntax

```
publicForm = form.makePublic();
```

where *form* is a Form object.

► Return value

A Form object that represents the same form with its scope changed to "Public".

► Exceptions

The `makePublic` method can throw the following exceptions.

Exception	Description
Form.NotFound	The specified form does not exist.
Form.NonUniqueName	A shared form with the same name as the one you are sharing already exists.
Form.NoPermission	You do not have permission to share the form.

► Example

```
var form= rapidResponse.forms.get({ name:"Finish Activity",  
scope:"Private" });  
form = form.makePublic();
```

rename method

Changes the name of a form.

► Syntax

```
form.rename(newName);
```

where *form* is a Form object.

Parameter	Type	Description
newName	String	The new name for the form.

► Return value

No return value.

► Exceptions

The `rename` method can throw the following exceptions.

Exception	Description
ArgumentError	A blank string, invalid string, or <code>null</code> value was specified for the <code>newName</code> parameter.
Form.NotFound	The specified form does not exist.
Form.NameValidationError	Either a form with the same name already exists or the specified name is not valid.
Form.NoPermission	You do not have permission to rename the form.
Form.RenamePublicError	The form is shared and cannot be renamed.

► Example

```
var form = rapidResponse.forms.get({ name:"Finish Activity",  
scope:"Private" });  
form.rename("Finish Order Input");
```

copy method

Creates a new form by copying an existing form.

► Syntax

```
form.copy(newName) ;
```

where *form* is a Form object.

Parameter	Type	Description
newName	String	The name of the new form.

► Return value

No return value.

► Exceptions

The `copy` method can throw the following exceptions.

Exception	Description
<code>ArgumentError</code>	A blank string, invalid string, or <code>null</code> value was specified for the <code>newName</code> parameter.
<code>Form.NotFound</code>	The specified form does not exist.
<code>Form.NameValidationError</code>	Either a form with the same name already exists or the specified name is not valid.
<code>Form.NoPermission</code>	The form cannot be copied or you do not have permission to copy it.

► Example

This example copies a form.

```
var form = rapidResponse.forms.get({ name:"Finished", scope:"Private"
});
form.copy("Finish Activity");
```

This example copies a form, and then gives the copy to a user who will modify it.

```
var form = rapidResponse.forms.get({ name:"Edit Scenario",
scope:"Public" });
form.copy("Edit My Scenario");
var formGive = rapidResponse.forms.get( {name:"Edit My Scenario",
scope:"Private"} );
formGive.give("formManagement");
```

okCancelNotice method

Creates a notice with OK and Cancel buttons.

► Syntax

```
form.okCancelNotice(message, okAction, cancelAction);
```

where *form* is a Form object.

Parameter	Type	Description
<code>message</code>	String	The message text that displays in the notice.
<code>okAction</code>	Object	The action performed when the OK button is clicked or tapped.
<code>cancelAction</code>	Object	The action performed when the Cancel button is clicked or tapped. This parameter is optional. If a value is not provided, the default value is <code>DismissNoticeAction</code> .

► Return value

A Notice object.

► Exceptions

No exceptions.

► Example

This example creates a notice that displays a warning message about overstocked inventory where the user can click OK to run the "Disregard" script action to continue or click Cancel to run the "Abort" script action, which cancels the action.

```
return myForm.response({  
  
    notice: myForm.okCancelNotice("Inventory overstock warning.",  
    myForm.runScriptAction ("Disregard"), myForm.runScriptAction  
    ("Abort")),  
    data: {  
        scenarioId: myScenario.id  
    }  
});
```

In this example, the Disregard function can ignore the overstock condition and commit the working scenario, and can be defined as follows:

```
function Disregard() {  
    var form = rapidResponse.scriptArguments["#Form"];  
    var scenario = rapidResponse.scenarios.get(form.data.scenarioId);  
  
    var name = scenario.name;  
  
    scenario.commit();  
    //Respond with an auto-closing message  
    return form.response ({  
        notice: form.autoCloseNotice("Committed the " + name + "  
        scenario.", "Success")  
    });  
}
```

The Abort function can then delete the working scenario, and can be defined as follows:

```
function Abort() {  
    var form = rapidResponse.scriptArguments["#Form"]  
    rapidResponse.scenarios.remove(form.data.scenarioId);  
    //Respond with an auto-closing message  
    return form.response ({  
        notice: form.autoCloseNotice("Deleted the working  
        scenario", "Success")  
    });  
}
```

yesNoNotice method

Creates a notice with Yes and No buttons.

► Syntax

```
form.yesNoNotice(message, yesAction, noAction);
```

where *form* is a Form object.

Parameter	Type	Description
message	String	The message text that displays in the notice.
yesAction	Object	The action performed when the Yes button is clicked or tapped.
noAction	Object	The action performed when the No button is clicked or tapped. This parameter is optional. If a value is not provided, the default value is <code>DismissNoticeAction</code> .

► Return value

A Notice object.

► Exceptions

No exceptions.

► Example

This example creates a notice that displays a warning message about being below a base threshold for inventory, where the user can click Yes to run the "Continue" script action to continue executing the script, or click "No" to cancel the action.

```
return myForm.response({
    notice: myForm.yesNoNotice("You are below the inventory base
    threshold. Do you want to continue?", myForm.runScriptAction
    ("Continue"), myForm.closeFormAction()),
    data: {
        scenarioId: myScenario.id
    }
});
```

The Continue function in this example could commit the working scenario, and can be defined as follows:

```
function Continue() {
    var form = rapidResponse.scriptArguments["#Form"]
    rapidResponse.scenarios.get(form.data.scenarioId).commit();
    //Respond with an auto-closing message
    return form.response ({
        notice: form.autoCloseNotice("Process complete", "Success")
    });
}
```



```
    });  
}
```

yesNoCancelNotice method

Creates a notice with Yes, No, and Cancel buttons.

► Syntax

```
form.yesNoCancelNotice(message, yesAction, noAction, cancelAction);
```

where *form* is a Form object.

Parameter	Type	Description
message	String	The message text that displays in the notice.
yesAction	Object	The action performed when the Yes button is clicked or tapped.
noAction	Object	The action performed when the No button is clicked or tapped.
cancelAction	Object	The action performed when the Cancel button is clicked or tapped. This parameter is optional. If a value is not provided, the default value is <code>DismissNoticeAction</code> .

► Return value

A Notice object.

► Exceptions

No exceptions.

► Example

This example creates a notice that displays a message about an option to automatically generate a new product number for the product you are inputting, where the user can click Yes to perform the "Yes" script action to automatically generate the number, click No to dismiss the form notice and specify different input values, or "Cancel" to cancel the action.

```
return myForm.response( {  
    notice: myForm.yesNoCancelNotice("Do you want to automatically  
        generate a new product number?", myForm.runScriptAction("Yes"),  
        myForm.dismissNoticeAction(), myForm.closeFormAction())  
});
```

In this example, the Yes function can generate a product number based on the user running the script and a random number between 1 and 1000, and can be defined as follows.

```
function Yes() {  
    var form = rapidResponse.scriptArguments["#Form"]
```

```

    var prodNumber = rapidResponse.loggedInUser.id + Math.floor
      ((Math.random() * 1000)+1);
    return form.response ({
      notice: form.autoCloseNotice("Created product number" +
        prodNumber, "Success")
    });
  }
}

```

The response that displays the notice does not require a `data` element because the function does not require any data from the script.

autoCloseNotice method

Creates a notice on the form that briefly displays and then automatically closes

► Syntax

```
form.autoCloseNotice(message, type);
```

where *form* is a Form object.

Parameter	Type	Description
message	String	The message text that displays in the notice.
type	String	Optionally, the type of notice to create. This value can be Error, Info, Question, Success, or Warning. If not specified, the Success type is displayed. For more information about notice types, see "Notices" on page 85 .

► Return value

A Notice object.

► Exceptions

No exceptions.

► Example

This example creates a notice that displays a message informing users that an activity in the S&OP cycle has finished. The notice then automatically closes.

```
myForm.autoCloseNotice("Activity finished.")
```

This example creates a warning message informing users records were inserted by the script and informing them that records inserted into a keyless table might not be unique. The notice then automatically closes.

```
myForm.autoCloseNotice("Records inserted. The table these records are
inserted into does not have key fields, so these records might not be
unique.", "Warning")
```

formDependencies collection

The formDependencies collection contains all forms that a workbook depends on, represented as [FormDependency](#) objects.

The dependencies collection implements only the collection iteration functions. For more information, see "[Resource collections](#)" on page 110.

FormDependency object

FormDependency objects define forms that a workbook depends on for functionality. The FormDependency object has the following properties.

Property	Type	Description
dependentId	String	The dependency identifier.
controlMappings	Object	A collection of FormDependencyControlMapping objects that define how the variables in the dependent form map to the variables in the workbook.
formKey	Object	The Form object the dependency exists for.

The FormDependencyControlMapping object has the following properties.

Property	Type	Description
targetId	String	The control identifier in the target form.
sourceId	String	The variable identifier in the source workbook.

Action objects

The Action object defines the type of action performed by a form. It is associated with a Button object and is used to specify the action that takes place when users press the button in a notice on the form.

The Action object has the following property.

Property	Type	Description
type	String	The type of action. This can be one of the following: dismissNoticeAction : Closes the form notice. The form itself remains open. runScriptAction : Reruns the script, calling the function specified by its globalFunctionName property. closeFormAction : Closes the form.

dismissNoticeAction method

The dismissNoticeAction method defines an action that closes the form notice. The form remains open. To close the form, the closeFormAction method must be invoked. For more information, see ["closeFormAction method" on page 365](#).

Syntax

```
myForm.dismissNoticeAction()
```

where *myForm* is a Form object.

Returns

No return value.

Exceptions

No exceptions.

Example

```
var notice = myForm.yesNoNotice("The specified values do not match a  
record. Specify a different combination?", myForm.dismissNoticeAction  
( ), myForm.closeFormAction());
```

runScriptAction method

The main script function is executed when the form is run. The runScriptAction method runs the script again and executes a function.

Syntax

```
myForm.runScriptAction(function)
```

where *myForm* is a Form object.

Argument	Description
function	The name of the script function to run.

Returns

No return value.

Exceptions

No exceptions.

Example

```
var notice = myForm.yesNoNotice("Operations complete. Do you want to  
commit the working scenario?", myForm.runScriptAction("commit"),  
myForm.closeFormAction());
```

closeFormAction method

The closeFormAction method defines an action that closes the form.

Syntax

```
myForm.closeFormAction()
```

where *myForm* is a Form object.

Returns

No return value.

Exceptions

No exceptions.

Example

```
var notice = myForm.yesNoNotice("The specified values do not match a  
record. Specify a different combination?", myForm.dismissNoticeAction  
( ), myForm.closeFormAction());
```



Note: This object was added in RapidResponse 2015.3.

Response object

The Response object defines the response returned from the script to a form. The form processes the response and, if specified in the object, it also displays a notice.

Property	Type	Description
notice	Object	The notice that displays to the user in the form. Notices can include information and buttons that allow users to interact with the script. For more information, see "Notice object" on page 366 .
data	Object	An optional string of data that might be sent back in the response if the users clicks or taps a button that performs a runScriptAction action. For more information, see "runScriptAction method" on page 364 .

ControlMessage object

The ControlMessage object defines a customized message that displays on a form control.

Property	Type	Description
argument	String	Defines the argument mapped to the form control.
message	String	The custom message to display on the mapped control.

Notice object

The Notice object defines an informative message or prompt to users at the top of a form after they have clicked the OK button.

Property	Type	Description
message	String	The message that displays in the notice on the form.
type	String	The type of notice. This value can be Error, Info, Question, Success, or Warning.
autoClose	Boolean	If true, no buttons display and the notice displays briefly before automatically closing.
buttons	Object	An array of Button objects that define the action performed by the script.

The Button object defines a customized button on the Notice and has the following properties

Property	Type	Description
label	String	The label that displays on the button.
action	Object	A collection of Action objects that define the script action performed when the notice button on the form is clicked or tapped.

Each button in a form notice can perform an action, which you can define.

ErrorResponse object

The ErrorResponse object defines the error response return from the script to a form.

Property	Type	Description
message	String	The error message that displays on the form.
controlMessages	Object	An array of ControlMessages objects that define customized messages that display on form controls.

CHAPTER 29: Scorecard objects

Scorecards collection	367
Scorecard object methods	370

The Scorecard object defines a scorecard. A scorecard is a type of report that allows users to compare scenarios' performance on key business metrics. Scorecard objects are contained in the [scorecards](#) collection.

Scorecard objects have the same [properties as Resource objects](#).



Note: This object was added in RapidResponse 2015.3.

Scorecards collection

The scorecards collection contains all scorecards available to the user running the script. The scorecards collection implements the following resource collection methods.

Method	Description
get (key)	Retrieves a scorecard.
exists (scorecard)	Determines whether the specified scorecard exists in the collection.
remove (scorecard)	Deletes the specified scorecard.

The scorecards collection also implements the [resource enumeration](#) functions.



Note: This collection was added in RapidResponse 2015.3.

get method

Retrieves an instance of a scorecard.

► Syntax

```
rapidResponse.scorecards.get( {name, scope} );
```

Parameter	Type	Description
name	String	The name of the scorecard.
scope	String	Whether the scorecard is public or private.

► Return value

A Scorecard object.

► Exceptions

The `get` method can throw the following exceptions.

Exception	Description
ArgumentError	A null value was passed to a parameter.
Scorecard.NotFound	The specified scorecard does not exist or you do not have access to it.

► Example

```
var scorecard = rapidResponse.scorecards.get( {name:"Clear to Build",  
scope: "Public" } );
```

remove method

Deletes a scorecard from the server, which also removes it from the `scorecards` collection. Deleted scorecards are no longer available to any user or process, and you can only delete scorecards you own or have permission to manage.

► Syntax

```
rapidResponse.scorecards.remove(scorecard);
```

Parameter	Type	Description
scorecard	Object	The scorecard to delete. This can be specified as a Scorecard object or using the <code>{ name, scope }</code> syntax.

► Return value

No return value.

► Exceptions

The `remove` method can throw the following exceptions.

Exception	Description
<code>Scorecard.NoPermission</code>	You do not have permission to delete the specified scorecard.
<code>Scorecard.NotFound</code>	The specified scorecard does not exist.

► Example

```
rapidResponse.scorecards.remove({name:"Inventory Metrics",  
scope:"Private"});
```

exists method

Determines whether a specific scorecard exists in the collection. However, because `RapidResponse` supports multiple users working with the same resources, a user might delete the specified scorecard after this method runs. If you are using this method to test whether a scorecard exists, you should still catch a `NotFound` exception.

► Syntax

```
rapidResponse.scorecards.exists({name: "Name", scope:"Public or  
Private"});
```

Parameter	Type	Description
<code>name</code>	String	The name of the scorecard.
<code>scope</code>	String	Whether the scorecard is public or private.

► Return value

Type	Description
Boolean	• true if the scorecard exists • false if the scorecard does not exist

► Exceptions

The `exists` method can throw the following exception.

Exception	Description
ArgumentError	One of the following has occurred: - The resource parameter is missing a name or scope value. - The scope value specified is not "Private" or "Public".

► Example

```
var scorecardExists = rapidResponse.scorecards.exists({name: "Clear to Build", scope: "Public"});
```

Scorecard object methods

The Scorecard object implements the following methods.

Method	Description
give (newOwner)	Gives the scorecard to the specified user.
makePublic()	Changes the scope of a scorecard to "Public", which is the equivalent of sharing the scorecard. Public scorecards can be accessed by administrators and users and groups with access. You can also share a scorecard with a particular user or group using the accessControlList collection's add method .
rename (newName)	Changes a scorecard's name.
copy (newName)	Creates a new scorecard by copying an existing scorecard.

give method

Gives a scorecard to the specified user.

► Syntax

```
scorecard.give(newOwner) ;
```

where *scorecard* is a Scorecard object.

Parameter	Type	Description
newOwner	String	The user to give the scorecard to. Note: You cannot give a scorecard to a group.

► Return value

No return value.

► Exceptions

The `give` method can throw the following exceptions.

Exception	Description
Scorecard.NotFound	The specified scorecard does not exist.
Principal.NotFound	The specified user does not exist.
Scorecard.NonUniqueName	The specified user already owns a scorecard with the same name as the one you are giving.
Scorecard.NoPermission	You do not have permission to give the scorecard.

► Example

```
var myScorecard = rapidResponse.scorecards.get({ name:"Order
Satisfaction", scope:"Private" });
myTaskFlow.give("myUser");
```

makePublic method

Changes the scope of the specified scorecard from "Private" to "Public", which shares the scorecard and allows other users to access it. This method does not specify other users or groups to share the scorecard with, so it is available to only you and administrators.

If you call this method on a variable that represents a scorecard , you must ensure you assign the returned public scorecard to the same variable. Otherwise, the variable retains the scope it was created with.

You can also share the scorecard with specific users and groups using the [accessControlList collection's add method](#).

Administrators can access any shared scorecard , and you can allow other users and groups to access a shared scorecard.

► Syntax

```
publicScorecard = scorecard.makePublic();
```

where *scorecard* is a Scorecard object.

► Return value

A Scorecard object that represents the same scorecard with its scope changed to "Public".

► Exceptions

The `makePublic` method can throw the following exceptions.

Exception	Description
<code>Scorecard.NotFound</code>	The specified scorecard does not exist.
<code>Scorecard.NonUniqueName</code>	A shared scorecard with the same name as the one you are sharing already exists.
<code>Scorecard.NoPermission</code>	You do not have permission to share the scorecard.

► Example

```
var myScorecard = rapidResponse.scorecards.get({ name:"Order  
Satisfaction", scope:"Private" });  
myScorecard = myScorecard.makePublic();
```

rename method

Changes the name of a scorecard.

► Syntax

```
scorecard.rename(newName);
```

where *scorecard* is a Scorecard object.

Parameter	Type	Description
<code>newName</code>	String	The new name for the scorecard.

► Return value

A Scorecard object with the new name.

► Exceptions

The `rename` method can throw the following exceptions.

Exception	Description
ArgumentError	A blank string, invalid string, or <code>null</code> value was specified for the <code>newName</code> parameter.
Scorecard.NotFound	The specified scorecard does not exist.
Scorecard.NameValidationError	Either a scorecard with the same name already exists or the specified name is not valid.
Scorecard.NoPermission	The scorecard cannot be renamed or you do not have permission to rename it.
Scorecard.RenamePublicError	The scorecard is shared and cannot be renamed.

► Example

```
var myScorecard = rapidResponse.scorecards.get({ name:"Order
Operations", scope:"Private" });
myScorecard = myScorecard.rename("Order Rescheduling Impact");
myScorecard.makePublic();
```

copy method

Creates a new scorecard by copying an existing scorecard. If you copy a public scorecard, the copy is private.

► Syntax

```
scorecard.copy(newName) ;
```

where *scorecard* is a Scorecard object.

Parameter	Type	Description
<code>newName</code>	String	The name for the new scorecard.

► Return value

No return value.

► Exceptions

The `copy` method can throw the following exceptions.

Exception	Description
ArgumentError	A blank string, invalid string, or <code>null</code> value was specified for the <code>newName</code> parameter.
Scorecard.NotFound	The specified scorecard does not exist.
Scorecard.NameValidationError	Either a scorecard with the same name already exists or the specified name is not valid.
Scorecard.NoPermission	The scorecard cannot be copied or you do not have permission to copy it.

► Example

```
var myScorecard = rapidResponse.scorecards.get({ name:"Order
Rescheduling Impact", scope:"Public" });
myScorecard.copy("Order Impacts over Time");
```

CHAPTER 30: Collaboration objects

collaborations collection	375
collaborationScenario collection	377

Collaboration objects allow you to communicate, share simulated data, and compare solutions in a single interface. Collaboration objects are contained in the [collaborations](#) collection.

Property	Type	Description
id	String	The unique identifier for the collaboration. This value is automatically generated for each collaboration. Note: This value is not the same as the publicly visible collaboration ID number.
collaborationScenario	Object	A collection of scenarios used in the collaboration. For more information, see " collaborationScenario collection " on page 377.



Note: This object was added in RapidResponse 2016.2.

collaborations collection

The collaborations collection contains all collaborations available to the user running the script. The collaborations collection implements the following resource collection methods.

Method	Description
get (collaborationId)	Retrieves the specified collaboration.
remove (collaborationId)	Deletes the specified collaboration.



Note: This collection was added in RapidResponse 2016.2.

get method

Retrieves an instance of a collaboration.

► Syntax

```
rapidResponse.collaborations.get( collaborationId );
```

Parameter	Type	Description
collaborationId	String	The unique identifier of the collaboration you want to retrieve. This is not the same as the publicly visible collaboration number. This value can be retrieved by the script from the hidden first column, named Guid, in the Collaborations worksheet in the Collaborations system workbook. An example of a collaborationId is 294d2262-0087-46dd-a319-cf076e40e98f.

► Example

```
var collaboration= rapidResponse.collaborations.get( "294d2262-0087-46dd-a319-cf076e40e98f" );
```

remove method

Deletes a collaboration. This function requires user administration or system administration permission to use.

Syntax

```
rapidResponse.collaborations.remove(collaborationId)
```

Parameter	Type	Description
collaborationId	String	A unique identifier for the collaboration. Note: This is not the same as the publicly visible collaboration number. An example of a collaborationId is 294d2262-0087-46dd-a319-cf076e40e98f.

Return value

No return value.

Example

In this example, the script retrieves information about collaborations from the Collaborations worksheet (id=DeleteCollaborations) in the Collaborations workbook. It then deletes collaborations that have not been accessed in the last 10 days. The value required for the remove method's collaborationId parameter can be found in the first column in the worksheet, which is hidden.

```
var collab = rapidResponse.workbooks.get({name:"Collaborations",
scope:"Public"}).worksheets.get("DeleteCollaborations");
var collabData = collab.getData(); //Get the collaborations from the
Collaborations workbook.
var goUntil = rapidResponse.collaborations.toArray().length; //Get the
number of collaborations.
for (var i=0; i < goUntil; i++){
    var position = collabData.rows[i]; //For each row, check the
    number of days since the collaboration was accessed.
```



```

    if (position.cells[5].value >= 10) { //Identify a collaboration
      older than the limit. In this case, the limit is 10 days.
      rapidResponse.collaborations.remove(position.cells
        [0].value); //Remove the collaboration using the remove
        method.
    }
  }
}

```



Note: This method was added in RapidResponse G1902.

collaborationScenario collection

The collaborationScenario object defines the scenarios in the collaboration and has the following property.

Property	Type	Description
scenario()	String	The specific scenario added to the collaboration.

The collaborationScenario has the following methods:

Method	Description
get(key)	Retrieves a scenario object in a collaboration.
has(key)	Determines if a collaboration has a specified scenario.
add(key)	Shares the scenario with participants in the specified collaboration by adding the scenario to the collection.
remove(name)	Deletes the specified scenario object from the collaboration.
setProperties()	Specifies values for the collaboration scenario properties.

get method

Retrieves an instance of a scenario in a collaboration.

► Syntax

```
Collaboration.scenarios.get({name:'Name', scope:'Public or Private'});
```

where *Collaboration* is a Collaboration object.

Parameter	Type	Description
name	String	The name of the scenario.
scope	String	Whether the scenario scope is public or private.

► Return value

A Scenario object.

► Exceptions

The `get` method can throw the following exceptions.

Exception	Description
ArgumentError	A null value was passed to a parameter.

► Example

```
var collaborationScenario = collaboration.scenarios.get(
{name:"Approved Actions", scope: "Public" } );
```



Note: The term "public" is used for shared scenarios in scripting because, in much earlier versions of RapidResponse, scenarios were referred to as private and public, rather than private and shared. "Public" is retained in reference to scenario scope to avoid breaking users' scripts.

has method

Determines if collaboration has a specific scenario. However, because RapidResponse supports multiple users working with the same resources, a user might delete the specified scenario from the collaboration after this method runs. If you are using this method to test if a scenario exists in a collaboration, you should still catch a `NotFound` exception.

► Syntax

```
rapidResponse.collaborationScenario.has({name: "Name", scope: "Public
or Private"});
```

Parameter	Type	Description
name	String	The name of the scenario in the collaboration.
scope	String	Whether the scenario is Private or Public.

► Returns

Type	Description
Boolean	• true if the scenario is in the collaboration. • false if the scenario is not in the collaboration.

► Exceptions

The has method can throw the following exceptions.

Exception	Description
ArgumentError	One of the following has occurred: • The resource parameter is missing a name or scope value. • The scope value specified is not "Private" or "Public".

► Example

```
var collaborationScenario = rapidResponse.collaborationScenario.has({name: "Baseline", scope: "Public"});
```



Note: The term "public" is used for shared scenarios in scripting because, in much earlier versions of RapidResponse, scenarios were referred to as private and public, rather than private and shared. "Public" is retained in reference to scenario scope to avoid breaking users' scripts.

add method

Shares the scenario with participants in a specified collaboration by adding the scenario to the collection.

► Syntax

```
rapidResponse.collaborationScenario.add(principal, level);
```

Parameter	Type	Description
principal	String	The collaboration to add the resource to. OR The users in the collaboration the scenario is shared with.
level	String	Determines what level of access collaboration participants have to the scenario. Can be "View only" or "Modify".

► Returns

No return value.

► Exceptions

The `add` method can throw the following exceptions.

Exception	Description
<code>ArgumentError</code>	An invalid or null value was specified for a parameter.
<code>AccessControl.InvalidPermissionLevel</code>	A value other than "View" or "Modify" was specified for the level parameter.
<code>Principal.NotFound</code>	The specified user or group does not exist.
<code>Scenario.NonUniqueName</code>	A shared scenario with the same name as the one you are sharing already exists.
<code>Scenario.NoPermission</code>	You do not have permission to share the scenario.

► Example

To share a scenario:

```
var myScenario = rapidResponse.scenarios.get({ name:"myScenario",  
scope:"Private" });  
myScenario.collaborationScenario.add("myUser", "Modify");
```

remove method

Deletes the specified scenario object from the collaboration. You can only delete scenarios that you own and you must own all the scenario's children.

► Syntax

```
rapidResponse.collaborationScenario.remove({name: "Name", scope:  
"Public or Private"});
```

Parameter	Type	Description
<code>scenario</code>	Object	The scenario to delete from the collaboration. This can be specified as a scenario object or using the <code>{name, scope}</code> syntax.

► Return value

No return value.

► Example

```
rapidResponse.collaborationScenario.remove({name:"Approved Actions",  
scope:"Public"});
```

setPropertyValues method

Specifies the scenario property values for scenarios added to a collaboration. This method allows you to modify the specified property values after retrieving a Scenario object.

If you specify values for multiple properties in a single call, the method succeeds only if all properties are set successfully. If any property cannot be set, the method fails and no values are changed.

► Syntax

```
collaborationScenarios.setPropertyValues( {shareLevel: 'None/View/Modify'}  
);
```

Parameter	Type	Description
shareLevel	String	How the scenario is shared with other collaboration participants. This value can be one of the following: • None: The scenario is not shared with any other users in the collaboration. They cannot view or modify data in the scenario. • View: Other users in the collaboration can view data in the scenario. • Modify: Other users in the collaboration can view or modify data in the scenario.

► Return value

No return value.

► Exceptions

The `setPropertyValues` method can throw the following exceptions.

Exception	Description
ArgumentException	A parameter is missing or a null value was passed to a parameter.
setPropertyValues.InvalidPermissionLevel	A value other than "None", "View", or "Modify" was specified for the shareLevel parameter.

► Example

```
scenario.setPropertyValues ({shareLevel: 'View'});
```



Note: This method was added in RapidResponse 2016.2.

CHAPTER 31: Responsibility objects

Responsibility objects define ownership of data elements. A responsibility can be used as part of a collaboration process by assigning responsibility for specific data records to a user, who should be included in collaborations involving those records. For more information about collaborating, see the [RapidResponse User Guide \(Java Client\)](#).

Responsibility objects have the same [properties as Resource objects](#).

responsibilities collection

The responsibilities collection contains all [Responsibility](#) objects the user running the script has access to. The responsibilities collection implements the following resource collection methods.

Method	Description
get({name, scope})	Retrieves the specified responsibility.
exists(hierarchy)	Determines whether the specified responsibility exists in the collection.

The responsibilities collection also implements the collection indexing and iteration functions. For more information, see "[Collection indexing](#)" on page 111.

get method

Retrieves a Responsibility object.

► Syntax

```
responsibility = rapidResponse.responsibilities.get({name, scope});
```

Parameter	Type	Description
name	String	The responsibility's name.
scope	String	Whether the responsibility is Public or Private.

► Returns

A Responsibility object.

► Exceptions

The `get` method can throw the following exceptions.

Exception	Description
Responsibility.NotFound	The specified responsibility does not exist or the user does not have access to it.
ArgumentError	One of the following has occurred: - The parameter is missing a name or scope value. - The scope value specified is not "Private" or "Public".

► Example

```
var responsible = rapidResponse.responsibilities.get({name:"Planner
Codes", scope:"Public"});
```

exists method

Determines whether a responsibility exists. However, because RapidResponse supports multiple users working with the same resources, a user might delete the specified responsibility after this method runs. If you are using this method to test whether a responsibility exists, you should still catch a `NotFound` exception for the responsibility you are using.

► Syntax

```
exists = rapidResponse.responsibilities.exists({name: "Name", scope:
"Public or Private"});
```

Parameter	Type	Description
name	String	The name of the responsibility to test for existence.
scope	String	Whether the responsibility is Private or Public.

► Returns

Type	Description
Boolean	• true if the responsibility exists • false if the responsibility does not exist

► Exceptions

The `exists` method can throw the following exception.

Exception	Description
ArgumentError	One of the following has occurred: • The parameter is missing a name or scope value. • The scope value specified is not "Private" or "Public".

► Example

```
var responsibilityExists = rapidResponse.responsibilities.exists
({name: "Planner Codes", scope: "Public"});

if (!responsibilityExists) {
    return "Responsibility does not exist.";
}
```

Responsibility object methods

The Responsibility object implements the following methods.

Method	Description
give (newOwner)	Gives the responsibility to the specified user.
makePublic()	Changes the scope of a responsibility to "Public", which is the equivalent of sharing the responsibility. Public responsibilities can be accessed by administrators and users and groups with access. You can also share a responsibility with a particular user or group using the accessControlList collection's add method .

give method

Gives a responsibility to the specified user.

► Syntax

```
responsibility.give(newOwner);
```

where *responsibility* is a Responsibility object.

Parameter	Type	Description
newOwner	String	The user to give the responsibility to. Note: You cannot give a responsibility to a group.

► Return value

No return value.

► Exceptions

The `give` method can throw the following exceptions.

Exception	Description
Responsibility.NotFound	The specified responsibility does not exist.
Principal.NotFound	The specified user does not exist.
Responsibility.NonUniqueName	The specified user already owns a responsibility with the same name as the one you are giving.
Responsibility.NoPermission	You do not have permission to give the responsibility.

► Example

```
var myResponsibility = rapidResponse.responsibilities.get({  
  name:"Customer Contacts", scope:"Private" });  
myResponsibility.give("myUser");
```

makePublic method

Changes the scope of the specified responsibility from "Private" to "Public", which shares the responsibility and allows other users to access it. This method does not specify other users or groups to share the responsibility with, so it is available to only you and administrators.

If you call this method on a variable that represents a responsibility, you must ensure you assign the returned public responsibility to the same variable. Otherwise, the variable retains the scope it was created with.

You can also share the responsibility with specific users and groups using the [accessControlList collection's add method](#).

Administrators can access any shared responsibility, and you can allow other users and groups to access a shared hierarchy.

► Syntax

```
publicResponsibility = responsibility.makePublic();
```

where *responsibility* is a Responsibility object.

► Return value

A Responsibility object that represents the same responsibility with its scope changed to "Public".

► Exceptions

The `makePublic` method can throw the following exceptions.

Exception	Description
Responsibility.NotFound	The specified responsibility does not exist.
Responsibility.NonUniqueName	A shared responsibility with the same name as the one you are sharing already exists.
Responsibility.NoPermission	You do not have permission to share the responsibility.

► Example

```
var myResponsibility = rapidResponse.responsibilities.get({  
  name:"Customer Contacts", scope:"Private" });  
myResponsibility = myResponsibility.makePublic();
```


CHAPTER 32: Workflow objects

workflows collection	389
Workflow object methods	392

The Workflow object defines workflows that build out automated processes in RapidResponse. All workflows available to the user running the script are contained in the [workflows collection](#).

Workflow objects have the same properties as [Resource objects](#).

workflows collection

The workflow collection contains all the workflow objects available to the user running the script.

The workflows collection implements the following resource collection methods:

Method	Description
get(key)	Retrieves a workflow object.
exists(workflow)	Determines whether the specified workflow exists.
remove(workflow)	Deletes the specified workflow, removing it from the collection.

get method

Retrieves a workflow object, which allows the workflow to be used in other methods and processes.

► Syntax

```
workflow = rapidResponse.workflows.get({name, scope});
```

Parameter	Type	Description
name	String	The name of the workflow to retrieve.
scope	String	Whether the workflow is private or public.

► Returns

A Workflow object.

► Exceptions

The `get` method can throw the following exceptions.

Exception	Description
<code>ArgumentError</code>	A blank string or null value was passed to the method.
<code>Workflow.NotFound</code>	The specified workflow does not exist or you do not have access to it.

► Example

```
var workflow = rapidResponse.workflows.get({name: 'Assess Orders',  
scope: 'Public'});
```

exists method

Determines whether a specific workflow exists. However, because `RapidResponse` supports multiple users working with the same resources, a user might delete the specified workflow after this method runs. If you are using this method to test whether a workflow exists, you should still catch a `NotFound` exception.

► Syntax

```
workflowExists = rapidResponse.workflows.exists({name: "Name", scope:  
"Public or Private"});
```

Parameter	Type	Description
<code>name</code>	String	The name of the resource to test for existence.
<code>scope</code>	String	Whether the resource is Private or Public.

► Returns

Type	Description
Boolean	• true if the workflow exists • false if the workflow does not exist

► Exceptions

The `exists` method can throw the following exceptions.

Exception	Description
ArgumentError	One of the following has occurred: - The resource parameter is missing a name or scope value. - The scope value specified is not "Private" or "Public".

► Example

```
var workflowExists = rapidResponse.workflows.exists({name: "Assess Orders", scope: "Public"});
```

remove method

Deletes a workflow from the server, which also removes it from the `workflows` collection. Deleted workflows are no longer available to any user or process, and you can only delete workflows you own or have permission to manage.

► Syntax

```
rapidResponse.workflows.remove(name);
```

Parameter	Type	Description
name	Object	The workflow you are deleting. This can be specified as either a Workflow object or by using the {name, scope} syntax.

► Returns

No return value.

► Exceptions

The `remove` method can throw the following exceptions.

Exception	Description
Workflow.NoPermission	You do not have permission to delete the specified workflow.
Workflow.NotFound	The specified workflow does not exist.

► Example

```
rapidResponse.workflows.remove({name: "Assess Orders", scope: "Public"});
```

Workflow object methods

The workflow object implements the following resource object methods:

Method	Description
give(newOwner)	Changes the owner of the workflow.
makePublic()	Changes the scope of the workflow to "Public", which is the equivalent of sharing the resource. Public resources can be accessed by administrators and users and groups who have been granted access.
rename(newName)	Changes the name of the workflow.
copy(newName)	Makes a copy of the workflow with the specified name.
startExecution()	Runs the workflow

give method

Gives a workflow to a specified user. You can only give resources to individual users, not groups of users. For more about users and groups, see the [RapidResponse Administration Guide](#).

► Syntax

```
workflows.give(newOwner);
```

Parameter	Type	Description
newOwner	String	The user you are giving the workflow to. You cannot give a workflow to a group.

► Return value

No return value.

► Exceptions

The `give` method can throw the following exceptions.

Exception	Description
Workflow.NotFound	The specified workflow does not exist.
Principal.NotFound	The specified user does not exist.

Exception	Description
Workflow.NonUniqueName	The specified user already owns a workflow with the same name as the one you are giving.
Workflow.NoPermission	You do not have permission to give the workflow.

► Example

```
var workflow = rapidResponse.workflows.get({ name:"Assess Orders",
scope:"Public" });
workflows.give("User101");
```

makePublic method

Changes the scope of the specified workflow from "Private" to "Public", which shares it with administrators. This method does not specify which users or groups to share the workflow with, so it is only available to you and administrators. To share the workflow with specific users and groups, use the [accessControllist collection's add method](#).

If you call this method on a variable that represents a workflow, you must ensure you assign the returned public workflow to the same variable. Otherwise, the variable retains the scope it was created with.



Note: Administrators can access any shared workflow.

► Syntax

```
workflows.makePublic();
```

► Return value

A Workflow object that represents the same workflow with its scope changed to "Public".

► Exceptions

The `makePublic` method can throw the following exceptions.

Exception	Description
Workflow.NotFound	The specified workflow does not exist.
Workflow.NonUniqueName	A shared workflow with the same name as the one you are sharing already exists.
Workflow.NoPermission	You do not have permission to share the workflow.

► Example

```
var myWorkflow = rapidResponse.workflows.get({ name:"Assess Orders",
scope:"Private" });
myWorkflow = myWorkflow.makePublic();
```

rename method

Changes the name of a workflow.

► Syntax

```
workflows.rename(newName);
```

Parameter	Type	Description
newName	String	The new name for the workflow.

► Return value

A Workflow object with the new name.

► Exceptions

The `rename` method can throw the following exceptions.

Exception	Description
ArgumentError	A blank string, invalid string, or <code>null</code> value was specified for the <code>newName</code> parameter.
Workflow.NotFound	The specified workflow does not exist.
Workflow.NameValidationError	Either a workflow with the same name already exists or the specified name is not valid.
Workflow.NoPermission	The workflow cannot be renamed or you do not have permission to rename it.
Workflow.RenamePublicError	The workflow is shared and cannot be renamed.

► Example

```
var myWorkflow = rapidResponse.workflows.get({ name:"Review Orders",
scope:"Private" });
myWorkflow = myWorkflow.rename("Assess Orders");
myWorkflow.makePublic();
```

copy method

Creates a new workflow by copying an existing workflow and giving it the name you specify in the method. If you copy a public workflow, the copy created is private.

► Syntax

```
workflows.copy(newName) ;
```

Parameter	Type	Description
newName	String	The name for the new workflow.

► Return value

No return value.

► Exceptions

The `copy` method can throw the following exceptions.

Exception	Description
ArgumentError	A blank string, invalid string, or <code>null</code> value was specified for the new Name parameter.
Workflow.NotFound	The specified workflow does not exist.
Workflow.NameValidationError	Either a workflow with the same name already exists or the specified name is not valid.
Workflow.NoPermission	The workflow cannot be copied or you do not have permission to copy it.

► Example

```
var myWorkflow = rapidResponse.workflows.get({ name:"Assess Orders",  
scope:"Public" });  
myWorkflow.copy("Order Review");
```

startExecution method

The `startExecution` method runs a specified workflow.

► Syntax

```
workflows.startExecution[(inputVariables)]
```

Parameter	Type	Description
inputVariables	array	Optionally, an array of name:value string pairs passed to the startExecution method to be input into the workflow when it is run.

► Return value

No return value.

► Exceptions

The `startExecution` method can throw the following exceptions.

Exception	Description
InputVariableNotFound	The name of an input variable provided does not map to an input variable in the workflow.
InvalidValue	The value of an input variable provided is not valid. For example, the value for a Boolean variable is a value other than "true" or "false".
ScenarioNotFound	The value provided for a scenario input variable does not map to a scenario that the user has access to.
ScenarioNameNotScoped	The value provided for a scenario input variable is not scoped with the <code>?private:</code> or <code>?public:</code> prefix.

► Examples

Execute the `startExecution` method without optional input variables:

```
workflows.startExecution();
```

Execute the `startExecution` method with optional input variables included:

```
var variables = [{name:"String", value:"Running from Script"},
                 {name:"Number", value:"42"}
                 {name:"Boolean", value:"true"}
                 {name:"Scenario", value:"?private:testScenario"}
                ];

workflows.startExecution(variables);
```

CHAPTER 33: Query object

The RapidResponse Query object is used to implement the following methods to retrieve data records:

Method	Type	Description
execute	Object	Returns a query object for multiple records.
executeSingleton	Object	Returns a cellsCollection object for a single record.

execute method

Returns multiple records from the query in the form of data rows and column information. The object returned by the execute method must be closed using the [close method](#).

► Syntax

```
rapidResponse.query.execute()
```

Parameters	Type	Description
scenario	Scenario	Scenario that the query runs in.
table	String	The table that the query will access to return the data.
filter	String	The filter to apply to the data.
queryParameters	Object	The queryParameters object that contains acceptable key values.
columns	Object	An array of column objects, each containing an id (string) and an expr (string) for a column.

► Return Value

Returns a [query object](#).

► Exceptions

The execute method can throw the following exception:

Exception	Description
BadNumberOfArgumentsException	More than one argument is specified.
ArgumentException	One of the following has occurred: • The argument is not a query exception. • A key in the query definition is invalid. Valid keys are "scenario", "table", "filter", "queryParameters", and "columns". • The queryParameters value is not an object. • A key in the queryParameters object is invalid. Valid keys are "variables", "source", or "recordLimit". • The queryParameters property recordLimit is not an Integer value. • The columns parameter is not an array. • A column object in the columns array is not a valid column definition. • A key in a column object is invalid. Valid keys are "id" and "expr".

► Example

In this example, the script multiplies the standard unit cost value by 15%.

```
function main() {
    var query = rapidResponse.query.execute({
        scenario: {name: "ChangeDataTest", scope: "Private"},
        queryParameters: {
            source: "This is testing how the execute method can
            change data in a query"
        },
        table: "ScheduledReceipt",
        columns: [
            {id: "Name", expr: "KeyString"},
            {id: "Site", expr: "InputString"},
            {id: "StdUnitCost", expr: "InputQuantity"},
        ]
    });
    query.columns.forEach(column => {
        rapidResponse.console.writeLine(column.id + ": " +
        column.dataType);
    });

    query.rows.forEach(row => {
        var cell = row.cells.get("StdUnitCost");
```

```

        cell.setValue(cell.value * 1.15);
    });

    query.commit();
    query.close();
}

```

executeSingleton method

Returns a single record from the query in the form of a data row and column information. This method automatically closes and does not require the [close method](#).

► Syntax

```
rapidResponse.query.executeSingleton()
```

Parameters	Type	Description
scenario	Scenario	Scenario that the query runs in.
table	String	The table that the query will access to return the data.
filter	String	The filter to apply to the data.
queryParameters	Object	The queryParameters object that contains acceptable key values.
columns	Object	An array of column objects, each containing an id (string) and an expr (string) for a column.

► Return Value

Returns a [cellsCollection object](#). If there is no data, a value of `null` is returned.

► Exceptions

The `executeSingleton` method can throw the following exception:

Exception	Description
BadNumberOfArgumentsException	More than one argument is specified.
ArgumentException	One of the following has occurred: • The argument is not a query exception. • A key in the query definition is invalid. Valid keys are "scenario", "table", "filter", "queryParameters", and "columns". • The queryParameters value is not an object. • A key in the queryParameters object is invalid. Valid keys are "variables", "source", or "recordLimit". • The queryParameters property recordLimit is not an Integer value. • The columns parameter is not an array. • A column object in the columns array is not a valid column definition. • A key in a column object is invalid. Valid keys are "id" and "expr".

► Example

In this example, the script gets data for a single record (name = AC001) and then uses the execute method to add a new record for AC_TEST1 at the site CM2 and commit that change.

```
function main() {
    var cellCollection = rapidResponse.query.executeSingleton({
        scenario: {name: "QuerySingletonTest", scope: "Private"},
        queryParameters: {
            source: "This is a test of the query singleton method"
        },
        table: "ScheduledReceipt",
        filter: "KeyString = 'AC001'",
        columns: [
            {id: "Name", expr: "KeyString"},
            {id: "Site", expr: "InputString"},
            {id: "StdUnitCost", expr: "InputQuantity"},
        ]
    });
}
```

query object

Provides access to a data set returned by a RapidResponse query.

The query object has the following properties:

Properties	Type	Description
columns	Object	A collection of column objects that contain information about the columns returned by the query. See "columns collection" on page 404 .
rows	Object	A collection of row objects that define the rows returned by the query. See "rows collection" on page 404 .
status	String	The status of the query. Any error that occurs during execution of the query displays here.

query object methods

The query object implements the following methods to manage the data from the query:

Method	Description
close	Closes the query after it has finished running.
commit	Commits data changes from a scenario to its parent scenario in the query.

queryParameters object

Contains additional key values to define the data retrieved by the query.

The queryParameters object has the following properties:

Properties	Type	Description
recordLimit	Integer	Specifies the maximum number of records to return.
source	String	String that you specify that is included in logs. Enables you to track this query in the log when multiple queries are run in a script.
variables	Object	The variable to retrieve values from. This is specified in key:value pairs.

close method

Closes the query after it has finished running. Queries that are closed are removed from memory and cannot be referenced.

The execute object must be closed and it is recommended that you close your queries as soon as possible to free up memory for other processes.

► Syntax

```
query.close()
```

where `query` is a query object.

► Returns

No return value

► Exceptions

No exceptions.

► Example

```
function main() {
    var query = rapidResponse.query.execute({
        scenario: {name: "DeleteRowTest", scope: "Private"},
        table: "ScheduledReceipt",
        columns: [
            {id: "Name", expr: "KeyString"},
            {id: "Site", expr: "InputString"},
            {id: "StdUnitCost", expr: "InputQuantity"},
        ]
    });
    query.close();
}
```

commit method

Saves the values in the current query to the scenario identified in the query. This method only applies if values have been changed in any of the cells or if rows were added or deleted in the query. You must have permission to edit the scenario and it must be up to date when the query is run.

► Syntax

```
query.commit()
```

where `query` is a query object.

► Returns

No return value.

► Exceptions

The commit method can throw the following exceptions:

Exception	Description
Scenario.NoPermission	Either you do not have permission to commit the specified scenario or the scenario is a change data capture scenario and cannot be committed.
Scenario.NotFound	The specified scenario does not exist.
Scenario.NeedsUpdate	The scenario is out of date with its parent and needs to be updated before it can be committed.
Scenario.NeedsReset	The scenario needs to be reset before it can be committed.
Scenario.NonRootOnly	The specified scenario is the root scenario, which cannot be committed.
Scenario.HasOpenQueries	The scenario has other queries running on it, or is being used in a workbook or scorecard.
Scenario.HasChildren	The scenario is the parent of one or more child scenarios.

► Example

In the following example, the script commits a scenario after multiple rows have been deleted.

```
function main() {
    var query = rapidResponse.query.execute({
        scenario: {name: "DeleteRowTest", scope: "Private"},
        table: "ScheduledReceipt",
        columns: [
            {id: "Name", expr: "KeyString"},
            {id: "Site", expr: "InputString"},
            {id: "StdUnitCost", expr: "InputQuantity"},
        ]
    });

    query.deleteRows(0, query.rows.count - 1);
    query.commit();
    query.close();
}
```

query data collections

Data used in queries is contained in collections of worksheet data rows and cells. The data is categorized in a collection of columns.

The rows collections contains the data rows, which are represented as an index of row objects. The rows and cells collections both have the following property:

Property	Type	Description
count	Number	The total number of rows or cells in the collection.

The cells within each row are contained in the cells collection, which contains an index of cell objects. These objects contain the data values in each column for a specific row.

Collections support the standard JavaScript iteration functions. See ["Collection iteration functions" on page 112](#).

columns collection

The columns collection contains a set of column objects that provide information about the columns returned by the query.

The columns object has the following properties:

Property	Type	Description
id	String	The identifier of the column.
dataType	String	The type of data in the column. Only Integer, Quantity, Money, String, Date, and DateTime data types support editing.

You can implement the `forEach()` standard JavaScript function to iterate over items in the collection. See ["Collection iteration functions" on page 112](#).

rows collection

The rows collection contains a set of row objects that defines the data in the rows returned by the query.

The rows object has the following properties:

Property	Type	Description
number	Number	The row number. This value is set in relation to the first row in the worksheet that the query returns the data from.
cell	Object	A collection of cell objects that contain all of the cells in that row of data. See "cells collection" on page 404 .

You can implement the `forEach()` standard JavaScript function to iterate over items in the collection. See ["Collection iteration functions" on page 112](#).

cells collection

The cells collection contains a set of cell objects that defines the data in the cells returned by the query.

The cells object has the following property:

Property	Type	Description
value	Object	The data stored in the cell. This value is dependent on the data type for that column. See "columns collection" on page 404 .

The cells collection implements the following methods:

Method	Description
get()	Retrieves the data from the cell.
setValue()	Changes the value of the data in the cell. This value must be compatible with the data type of the cell. Only Integer, Quantity, Money, String, Date, and DateTime data types support editing.

You can also implement the `forEach()` standard JavaScript function to iterate over items in the collection. See ["Collection iteration functions" on page 112](#).

query data methods

The cells collection implements the following resource collection method:

Method	Description
get()	Retrieves the value for the specified worksheet cell.

The rows collection implements the following methods:

Method	Description
deleteRow()	Deletes a single row from the query.
deleteRows()	Deletes multiple rows from the query.
insertRow()	Inserts a single new row into the query.

get method

Retrieves the value of a worksheet cell.

► Syntax

```
cellCollection.get().value
```

► Returns

Returns the value for the specified cell.

► Exceptions

The get method can throw the following exception:

Exception	Description
ArgumentError	A blank string or null value was passed to the method.

► Example

```
var unitCost = cellCollection.get("StdUnitCost").value;
```

deleteRow method

Deletes a single row of data from the query.

This method must be followed by the commit method for the data to be saved to the server. See ["commit method" on page 402](#).

► Syntax

```
query.deleteRow(rowIndex)
```

where `query` is a query object.

Parameter	Type	Description
rowIndex	Integer	The row to delete in the data that was returned by the query.

► Returns

No return value

► Exceptions

The deleteRow method can throw the following exception:

Exception	Description
ArgumentException	The number of rows is less than 0 or greater than the number of rows in the query.

► Example

In this example, the script deletes any rows with the part name AC001 or AC002.

```
function main() {  
    var query = rapidResponse.query.execute({  
        scenario: {name: "DeleteRowTest", scope: "Private"},  
        table: "ScheduledReceipt",  
        columns: [  

```

```

        {id: "Name", expr: "KeyString"},
        {id: "Site", expr: "InputString"},
    ]
});

for (var x = query.rows.count - 1; x >= 0; --x) {
    var partName = query.rows[x].cells.get("Name");
    if ((partName.value == "AC001") || (partName.value ==
"AC002")) {
        query.deleteRow(x);
    }
}
query.commit();
query.close();
}

```

deleteRows method

Deletes multiple rows of data from the query.

This method must be followed by the commit method for the data to be saved to the server. See ["commit method" on page 402](#).

► Syntax

```
query.deleteRows(startRowIndex, endRowIndex)
```

where `query` is a query object.

Parameter	Type	Description
endRowIndex	Integer	The last row to delete. This value is set in relation to the first row returned by the query.
startRowIndex	Integer	The first row to delete. This value is set in relation to the first row returned by the query.

► Returns

No return value

► Exceptions

The deleteRows method can throw the following exceptions:

Exception	Description
ArgumentException	One of the following has occurred: - The number of rows is less than 0 or greater than the number of rows in the query. - The end index value is greater than the start index value.

► Example

In this example, the script deletes all rows returned by the query.

```
function main() {
    var query = rapidResponse.query.execute({
        scenario: {name: "DeleteRowTest", scope: "Private"},
        table: "ScheduledReceipt",
        columns: [
            {id: "Name", expr: "KeyString"},
            {id: "Site", expr: "InputString"},
            {id: "StdUnitCost", expr: "InputQuantity"},
        ]
    });

    query.deleteRows(0, query.rows.count - 1);
    query.commit();
    query.close();
}
```

insertRow method

Inserts one row of data into the query. To successfully insert a row, the query must include columns for all key fields in the table and each cell in the new row must have data specified for it.

This method must be followed by the commit method for the data to be saved to the server. See ["commit method" on page 402](#).

► Syntax

```
query.insertRow()
```

where `query` is a query object.

► Returns

Returns a row object for each new row.

► Exceptions

No exceptions

► Example

In the example, the script inserts a new row for AC_TEST at the site CM2 with a standard unit cost of 45.

```
function main() {
    var query = rapidResponse.query.execute({
        scenario: {name: "InsertRowTest", scope: "Private"},
        table: "ScheduledReceipt",
```



```

        columns: [
            {id: "Name", expr: "KeyString"},
            {id: "Site", expr: "InputString"},
            {id: "StdUnitCost", expr: "InputQuantity"},
        ]
    });

    var newRow = query.insertRow();
    newRow.cells.get("Name").setValue("AC_TEST");
    newRow.cells.get("Site").setValue("CM2");
    newRow.cells.get("StdUnitCost").setValue("45");

    query.commit();
    query.close();
}

```


CHAPTER 34: ProcessTemplate objects

ProcessTemplate objects define high-level processes that multiple users interact with to perform their jobs.

ProcessTemplate objects have the same [properties as Resource objects](#).

processTemplates collection

The processTemplates collection contains all [ProcessTemplate](#) objects the user running the script has access to. The processTemplates collection implements the following resource collection methods.

Method	Description
get({name, scope})	Retrieves the specified process.
exists(processTemplate)	Determines whether the specified process exists in the collection.

The processTemplates collection also implements the collection indexing and iteration functions. For more information, see "[Collection indexing](#)" on page 111.

get method

Retrieves a ProcessTemplate object.

► Syntax

```
process = rapidResponse.processTemplates.get({name, scope});
```

Parameter	Type	Description
name	String	The process's name.
scope	String	Whether the process is Public or Private.

► Returns

A ProcessTemplate object.

► Exceptions

The `get` method can throw the following exceptions.

Exception	Description
ProcessTemplate.NotFound	The specified process does not exist or the user does not have access to it.
ArgumentError	One of the following has occurred: - The parameter is missing a name or scope value. - The scope value specified is not "Private" or "Public".

► Example

```
var process = rapidResponse.processTemplates.get({name:"S&OP Cycle",
scope:"Public"});
```

exists method

Determines whether a process exists. However, because RapidResponse supports multiple users working with the same resources, a user might delete the specified process after this method runs. If you are using this method to test whether a process exists, you should still catch a NotFound exception for the process you are using.

► Syntax

```
exists = rapidResponse.processTemplates.exists({name: "Name", scope:
"Public or Private"});
```

Parameter	Type	Description
name	String	The name of the process to test for existence.
scope	String	Whether the process is Private or Public.

► Returns

Type	Description
Boolean	• true if the process exists • false if the process does not exist

► Exceptions

The `exists` method can throw the following exception.

Exception	Description
ArgumentError	One of the following has occurred: • The parameter is missing a name or scope value. • The scope value specified is not "Private" or "Public".

► Example

```
var processExists = rapidResponse.processTemplates.exists({name: "S&OP  
Cycle", scope: "Public"});  
  
if (!processExists) {  
    return "Process does not exist."  
}
```

ProcessTemplate object methods

The ProcessTemplate object implements the following methods.

Method	Description
give (newOwner)	Gives the process to the specified user.
makePublic()	Changes the scope of a process to "Public", which is the equivalent of sharing the process. Public processes can be accessed by administrators and users and groups with access. You can also share a process with a particular user or group using the accessControlList collection's add method .
rename (newName)	Changes the process's name.
copy (newName)	Creates a new process by copying an existing one.

give method

Gives a process to the specified user.

► Syntax

```
process.give(newOwner);
```

where *process* is a *ProcessTemplate* object.

Parameter	Type	Description
newOwner	String	The user to give the process to. Note: You cannot give a process to a group.

► Return value

No return value.

► Exceptions

The *give* method can throw the following exceptions.

Exception	Description
ProcessTemplate.NotFound	The specified process does not exist.
Principal.NotFound	The specified user does not exist.
ProcessTemplate.NonUniqueName	The specified user already owns a process with the same name as the one you are giving.
ProcessTemplate.NoPermission	You do not have permission to give the process.

► Example

```
var myProcess = rapidResponse.processTemplates.get({  
  name:"Manufacturing Process", scope:"Private" });  
myProcess.give("myUser");
```

makePublic method

Changes the scope of the specified process from "Private" to "Public", which shares the process and allows other users to access it. This method does not specify other users or groups to share the process with, so it is available to only you and administrators.

If you call this method on a variable that represents a process, you must ensure you assign the returned public process to the same variable. Otherwise, the variable retains the scope it was created with.

You can also share the process with specific users and groups using the [accessControlList collection's add method](#).

Administrators can access any shared process, and you can allow other users and groups to access a shared process.

► Syntax

```
publicProcess = process.makePublic();
```

where *process* is a *ProcessTemplate* object.

► Return value

A *ProcessTemplate* object that represents the same process with its scope changed to "Public".

► Exceptions

The `makePublic` method can throw the following exceptions.

Exception	Description
<code>ProcessTemplate.NotFound</code>	The specified process does not exist.
<code>ProcessTemplate.NonUniqueName</code>	A shared process with the same name as the one you are sharing already exists.
<code>ProcessTemplate.NoPermission</code>	You do not have permission to share the process.

► Example

```
var myProcess = rapidResponse.processTemplates.get({  
  name:"Manufacturing Process", scope:"Private" });  
myProcess = myProcess.makePublic();
```

rename method

Changes the name of a process.

► Syntax

```
process.rename(newName);
```

where *process* is a *ProcessTemplate* object.

Parameter	Type	Description
<code>newName</code>	String	The new name for the process.

► Return value

A *ProcessTemplate* object with the new name.

► Exceptions

The `rename` method can throw the following exceptions.

Exception	Description
<code>ArgumentError</code>	A blank string, invalid string, or <code>null</code> value was specified for the <code>newName</code> parameter.
<code>ProcessTemplate.NotFound</code>	The specified process does not exist.
<code>ProcessTemplate.NameValidationError</code>	Either a process with the same name already exists or the specified name is not valid.
<code>ProcessTemplate.NoPermission</code>	The process cannot be renamed or you do not have permission to rename it.
<code>ProcessTemplate.RenamePublicError</code>	The process is shared and cannot be renamed.

► Example

```
var myProcess = rapidResponse.processTemplates.get({
name:"Manufacture", scope:"Private" });
myProcess = myProcess.rename("New Part Production");
myProcess.makePublic();
```

copy method

Creates a new process by copying an existing process. If you copy a public process, the copy is private.

► Syntax

```
process.copy(newName) ;
```

where *process* is a `ProcessTemplate` object.

Parameter	Type	Description
<code>newName</code>	<code>String</code>	The name for the new process.

► Return value

No return value.

► Exceptions

The `copy` method can throw the following exceptions.

Exception	Description
ArgumentError	A blank string, invalid string, or <code>null</code> value was specified for the <code>newName</code> parameter.
ProcessTemplate.NotFound	The specified process does not exist.
ProcessTemplate.NameValidationError	Either a process with the same name already exists or the specified name is not valid.
ProcessTemplate.NoPermission	The process cannot be copied or you do not have permission to copy it.

► Example

```
var myProcess = rapidResponse.processTemplates.get({ name:"New Part
Production", scope:"Public" });
myProcess.copy("Manufacturing Process");
```


CHAPTER 35: charting property

JsChart objects	419
ChartEngine object methods	419

The charting property defines a ChartEngine object, which generates charts for dashboards and crosstab worksheets.

JsChart objects

The JsChart object defines a chart. The JsChart object has the following properties.

Property	Type	Description
actualHeight	Integer	The height of the chart.
actualWidth	Integer	The width of the chart.
height	Integer	The height of the chart as displayed.
image	String	A base64-encoded String that represents an image file. This string can be displayed as an image in an HTML document.
width	Integer	The width of the chart as displayed.

ChartEngine object methods

The ChartEngine object implements the following method.

Method	Description
generate(definition, options)	Creates a chart with the specified definition and options.

generate method

Creates a chart.

► Syntax

```
rapidResponse.charting.generate(definition, options)
```

Parameter	Type	Description
definition	Object	Defines how the chart is created using a chart definition. See "getChartDefinition method" on page 204 .
options	Object	Optionally defines the dimensions of the chart. This is an object with the following properties. - height: The height of the chart, in pixels. This must be a positive number. - width: The width of the chart, in pixels. This must be a positive number.

► Returns

A [JsChart](#) object.

► Exceptions

The `generate` method can throw the following exception.

Exception	Description
ArgumentException	One of the following has occurred. - One or both of the parameters have not been specified, or a null value was provided. - An invalid chart definition was provided for the definition parameter. - A negative value or a non-number value was provided for the options parameter.

► Example

```
var myWorkbook = rapidResponse.workbooks.get({name:"Periodic Revenue Review", scope:"Public"},
{
    scenarios: [{ name:"myScenario", scope:"Private" }],
    filter: {name: "Buy Parts", scope:"Public"},
    siteGroup: {name: "DC-Asia", scope: "Public"}
});
var myWorksheet = myWorkbook.worksheets.get("Monthly Totals");
```

```
var chartDef = myWorksheet.getChartDefinition();  
var newChart = rapidResponse.charting.generate(chartDef, {height:500,  
width:800});
```


CHAPTER 36: integrationPlatformService property

Real-time message objects	424
runTask method	425
sendMessage method	426
getEndpointIds method	427
getEndpointPathParameterNames method	428
invokeEndpoint method	429

The integrationPlatformService property defines connections to external systems that you can integrate with your RapidResponse instance. This includes a connection to the RapidResponse Integration Platform and Talend Administration Center (TAC), and any external endpoints that send or receive data for use in your RapidResponse processes. You can use these connections to run tasks defined on your TAC or send real-time messages to your external systems, using either the Real-time Integration Platform or a predefined external system endpoint.

The integrationPlatformService property defines the following methods:

Method	Description
runTask(name)	Runs the specified task on the Talend Administration Center.
sendMessage(messageDefinition, data, set)	Sends a message using the Real-time Integration Platform's outbound message flow.
getEndpointIds()	Retrieves a list of endpoints you can send requests to.

Method	Description
getEndpointPathParameterNames(endpoint)	Retrieves the names of any path parameters defined for a specific endpoint.
invokeEndpoint(endpoint, method)	Sends a request to an external endpoint.

The `integrationPlatformService` property is available to run tasks on the TAC only if the RapidResponse Integration Platform Service has been configured. Otherwise, a script that uses this property and the `runTask()` method fails. For more information, see the [RapidResponse Data Integration Guide](#).



Note: This property was modified in Service Update 2103.

Real-time message objects

Messages sent using the Real-time Integration Platform are defined using the `MessageDefinition` object, which contains the following properties:

Property	Type	Description
<code>name</code>	String	The message definition name.
<code>groupId</code>	String	An optional identifier used to ensure a message split into multiple parts is delivered together. You should ensure this value is unique by constructing it from randomly-generated components, such as a GUID.
<code>worksheet</code>	String	A worksheet identifier for the worksheet the data for this message is taken from.
<code>test</code>	String	Optionally, whether the message is being sent as a test. This property should be set to "Y" if you are testing the message, and set to "N" otherwise. If the <code>test</code> property is set to "Y", the message output is displayed in the script output console, and the message is not sent.
<code>sets</code>	Object[]	An array of Set objects, which define a set of data records from another table to be included in the outbound data.

The Set object represents the data from a referenced table that is included with the outbound data. and contains the following properties:

Property	Type	Description
worksheet	String	The worksheet identifier for the worksheet that contains the set records.
name	String	A name assigned to the set, which is used in the outbound XML data to define the set. It is recommended you set this value to the name of the table the set records are taken from or the name of the set field in the table.
keys	String[]	An array of the column identifiers for the key fields that link the table and the set.

runTask method

The `runTask` method accesses the Talend Administration Center (TAC) and runs the specified task. The task performs the actions defined in its underlying job, executing on the job server.

► Syntax

```
rapidResponse.integrationPlatformService.runTask(taskName);
```

Argument	Type	Description
taskName	String	The name of the task to run.

► Returns

No return value.

► Exceptions

The `runTask` method returns exceptions from the TAC, and does not use the `RapidResponse` Exception object. The following error conditions might occur.

- The specified task does not exist.
- The user configured for the Integration Platform Service does not have access to the specified task.
- The Integration Platform Service is not configured.

► Example

This example runs a workbook command to prepare data, and then runs the `OutboundData` task on your hosted TAC. This task takes data from the workbook and prepares it to be loaded back into the originating data source.

```
var command = rapidResponse.workbooks.get({name:"Data Preparation",
scope:"Public"}, {scenarios:[{name:"Approved Actions",
scope:"Public"}], filter:{name:"Modified", scope:"Public"}, siteGroup:
{name:"HQ", scope:"Public"}}).commands.get("Prepare");
command.execute();
```

```
rapidResponse.integrationPlatformService.runTask("OutboundData");
```

sendMessage method

Sends a message to an external system using the Real-time Integration Platform.

Syntax

```
var messageSent = rapidResponse.integrationPlatformService.sendMessage(definition, data, set);
```

Argument	Description
definition	The MessageDefinition object that defines the message to send.
data	The worksheet data to send to the external destination.
set	Optionally, the worksheet data representing a set of related data to send to the external destination.

Return value

An object that defines the replies for the message.

Example

This example sends a test message that contains supply orders and associated order lines.

```
var dataWorkbook = rapidResponse.workbooks.get({name:"Supply Orders",
scope:"Public"}, {
    scenarios: [{name:"Supply Out", scope: "Public"}],
    filter: {name:"All Parts", scope: "Public"},
    siteGroup: {name:"HQ", scope:"Public"}
});
var messageDef = {
    name: "SupplyOrdersWithLines",
    worksheet: "SupplyOrders",
    test:"Y",
    sets: [{
        worksheet:"Lines",
        name:"ScheduledReceipt",
        keys:["OrderId", "OrderSite","OrderType"]
    }]
};
var orderData = dataWorkbook.worksheets.get("SupplyOrders").getData();
var setData = dataWorkbook.worksheets.get("Lines").getData();
var result = rapidResponse.integrationPlatformService.sendMessage(
messageDef, orderData, setData);
```

This example sends a message that contains supply orders. This example uses a group identifier that is generated from two randomly-generated numbers.

```
var dataWorkbook = rapidResponse.workbooks.get({name:"Supply Orders",
scope:"Public"}, {
    scenarios: [{name:"Supply Out", scope: "Public"}],
    filter: {name:"All Parts", scope: "Public"},
    siteGroup: {name:"HQ", scope:"Public"}
});
var messageDef = {
    name: "SupplyOrders",
    worksheet: "SupplyOrders",
    groupId: "SupplyOrder-" + Math.random() + "-" + Math.random()
};
var orderData = dataWorkbook.worksheets.get("SupplyOrders").getData();
var result = rapidResponse.integrationPlatformService.sendMessage
(messageDef, orderData);
```

getEndpointIds method

Returns a list of endpoints defined for you to use for sending requests to external systems or services. These endpoints are defined by your RapidResponse administrator, and define the location and authentication to access the corresponding system.

You can create a script that calls this method and run it periodically to ensure the endpoint you intend to use has not changed

► Syntax

```
endpoints = rapidResponse.integrationPlatformService.getEndpointIds();
```

► Returns

An array of the endpoint identifiers, as strings.

► Exceptions

The getEndpointIds method can throw the following exceptions.

Exception	Description
ArgumentException	An argument was passed to the method.
CalloutError	An error occurred when accessing the endpoints.

► Example

This example returns all endpoints your RapidResponse administrator has defined.

```
var endpoints =  
rapidResponse.integrationPlatformService.getEndpointIds();  
return endpoints;
```

This example takes an endpoint as an argument, then retrieves endpoints and sets a variable to that endpoint if it exists.

```
var target = rapidResponse.scriptArguments["endpoint"];  
var finalEndpoint = "None";  
var endpoints =  
rapidResponse.integrationPlatformService.getEndpointIds();  
for (var i=0; i < endpoints.length; i++){  
    if (endpoints[i] = target; {  
        finalEndpoint=endpoints[i];  
    }  
}  
if (finalEndpoint != "None") {  
    return finalEndpoint;  
} else {  
    return "Not found";  
}
```



Note: This method was added in Service Update 2011.

getEndpointPathParameterNames method

Endpoints can have path parameters defined, which you can pass values for when you invoke the endpoint. These parameters are not visible as part of the endpoint's identifier, so you must run this method to return the parameters defined for an endpoint.

► Syntax

```
parmList =  
rapidResponse.integrationPlatformService.getEndpointPathParameterNames  
(endpoint)
```

where *endpoint* is an endpoint identifier.

► Returns

An object that contains a list of the names of all path parameters defined for the endpoint. If the endpoint has no path parameters, returns a blank list.

► Exceptions

The `getEndpointPathParameterNames` method can throw the following exceptions.

Exception	Description
ArgumentException	A null value was passed to a parameter.
EndpointNotFound	The specified endpoint does not exist.

► Examples

```
var parameterList =
rapidResponse.integrationPlatformService.getEndpointPathParameterNames
("GetServiceStatus");
rapidResponse.console.WriteLine(JSON.stringify(parameterList));
```



Note: This method was added in Service Update 2103.

invokeEndpoint method

Uses a predefined endpoint to send notifications or data to external systems. Typically these are RESTful services that you can use to process data externally to RapidResponse, or to provide external systems with results calculated by RapidResponse resources or processes.

Endpoints are defined by your RapidResponse administrator, and include the URL to access and authentication to access that external server or system. You can use the endpoint to send a request to that system. You can also specify the HTTP method to perform for the call, such as GET to retrieve some data from the endpoint or POST or PUT to send a message or data to the endpoint.

For an endpoint that takes data, you can specify the data to include in the payload and its format. The payload requirements depend on the input required by the external system. You can define the payload data using a worksheet that is configured to provide the data in the format the external system requires.

For an endpoint that sends a notification, you can call the endpoint with no payload. Your external system or the processes that use it determine how this signal is processed and used.



Notes:

- The payloads sent using this method can be up to 50 MB, and requests sent using this method have a 2 minute timeout.
- RapidResponse includes endpoints for managing data in workbooks and tables, which you can use in addition to external endpoints created by your RapidResponse administrator. For more information, see the [RapidResponse Web Services Guide](#).

► Syntax

```
response = rapidResponse.integrationPlatformService.invokeEndpoint
({endpointId, method, payload, contentType, headers, parameters,
pathParameters})
```

Parameter	Type	Description
endpointId	String	The endpoint to connect to.
method	String	The HTTP method to perform using the endpoint. This must be one of the standard HTTP methods (such as GET, PUT, POST, or DELETE), and must also be a method that is implemented by the endpoint.
payload	String	Optionally, a string representation of a payload to send to the endpoint. The format of the payload should match the content type required by the endpoint. For example, if your endpoint requires a JSON payload, the payload should be a JSON string, which you can create by converting an object using the <code>JSON.stringify()</code> method or by converting a worksheet's data to a JSON object using the convertToJSON() method. A payload is not required if you are using the endpoint to send a notification.
contentType	String	Optionally, the HTTP content type of data the payload contains, such as text/plain or application/json. This is required only if you specify a payload.
headers	Object	Optionally, custom headers or custom values for standard headers to include in the request, specified in key:value pairs.
parameters	Object	Optionally, additional parameters for the request, specified in key:value pairs.
pathParameters	Object	Values for path variables defined for the endpoint, which are used to specify a dynamic location at the endpoint's destination. All path parameters for an endpoint are required, and are specified in key:value pairs. Depending on the endpoint and the parameter, you might require a specific value, or you might be able to pass a value from another endpoint's response. Whether you must specify a value or pass a value requires some knowledge of the endpoint and its requirements.

► Returns

The response from the endpoint. Depending on how the endpoint is defined, this might be a specific message, a message with a payload, or an HTTP response code (such as 200 for successful messages or 400 for errors).

The return object has the following properties.

Property	Description
status	The HTTP status for the response.
response	The endpoint's return message, payload, or error message if the request failed.

► Exceptions

The `invokeEndpoint` method can throw the following exceptions.

Exception	Description
ArgumentException	A null value was passed to a parameter, or a value was not provided for a required path parameter.
CalloutError	An error occurred contacting the endpoint or the request timed out.
EndpointNotFound	The specified endpoint does not exist.

► Examples

This example sends a notification to an endpoint named `ServiceReady`.

```
var response = rapidResponse.integrationPlatformService.invokeEndpoint(
({
    endpointId:"ServiceReady",
    method:"PUT"
}));
```

This example sends a message to an internal communication channel to indicate a workbook operation is complete.

```
var workbookData = rapidResponse.workbooks.get({name:"Demand
Operations", scope:"Public"}, {
    scenarios:[{name:"Baseline", scope:"Public"}],
    siteGroup:(name: "HQ", scope:"Public"),
    filter:{name: "All Parts", scope: "Public"}
});
var dataMod = workbookData.commands.get("Allocation Balance").execute
();
var message = workbookData.name + " workbook operation completed with
"+ dataMod.modified +" modifications.";
var response = rapidResponse.integrationPlatformService.invokeEndpoint
({
    endpointId: "Communications",
    method: "POST",
    payload: message;
    contentType: "text/plain"
});
```

This example sends data from a worksheet formatted to provide data to an external service, which has an endpoint named `ExternalService` defined and expects a JSON payload. The endpoint expects each row as an object named `DataRow`, with the value represented as a comma-delimited string.

```
var workbookData = rapidResponse.workbooks.get({name:"Outbound Supply
Orders", scope:"Public"}, {
    scenarios:[{name:"Baseline",scope:"Public"}],
```

```

        siteGroup: {name: "Albany", scope: "Public"},
        filter: {name: "All Parts", scope: "Public"}
    });
    var worksheetData = workbookData.worksheets.get
    ("ExternalData").getData();
    var sendData = worksheetData.convertToJSON(0, 200, {
        rowName: "DataRow",

        rowOutputFormat: "AsDelimitedString"
    });
    var result = rapidResponse.integrationPlatformService.invokeEndpoint({
        endpointId: "ExternalService",

        payload: sendData,

        contentType: "application/json"

        method: "POST"
    });

```

This example constructs a JSON payload based on arguments and values from a single worksheet row, and then sends it to an endpoint that defines a single HTTP method.

```

var sendRow = rapidResponse.scriptArguments["start"];
var ident = rapidResponse.scriptArguments["id"];
var site = rapidResponse.scriptArguments["site"]
var workbookData = rapidResponse.workbooks.get({name: "Outbound Supply
Orders", scope: "Public", {
    scenarios: [{name: "Baseline", scope: "Public"}],

    siteGroup: site,

    filter: {name: "All Parts", scope: "Public"}

}
});
var worksheetData = workbookData.worksheets.get
("ExternalData").getData().rows[sendRow];
var payloadConstructor = {name: ident, location: site,
due: worksheetData.cells[3].value, source: "External",
total: worksheetData.cells[5].value};
var payload = JSON.stringify(payloadConstructor);
var result = rapidResponse.integrationPlatformService.invokeEndpoint({
    endpointId: "OneRowJsonPost",

    contentType: "application/json",

    payload: payload,

    method: "POST"
});

```

This example retrieves data from an endpoint named `GetStatus` and outputs it to the console.

```

var result = rapidResponse.integrationPlatformService.invokeEndpoint({

```



```

        endpointId:"GetStatus",
        method: "GET"
    });
    rapidResponse.console.WriteLine(result.response);

```

This example retrieves the status from a service defined using a dynamic path parameter. In this case, the endpoint has some specific values that can be passed for the parameter, and you know the one you want to use.

```

var result = rapidResponse.integrationPlatformService.invokeEndpoint({
    endpointId:"GetServiceStatus",
    method: "GET"
    pathParameters: {
        serviceName: "DataMonitor"
    }
});
rapidResponse.console.WriteLine(result.response);

```

This example uses the response from an endpoint to obtain the dynamic path defined for another endpoint. In this case, the endpoint does not have specific values that can be passed, and the value is dynamically generated.

```

var callService =
    rapidResponse.integrationPlatformService.invokeEndpoint({
        endpointId:"GetServicePath",
        method: "GET"
    });
var result = rapidResponse.integrationPlatformService.invokeEndpoint({
    endpointId:"GetServiceStatus",
    method: "GET"
    pathParameters: {
        servicePath: JSON.stringify(callService.path)
    }
});

```



Note: This method was added in Service Update 2103.

CHAPTER 37: Link object

links collection	435
Link object methods	436

The Link object defines a link to a resource, such as a workbook, dashboard, or form. All links you have access to are contained in the links collection. The Link object contains only the [resource object properties](#), however, the scope property for links is always "Public".

The Link object has the following property.

Property	Type	Description
targetType	String	The type of resource defined in the link.



Note: This object was modified in RapidResponse 2014.4.

links collection

All workbook and dashboard links that you have access to are contained in the links collection.

The links collection implements the following method, in addition to the [resource collection enumeration methods](#).

Method	Description
get(link)	Retrieves an instance of a link object.



Note: This collection was added in RapidResponse 2013.4.

get method

Retrieves an instance of a link object.

► Syntax

```
link = rapidResponse.links.get(linkKey);
```

where

Parameter	Type	Description
linkKey	Object	Either the identifier of a link object in the <code>links</code> collection, or a link object specified using the <code>{name, scope}</code> syntax. The <code>scope</code> for links is always "Public".

► Returns

A [Link](#) object.

► Exceptions

The `get` method can throw the following exceptions.

Exception	Description
<code>ArgumentError</code>	A blank or null value was passed for a parameter.
<code>Link.NotFound</code>	The specified link does not exist or you do not have access to it.

► Example

```
var myLink = rapidResponse.links.get({name:"dash", scope:"Public"});
```



Note: This method was added in RapidResponse 2013.4.

Link object methods

The Link object defines the following method.

Method	Description
getResource()	Retrieves the resource a link opens.
createEmbeddedLinkUrl()	Retrieves an instance of a link object in a collaboration.

getResource method

Retrieves the name of the resource a link opens.

► Syntax

```
resourceName = link.getResource();
```

where *link* is a [Link](#) object.

► Returns

A String value.

► Exceptions

The `getResource` method can throw the following exception.

Exception	Description
<code>ArgumentError</code>	A parameter was provided for the method.

► Example

This example creates an instance of a link object, then retrieves the name of the resource. However, this example returns a script run-time error because the `getResource` method must be called on a `Link` object. The `createEmbeddedLinkUrl` method returns a string value, not a link object.

```
var dashLink = myDashboard.createEmbeddedLinkUrl();  
var resourceName = dashLink.getResource();
```



Note: This method was added in RapidResponse 2013.4.

createEmbeddedLinkUrl method

Creates an instance of a link object in a collaboration.

► Syntax

```
link = rapidResponse.links.createEmbeddedLinkUrl();
```

► Returns

A String value.

► Exceptions

The `create` method can throw the following exceptions.

Exception	Description
ArgumentError	A blank or null value was passed for a parameter.
Link.NotFound	The specified link does not exist or you do not have access to it.

► Example

```
var dash = rapidResponse.dashboards.get({name: "My Dashboard", scope:
"Public"});
var link = dash.createEmbeddedLinkUrl();
```



Note: This method was added in RapidResponse 2016.2

CHAPTER 38: dateTime property

toCalendarDate method	440
add method	440
subtract method	441
difference method	442

The `dateTime` property provides methods for date and time calculations, and has the following properties.

Property	Type	Description
NOW	Date	The RapidResponse Now DateTime constant.
TODAY	Date	The RapidResponse Today date constant.
PAST	Date	The RapidResponse Past date constant.
FUTURE	Date	The RapidResponse Future date constant.
UNDEFINED	Date	The RapidResponse Undefined date constant.

The `Now` `DateTime` constant reports the time and date when the script begins executing, and is not updated during script execution. If you require an updated time for your script operations, you can use the JavaScript date functions, such as `Date (Date . now ( ) )`.

The following `dateTime` methods are available.

Method	Description
toCalendarDate(date, calendar)	Converts a value to a calendar date.
add(date, number, calendar)	Adds the specified number of calendar units to the specified date.
subtract(date, number, calendar)	Subtracts the specified number of calendar units from the specified date.
difference(firstDate, lastDate, calendar)	Determines the number of calendar units between two dates.

toCalendarDate method

Calculates the closest calendar date to a specified date.

► Syntax

```
rapidResponse.dateTime.toCalendarDate(date, calendar);
```

Parameter	Type	Description
date	Date	The date to be calculated.
calendar	String	The calendar the date is calculated in. This parameter is optional. If no calendar is specified, business days are used.

► Returns

Type	Description
Date	The calendar date closest to the specified date

► Exceptions

The `toCalendarDate` method can throw the following exceptions.

Exception	Description
ArgumentError	The specified calendar does not exist.

► Example

```
var qStart = rapidResponse.dateTime.toCalendarDate
(rapidResponse.dateTime.TODAY, "Quarter");
```

add method

Adds the specified number of calendar dates to a specified date.

► Syntax

```
rapidResponse.dateTime.add(date, number, calendarName);
```

Parameter	Type	Description
date	Date	The date to add calendar units to.
number	Number	The number of calendar units to add.
calendarName	String	The calendar used to add dates. This parameter is optional. If no calendar is specified, business days are used.

► Returns

Type	Description
Date	The resulting date.

► Exceptions

The `add` method can throw the following exceptions.

Exception	Description
ArgumentError	The specified calendar does not exist.

► Example

```
var nextMonth = rapidResponse.dateTime.add  
(rapidResponse.dateTime.TODAY, 1, "Month");
```

subtract method

Subtracts the specified number of calendar units from the specified date.

► Syntax

```
rapidResponse.dateTime.subtract(date, number, calendarName);
```

Parameter	Type	Description
date	Date	The date to subtract calendar units from.
number	Number	The number of calendar units to subtract.
calendarName	String	The calendar used to subtract dates. This parameter is optional. If no calendar is specified, business days are used.

► Returns

Type	Description
Date	The resulting date.

► Exceptions

The `subtract` method can throw the following exceptions.

Exception	Description
<code>ArgumentError</code>	The specified calendar does not exist.

► Example

```
var lastMonth = rapidResponse.dateTime.subtract  
(rapidResponse.dateTime.TODAY, 1, "Month");
```

difference method

Reports the difference between two dates.

► Syntax

```
rapidResponse.dateTime.difference(date, dateToSubtract, calendarName);
```

Parameter	Type	Description
<code>date</code>	Date	The date to subtract calendar units from. This date must be later than the date specified for the <code>dateToSubtract</code> parameter.
<code>dateToSubtract</code>	Date	The date to subtract to determine the difference.
<code>calendarName</code>	String	The calendar used to report the difference. This parameter is optional. If no calendar is specified, business days are used.

► Returns

Type	Description
Number	The number of calendar periods between the two dates.

► Exceptions

The `difference` method can throw the following exceptions.

Exception	Description
ArgumentError	The specified calendar does not exist.

► Example

```
var nextMonth = rapidResponse.dateTime.add
(rapidResponse.dateTime.TODAY, 1, "Month");
var lastMonth = rapidResponse.dateTime.subtract
(rapidResponse.dateTime.TODAY, 1, "Month");
var numberOfWeeks = rapidResponse.dateTime.difference
(nextMonth, lastMonth, "Week");
```


CHAPTER 39: console property

writeLine method445

The console property defines an output console, which you can write information to debug a script. The console property has the following method.

Method	Description
writeLine(text)	Writes the specified text to the output console.

writeLine method

Writes the specified text to the output console.

► Syntax

```
rapidResponse.console.writeLine(text);
```

Parameter	Type	Description
text	String	The text to be output to the console.

► Returns

No return value

► Example

```
rapidResponse.console.writeLine("value: " + value.toString());
```


CHAPTER 40: Message objects

SendMessage object

SendBroadcastMessage object

messages collection

448

448

449

The Message object defines a message that you have received in Message Center. Message objects are contained in the `messages` collection. For more information, see ["messages collection" on page 449](#).

The Message object has the following properties.

Property	Type	Description
to	String	The recipients for the message.
from	String	The message's sender.
body	String	The message's content.
sentAt	Date	When the message was sent.
messageId	String	The message's identifier.
cc	String	Users who were carbon copied on the message.
subject	String	The message's subject line.
read	Boolean	Whether the message has been read.

SendMessage object

This object defines a message, which can be sent to other users' Message Center. Messages are sent by any user who runs the script, and sending messages does not require other permissions. All users, including those without access to Message Center, can send messages through scripts.

This object contains the following properties.

Property	Type	Description
to	String	The users, group, or email addresses the message is sent to. Users must be specified by user ID.
cc	String	The users, group, or email addresses that are copied on the message. Users must be specified by user ID.
bcc	String	The users, group, or email addresses that are blind copied on the message. Users must be specified by user ID.
subject	String	The message's subject.
message	String	The message's body.
attachments	Object	A file attached to the message. This property was added in RapidResponse 2014.1.

Values specified for the `to`, `cc`, or `bcc` properties can be arrays of strings, or a single string with multiple values separated by semicolons. For example,

```
to: ["user1", "user2"]
cc: "user3; user4"
```

The object defined by the `attachments` property contains the following properties.

Property	Type	Description
fileName	String	The name of the file that is attached to the message.
content	Object	A ReportOutput object that represents the content of the file to be attached.

SendBroadcastMessage object

This object defines a broadcast message, which is sent to all logged-on users. This object contains the following properties.

Property	Type	Description
subject	String	The message's subject.
message	String	The message's body.
to	String	Optionally, the user or users to send the broadcast message to. If no value is specified, the message is sent to all logged-on users. If multiple users are specified, the user names must be separated by semicolons.
expiry	Date	Optionally, the date after which the message's pop-up notification is not displayed. If a recipient signs in after the specified date, the message displays only in Message Center. Otherwise, the user sees the pop-up notification when they sign in. If no value is specified, the pop-up notification is displayed regardless of how old the message is.

messages collection

The messages collection contains all RapidResponse messages. You can use messages to send notifications when a script finishes or broadcast to all logged-on users. You can send messages to any user, group, or email address. These messages are received in Message Center and can be sent to email addresses.

Messages you send are defined using a SendMessage object, as described in ["SendMessage object" on page 448](#). Broadcast messages are defined using a SendBroadcastMessage object, as described in ["SendBroadcastMessage object" on page 448](#).

The messages collection has the following methods.

Method	Description
sendMessage(message)	Sends the specified message to the users, groups, or email addresses specified in the message.
sendBroadcastMessage(message)	Sends a broadcast message to all logged-on users.

sendMessage method

Sends the specified message to the specified users or groups. Users can send messages using this method if they do not have access to Message Center.

This method's availability is controlled by the user's visibility level. Messages include information about the sender, so users with a visibility level of None cannot send messages. Users with a visibility level of Limited can send messages only to users with the Full (or All) visibility level. Otherwise, sending the message fails. For more information about user visibility, see the [RapidResponse Administration Guide](#)

► Syntax

```
rapidResponse.messages.sendMessage(message);
```

Parameter	Type	Description
message	Object	A SendMessage object that defines the message to be sent. For more information, see "SendMessage object" on page 448 .

► Returns

No return value.

► Example

This example sends a message to the members of the Buyers group.

```
rapidResponse.messages.sendMessage( {to:"Buyers", subject:"Daily  
scenario is ready", message: "Today's daily usage scenario has been  
created and can now by used."} );
```

This example sends a message to the user running the script to inform them a workbook command has finished.

```
var workbookWithCommand = rapidResponse.workbooks.get({name:"Order  
Quantity Comparison", scope:"Public"},  
{  
  scenarios: [{ name:"myScenario", scope:"Private" }],  
  filter: {name: "All Parts", scope:"Public"},  
  siteGroup: {name: "HQ", scope: "Public"},  
  variables: [  
    {name: "showLate", value: true},  
    {name: "onDate", value: rapidResponse.dateTime.TODAY},  
  ]  
});  
var command = workbookWithCommand.commands.get("QtyDividedBy2");  
command.execute();  
rapidResponse.messages.sendMessage(  
{to:rapidResponse.loggedInUser.userid, subject:"Command complete",  
message:"The "+command.name+" command has completed."});
```

This example sends a worksheet report to the Buyers group.

```
var workbookReport = rapidResponse.workbooks.get({name:"Order  
Quantities", scope:"Private"},  
{  
  scenarios: [{ name:"myScenario", scope:"Private" }],  
  filter: {name: "All Parts", scope:"Public"},  
  siteGroup: {name: "HQ", scope: "Public"}  
});  
var report = workbookReport.generateReport("demandSummary", "xlsx");
```

```
rapidResponse.messages.sendMessage( {to:"Buyers", subject:"Report generated", message:"Demand summary report is attached.", attachments: [ {fileName: "DemandReport.xlsx", content: report} ]});
```

sendBroadcastMessage method

Sends a broadcast message to users. You must be an administrator to call this method.

► Syntax

```
rapidResponse.messages.sendBroadcastMessage(message);
```

Parameter	Type	Description
message	Object	A SendBroadcastMessage object that defines the message to be broadcast to users. You must specify a value for the subject property. If no recipients are specified, the message is broadcast to all logged-on users. For more information, see "SendBroadcastMessage object" on page 448 .

► Returns

No return value.

► Exceptions

The sendBroadcastMessage method can throw the following exceptions.

Exception	Description
ArgumentError	A value was not specified for the subject property.
Message.NoPermission	You must be an administrator to send a broadcast message.

► Example

```
rapidResponse.messages.sendBroadcastMessage(  
    {  
        subject:"RapidResponse availability",  
        message:"Due to hardware upgrades, RapidResponse will be  
        sporadically unavailable for the next day.",  
        expiry:new Date("2017-07-26T07:30:00")  
    }  
);
```


CHAPTER 41: User objects

UserProfile object453

ContactInfo object455

users collection455

UserCollectionObject object459

LoggedInUser object463

The User object defines a user, and all users are contained in the [users collection](#). The User object contains the following properties.

Property	Type	Description
id	String	The user's name or ID.
name	String	The user's name or ID.
displayName	String	The name displayed for the user.
email	String	The user's email address.
forwardToEmail	Boolean	If true, messages sent to the user are also sent to their email address.

UserProfile object

The UserProfile object is used when creating a user account, and defines the properties of a user account. The UserProfile object has the following properties.

Property	Type	Description
name	String	The user's name.
password	String	The user's password.
email	String	The user's email address.
profilePicture	String	A base64-encoded String that represents an image file. This string is used to set the user's profile picture. This property was added in RapidResponse 2015.3.
forwardToEmail	Boolean	If true, messages sent to the user are also sent to their email address.
licenseType	String	The user's type. This can be one of the following values: • Internal (corresponds to Full license type) • External (corresponds to Light license type) • Alert As of RapidResponse 2016.2, Service Update 21, this property is deprecated. It is replaced by userLicenseType. When a UserProfile object has the userLicenseType property set, licenseType is ignored.
userLicenseType	String	The user's type. This can be one of the following values: • Full • Light • Report • Alert When the userLicenseType = Alert, passwords cannot be set for this user profile because alert users do not have passwords specified in RapidResponse.
visibilityLevel	String	The user's visibility level, which determines which users can see information about each other. The following values are valid: • Full (corresponds to visibility level of All) • Limited • None If a value is not specified for visibilityLevel, the All visibility level is assigned by default.
isAccountDisabled	Boolean	If true, the user account cannot be used to sign in.

Property	Type	Description
isSSOOnly	Boolean	If true, the user can sign in only using a single sign-in infrastructure. These users do not have passwords specified in RapidResponse. You cannot set a password for a user that has this property set to true, and you must provide a password for the user if you set this property to false. For users with userLicenseType = Alert, this property can only be set to false.
contactInfo	Object	The user's contact information. For more information, see "ContactInfo object" on page 455 .



Note: This object was added in RapidResponse 2014.4.

ContactInfo object

The ContactInfo object defines the contact information for a user account. The ContactInfo object has the following properties.

Property	Type	Description
title	String	The user's job title.
phone	String	The user's telephone number.
fax	String	The user's fax number.
mobile	String	The user's mobile telephone number.
pager	String	The user's pager number.
location	String	Where the user is located.
country	String	The country the user is located in.
notes	String	Information about the user.

users collection

The users collection contains all [User](#) objects defined in the system.

The users collection implements the following methods.

Method	Description
create(id, profile)	Creates a user account.
generatePassword()	Generates a password, which can be used as the password for a new user.
remove()	Deletes a user.

The users collection also implements the resource collection iteration methods. For more information, see "[Resource collections](#)" on page 110.



Note: If a script is run using the credentials of a user who has a visibility level of Limited or None, the users whose information they do not have permission to see are not included in the users collection.

create method

The create method creates a user account and adds it to the collection.

► Syntax

```
rapidResponse.users.create(userId, Profile);
```

Parameter	Type	Description
userId	String	The user ID to create the user with.
profile	Object	Specifies the properties of the created user, specified using a UserProfile object. Each of the individual properties is optional.

► Returns

Type	Description
Object	A UserCollectionObject object.

► Exceptions

The `create` method can throw the following exceptions.

Exception	Description
ArgumentError	A blank string or <code>null</code> value was specified for the <code>id</code> parameter. This exception also indicates if a password is specified for a user that can sign in only from the single sign-in infrastructure.
User.NoPermission	You do not have permission to create user accounts.
User.NonUniqueName	A user with the specified <code>id</code> already exists.
PasswordPolicy.TooShort	If you specified a password and password policies are used, the specified password does not meet the length requirement of the password policy.
PasswordPolicy.NotComplexEnough	If you specified a password and password policies are used, the specified password does not meet the complexity requirement of the password policy.
PasswordPolicy.SameAsUserID	If you specified a password, the specified password is the same as the user ID.

► Examples

This example creates a basic user with no permissions or resources and takes the user ID from a script argument.

```
var idToUse = rapidResponse.scriptArguments["user"];
var newUser = rapidResponse.users.create(idToUse, {});
return newUser;
```

This example creates a user with a randomly-generated password and takes the user ID from a script argument.

```
var idToUse = rapidResponse.scriptArguments["user"];
var pwr = rapidResponse.users.generatePassword();
var newUser = rapidResponse.users.create(idToUse, {
  name: idToUse,
  password: pwr,
  email: idToUse + "@companyx.com"
});
```

This example creates a user that can sign in only using a single sign-in infrastructure, and takes the user ID from a script argument.

```
var idToUse = rapidResponse.scriptArguments["user"];
var newUser = rapidResponse.users.create(idToUse, {
  name: idToUse,
  email: idToUse + "@companyx.com",
  isSSOOnly: true
});
```



Note: This method was added in RapidResponse 2014.4.

generatePassword method

The `generatePassword` method creates a random-generated and cryptographically strong password that can be used to set the password for a user account.

► Syntax

```
password = rapidResponse.users.generatePassword();
```

► Returns

A String value that contains the password.

► Examples

This example generates a password.

```
var pword = rapidResponse.users.generatePassword();
```



Note: This method was added in RapidResponse 2014.4.

remove method

The remove method deletes the specified user.



Note: In most cases it is recommended that you disable the user account rather than delete the user. Disabling the user account allows you to keep all information related to the user, including data changes and the user's resources, while preventing the user from signing in to RapidResponse. For example:

```
user.setProperties({isAccountDisabled:true});
```

► Syntax

```
rapidResponse.users.remove(userId);
```

Parameter	Type	Description
userId	String	The user to delete.

► Returns

No return value.

► Exceptions

The `remove` method can throw the following exceptions.

Exception	Description
User.NotFound	The specified user does not exist.
User.NoPermission	You do not have permission to delete users.

► Example

This example deletes the user specified by a script argument.

```
var idToRemove = rapidResponse.scriptArguments["user"];
rapidResponse.users.remove(idToRemove);
```



Note: This method was added in RapidResponse 2014.4.

UserCollectionObject object

The `UserCollectionObject` object inherits all of the properties and functions of the `User` object, and adds properties and functions which are only available to system or user administrators.

The `UserCollectionObject` has the following properties.

Property	Type	Description
id	String	The user's name or ID.
name	String	The user's name or ID.
displayName	String	The name displayed for the user.
profilePicture	String	A base64-encoded String that represents an image file. This string is used to set the user's profile picture. This property was added in RapidResponse 2015.3.
email	String	The user's email address.
forwardToEmail	Boolean	If true, messages sent to the user are also sent to their email address.

Property	Type	Description
licenseType	String	The user's type. This can be one of the following values: • Alert • Internal (corresponds to Full license type) • External (corresponds to Light license type) • System As of RapidResponse 2016.2, Service Update 21, this property is deprecated. It is replaced by userLicenseType. When a UserProfile object has the userLicenseType property set, licenseType is ignored.
userLicenseType	String	The user's type. This can be one of the following values: • Full • Light • Report • Alert • System
visibilityLevel	String	The user's visibility level, which determines which users can see information about each other. The following values are valid: • Full (corresponds to visibility level of All) • Limited • None
isAccountDisabled	Boolean	If true, the user's account is disabled.
isLoggedIn	Boolean	If true, the user is logged on to their account.
title	String	The user's job title.
phone	String	The user's telephone number.
fax	String	The user's fax number.
mobile	String	The user's mobile telephone number.
pager	String	The user's pager number.
location	String	Where the user is located.
country	String	The country the user is located in.
notes	String	Information about the user.

The UserCollectionObject object contains the following methods.

Method	Description
setProperties(profile)	Sets the specified user properties.
setPassword(password)	Changes the user's password.



Note: This object was added in RapidResponse 2014.4.

setProperties method

The `setProperties` method modifies the user's properties to match the provided settings.

► Syntax

```
user.setProperties(profile)
```

Parameter	Type	Description
profile	Object	A UserProfile object representing the properties you want to apply to the user.

► Returns

Type	Description
Object	A new UserCollectionObject object that reflects the property changes.

► Exceptions

The `setProperties` method can throw the following exceptions.

Exception	Description
ArgumentError	A blank string, undefined value, or <code>null</code> value was specified for the property of the profile object.
User.NoPermission	You do not have permission to modify user accounts.
User.NotFound	The specified user does not exist.

► Example

This example adds a new phone number to the user's contact information.

```
rapidResponse.users["testuser"]
    .setProperties({contactInfo: {phone: "6131234567"}});
```

This example modifies a user to allow it to sign in only from a single sign-in infrastructure.

```
rapidResponse.users["testuser"].setProperties({isSSOOnly: true,  
password:""});
```

This example takes an image file and user from script arguments and sets the user's profile picture to the image.

```
var user = rapidResponse.scriptArguments["userName"];  
var image = rapidResponse.scriptArguments["imageName"].toBase64();  
rapidResponse.users[user].setProperties({profilePicture:image});
```



Note: This method was added in RapidResponse 2014.4.



Note: This method cannot assign a password to users assigned an Alert license type because alert users do not have passwords specified in RapidResponse.

setPassword method

The setPassword method sets the password of an existing user.

► Syntax

```
user.setPassword(password)
```

Parameter	Type	Description
password	String	The new password

► Returns

A Boolean value indicating whether the password was changed.

► Exceptions

The setPassword method can throw the following exceptions.

Exception	Description
ArgumentError	A blank string, undefined value, or null value was specified for the password parameter.
User.NoPermission	You do not have permission to modify user accounts.
User.NotFound	The specified user does not exist.

Exception	Description
User.SSOOnlyRestriction	The user can sign in only from a single sign-in infrastructure, and cannot have its password specified in RapidResponse.
PasswordPolicy.TooShort	If password policies are used, the specified password does not meet the length requirement of the password policy
PasswordPolicy.NotComplexEnough	If password policies are used, the specified password does not meet the complexity requirement of the password policy.
PasswordPolicy.UsedPreviously	If password policies are used, the specified password has already been used .
PasswordPolicy.SameAsOldPassword	If password policies are used, the specified password is identical to the current password .
PasswordPolicy.SameAsUserID	The specified password is the same as the user's ID.

► Example

This example changes the password of user "jsmith".

```
rapidResponse.users["jsmith"].setPassword("newpassword123");
```



Note: This method was added in RapidResponse 2014.4.

LoggedInUser object

The LoggedOnUser object represents the user running the script. The LoggedOnUser object contains the following properties.

Property	Type	Description
displayName	String	The user's name.
email	String	The user's email address.
forwardToEmail	Boolean	Whether copies of messages the user receives are forwarded to the email address. This property was added in RapidResponse 2013.4.
name	String	The user's name.
profilePicture	String	A base64-encoded String that represents an image file. This string is used to set the user's profile picture. This property was added in RapidResponse 2015.3.

Property	Type	Description
openResourceOnSignIn	Object	The resource that is opened when the user signs in.
userId	String	The user's ID.

The LoggedOnUser object defines the following methods.

Method	Description
hasPermission(permission)	Determines whether the user has the specified permission.
setProperties(properties)	Specifies values for the user's modifiable properties.

Permissions object

The Permissions object defines the permissions the LoggedOnUser has for performing operations. The Permissions object has the following properties.

Property	Type	Description
messageCenter	Boolean	Whether the LoggedOnUser has access to Message Center.
sendLinks	Boolean	Whether the LoggedOnUser has permission to create and send dashboard links to other users.

hasPermission method

Determines if the user running the script has the specified permission.

► Syntax

```
rapidResponse.loggedOnUser.hasPermission(permissionId)
```

Parameter	Type	Description
permissionId	String	The ID of the permission. Click here for a list of permission IDs.

► Returns

Type	Description
Boolean	• true if the user or group has the permission • false if the user or group does not have the permission

► Exceptions

Exception	Description
Permission.NotFound	The permission ID does not exist.

► Example

This example determines if the user running the script has permission to author workbooks.

```
rapidResponse.loggedInUser.hasPermission("AuthorWorkbooks");
```



Note: This method was added in RapidResponse 2014.4.

setProperty method

Specifies user properties for the user running the script.

► Syntax

```
rapidResponse.loggedInUser.setProperties(properties);
```

where

Parameter	Type	Description
properties	Object	A list of properties to set for the user. These properties can be defined as an object that contains a list of key:value pairs.

► Returns

No return value.

► Exceptions

The `setProperty` method can throw the following exception.

Exception	Description
ArgumentError	A blank or null value was passed for a parameter.

► Example

This example modifies the user's properties to forward a copy of each message to their email address.

```
rapidResponse.loggedInUser.setProperties({forwardToEmail:true});
```

This example takes an image file the from script arguments and sets the user's profile picture to the image.

```
var image = rapidResponse.scriptArguments["imageName"].toBase64();  
rapidResponse.loggedInUser.setProperties({profilePicture:image});
```



Note: This method was added in RapidResponse 2013.4.

CHAPTER 42: Group object

GroupProfile object	467
groups collection	468
GroupCollectionObject object	470
Group object methods	470

The Group object defines a group, and all groups are contained in the [groups collection](#). The Group object contains the following properties.

Property	Type	Description
name	String	The group name.
displayName	String	The name displayed for the group.
id	String	The group ID.

GroupProfile object

The GroupProfile object contains all properties of a group, including its members, responsibility, and permissions. The GroupProfile object contains the following properties.

Property	Type	Description
description	string	A description of the group.
notifyNewMembers	Boolean	If true, members are sent a notification message when they join the group.

Property	Type	Description
notificationSubject	String	The subject of the notification message sent to new members.
notificationBody	String	The main text of the notification message sent to new members.



Note: This object was added in RapidResponse 2014.4.

groups collection

The groups collection contains all groups defined in RapidResponse.

The groups collection contains the following methods.

Method	Description
create (name, profile)	Creates a user group with a specific name and group profile.
remove(name)	Deletes the user group with the specified name.

create method

The create method creates a user group and adds it to the collection.

► Syntax

```
rapidResponse.groups.create(name, [profile]);
```

Parameter	Type	Description
name	String	The name of the group.
profile	Object	Optionally specifies the properties of the created group, specified using a GroupProfile object.

► Returns

Type	Description
Object	A GroupCollectionObject object.

► Exceptions

Exception	Description
Group.NonUniqueName	A group with the specified ID already exists.
Group.NoPermission	You do not have permission to create groups.

► Example

This example creates a basic user group called BuyersNA.

```
rapidResponse.groups.create("BuyersNA");
```



Note: This method was added in RapidResponse 2014.4.

remove method

The remove method deletes the specified user group.

► Syntax

```
RapidResponse.groups.remove(name);
```

Parameter	Type	Description
name	String	The user group to delete.

► Returns

No return value.

► Exceptions

The `remove` method can throw the following exceptions.

Exception	Description
Group.NotFound	The specified user group could not be found.
Group.NoPermission	You do not have permission to delete user groups.

► Example

This example deletes the user group specified by a script argument.

```
var idToRemove = rapidResponse.scriptArguments["group"];  
rapidResponse.groups.remove(idToRemove);
```



Note: This method was added in RapidResponse 2014.4.

GroupCollectionObject object

The GroupCollectionObject object defines the properties of a group.

Property	Type	Description
name	String	The group name.
displayName	String	The name displayed for the group.
id	String	The group ID.
members	Object	A collection of users and user groups that belong to the group. This property was added in RapidResponse 2014.4.

The members property refers to a collection of user and group objects. Each object has the following properties.

Property	Type	Description
isGroup	Boolean	Whether the group member is a user group.
isUser	Boolean	Whether the group member is a user.



Note: This object was added in RapidResponse 2014.4.

Group object methods

The Group object contains the following methods.

Method	Description
setProperties(profile)	Sets the specified user group properties.
add(principalId, principal)	Adds the user or group to the user group.
remove(principalId, principal)	Removes the user or group from the user group.

setProperties method

The setProperties method modifies the group's properties to match the settings provided by a GroupProfile object.

► Syntax

```
group.setProperties(profile);
```

where *group* is a Group object.

Parameter	Type	Description
profile	Object	A GroupProfile object representing the properties you want to apply to the user group. If you are setting only a few properties with the provided GroupProfile object, the other properties are not changed.

► Returns

Type	Description
Object	A new GroupCollectionObject object that reflects the property changes.

► Exceptions

The `setProperties` method can throw the following exceptions.

Exception	Description
Group.NoPermission	You do not have permission to modify user groups.
Group.NotFound	The specified user group does not exist.

► Example

This example adds the description "Buyers North America" to the BuyersNA user group.

```
rapidResponse.groups["BuyersNA"].setProperties({description:"Buyers  
North America"});
```



Note: This method was added in RapidResponse 2014.4.

add method

Adds the user or group to the user group.

► Syntax

```
group.add(principalId);
```

```
group.add(principal);
```

where *group* is a Group object.

Parameter	Type	Description
principalId	string	The ID of the user or group to be added to the user group.
principal	object	The user object or group object to be added to the user group.

► Returns

No return value.

► Exceptions

The `add` method can throw the following exceptions.

Exception	Description
Principal.NotFound	The user or group does not exist.
Principal.NoPermission	You do not have permission to modify user groups.

► Example

This example adds the user with the user ID "jsmith" to the Buyers group.

```
rapidResponse.groups["Buyers"].add("jsmith");
```



Note: This method was added in RapidResponse 2014.4.

remove method

Removes the user or group from the user group.

► Syntax

```
group.remove(principalId);
```

```
group.remove(principal);
```

where *group* is a Group object.

Parameter	Type	Description
principalId	string	The ID of the user or group to be removed from the user group.
principal	object	The user object or group object to be removed from the user group.

► Returns

No return value.

► Exceptions

The `remove` method can throw the following exceptions.

Exception	Description
<code>Principal.NotFound</code>	The user or group does not exist.
<code>Principal.NoPermission</code>	You do not have permission to modify user groups.

► Example

This example removes the user with the user ID "jsmith" from the Buyers user group.

```
rapidResponse.groups["Buyers"].remove("jsmith");
```



Note: This method was added in RapidResponse 2014.4.

CHAPTER 43: server property

executeCheckpoint method	477
executeSnapshot method	477
executeCapture method	478
executeCondense method	479
executeCreateDataPackage method	480
executeDataExpiry method	480
executeDataIntegrityCheck method	481
executeDataSynchronize method	482
executeDataUpdate method	483
executeCommitDataUpdate method	485
executeCancelDataUpdate method	485
executeDeleteFiles method	486
executeExtractData method	488
executeExtractNetChangeData method	490
executeRestart method	493
executeVersionReclaim method	494
Defining tables for extracting data	495

The server property provides methods that execute some RapidResponse server commands. These commands are also used in RapidResponse server scheduled tasks. You must be a system administrator to create or run scripts that call these methods. In addition, these methods are available only if your company uses On-Premises RapidResponse.

The server commands executed by these methods do not send notification messages to your Message Center. For example, a data update executed through a scheduled system task sends all data administrators a summary of the records inserted, modified, or deleted by the data update, but a data update executed through a script does not. Instead, you can define how you want to be notified of any

server operation you execute in your scripts, or execute several server commands in a script and send a notification that addresses all operations performed.

Regardless of how you define notifications, server operations are recorded in the Administration Log workbook. For information about the RapidResponse Server commands and logging, see the [RapidResponse Administration Guide](#).

The server property has the following methods.

Method	Description
executeSnapshot()	Closes the active transaction data file, and then creates a new active file. The timestamp in the file name indicates when the file was originally created.
executeCapture()	Writes transactional data changes to the RapidResponse database, which is comprised of one or more DFS files.
executeCondense()	Creates a new main database file by combining all the DFS files in the hive. When run, it forces a Capture command. After the condense operation completes, the earlier hive fragment DFS files are deleted and the new main DFS file contains the complete database as of the time the operation began.
executeCreateDataPackage(packageOptions)	Creates a data package, using the optional values supplied for the packageOptions parameter to determine how the package is created.
executeDataExpiry()	Performs a data expiry operation.
executeDataIntegrityCheck(checkOptions)	Performs a data integrity check operation, using the optional values supplied for the checkOptions parameter to determine which scenarios are checked.
executeDataSynchronize(namedExtractSet)	Performs a data synchronize operation, which performs a data update on folders in the specified named extract location.
executeDataUpdate(options)	Performs a data update operation, using an optional set of parameters to customize how the update operation is performed.
executeCancelDataUpdate()	For a data update that fails a tolerance test, cancels the data update.
executeCommitDataUpdate()	For a data update that fails a tolerance test, commits the data update.
executeDeleteFiles(path)	Deletes files from the specified file path.
executeExtractData(scenario, folder, tables)	Performs a generate extract operation, which extracts the specified table data to the specified location.
executeExtractNetChangeData(scenario, baselineScenario, folder, tables)	Performs a generate net change extract operation, which extracts the data changes from the specified tables to the specified location

Method	Description
executeRestart()	Restarts the RapidResponse server
executeVersionReclaim(threshold)	Performs a version reclaim operation. If the number of used scenario versions is greater than the specified threshold, the version reclaim runs. As of RapidResponse 2014.1, the VersionReclaim command is no longer supported, because version reclaim operations are performed automatically as they are needed. This method continues to function, however, no operation is performed. RapidResponse 2016.2 introduced the VersionReclaimSweep command. This command is similar to the VersionReclaim command.

executeCheckpoint method

As of RapidResponse 11.0, the `executeCheckpoint` method has been replaced by the following methods:

- [executeSnapshot](#)
- [executeCapture](#)
- [executeCondense](#)

The `executeCheckpoint` method is no longer supported. It is recommended you modify existing scripts to use [executeCapture](#).

The `versionReclaim` parameter is no longer supported and is ignored. If you had been using this parameter, consider using the [executeVersionReclaim](#) method instead.

executeSnapshot method

As users or automated processes (for example, scheduled tasks) modify data or resources, the changes are stored in-memory. The changes are also written to transactional database files named XACT. This helps to maintain system performance and makes the changes available for recovery in case of system failure.

The XACT files are stored in the `C:\RapidResponse\Data` folder (default location). The data changes are subsequently written to the RapidResponse database by a RapidResponse Server command named Capture.

Running the Snapshot command closes the active transaction data file. A new active file is then created. Once the transaction data files are written to the RapidResponse database, the files are deleted and a new active transaction data file is created.

If you are using a RapidResponse H Service Update, this command is not supported, and performs no operations.

► Syntax

```
serverObject.executeSnapshot();
```

► Return value

No return value.

► Exceptions

The `executeSnapshot` method can throw the following exceptions.

Exception	Description
<code>Server.Snapshot.NoPermission</code>	You do not have permission to run this command.
<code>Server.Snapshot.ObsoleteCommand</code>	This command is not supported in RapidResponse H Service Updates.

► Example

```
rapidResponse.server.executeSnapshot();
```

executeCapture method

The Capture command writes transactional data changes to the RapidResponse database, which includes one or more DFS folders. When run, it forces a Snapshot command, applies the transactional data from the XACT files to a new single DFS hive folder, and deletes all the processed XACT files. A new active transaction file is then created.

If the total size of the transaction data files (XACT files) exceeds 1 GB, the Capture command automatically runs and a new active transaction data file is created.

The Capture command also runs when the RapidResponse Server service is started.

If you are using a RapidResponse H Service Update, this command is not supported, and performs no operations.

► Syntax

```
serverObject.executeCapture();
```

► Return value

No return value.

► Exceptions

The `executeCapture` method can throw the following exceptions.

Exception	Description
Server.Capture.NoPermission	You do not have permission to run this command.
Server.Capture.ObsoleteCommand	This command is not supported in RapidResponse H Service Updates.

► Example

```
rapidResponse.server.executeCapture();
```

executeCondense method

The Condense command creates a new main database file by combining all the DFS files in the hive. When run, it forces a Capture command. After the condense operation completes, the earlier hive fragment DFS files are deleted and the new main DFS file contains the complete database as of the time the operation began.

Running the Condense command can significantly reduce the amount of disk space in use.

If you are using a RapidResponse H Service Update, this command is not supported, and performs no operations.

► Syntax

```
serverObject.executeCondense();
```

► Return value

No return value.

► Exceptions

The `executeCondense` method can throw the following exceptions.

Exception	Description
Server.Condense.NoPermission	You do not have permission to run this command.
Server.Condense.ObsoleteCommand	This command is not supported in RapidResponse H Service Updates.

► Example

```
rapidResponse.server.executeCondense();
```

executeCreateDataPackage method

The `executeCreateDataPackage` method creates a backup data package, which contains copies of your workbooks, data, and log files. You can use this package to back up your resources, or to provide information for diagnosing server issues.

► Syntax

```
serverObject.executeCreateDataPackage(packageOptions);
```

Parameter	Type	Description
packageOptions	Object	Optional arguments that control how the data package is created. The arguments are: • <code>type</code>: The type of data package that is created. The type can be either "Debug", which includes all resources and data, or "Log", which includes only copies of your log files. • <code>path</code>: The file location the data package is saved in. Backslashes in the path must be preceded by the escape character (<code>\</code>).

► Returns

No return value.

► Exceptions

The `executeCreateDataPackage` method can throw the following exception.

Exception	Description
<code>Server.CreateDataPackage.NoPermission</code>	You do not have permission to run this command.

► Example

```
rapidResponse.server.executeCreateDataPackage(  
    {  
        path: "C:\\package_directory"  
    }  
);
```

executeDataExpiry method

The `executeDataExpiry` method performs a data expiry command, which deletes system data that is older than the defined expiry rules.

► Syntax

```
serverObject.executeDataExpiry();
```

► Returns

No return value.

► Exceptions

The `executeDataExpiry` method throws the following exception.

Exception	Description
<code>Server.DataExpiry.NoPermission</code>	You do not have permission to run this command.

► Example

```
rapidResponse.server.executeDataExpiry();
```

executeDataIntegrityCheck method

The `executeDataIntegrityCheck` method performs a data integrity check, which determines whether data in scenarios contains errors.

► Syntax

```
serverObject.executeDataIntegrityCheck(checkOptions);
```

Parameter	Type	Description
<code>checkOptions</code>	Object	Optional arguments that control how the data integrity is checked. The options are: • <code>maxScenarios</code>: The maximum number of scenarios to run the data integrity check on. • <code>maxScenarioDepth</code>: The maximum number of scenarios in each branch of the scenario tree to run the data integrity check on.

► Returns

No return value.

► Exceptions

The `executeDataIntegrityCheck` method can throw the following exception.

Exception	Description
Server.DataIntegrityCheck.NoPermission	You do not have permission to run this command.

► Example

```
rapidResponse.server.executeDataIntegrityCheck();
```

executeDataSynchronize method

This method performs a data update using every data extract folder in a specified named extract location. The folders included in the data synchronize operation must contain timestamps. In a standard data update, the parent folder would be removed as part of the update process. The DataSynchronize process retains the parent folder, and removes only the folders that are used in the data update, so you do not have to re-create the parent folder as part of your data extraction process. For more information about data updates, see ["executeDataUpdate method" on page 483](#).

This method is typically used to perform data updates that bring changes from one RapidResponse instance server into another, and can be used in any case where you have multiple named extract folders and want to bring all of them in to RapidResponse without deleting the parent folder. For more information, see the [RapidResponse Data Integration Guide](#).

A data synchronize operation cannot run on a folder that is already being processed by another data synchronize operation. You must wait until the process is complete before running the next data synchronize operation.

► Syntax

```
rapidResponse.server.executeDataSynchronize({options});
```

Parameter	Type	Description
options	Object	Optionally, the options passed to the DataSynchronize command. This can contain the following options, which are both specified as String values: "NamedExtractSet": Mandatory option that specifies the name of the folder that contains the data extract folders to be brought in to RapidResponse through data updates. This folder is specified relative to the NamedExtractSets folder under the RapidResponse folder. "Scenario": Optionally, the name of the scenario to update data in. If not specified, the root scenario is used. Only the name of the scenario should be provided.

► Returns

No return value.

► Exceptions

The `executeDataSynchronize` method can throw the following exceptions.

Exception	Description
<code>ArgumentError</code>	An invalid parameter or <code>null</code> value was provided as a parameter.
<code>Server.DataSynchronize.NoPermission</code>	You do not have permission to perform a <code>DataSynchronize</code> operation.
<code>Server.DataSynchronize.ToleranceFailed</code>	The data in an extract folder fails a data tolerance test.
<code>Server.DataSynchronize.DataNotAvailable</code>	There is no data ready to be updated.
<code>Server.DataSynchronize.DataUpdateError</code>	The data update process failed.
<code>Server.DataSynchronize.AnotherRunning</code>	A <code>DataSynchronize</code> operation is already in progress.

► Example

This example performs a data update using data from all folders under the `NamedExtractSets` folder.

```
rapidResponse.server.executeDataSynchronize({"NamedExtractSet":"."});
```

This example performs a data update using data from folders under a folder named `Demands`, and updates data in the `Pending Update` scenario.

```
rapidResponse.server.executeDataSynchronize  
({"NamedExtractSet":"Demands", "Scenario":"Pending Update"});
```

This example performs data updates from multiple folders.

```
rapidResponse.server.executeDataSynchronize  
({"NamedExtractSet":"Demands"});  
rapidResponse.server.executeDataSynchronize  
({"NamedExtractSet":"Supplies"});
```



Note: This method was added in RapidResponse 2014.4.

executeDataUpdate method

This method performs a data update, which brings extract data into RapidResponse.

► Syntax

```
returnValue = rapidResponse.server.executeDataUpdate({options});
```

Argument	Type	Description
options	Object	An optional list of data update options, represented as a set of name:value pairs, both of which are specified as String values. For information about the options available for performing data updates, see the RapidResponse Administration Guide.

► Returns

Log information about the data update's operations. This information is also displayed in the Data Import and Update Log workbook. For more information, see the [RapidResponse Administration Guide](#).

► Exceptions

The `executeDataUpdate` method can throw the following exceptions.

Exception	Description
<code>ArgumentError</code>	An invalid data update parameter or <code>null</code> value was provided as a parameter.
<code>Server.DataUpdate.ToleranceFailed</code>	The data included in the data update fails a data tolerance test.
<code>Server.DataUpdate.DataNotAvailable</code>	There is no data ready to be updated.
<code>Server.DataUpdate.DataUpdateError</code>	The data update process failed.
<code>Server.DataUpdate.NoPermission</code>	You do not have permission to perform the data update operation.
<code>Server.DataUpdate.AnotherRunning</code>	A data update operation is already in progress.

► Examples

The following example performs a data update.

```
var updateInfo = rapidResponse.server.executeDataUpdate();
```

The following example performs a data update that only inserts records in the Transaction Data scenario.

```
var updateInfo = rapidResponse.server.executeDataUpdate({
    "Scenario": "Transaction Data",
    "AllowDeletes": "False",
    "AllowModifications": "False"
});
```

The following example performs a data update test in the Transaction Data scenario, which brings in only 50 changes for each table and creates a detailed transaction log.

```
var updateInfo = rapidResponse.server.executeDataUpdate({
    "Scenario": "Transaction Data",
    "MaxRecords": "50",
```

```
        "DetailTransactionLog": "True"
    });
```



Note: This method was added in RapidResponse 2014.4.

executeCommitDataUpdate method

For a data update that has failed a tolerance test, commits the data update. This method allows you to commit the data and ignore the data tolerance tests. This can result in invalid data.

► Syntax

```
rapidResponse.server.executeCommitDataUpdate();
```

► Returns

A String value that indicates whether the operation was successful.

► Exceptions

The `executeCommitDataUpdate` method can throw the following exceptions.

Exception	Description
ArgumentError	A parameter was passed to the method.
Server.DataUpdate.NoDataPending	There is no pending data update to commit.
Server.DataUpdate.NoPermission	You do not have permission to commit the data update.
Server.DataUpdate.CommitDataUpdateError	The commit operation could not be completed.

► Examples

The following example commits a data update.

```
var commitUpdate = rapidResponse.server.executeCommitDataUpdate();
```



Note: This method was added in RapidResponse 2014.4.

executeCancelDataUpdate method

For a data update that has failed a tolerance test, cancels the data update. This deletes the temporary scenario that stores the data to be used in the update.

► Syntax

```
result = rapidResponse.server.executeCancelDataUpdate();
```

► Returns

A String value that indicates whether the operation was successful.

► Exceptions

The `executeCancelDataUpdate` method can throw the following exceptions.

Exception	Description
<code>ArgumentError</code>	A blank string or <code>null</code> value was specified for the name parameter.
<code>Server.DataUpdate.NoDataPending</code>	There is no pending data update to cancel.
<code>Server.DataUpdate.NoPermission</code>	You do not have permission to cancel the data update.
<code>Server.DataUpdate.CancelDataUpdateError</code>	The cancel update operation failed.

► Examples

The following example cancels a pending data update.

```
var cancelResult = rapidResponse.server.executeCancelDataUpdate();
```



Note: This method was added in RapidResponse 2014.4.

executeDeleteFiles method

Deletes files and folders from a specified path, which must be accessible from RapidResponse.

You must be a data or system administrator to call this method.

► Syntax

```
result = rapidResponse.server.executeDeleteFiles({path});
```

Parameter	Type	Description
path	String	Optionally, the file path to delete files from. This must be a file location that has been defined in the RapidResponse Configuration workbook. For more information, see the RapidResponse Administration Guide. You can specify a file to delete or use a wildcard search to delete a set of specific files in this parameter. The path cannot contain '..', which is interpreted as a higher folder level. You can access only folders in or below the specified file location.

► Returns

An object that contains the return code from the DeleteFiles command.

► Exceptions

The `executeDeleteFiles` method can throw the following exceptions.

Exception	Description
<code>ArgumentError</code>	A blank string or <code>null</code> value was specified for the <code>path</code> parameter.
<code>Server.DeleteFiles.InvalidPath</code>	The specified path is not a valid file path, or contains unsupported characters.
<code>Server.DeleteFiles.DiskError</code>	The files could not be deleted.
<code>Server.DeleteFiles.NoPermission</code>	You do not have permission to call this method.
<code>Server.DeleteFiles.MissingPrefixKey</code>	The <code>path</code> parameter does not contain a file location.
<code>Server.DeleteFiles.BadPrefixKey</code>	The <code>path</code> parameter does not contain a valid file location.
<code>Server.DeleteFiles.IllegalPath</code>	The <code>path</code> parameter contains invalid characters.

► Examples

This example deletes all files and folders from the `Files` file location.

```
var result = rapidResponse.server.executeDeleteFiles({path: '[Files]'});
```

This example deletes all files and folders from the `Demands` and `Supplies` folders under the `ExtractedFiles` file location.

```
var resultDemand = rapidResponse.server.executeDeleteFiles({path: '[ExtractedFiles]\Demands'});
var resultSupply = rapidResponse.server.executeDeleteFiles({path: '[ExtractedFiles]\Supplies'});
```

This example deletes all `.csv` files in the `Demands` folder under the `ExtractedFiles` file location.

```
var result = rapidResponse.server.executeDeleteFiles({path: '[ExtractedFiles]\Demands\*.csv'});
```

This example deletes any file that ends with 'Notes' from the Master folder under the Files file location.

```
var result = rapidResponse.server.executeDeleteFiles({path: '[Files]\Master\*Notes.*'});
```



Note: This method was added in RapidResponse 2014.4.

executeExtractData method

Performs an ExtractData command, which creates tab files representing the data in the RapidResponse database. The data is extracted from a specific set of tables. A full extract is created using this method, which includes all data from the tables. For more information, see the [RapidResponse Data Integration Guide](#).

You must be a RapidResponse system administrator to call this method.

► Syntax

```
rapidResponse.server.executeExtractData({scenario, folder, definitionId});
```

Parameter	Type	Description
scenario	Object	The scenario that data is extracted from.
folder	String	The file location where the extracted files are saved. This must be a file location that has been defined in the RapidResponse Configuration workbook. For more information, see the RapidResponse Administration Guide .
definitionId	String	A String that defines the extract definition that contains set of namespaces and tables to extract. For more information, see " Defining tables for extracting data " on page 495.

► Returns

An object that contains the following:

Property	Description
GenExHistoryId	The extract identifier. This is a GUID that can be used to identify the extract in other processes.
ReturnCode	Whether the extract succeeded or failed. Successful extracts return the value "Ok".
Scenario	The name of the scenario the data was extracted from.

Property	Description
Folder	The folder that data was extracted into.
ExtractDefinition	The name of the extract definition that defined the data to extract.

► Exceptions

The `executeExtractData` method can throw the following exceptions.

Exception	Description
<code>ArgumentError</code>	A blank string or <code>null</code> value was specified for the <code>name</code> parameter.
<code>Scenario.NotFound</code>	The specified scenario does not exist or is no longer available to you.
<code>User.NoPermission</code>	You do not have permission to call this method.
<code>Server.ExtractData.MissingFolderParam</code>	The <code>folder</code> parameter was not provided.
<code>Server.ExtractData.FolderDoesNotExist</code>	The specified folder does not exist and must be created before the method is called.
<code>Server.ExtractData.MissingScenarioParam</code>	A <code>scenario</code> parameter was not provided
<code>Server.ExtractData.MissingExtractDefinitionParam</code>	A <code>definitionId</code> parameter was not provided.
<code>Server.ExtractData.InvalidExtractDefinition</code>	The value specified for the <code>definitionId</code> is not a valid definition, or the definition either does not contain a valid expression or includes an invalid table.
<code>Server.ExtractData.DiskError</code>	An error occurred accessing or writing to the disk.
<code>Server.ExtractData.FailToUpdateExtractDataHistory</code>	The history for this extract could not be saved.
<code>Server.ExtractData.FailToCreateExtractDoneFile</code>	The extract.done file that indicates the extract is complete could not be created.
<code>Server.ExtractData.FailToExtractTable</code>	A specified table could not be extracted.
<code>Server.ExtractData.FailToExtractRecord</code>	A record in a specified table could not be extracted.
<code>Server.ExtractData.MissingPrefixKey</code>	The value specified for the <code>folder</code> parameter does not contain a file system name in its prefix.
<code>Server.ExtractData.InvalidPrefixKey</code>	An invalid file system record was provided for the <code>folder</code> parameter.
<code>Server.ExtractData.FailToCreateExtractFolder</code>	A folder to store the generated extract files could not be created.

► Examples

The following example extracts data from all tables defined in the "All_Mfg" definition.

```
var extractScenario = rapidResponse.scenarios.get({name:"Transformed
Data", scope:"Public"});
rapidResponse.server.executeExtractData({
    scenario: extractScenario,

    folder: "[extract]\\Demands",

    definitionId: "All_Mfg"
});
```

The following example extracts data from all tables defined in the "No_Historical" definition.

```
var extractScenario = rapidResponse.scenarios.get({name:"Transformed
Data", scope:"Public"});
var tableList = "No_Historical";
rapidResponse.server.executeExtractData( {
    scenario: extractScenario,

    folder: "[extract]\\Demands",

    definitionId: tableList
});
```

The following example returns the extract identifier for an extract using the All_Mfg definition.

```
var extractScenario = rapidResponse.scenarios.get({name:"Transformed
Data", scope:"Public"});
var extract = rapidResponse.server.executeExtractData({
    scenario: extractScenario,

    folder: "[extract]\\Demands",

    definitionId: "All_Mfg"
});
return (extract.GenExHistoryId);
```



Note: This method was modified in RapidResponse Service Update 2208.

executeExtractNetChangeData method

Performs an ExtractNetChangeData command, which creates tab files representing differences between a scenario and its parent for a set of database tables. For more information, see the [RapidResponse Data Integration Guide](#).

This method creates .partial tab files for tables with keys, and .full tab files for tables that do not have keys. The .partial files contain only records that are different from the specified scenario and its parent, but .full files contain all records in the table.

You must be a RapidResponse system administrator to call this method.

► Syntax

```
rapidResponse.server.executeExtractNetChangeData({scenario,  
baselineScenario, folder, definitionId});
```

Parameter	Type	Description
scenario	Object	The scenario that data is extracted from.
baselineScenario	Object	The scenario that is used to determine the changes to extract. This scenario must be a child of the scenario specified for the <code>scenario</code> parameter.
folder	String	The file location where the extracted files are saved. This must be a file location that has been defined in the RapidResponse Configuration workbook. For more information, see the RapidResponse Administration Guide .
definitionId	String	A String that defines the extract definition that contains set of namespaces and tables to extract. For more information, see " Defining tables for extracting data " on page 495.

► Returns

An object that contains the following:

Property	Description
GenExHistoryId	The extract identifier. This is a GUID that can be used to identify the extract in other processes.
ReturnCode	Whether the extract succeeded or failed. Successful extracts return the value "Ok".
Scenario	The scenario the data was extracted from.
BaselineScenario	The scenario used to compare current data to the previously extracted data.
Folder	The folder that data was extracted into.
ExtractDefinition	The name of the extract definition that defined the data to extract.

► Exceptions

The `executeExtractNetChangeData` method can throw the following exceptions.

Exception	Description
ArgumentError	A blank string or <code>null</code> value was specified for the name parameter.
Scenario.NotFound	The specified scenario does not exist or is no longer available to you.
User.NoPermission	You do not have permission to call this method.

Exception	Description
Server.ExtractNetChangeData.MissingFolderParam	The <code>folder</code> parameter was not provided.
Server.ExtractNetChangeData.FolderDoesNotExist	The specified folder does not exist and must be created before the method is called.
Server.ExtractNetChangeData.MissingScenarioParam	A <code>scenario</code> parameter was not provided
Server.ExtractNetChangeData.MissingBaselineParam	A <code>baselineScenario</code> parameter was not provided.
Server.ExtractNetChangeData.InvalidScenarioRelationship	The scenario specified for the <code>baselineScenario</code> parameter is not a child of the scenario specified for the <code>scenario</code> parameter.
Server.ExtractNetChangeData.MissingExtractDefinitionParam	A <code>definitionId</code> parameter was not provided.
Server.ExtractNetChangeData.InvalidExtractDefinition	The value specified for the <code>definitionId</code> is not a valid definition, or the definition either does not specify a valid expression in the definition or filter, or includes an invalid table.
Server.ExtractNetChangeData.DiskError	An error occurred accessing or writing to the disk.
Server.ExtractNetChangeData.FailToUpdateExtractDataHistory	The history for this extract could not be saved.
Server.ExtractNetChangeData.FailToCreateExtractDoneFile	The extract.done file that indicates the extract is complete could not be created.
Server.ExtractNetChangeData.FailToExtractTable	A specified table could not be extracted.
Server.ExtractNetChangeData.FailToExtractRecord	A record in a specified table could not be extracted.
Server.ExtractNetChangeData.MissingPrefixKey	The value specified for the <code>folder</code> parameter does not contain a file system name in its prefix.
Server.ExtractNetChangeData.InvalidPrefixKey	An invalid file system record was provided for the <code>folder</code> parameter.
Server.ExtractNetChangeData.FailToCreateExtractFolder	A folder to store the generated extract files could not be created.

► Examples

The following example generates a net change extract for a private scenario for only the tables defined in the "All_Demand" definition.

```
var compareScn = rapidResponse.scenarios.get({name:"Pending Actions",
scope:"Private"});
var tableList = "All_Demand";
rapidResponse.server.executeExtractNetChangeData({
    scenario: compareScn.parentScenario,
```

```

        baselineScenario: compareScn,
        folder: "[extract]\\Demands",
        definitionId: tableList
    });

```

The following example generates a net change extract for all tables defined in the "NoHistory" definition.

```

var compareScn = rapidResponse.scenarios.get({name:"Pending Actions",
scope:"Private"});
var fromScn = rapidResponse.scenarios.get({name: "Approved Actions",
scope:"Public"});
var tableList = "NoHistory";
rapidResponse.server.executeExtractNetChangeData({
    scenario: fromScn,

    baselineScenario: compareScn,
    folder: "[extract]\\Demands",
    definitionId: tableList
});

```

The following example returns the extract identifier if the extract was successful.

```

var compareScn = rapidResponse.scenarios.get({name:"Pending Actions",
scope:"Private"});
var fromScn = rapidResponse.scenarios.get({name: "Approved Actions",
scope:"Public"});
var tableList = "NoHistory";
var extract = rapidResponse.server.executeExtractNetChangeData({
    scenario: fromScn,

    baselineScenario: compareScn,
    folder: "[extract]\\Demands",
    definitionId: tableList
});
if (extract.ReturnCode == "Ok") {
    return (extract.GenExHistoryId);
}

```



Note: This method was modified in RapidResponse Service Update 2208.

executeRestart method

The executeRestart method shuts down and restarts the RapidResponse server.

► Syntax

```
serverObject.executeRestart(checkpoint);
```

Argument	Type	Description
checkpoint	Boolean	Optionally, indicates whether a checkpoint operation is performed when the server restarts. For more information about the operations performed by a checkpoint, see the RapidResponse Administration Guide .

► Returns

No return value.

► Exceptions

The `executeRestart` method can throw the following exception.

Exception	Description
<code>Server.Restart.NoPermission</code>	You do not have permission to run this command.

► Example

```
rapidResponse.server.executeRestart();
```



Note: This method was modified in RapidResponse 2016.2, Service Update 6.

executeVersionReclaim method

Runs a version reclaim operation, which removes inactive scenario version numbers, and allows users to create new scenarios using those version numbers. If all scenario version numbers are used, users are unable to create new scenarios.

As of RapidResponse 2014.1, the VersionReclaim command is no longer supported, because version reclaim operations are performed automatically as they are needed. This method continues to function, however, no operation is performed. RapidResponse 2016.2 introduced the VersionReclaimSweep command. For more information, see the [RapidResponse Administration Guide](#).

► Syntax

```
serverObject.executeVersionReclaim(threshold);
```

Parameter	Type	Description
threshold	Number	Optionally, the number of scenario versions that determine whether a version reclaim runs. The version reclaim runs only if the number of used version numbers is greater than the threshold value. The value you specify for this parameter can be in one of the following ranges: 0 .. 1: Specifies the threshold as a percentage of used scenarios. For example, specifying 0.75 sets the threshold to 75% of all scenario version numbers. 2 .. 8192: Specifies the threshold as the number of used scenarios. This syntax is deprecated, and Kinaxis recommends using the percentage syntax instead.

► Returns

No return value.

► Exceptions

The `executeVersionReclaim` method can throw the following exception.

Exception	Description
<code>Server.VersionReclaim.NoPermission</code>	You do not have permission to run this command.

► Example

```
rapidResponse.server.executeVersionReclaim(0.8);
```

Defining tables for extracting data

The `executeExtractData` and `executeExtractNetChangeData` methods use a list of tables to determine the data that is extracted.

The lists of tables that are extracted are defined using the Scheduled System Tasks workbook, which contains worksheets that allow you to define named definitions that contain a table list and, optionally, filters that can be applied to the tables in the list.

The extract definition contains a query expression that defines the set of namespaces and tables that are extracted. This is specified as a string, which can include wildcards for the table names, but cannot include wildcards for the namespaces. You can specify multiple sets of tables by separating the sets of tables using commas. For more information about wildcards, see the [RapidResponse Resource Authoring Guide](#).

Examples of extract definition expressions are shown below.

Expression	Description
"Mfg::*"	Extracts all tables from the Mfg namespace.
"Mfg::*Demand*, User::*Demand*"	Extracts data from all tables in the Mfg and User namespaces that have 'Demand' in their name.
"Mfg::*, !Mfg::Historical*"	Extracts data from all tables in the Mfg namespace, except for any table whose name begins with 'Historical'.

You can also specify filter expressions to apply to the tables present in an extract definition. These filters limit the records included in the extract, and are used every time the extract definition is used to extract data. Each filter expression applies to a single table, and must be a valid filter expression that contains a comparison operator. A filter expression for a table is associated with a specific extract definition, and is used only when that extract definition is used.

Because the filter is specified in the definition, it cannot be validated before being used. It is recommended you develop a filter expression either in a worksheet to verify it returns the correct results, or use it to extract data on a test system. The filter expression cannot contain square brackets ([]).

When you create an extract definition, you can use it in a script by passing its name as a parameter to the `executeExtractData` or `executeExtractNetChangeData` method. For more information, see ["executeExtractData method" on page 488](#) and ["executeExtractNetChangeData method" on page 490](#).

► Create an extract definition

1. In the **Administration** pane, under **System Settings**, click **Scheduled System Tasks**.
2. Click the **Extract Definitions** worksheet tab.
3. On the **Edit** menu, click **Insert Record**.
4. For the new record, specify the following:
 - **Extract ID:** The name of the definition. This name will be used in method calls.
 - **Table Selection String:** The expression that defines the tables to be extracted.
 - **Description:** Information about the definition.
5. In the **Auto-managed** list, ensure **N** is selected.
6. If you are creating multiple extract definitions, click **Save**, and then repeat step 4. Otherwise, click **Save and Close**.

► Apply a filter to an extract definition

1. In the **Administration** pane, under **System Settings**, click **Scheduled System Tasks**.
2. Click the **Extract Configuration** worksheet tab.
3. On the **Edit** menu, click **Insert Record**.

4. For the new record, specify the following:
 - **Extract ID:** The definition this filter is used by.
 - **Table:** The namespace and table the filter applies to.
 - **Filter Expression:** The filter expression to apply to the specified table. For more information, see the [RapidResponse Resource Authoring Guide](#)
5. If you are applying filters to multiple tables, click **Save**, and then repeat step 4 for each table you want to apply a filter to. Otherwise, click **Save and Close**.



Note: If your extract definition includes tables from only one namespace, you can specify only the table name for the **Table** value. However, it is recommended you include the namespace to ensure the correct table is filtered if the extract definition is modified.

Object index

rapidResponse	103
Resource	109
HelpInformation	110
Resource collection	110
accessControlList collection	121
Principal	123
PermissionCollection	124
Permission	126
Script	129
scripts collection	129
Scenario	139
Perspective	141
ActivityLog	141
scenarios collection	142
Options	147
Options	159
cdcInstances collection	164
CDCInstance	165
Workbook	169
WorkbookInitialization	170
WorkbookHierarchyInitialization	171
Path	171
Variable	172
WorkbookPreferences	172
workbooks collection	172
dependencies collection	185
WorkbookDependency	186

WorkbookDependencyVariableMapping	186
variables collection	186
WorkbookVariable	186
VariableValueQuantity	187
VariableValueBoolean	188
VariableValueList	188
VariableValueListFixed	188
VariableValueListFixedFixedValue	188
VariableValueListQuery	189
NameValuePair	189
commands collection	189
Command	191
DataUpdateCommand	192
Action	192
ScriptCommand	192
OpenFormCommand	193
execute (Command)	193
Worksheet	197
CrosstabWorksheet	199
worksheets collection	200
columns collection	216
WorksheetColumn	216
imageMap	216
WorksheetColumnImageMapping	217
rows collection (worksheet)	217
cells collection (worksheet)	217
WorksheetRow	218
WorksheetCell	218
WorkbookStyle	218
WorksheetData	219
WorksheetDataRow	219

WorksheetDataCell	220
ImportData	220
Bucketing	221
WorksheetDataModification	224
BucketSettings	225
BasicBucketSettings	226
AdvancedBucketSettings	226
BucketInterval	227
BucketOption	227
BucketAnchorDate	228
BucketDateOffset	228
BucketStartDate	229
BucketCalendarToDate	229
AutoBucketInfo	230
DateBucket	230
WorksheetColumnDrillToDetail	231
WorksheetColumnDrillToDetailCondition	232
columnMappings collection (worksheet)	232
DrillToDetailColumnMapping	232
WorksheetColumnDrillToDetailBucketVariableMappings	232
TreemapQuery	235
TreemapQueryColorMeasure	236
TreemapQueryDimension	236
dimensions collection	236
TreemapQueryData	237
TreemapQuerySummaryData	237
TreemapQueryRow	237
TreemapQueryCell	238
rows collection (treemap)	238
cells collection (treemap)	238
TreemapDrillToDetail	238
columnMappings collection (treemap)	239
TreemapDrillToDetailMapping	239
Filter	241
Table	241

filters collection	242
CompositeFilterData	249
FilterItemGroup	249
ScenarioGroup	249
HierarchyGroup	250
BucketSettingsGroup	250
FilterManagerFactory	250
FilterManager	253
FilterManagerFilterItem	254
FilterManagerFilterItemValue	254
CompositeFilterManager	262
Hierarchy	265
hierarchies collection	265
SiteGroup	273
siteGroups collection	273
Alert	281
alerts collection	281
ScheduledTask	291
scheduledTasks collection	291
AutomationChain	301
automationChains collection	301
Schedule	311
TaskFlow	315
taskFlows collection	315
Exception	323
ProfileVariable	325
profileVariables collection	325
ProcessingRule	329
processingRules collection	329
Dashboard	331
WidgetStyleSettings	331
widgetReferences collection	332
WidgetReference	332
dashboards collection	333
Widget	341

WidgetWorksheetSettings	341
ControlSettings	342
NameValueDisplayValue	342
widgets collection	343
get (widgets collection)	344
form	351
forms collection	351
scorecard	367
scorecards collection	367
collaboration	375
collaborations collection	375
collaborationScenario	377
Responsibility	383
responsibilities collection	383
Workflow	389
Query	397
query	400
columns collection	404
rows collection	404
cells collection	404
ProcessTemplate	411
processTemplates collection	411
ChartEngine	419
JsChart	419
integrationPlatformService property	423
MessageDefinition	424
Set	425
Link	435
links collection	435
dateTime	439
console	445
Message	447
SendMessage	448
Attachment	448
SendBroadcastMessage	448

messages collection	449
User	453
UserProfile	453
ContactInfo	455
users collection	455
UserCollectionObject	459
LoggedInUser	463
Permissions	464
Group	467
GroupProfile	467
groups collection	468
GroupCollectionObject	470
groupMember collection	470
server	475

Method index

forEach (Resource collection)	112
forEachId (Resource collection)	112
get (Resource collection)	112
exists (Resource collection)	113
create (Resource collection)	114
remove (Resource collection)	115
give (Resource)	116
makePublic (Resource)	117
rename (Resource)	119
copy (Resource)	120
remove (accessControlList collection)	121
add (accessControlList collection)	122
allow (PermissionCollection)	124
deny (PermissionCollection)	125
hasPermission (PermissionCollection)	126
get (scripts collection)	130
exists (scripts collection)	130
remove (scripts collection)	131
give (Script)	133
makePublic (Script)	134
rename (Script)	134
copy (Script)	135
call (Script)	136
get (scenarios collection)	142
exists (scenarios collection)	144
create (scenarios collection)	145
remove (scenarios collection)	147

createTemporary (scenarios collection)	148
createWorkflowScenario (scenarios collection)	149
give (Scenario)	154
makePublic (Scenario)	155
rename (Scenario)	156
update (Scenario)	157
reset (Scenario)	158
commit (Scenario)	159
addResponse (ActivityLog)	161
setProperties (Scenario)	163
get (cdcInstances collection)	165
captureChanges (CDCInstance)	166
clearChanges (CDCInstance)	167
resetChanges (CDCInstance)	167
get (workbooks collection)	172
exists (workbooks collection)	176
remove (workbooks collection)	176
give (Workbook)	178
makePublic (Workbook)	179
rename (Workbook)	180
copy (Workbook)	181
generateReport (Workbook)	181
get (commands collection)	189
exists (commands collection)	190
get (worksheets collection)	200
exists (worksheets collection)	201
getAvailableHierarchies (Worksheet)	203
getChart (Worksheet)	203
getChartDefinition (Worksheet)	204
getData (Worksheet)	205
getFilterManager (Worksheet)	206

getPossibleControlValues (Worksheet)	206
getTreemapQuery (Worksheet)	208
importData (Worksheet)	208
convertToJSON (Worksheet)	210
get (cells)	221
slice (rows)	222
close (rows)	223
execute (WorksheetDataModification)	224
getData (TreemapQuery)	240
get (filters collection)	242
exists (filters collection)	243
remove (filters collection)	244
give (Filter)	245
makePublic (Filter)	246
rename (Filter)	247
copy (Filter)	248
createFilterManager (FilterManagerFactory)	250
createCompositeFilterManager (FilterManagerFactory)	251
getWorkbookBaseTable (FilterManager)	254
getUsesSelectedFilter (FilterManager)	255
getMultiScenarioCount (FilterManager)	256
getWorkbookVariables (FilterManager)	257
getBucketSettings (FilterManager)	257
getFilterValues (FilterManager)	258
getFilterValues (FilterManager)	259
getHierarchyNode (FilterManager)	260
isMultiScenario (FilterManager)	261
validate (FilterManager)	261
getData (CompositeFilterManager)	262
get (hierarchies collection)	265
exists (hierarchies collection)	266
give (Hierarchy)	268
makePublic (Hierarchy)	268
rename (Hierarchy)	269
copy (Hierarchy)	270

get (siteGroups collection)	274
exists (siteGroups collection)	275
remove (siteGroups collection)	276
give (SiteGroup)	277
makePublic (SiteGroup)	278
rename (SiteGroup)	278
copy (SiteGroup)	279
get (alerts collection)	281
exists (alerts collection)	282
remove (alerts collection)	283
give (Alert)	285
makePublic (Alert)	285
rename (Alert)	286
copy (Alert)	287
runAsync (Alert)	288
get (scheduledTasks collection)	291
exists (scheduledTasks collection)	292
remove (scheduledTasks collection)	293
give (ScheduledTask)	295
makePublic (ScheduledTask)	295
rename (ScheduledTask)	296
copy (ScheduledTask)	297
runAsync (ScheduledTask)	298
remove (automationChains collection)	302
get (automationChains collection)	302
exists (automationChains collection)	303
give (AutomationChain)	305
makePublic (AutomationChain)	306
rename (AutomationChain)	307
copy (AutomationChain)	308
runAsync (AutomationChain)	309
get (schedules collection)	311
exists (schedules collection)	312
get (taskFlows collection)	315
exists (taskFlows collection)	316

give (TaskFlow)	318
makePublic (TaskFlow)	318
rename (TaskFlow)	319
copy (TaskFlow)	320
set (profileVariables collection)	326
setConstantVariable (profileVariables collection)	327
get (profileVariables collection)	327
get (processingRules collection)	330
get (dashboards collection)	333
exists (dashboards collection)	334
give (Dashboard)	335
makePublic (Dashboard)	336
rename (Dashboard)	337
copy (Dashboard)	338
getLayout (Dashboard)	338
createEmbeddedLinkUrl (Dashboard)	339
createEmbeddedLinkUrl (Dashboard)	339
exists (widgets collection)	343
give (Widget)	345
makePublic (Widget)	346
rename (Widget)	347
copy (Widget)	348
getAttachments (Widget)	348
get (forms collection)	352
exists (forms collection)	353
remove (forms collection)	353
give (form)	355
makePublic (form)	356
rename (form)	356
copy (form)	357
okCancelNotice (Form)	358
yesNoNotice (Form)	360
yesNoCancelNotice (Form)	361
autoCloseNotice (Form)	362
get (scorecards collection)	368

remove (scorecards collection)	368
exists (scorecards collection)	369
give (Scorecard)	370
makePublic (Scorecard)	371
rename (Scorecard)	372
copy (TaskFlow)	373
get (collaborations collection)	375
remove method	376
get (collaborationScenario collection)	377
has (collaborationScenario collection)	378
add (accessControlList collection)	379
remove (scorecards collection)	380
setProperties (Scenario)	381
get (responsibilities collection)	383
exists (responsibilities collection)	384
give (Responsibility)	385
makePublic (Responsibility)	386
get (workflows collection)	389
exists workflows collection)	390
remove (workflows collection)	391
give (Workflow)	392
makePublic (Workflow)	393
rename (Workflow)	394
copy (Workflow)	395
startExecution(Workflow)	395
execute (Query)	397
executeSingleton (Query)	399
queryParameters (query)	401
close (query)	401
commit (query)	402
get (query)	405
deleteRow (query)	406
deleteRows (query)	407
insertRow (query)	408
get (processTemplates collection)	411

exists (processTemplates collection)	412
give (ProcessTemplate)	414
makePublic (ProcessTemplate)	414
rename (ProcessTemplate)	415
copy (ProcessTemplate)	416
generate (ChartEngine)	420
runTask (integrationPlatformService)	425
sendMessage (integrationPlatformService)	426
getEndpointIds (IntegrationPlatformService)	427
getEndpointPathParameterNames (IntegrationPlatformService)	428
invokeEndpoint (IntegrationPlatformService)	429
get (links collection)	436
getResource (Link)	437
createEmbeddedLinkUrl (Dashboard)	437
toCalendarDate (dateTime)	440
add (dateTime)	440
subtract (dateTime)	441
difference (dateTime)	442
writeLine (console)	445
sendMessage (messages collection)	449
sendBroadcastMessage (messages collection)	451
create (users collection)	456
generatePassword (users collection)	458
remove (users collection)	458
setProperties (UserCollectionObject)	461
setPassword (UserCollectionObject)	462
hasPermission (LoggedInUser)	464
setProperties (LoggedInUser)	465
create (groups collection)	468
remove (groups collection)	469
setProperties (group)	470
add (group)	471
remove (group)	472
executeCheckpoint (server)	477
executeSnapshot (server)	477

executeCapture (server)	478
executeCondense (server)	479
executeCreateDataPackage (server)	480
executeDataExpiry (server)	480
executeDataIntegrityCheck (server)	481
executeDataSynchronize (server)	482
executeDataUpdate (server)	483
executeCommitDataUpdate (server)	485
executeCancelDataUpdate (server)	485
executeDeleteFiles (server)	486
executeExtractData (server)	488
executeExtractNetChangeData (server)	490
executeRestart (server)	493
executeVersionReclaim (server)	494

Property index

alerts (rapidResponse)	103
automationChains (rapidResponse)	103
charting (rapidResponse)	103
collaborations (rapidResponse)	103
console (rapidResponse)	103
dashboards (rapidResponse)	104
dateTime (rapidResponse)	104
exceptions(rapidResponse)	104
filters (rapidResponse)	104
filtering (rapidResponse)	104
forms (rapidResponse)	104
groups (rapidResponse)	104
help (rapidResponse)	104
hierarchies(rapidResponse)	104
integrationPlatformService (rapidResponse)	104
links (rapidResponse)	104
loggedInUser (rapidResponse)	104
messages (rapidResponse)	105
permissions (rapidResponse)	105
profileVariables (rapidResponse)	105
processingRule (rapidResponse)	105
processTemplate(rapidResponse)	105
queries(rapidResponse)	105
resources(rapidResponse)	105
responsibilities(rapidResponse)	105
scenarios (rapidResponse)	105
scheduledTasks (rapidResponse)	105

schedules (rapidResponse)	105
scriptArguments (rapidResponse)	105
scripts (rapidResponse)	106
scorecards (rapidResponse)scorecards	106
server (rapidResponse)	106
siteGroups (rapidResponse)	106
taskflows(rapidResponse)	106
treemapQueries(rapidResponse)	106
users (rapidResponse)	106
widgets (rapidResponse)	106
workbooks (rapidResponse)	106
workflows (rapidResponse)	106
worksheets (rapidResponse)	106
name (Resource)	109
scope (Resource)	109
id (Resource)	109
owner (Resource)	109
creator (Resource)	109
creationDate (Resource)	110
lastModifiedDate (Resource)	110
lastUsedDate (Resource)	110
usageCount (Resource)	110
accessControlList (Resource)	110
customHelpEnabled (HelpInformation)	110
customHelpLabel (HelpInformation)	110
customHelpUrl (HelpInformation)	110
helpUrl (HelpInformation)	110
LOCAL_URL (HelpInformation)	110
ONDEMAND_URL (HelpInformation)	110
name (Resource)	111
scope (Resource)	111

principal (accessControlList collection)	121
level (accessControlList collection)	121
isUser (Principal)	123
isGroup (Principal)	123
displayName (Principal)	123
permissions (Principal)	123
id (Permission)	127
hasPermission (Permission)	127
isDenied (Permission)	127
isInherited (Permission)	127
accessPermission (Scenario)	139
activityLog (Scenario)	139
cdcInstances (Scenario)	140
childScenarios (Scenario)	140
isAutoUpdate (Scenario)	140
isPermanent (Scenario)	140
isUpToDate (Scenario)	140
lastUpdatedDate (Scenario)	140
lastCommittedDate (Scenario)	140
parentScenario (Scenario)	140
perspective (Scenario)	140
purpose (Scenario)	140
scopedName (Scenario)	140
status (Scenario)	140
suppressEditWarning (Scenario)	140
type (Scenario)	141
usersWithOpenQueries (Scenario)	141
name (Perspective)	141
isInherited (Perspective)	141
subject (ActivityLog)	141
type (ActivityLog)	141
body (ActivityLog)	141
notifyAll (ActivityLog)	141
processOrchestrationScenario (Scenario)	142
systemRootScenario (Scenario)	142

failIfScenarioHasChildren (Options)	147
failIfScenarioHasOpenQueries (Options)	147
failIfScenarioHasChildren (Options)	159
failIfScenarioHasOpenQueries (Options)	159
failOnUpdateMergeConflicts (Options)	159
forceUpdate (Options)	160
keepScenarioAfterCommit (Options)	160
name (CDCInstance)	166
commands (Workbook)	169
dependencies (Workbook)	169
variables (Workbook)	170
worksheets (Workbook)	170
scenarios (WorkbookInitialization)	170
filter (WorkbookInitialization)	170
siteGroup (WorkbookInitialization)	170
hierarchies (WorkbookInitialization)	170
variables (WorkbookInitialization)	170
currency (WorkbookInitialization)	170
unitOfMeasure (WorkbookInitialization)	170
model (WorkbookInitialization)	170
pool (WorkbookInitialization)	170
partType (WorkbookInitialization)	171
part (WorkbookInitialization)	171
referencePart (WorkbookInitialization)	171
constraint (WorkbookInitialization)	171
workCenter (WorkbookInitialization)	171
processInstance (WorkbookInitialization)	171
cdcInstance (WorkbookInitialization)	171
hierarchy (WorkbookHierarchyInitialization)	171
path (WorkbookHierarchyInitialization)	171
name (Path)	171
childPath (Path)	171
name (Variable)	172
value (Variable)	172
dependentId (WorkbookDependency)	186

dependentType (WorkbookDependency)	186
variableMappings (WorkbookDependency)	186
workbookKey (WorkbookDependency)	186
targetId (WorkbookDependencyVariableMapping)	186
sourceId (WorkbookDependencyVariableMapping)	186
isVisible (WorkbookVariable)	186
description (WorkbookVariable)	186
label (WorkbookVariable)	186
name (WorkbookVariable)	187
defaultType (WorkbookVariable)	187
isLabelOnToolbar (WorkbookVariable)	187
defaultValue (WorkbookVariable)	187
isOnToolbar (WorkbookVariable)	187
type (WorkbookVariable)	187
quantityValue (WorkbookVariable)	187
booleanValue (WorkbookVariable)	187
listvalue (WorkbookVariable)	187
min (VariableValueQuantity)	187
minSpecified (VariableValueQuantity)	187
max (VariableValueQuantity)	187
maxSpecified (VariableValueQuantity)	187
falseDataValue (VariableValueBoolean)	188
falseDisplayValue (VariableValueBoolean)	188
trueDataValue (VariableValueBoolean)	188
trueDisplayValue (VariableValueBoolean)	188
fixed (VariableValueList)	188
query (VariableValueList)	188
fixedValues (VariableValueListFixed)	188
sortFixedValues (VariableValueListFixed)	188
useFixedValues (VariableValueListFixed)	188
dataValue (VariableValueListFixedFixedValue)	188
displayValue (VariableValueListFixedFixedValue)	188
useQueryValues (VariableValueListQuery)	189
name (NameValuePair)	189
value (NameValuePair)	189

name (Command)	192
type (Command)	192
actions (DataUpdateCommand)	192
name (DataUpdateCommand)	192
type (DataUpdateCommand)	192
index (Action)	192
name (ScriptCommand)	193
type (ScriptCommand)	193
id (Worksheet)	197
name (Worksheet)	197
displayName (Worksheet)	197
scenarios (Worksheet)	198
filter (Worksheet)	198
siteGroup (Worksheet)	198
model (Worksheet)	198
pool (Worksheet)	198
partType (Worksheet)	198
part (Worksheet)	198
referencePart (Worksheet)	198
hierarchies (Worksheet)	198
variables (Worksheet)	198
columns (Worksheet)	198
sortedColumns (Worksheet)	198
parentWorkbook (Worksheet)	198
isHidden (Worksheet)	198
canSort (Worksheet)	198
dataModification (Worksheet)	198
worksheetData (Worksheet)	198
importData (Worksheet)	199
crosstabWorksheet (Worksheet)	199
autobucketInfo (Worksheet)	199
bucketSettings (Worksheet)	199
rawBucketValues (CrosstabWorksheet)	199
bucketColumnId (CrosstabWorksheet)	199
bucketDataType (CrosstabWorksheet)	199

index (CrosstabWorksheet)	199
id (CrosstabWorksheet)	199
start (WorksheetSelection)	200
end (WorksheetSelection)	200
columnId (WorksheetColumn)	216
drillToDetails (WorksheetColumn)	216
groupRule (WorksheetColumn)	216
header (WorksheetColumn)	216
hidden (WorksheetColumn)	216
hideZeroValues (WorksheetColumn)	216
imageMap (WorksheetColumn)	216
width (WorksheetColumn)	216
count (imageMap)	216
dataValue (WorksheetColumnImageMapping)	217
image (WorksheetColumnImageMapping)	217
imageName (WorksheetColumnImageMapping)	217
index (WorksheetColumnImageMapping)	217
toolTip (WorksheetColumnImageMapping)	217
count (worksheet rows)	217
isPivoted (WorksheetRow)	218
recordId (WorksheetRow)	218
number (WorksheetRow)	218
pivotedColumnIndex (WorksheetRow)	218
cells (WorksheetRow)	218
pivotedColumnId (WorksheetRow)	218
type (WorksheetRow)	218
formattedValue (WorksheetCell)	218
isTransposedHeader (WorksheetCell)	218
width (WorksheetCell)	218
columnId (WorksheetCell)	218
isCrosstab (WorksheetCell)	218
isPivotedandBucketed (WorksheetCell)	218
value (WorksheetCell)	218
dataAlignment (WorksheetCell)	218
dataType (WorksheetCell)	218

style (WorksheetCell)	218
fontStyle (WorkbookStyle)	218
foregroundColor (WorkbookStyle)	218
backgroundColor (WorkbookStyle)	218
rows (WorksheetData)	219
status (WorksheetData)	219
index (WorksheetDataRow)	219
number (WorksheetDataRow)	219
type (WorksheetDataRow)	220
cells (WorksheetDataRow)	220
index (WorksheetDataCell)	220
value (WorksheetDataCell)	220
bucketing (ImportData)	220
data (ImportData)	220
buckets (Bucketing)	221
calendar (bucketing)	221
type (WorksheetDataModification)	224
type (BucketSettings)	226
settings (BucketSettings)	226
bucketStartDate (BasicBucketSettings)	226
bucketStartDateEnabled (BasicBucketSettings)	226
bucketStartDateModifiable (BasicBucketSettings)	226
intervals (BasicBucketSettings)	226
lockBucketing (BasicBucketSettings)	226
options (BasicBucketSettings)	226
anchorDate (AdvancedBucketSettings)	227
calendarToDateSubtotals (AdvancedBucketSettings)	227
includePastBucket (AdvancedBucketSettings)	227
includeFutureBucket (AdvancedBucketSettings)	227
intervals (AdvancedBucketSettings)	227
lockBucketing (AdvancedBucketSettings)	227
bucketCalendar (BucketInterval)	227
numberOfBuckets (BucketInterval)	227
direction (BucketInterval)	227
UseSubtotalCalendar1 (BucketInterval)	227

useSubtotalCalendar2 (BucketInterval)	227
useSubtotalCalendar 3 (BucketInterval)	227
useSubtotalCalendar4 (BucketInterval)	227
isDefault (BucketOption)	228
numberOfBuckets (BucketOption)	228
bucketCalendar (BucketOption)	228
subtotalCalendar (BucketOption)	228
index (BucketOption)	228
id (BucketOption)	228
type (BucketAnchorDate)	228
offset (BucketAnchorDate)	228
currentCalendar (BucketAnchorDate)	228
rawDate (BucketAnchorDate)	228
index (BucketAnchorDate)	228
id (BucketAnchorDate)	228
adjustment (BucketDateOffset)	229
calendar (BucketDateOffset)	229
index (BucketDateOffset)	229
id (BucketDateOffset)	229
calendar (BucketStartDate)	229
offset (BucketStartDate)	229
type (BucketStartDate)	229
balanceBucketLabel (BucketCalendarToDate)	229
bucketLabel (BucketCalendarToDate)	229
calendar (BucketCalendarToDate)	229
origin (BucketCalendarToDate)	230
originCalendar (BucketCalendarToDate)	230
includeFutureBucket (AutoBucketInfo)	230
includePastBucket (AutoBucketInfo)	230
buckets (AutoBucketInfo)	230
autoBucketColumnId (AutoBucketInfo)	230
allowPartialTimeSubtotals (AutoBucketInfo)	230
index (AutoBucketInfo)	230
id (AutoBucketInfo)	230
end (DateBucket)	230

start (DateBucket)	230
index (DateBucket)	231
id (DateBucket)	231
condition (WorksheetColumnDrillToDetail)	231
customLabel (WorksheetColumnDrillToDetail)	231
drillTarget(WorksheetColumnDrillToDetail)	231
labelSource (WorksheetColumnDrillToDetail)	231
columnMappings (WorksheetColumnDrillToDetail)	231
dependencyId (WorksheetColumnDrillToDetail)	231
bucketVariableMappings (WorksheetColumnDrillToDetail)	231
isTargetWorksheetTransformation (WorksheetColumnDrillToDetail)	231
isTargetWorksheetTreemap (WorksheetColumnDrillToDetail)	231
label (WorksheetColumnDrillToDetail)	231
value (WorksheetColumnDrillToDetailCondition)	232
columnId (WorksheetColumnDrillToDetailCondition)	232
index (DrillToDetailColumnMapping)	232
sourceId (DrillToDetailColumnMapping)	232
targetId (DrillToDetailColumnMapping)	232
bucketStartDate (WorksheetColumnDrillToDetailBucketVariableMappings)	233
bucketEndDate (WorksheetColumnDrillToDetailBucketVariableMappings)	233
sourceId (WorksheetColumnDrillToDetailColumnMapping)	233
targetId (WorksheetColumnDrillToDetailColumnMapping)	233
defaults (TreemapQuery)	235
dimensions (TreemapQuery)	235
measures (TreemapQuery)	235
supportsHierarchies (TreemapQuery)	235
colors (TreemapQueryColorMeasure)	236
name (TreemapQueryColorMeasure)	236
header (TreemapQueryDimension)	236
index (TreemapQueryDimension)	236
hasDimension1 (TreemapQueryData)	237
hasDimension2 (TreemapQueryData)	237
rows (TreemapQueryData)	237
summary (TreemapQueryData)	237
name (TreemapQuerySummaryData)	237

values (TreemapQuerySummaryData)	237
index (TreemapQueryRow)	237
cells (TreemapQueryRow)	237
formattedValue (TreemapQueryCell)	238
index (TreemapQueryCell)	238
value (TreemapQueryCell)	238
count (treemap rows)	238
count (treemap cells)	238
columnMappings (TreemapDrillToDetail)	238
dependencyId (TreemapDrillToDetail)	238
isTargetWorksheetTransformation (TreemapDrillToDetail)	239
isTargetWorksheetTreemap (TreemapDrillToDetail)	239
worksheetId (TreemapDrillToDetail)	239
index (TreemapDrillToDetailMapping)	239
associatedTable (Filter)	241
name (Table)	241
filterItemGroups (CompositeFilterData)	249
scenarioGroups (CompositeFilterData)	249
hierarchyGroups (CompositeFilterData)	249
bucketSettingsGroups (CompositeFilterData)	249
filterItemDisplayValue (FilterItemGroup)	249
filterItemTypeId (FilterItemGroup)	249
filterItemValue (FilterItemGroup)	249
filterManagerIds (FilterItemGroup)	249
isValid (FilterItemGroup)	249
scenarios (ScenarioGroup)	249
filterManagerIds (ScenarioGroup)	249
hierarchies (HierarchyGroup)	250
filterManagerIds (HierarchyGroup)	250
bucketSettings (BucketSettingsGroup)	250
filterManagerIds (BucketSettingsGroup)	250
id (FilterManager)	253
type(FilterManager)	254
availableValues(FilterManager)	254
level(FilterManager)	254

isValid(FilterManager)	254
isSelectable(FilterManager)	254
displayValue(FilterManager)	254
dataValue(FilterManager)	254
level(FilterManager)	254
isValid(FilterManager)	254
isSelectable(FilterManager)	254
displayValue(FilterManager)	254
dataValue(FilterManager)	254
imageId(siteGroup)	273
name (Exception)	323
errorCode (Exception)	323
message (Exception)	323
stack (Exception)	323
name (ProfileVariable)	325
value (ProfileVariable)	325
id (ProcessingRule)	329
value (ProcessingRule)	329
backgroundColor (Dashboard)	331
preferences (Dashboard)	331
showDataSettingsDialogOnOpen (Dashboard)	331
widgetReferences (Dashboard)	331
widgetStyleSettings (Dashboard)	331
showTitle (WidgetStyleSettings)	331
showBorder (WidgetStyleSettings)	331
paddingTop (WidgetStyleSettings)	332
paddingLeft (WidgetStyleSettings)	332
paddingBottom (WidgetStyleSettings)	332
paddingRight (WidgetStyleSettings)	332
widgetKey (WidgetReference)	332
referenceType (WidgetReference)	332
controlSettings (WidgetReference)	332
bucketSettings (WidgetReference)	332
workbookPreferences (WidgetReference)	332
title (WidgetReference)	332

height (WidgetReference)	332
layoutId (WidgetReference)	332
settings (Widget)	341
type (Widget)	341
title (Widget)	341
help (Widget)	341
workbookKey (WidgetWorksheetSettings)	341
displayType (WidgetWorksheetSettings)	341
worksheetId (WidgetWorksheetSettings)	341
controlSettings (WidgetWorksheetSettings)	341
hierarchies (ControlSettings)	342
siteGroup (ControlSettings)	342
filter (ControlSettings)	342
pool (ControlSettings)	342
processInstance (ControlSettings)	342
constraint (ControlSettings)	342
referencePart (ControlSettings)	342
scenarios (ControlSettings)	342
variables (ControlSettings)	342
model (ControlSettings)	342
project (ControlSettings)	342
workCenter (ControlSettings)	342
part (ControlSettings)	342
name (NameValueDisplayValue)	342
value (NameValueDisplayValue)	342
displayValue (NameValueDisplayValue)	342
collaborationScenario	375
columns (query)	401
rows (query)	401
status (query)	401
recordLimit (queryParameters)	401
source (queryParameters)	401
variables (queryParameters)	401
count (query)	404
id (columns collection)	404

dataType (columns collection)	404
number (rows collection)	404
cell (rows collection)	404
value (cells collection)	405
ActualHeight (JsChart)	419
actualWidth (JsChart)	419
height (JsChart)	419
image (JsChart)	419
width (JsChart)	419
name (MessageDefinition)	424
groupId (MessageDefinition)	424
worksheet (MessageDefinition)	424
test (MessageDefinition)	424
sets (MessageDefinition)	424
worksheet (Set)	425
name (Set)	425
keys (Set)	425
NOW (dateTime)	439
TODAY (dateTime)	439
PAST (dateTime)	439
FUTURE (dateTime)	439
UNDEFINED (dateTime)	439
to (Message)	447
from (Message)	447
body (Message)	447
sentAt (Message)	447
messageId (Message)	447
cc (Message)	447
subject (Message)	447
read (Message)	447
to (SendMessage)	448
cc (SendMessage)	448
bcc (SendMessage)	448
subject (SendMessage)	448
message (SendMessage)	448

attachments (SendMessage)	448
fileName (Attachment)	448
content (Attachment)	448
subject (SendBroadcastMessage)	449
message (SendBroadcastMessage)	449
to (SendBroadcastMessage)	449
expiry (SendBroadcastMessage)	449
id (User)	453
name (User)	453
displayName (User)	453
email (User)	453
forwardToEmail (User)	453
name (UserProfile)	454
password (UserProfile)	454
email (UserProfile)	454
profilePicture (UserProfile)	454
forwardToEmail (UserProfile)	454
licenseType (UserProfile)	454
userLicenseType (UserProfile)	454
visibilityLevel (UserProfile)	454
isAccountDisabled (UserProfile)	454
isSSOOnly (UserProfile)	455
contactInfo (UserProfile)	455
title (ContactInfo)	455
phone (ContactInfo)	455
fax (ContactInfo)	455
mobile (ContactInfo)	455
pager (ContactInfo)	455
location (ContactInfo)	455
country (ContactInfo)	455
notes (ContactInfo)	455
id (UserCollectionObject)	459
name (UserCollectionObject)	459
displayName (UserCollectionObject)	459
profilePicture (UserCollectionObject)	459

email (UserCollectionObject)	459
forwardToEmail (UserCollectionObject)	459
licenseType (UserCollectionObject)	460
userLicenseType (UserCollectionObject)	460
visibilityLevel (UserCollectionObject)	460
isAccountDisabled (UserCollectionObject)	460
isloggedIn (UserCollectionObject)	460
title (UserCollectionObject)	460
phone (UserCollectionObject)	460
fax (UserCollectionObject)	460
mobile (UserCollectionObject)	460
pager (UserCollectionObject)	460
location (UserCollectionObject)	460
country (UserCollectionObject)	460
notes (UserCollectionObject)	460
displayName (LoggedInUser)	463
email (LoggedInUser)	463
forwardToEmail (LoggedInUser)	463
name (LoggedInUser)	463
profilePicture (LoggedInUser)	463
openResourceOnSignIn (LoggedInUser)	464
userId (LoggedInUser)	464
messageCenter (Permissions)	464
sendLinks (Permissions)	464
name (Group)	467
displayName (Group)	467
id (Group)	467
description (GroupProfile)	467
notifyNewMembers (GroupProfile)	467
notificationSubject (GroupProfile)	468
notificationBody (GroupProfile)	468
name (GroupCollectionObject)	470
displayName (GroupCollectionObject)	470
id (GroupCollectionObject)	470
members (GroupCollectionObject)	470

isGroup (Members)470

isUser (Members) 470

World Headquarters

3199 Palladium Drive
Ottawa, Ontario
Canada K2T 0N9

Tel. +1 613.592.5780
Toll free. +1 866.236.3249
Support. +1 866.463.7877

Email. info@kinaxis.com



Want knowledge about
RapidResponse from Kinaxis-
vetted experts? Join the Kinaxis
Knowledge Network!

<https://knowledge.kinaxis.com>